# UDCT v7.13 — Post-registration Research Findings and Technical Record

**Won Shik Paik** · Independent Researcher · Auckland, New Zealand
22 October 2026 · Complete Research Report

## Abstract

This retrospective technical report audits an archived six-arm numerical investigation of Jeans-equation sign boundaries in screened modified-gravity models, using observation-informed Hydrus I modelling inputs. Twenty-four inherited targets (B01–B24) comprise 48 endpoints and 288 saved profiles. Under the archived strict rule, 18 endpoints are robust positive, 30 are unresolved within the numerical margin, and none is robust negative. No one of the 24 proposed strict sign-transition brackets is validated. The negative native-grid minima seen in selected inner-radius samples do not establish a negative witness at a shared comparison radius. Separate numerical Hessian-stencil and discrete-action checks document implementation behaviour, but the full physical three-dimensional total-potential Hessian, validated exterior boundary conditions, and direct fits to individual stellar velocities remain unverified. The outcome is an auditable constraint on numerical interpretation, not validation or falsification of modified gravity.

## 1. Research question and scope

The programme asks whether apparent changes in the sign of an anisotropic Jeans second-moment solution persist when radial-domain alignment, resolution changes, numerical tolerances and the distinction between one-dimensional sign diagnostics and a genuine three-dimensional Hessian are enforced. The v7.13 archive inherits candidate boundaries from v7.12; it does not independently establish a new gravitational field law.

## 2. Mathematical framework

For a steady spherical tracer system, the anisotropic Jeans equation is d(nu sigma_r^2)/dr + 2 beta nu sigma_r^2/r = -nu g(r). Here nu is tracer density, sigma_r^2 is radial velocity dispersion, beta is anisotropy and g is the model radial acceleration. The sign of a numerical second-moment solution must not be conflated with an observed negative velocity dispersion. A physical Hessian H_ij = partial_i partial_j Phi_total requires a consistent full 3-D potential and boundary conditions; it is not supplied by differentiating a 1-D Jeans-sign curve.

## 3. Method and decision rule

Six arms compare default, refined and validation grids at two inner cutoffs. The common comparison domain starts at 0.002 half-light radii. The archived error envelope uses E, the maximum pointwise difference across normalized profiles, and tau = max(1e-8, 5E). Robust positive requires F > tau; robust negative additionally requires F < -tau and a common sampled radius at which every arm lies below -tau. Otherwise the endpoint is unresolved. The minimum F includes native-grid information that is not wholly recoverable from the common-domain NPZ.

ARCHIVE AUDIT: 18 robust-positive / 30 unresolved / 0 robust-negative endpoints.
B18 at 2.0 is robust-positive (F=0.263410 > tau=0.121287).

## 4. Principal numerical findings

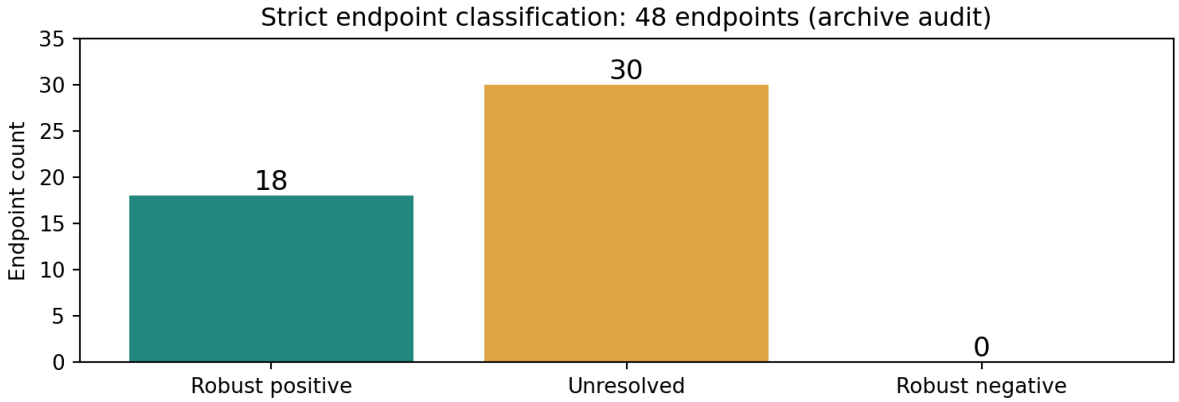



**Figure 1.** Endpoint counts under the archived strict decision rule. These are numerical classifications, not observational success probabilities or evidence that negative solutions are absent.

All 24 candidate brackets remain unresolved under the strict opposite-sign requirement. The archive contains 288 six-arm profiles with 1,200 common-domain samples per profile. Seven native-minimum F entries lack independent reconstruction from the common-domain archive and are conservatively left unresolved.

## 5. Illustrative boundary: B01

At beta = 0.35 and screening length 1 pc, the archived B01 native-grid minimum is F = -0.158964, while E = 0.0235645 and tau = 0.117822. Yet the common-domain minimum is +0.231148. The inner negative value therefore does not establish a robust negative common-radius witness. The beta = 0.40 endpoint is likewise unresolved (F = +0.078230, tau = 0.117165).

## 6. Interpretation and limitations

The main methodological distinction is between an inner native-grid minimum and a comparable negative band shared by all six arms. These are not interchangeable. Six-arm comparisons also share a numerical implementation and cannot substitute for an independent solver. Manufactured-field action-variation relative errors of 1.074e-9 and 2.374e-9 support particular code checks, not a verified physical 3-D Hydrus I Hessian. The full total-potential Hessian convergence gate remains NOT EVALUATED.

## 7. Observational relevance

Hydrus I observation-informed inputs were used in the modelling lineage. This is not the same as directly fitting the complete sample of individual stellar line-of-sight velocities. Neither a positive distribution function nor dynamical stability is established. The report makes no claim that MOND generally fails or that UDCT has been empirically confirmed.

## 8. Future research programme

**First:** recover or recompute all missing native-grid minima from original integrations, preserving the input configuration and checksums. **Second:** repeat the six-arm analysis using an independently implemented solver and verify common-radius negative witnesses. **Third:** test radial cutoff, boundary closure, anisotropy and screening-length sensitivity with preregistered acceptance thresholds.

**Fourth:** solve the complete 3-D total potential with physically justified exterior boundary conditions, and perform the specified Hessian convergence sequence at h/r_h = 0.02, 0.01 and 0.005. **Fifth:** where data access and membership modelling permit, fit the individual Hydrus I stellar line-of-sight velocity observations and quantify uncertainty rather than treating input calibration as direct validation.

## 9. Conclusion

The archived v7.13 results narrow the interpretation of apparent sign changes but do not validate a strict positive-to-negative bracket. Eighteen of 48 endpoints meet the robust-positive numerical criterion, 30 remain unresolved, and none meets the robust-negative criterion. None of the 24 proposed strict sign-transition brackets is validated. This is a documented unresolved boundary problem, not proof of a new gravity law, a universal exclusion of MOND, a completed 3-D Hessian calculation, or a direct observational test of stellar velocities.

## 10. Reproducibility and reading guide

The subsequent pages contain the full original report, figures, code inventory and detailed per-target discussion. The interactive contents provide direct access to B01–B24, mathematical explanations, numerical tests and Python source documentation. Tap the navigation link on the next page to jump to the interactive contents. Internal links depend on the PDF viewer.

## Source records

Paik, UDCT v7.13 preregistration (DOI: 10.5281/zenodo.23342971); v7.12 post-registration (DOI: 10.5281/zenodo.23296444); archived v7.13 post-registration evidence report and Hessian research archive. Chronology of preregistration amendments versus exploratory runs has not been independently established.

### Publication and reproducibility links

Zenodo reserved DOI: https://doi.org/10.5281/zenodo.23539127

GitHub: https://github.com/wspaik5/UDCT_v7.13_Postregistration_Hessian_Archive

The reserved DOI becomes registered upon Zenodo publication.

# Interactive Navigation

The preceding three pages form a self-contained, concise research paper: abstract, mathematical model, numerical results, interpretation, future work and conclusion.

For full detail, continue into the original technical document. Its interactive table of contents contains links to the B01–B24 analyses, figures, mathematical sections, Hessian implementation checks and code descriptions.

## OPEN INTERACTIVE CONTENTS →

If your PDF reader does not follow internal links, scroll forward to the INTERACTIVE CONTENTS pages.

# INTERACTIVE CONTENTS

Verified PDF page numbers | tap a title to open its section


Cover 10

Contents 11

Part I · Main Report 12

- Title and overview 13
- Executive abstract 14
- Mathematical background 15
- Global graphical summary 16
- B01·B24: individual profiles 17
- B01 · Six-arm profile 17
- B02 · Six-arm profile 18
- B03 · Six-arm profile 19
- B04 · Six-arm profile 20
- B05 · Six-arm profile 21
- B06 · Six-arm profile 22
- B07 · Six-arm profile 23
- B08 · Six-arm profile 24
- B09 · Six-arm profile 25
- B10 · Six-arm profile 26
- B11 · Six-arm profile 27
- B12 · Six-arm profile 28
- B13 · Six-arm profile 29
- B14 · Six-arm profile 30
- B15 · Six-arm profile 31
- B16 · Six-arm profile 32
- B17 · Six-arm profile 33
- B18 · Six-arm profile 34
- B19 · Six-arm profile 35
- B20 · Six-arm profile 36
- B21 · Six-arm profile 37

# INTERACTIVE CONTENTS

Verified PDF page numbers | tap a title to open its section


B22 · Six-arm profile 38

B23 · Six-arm profile 39

B24 · Six-arm profile 40

Source-code architecture 41

3-D Hessian verification status 44

Appendix A · File manifest 46

Appendix B · Source excerpts 50

Part II · Technical Supplement 56

Title and overview 57

Reading map and conventions 58

Part I · Mathematical derivation 59

Part II · B01·B24 numerical dossiers 61

B01 · Numerical dossier 61

B02 · Numerical dossier 62

B03 · Numerical dossier 63

B04 · Numerical dossier 64

B05 · Numerical dossier 65

B06 · Numerical dossier 66

B07 · Numerical dossier 67

B08 · Numerical dossier 68

B09 · Numerical dossier 69

B10 · Numerical dossier 70

B11 · Numerical dossier 71

B12 · Numerical dossier 72

B13 · Numerical dossier 73

B14 · Numerical dossier 74

B15 · Numerical dossier 75

B16 · Numerical dossier 76

B17 · Numerical dossier 77

# INTERACTIVE CONTENTS

Verified PDF page numbers | tap a title to open its section


B18 · Numerical dossier **78**

B19 · Numerical dossier **79**

B20 · Numerical dossier **80**

B21 · Numerical dossier **81**

B22 · Numerical dossier **82**

B23 · Numerical dossier **83**

B24 · Numerical dossier **84**

Part III · Annotated Python source atlas **85**

Source 01/77 · UDCT_v7_13_3D_Constitutive_Poisson_Coupling_Exploratory **86**

Source 02/77 · UDCT_v7_13_3D_Constitutive_Poisson_Coupling_Factor_Corrected **87**

Source 03/77 · UDCT_v7_13_3D_Poisson_Solver_Manufactured_Control **88**

Source 04/77 · UDCT_v7_13_3D_Potential_Implementation_Gate **89**

Source 05/77 · UDCT_v7_13_Acceleration_Richardson_Diagnostic **90**

Source 06/77 · UDCT_v7_13_Action_Variation_Factor_Audit **91**

Source 07/77 · UDCT_v7_13_B01_B24_Trace_Inventory **92**

Source 08/77 · UDCT_v7_13_B01_Explicit_Hessian_Ablation **93**

Source 09/77 · UDCT_v7_13_B01_Hessian_Law_Control_Exploratory **94**

Source 10/77 · UDCT_v7_13_B01_Independent_Jeans_Quadrature_Audit **95**

Source 11/77 · UDCT_v7_13_B01_Inner_Cutoff_Stability **96**

Source 12/77 · UDCT_v7_13_B01_Radial_Domain_Audit **97**

Source 13/77 · UDCT_v7_13_B01_Radial_Zero_Exploratory **98**

Source 14/77 · UDCT_v7_13_B01_Stress_Derivative_Decomposition **99**

Source 15/77 · UDCT_v7_13_B02_B24_Full_Profile_Trace **100**

Source 16/77 · UDCT_v7_13_Boundary_Moment_Cancellation_Audit **101**

Source 17/77 · UDCT_v7_13_Common_Coordinate_Acceleration_Audit **102**

Source 18/77 · UDCT_v7_13_Confirmatory_Readiness_Audit **103**

Source 19/77 · UDCT_v7_13_Constitutive_3D_Source_Prototype **104**

Source 20/77 · UDCT_v7_13_Constitutive_3D_Source_Revalidated **105**

Source 21/77 · UDCT_v7_13_Direct_Boundary_59cube_Audit **106**

# INTERACTIVE CONTENTS

Verified PDF page numbers | tap a title to open its section


Source 22/77 · UDCT_v7_13_Direct_Boundary_Radial_Audit **107**

Source 23/77 · UDCT_v7_13_Direct_Boundary_Resolution_Audit **108**

Source 24/77 · UDCT_v7_13_Direct_Source_Boundary_Audit **109**

Source 25/77 · UDCT_v7_13_Fixed_Box_Resolution_Audit **110**

Source 26/77 · UDCT_v7_13_Green_Cell_Quadrature_Audit **111**

Source 27/77 · UDCT_v7_13_Hessian_Equivalent_Density_Audit **112**

Source 28/77 · UDCT_v7_13_Hessian_External_Field_Audit **113**

Source 29/77 · UDCT_v7_13_Hessian_Factor_Grid_Convergence **114**

Source 30/77 · UDCT_v7_13_Hessian_Factor_Stress_Audit **115**

Source 31/77 · UDCT_v7_13_Hessian_Preflight_and_Estimator_Selftest **116**

Source 32/77 · UDCT_v7_13_Hessian_Preflight_and_Estimator_Selftest **117**

Source 33/77 · UDCT_v7_13_Hydrus_3D_Angular_Newtonian_Gate **118**

Source 34/77 · UDCT_v7_13_Hydrus_3D_DeltaPotential_Exploratory **119**

Source 35/77 · UDCT_v7_13_Hydrus_Box_Boundary_Sensitivity **120**

Source 36/77 · UDCT_v7_13_Hydrus_Common_Frame_Newtonian_Audit **121**

Source 37/77 · UDCT_v7_13_Hydrus_Ell_Sensitivity_Local_Source **122**

Source 38/77 · UDCT_v7_13_Hydrus_External_Field_3D_Input_Gate **123**

Source 39/77 · UDCT_v7_13_Hydrus_External_Field_vs_Tide_Audit **124**

Source 40/77 · UDCT_v7_13_Hydrus_External_Grid_Source_Audit **125**

Source 41/77 · UDCT_v7_13_Hydrus_Hessian_Increment_Convergence_Audit **126**

Source 42/77 · UDCT_v7_13_Hydrus_Hessian_Orientation_Bounds **127**

Source 43/77 · UDCT_v7_13_Hydrus_Hessian_Scale_Comparison **128**

Source 44/77 · UDCT_v7_13_Hydrus_Local_3D_Source_Gate **129**

Source 45/77 · UDCT_v7_13_Hydrus_Plummer_3D_Hessian_Control **130**

Source 46/77 · UDCT_v7_13_Hydrus_Radial_Tidal_Scan **131**

Source 47/77 · UDCT_v7_13_Independent_Endpoint_Decision_Audit **132**

Source 48/77 · UDCT_v7_13_Joint_Grid_Box_StopGate_Audit **133**

Source 49/77 · UDCT_v7_13_Multipole_Boundary_Exploratory **134**

Source 50/77 · UDCT_v7_13_Poisson_Sign_Units_Boundary_Gate **135**

# INTERACTIVE CONTENTS

Verified PDF page numbers | tap a title to open its section


Source 51/77 · UDCT_v7_13_Quadrupole_Boundary_Diagnostic **136**

Source 52/77 · UDCT_v7_13_Radial_Acceleration_Convergence **137**

Source 53/77 · UDCT_v7_13_Radial_Box_Size_Audit **138**

Source 54/77 · UDCT_v7_13_Source_Localization_Boundary_Audit **139**

Source 55/77 · v713_b01_six_arm_profile_audit **140**

Source 56/77 · v713_b02_six_arm_profile_audit **141**

Source 57/77 · v713_b03_six_arm_profile_audit **142**

Source 58/77 · v713_b04_six_arm_profile_audit **143**

Source 59/77 · v713_b05_six_arm_profile_audit **144**

Source 60/77 · v713_b07_six_arm_profile_audit **145**

Source 61/77 · v713_b08_six_arm_profile_audit **146**

Source 62/77 · v713_b09_six_arm_profile_audit **147**

Source 63/77 · v713_b10_six_arm_profile_audit **148**

Source 64/77 · v713_b11_six_arm_profile_audit **149**

Source 65/77 · v713_b12_six_arm_profile_audit **150**

Source 66/77 · v713_b13_six_arm_profile_audit **151**

Source 67/77 · v713_b14_six_arm_profile_audit **152**

Source 68/77 · v713_b15_six_arm_profile_audit **153**

Source 69/77 · v713_b16_six_arm_profile_audit **154**

Source 70/77 · v713_b17_six_arm_profile_audit **155**

Source 71/77 · v713_b19_six_arm_profile_audit **156**

Source 72/77 · v713_b20_six_arm_profile_audit **157**

Source 73/77 · v713_b21_six_arm_profile_audit **158**

Source 74/77 · v713_b22_six_arm_profile_audit **159**

Source 75/77 · v713_b23_six_arm_profile_audit **160**

Source 76/77 · v713_b24_six_arm_profile_audit **161**

Source 77/77 · v713_validate_b01 **162**

Part IV · Evidence ledger **163**

# UDCT v7.13

## Complete Research Report

### Main Report + Mathematical & Code Supplement

Won Shik Paik | 22 October 2026

LinkedIn reading edition

**RESEARCH STATUS**

Retrospective exploratory technical documentation.

The strict sign brackets and physical 3-D Hessian are not validated.

The two original PDFs are reproduced unchanged after the front matter.

# Contents

**Part I | Main Report** 4

43 original pages | consolidated pages 4-46

**Part II | Technical Supplement** 48

114 original pages | consolidated pages 48-161

## Reading guide

Begin with Part I for the research questions, B01-B24 results and limitations.

Use Part II for mathematical derivations, archived source-code descriptions, numerical diagnostics and the reproducibility inventory.

Page numbers printed within each original part restart at page 1.

PDF bookmarks identify the start of each part.

PAGE NUMBER CORRECTION: Earlier consolidated page labels refer to the original preface-free edition. Use the verified INTERACTIVE CONTENTS on PDF pages 5?9. Part I starts on PDF page 12; Part II follows later.

PART I

# Main Report

## Comprehensive Technical Monograph

43 pages | B01-B24 profiles, numerical findings and limitations

# UDCT v7.13: Comprehensive Technical Monograph

A source-linked, illustrated audit of B01–B24 six-arm Jeans profiles, sign-boundary criteria, 3-D Hessian implementation studies, numerical diagnostics and reproducibility limitations

Won Shik Paik | Independent research archive | 22 October 2026

Status: retrospective exploratory technical documentation. NOT a successful confirmatory preregistered Hessian validation.

Primary source: user-supplied 7.13 Hessian.zip (162 archived files: 81 JSON, 77 Python, 2 NPZ, 2 TXT). Source filenames are preserved verbatim. The original archive is not modified.

# Executive abstract

This monograph documents all 24 inherited sign-transition targets without selecting cases based on their outcome. The archived six-arm inventory marks every bracket unresolved and records zero completed physical 3-D Hessian tests. A subsequent audit of the stored common-domain profiles reports 18 robust-positive endpoints, 30 unresolved endpoints , zero robust-negative endpoints and zero validated strict sign brackets. The distinction between negative inner native-grid minima and nonnegative or uncertain shared-domain profiles is essential: a local negative value does not establish a reproducible common-radius negative witness.

The purpose is traceability, not advocacy. Each target receives a figure, the preserved endpoint labels and a discussion of what can and cannot be inferred. The second half maps the source-code families and recorded implementation tests, explains the physical/mathematical gates, and provides a file-level inventory. Execution timestamps embedded in names are metadata only, not proof of scientific chronology or preregistration precedence.

# 1. Scope, epistemic status and terminology

## Archived numerical evidence

Profiles and audit JSON are stored results. A graph made from these data is a visualization of saved outputs, not an independently rerun galaxy simulation.

## Robust positive

A positive endpoint requires the registered minimum F to exceed the numerical tolerance tau.

## Robust negative

A negative endpoint requires F below minus tau AND a common sampled radius where all six profiles are below minus tau.

## Unresolved

An endpoint that does not meet either strict criterion. This includes missing native minima, domain mismatch, and insufficient numerical margin.

## Physical Hessian

H_ij is the 3-by-3 matrix of second spatial derivatives of the full three-dimensional total potential. A radial Jeans derivative or an externally imposed tidal tensor is not interchangeable with it.

## Observational boundary

Hydrus I observation-informed model inputs are distinct from direct likelihood fitting to the complete individual stellar line-of-sight velocity sample.

# 2. Theoretical framework and sign problem

For a spherical steady-state tracer population, d(nu sigma_r^2)/dr + 2 beta nu sigma_r^2/r = -nu g_r, with beta = 1 - sigma_t^2/(2 sigma_r^2) under the usual two-component tangential convention. The solution depends on tracer density nu, gravitational acceleration g_r, anisotropy closure, and integration/boundary conditions. A negative calculated second moment signals a failure of that mathematical candidate under its assumptions; it is not a measurement of a negative stellar velocity variance.

In the archived B01 decomposition, the effective acceleration is written g_full = g_lead - 0.5 d(p_nn)/dr - 0.5 (3 p_nn - tr(p))/r. The stress derivative and geometric term must be retained with their signs. Their integrated contributions are not pointwise accelerations and should not be conflated with them. This relation is an archived exploratory model expression, not a verified universal gravity law.

A Hessian-sensitive constitutive law requires differentiating the full total potential in Cartesian coordinates, establishing a physically justified exterior boundary, and showing resolution convergence. Manufactured-field stencil tests can check differentiation algebra but cannot establish that a galaxy potential is physically solved.

## 3. Registered numerical decision logic

The archived six-arm comparisons use default 360 x 24 x 36, refined 720 x 48 x 72 and validation 1440 x 96 x 144 settings, each at inner cutoffs 0.002 and 0.0005 r_h. The common comparison domain starts at 0.002 r_h. For each endpoint E is the maximum pointwise six-profile difference and tau = max(1e-8, 5E). The archived decision F combines six common-domain minima and six native-grid minima. Missing native minima cannot be recovered from common-domain NPZ alone.

A strict sign bracket needs two opposite robust endpoint signs. An earlier N/P label change or an apparent zero crossing does not suffice. Six arms built on one underlying numerical approach are a resolution sensitivity check, not a fully independent solver cross-validation.

## 4. Global graphical summary

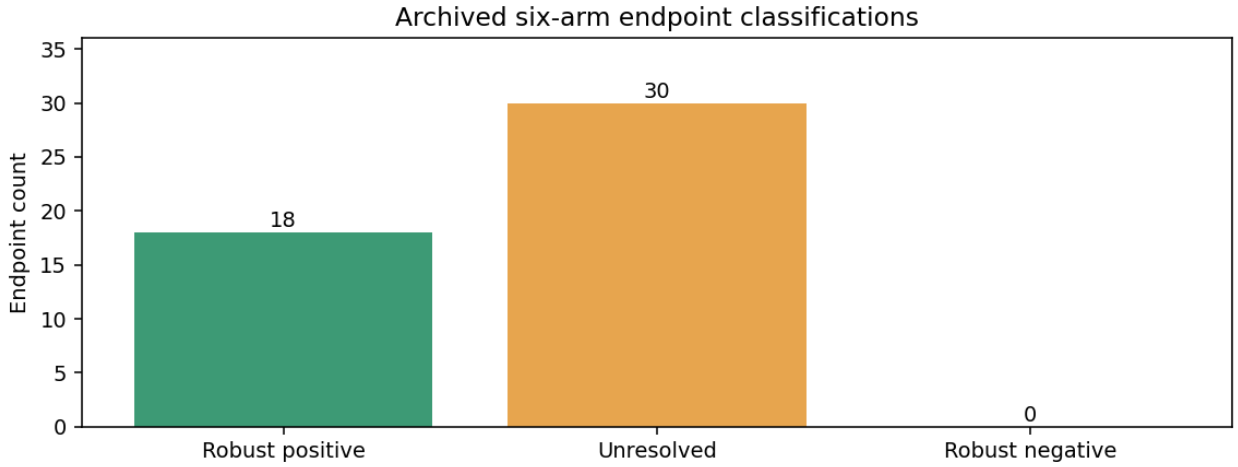



Figure 1. Summary visualization from the archived technical-chronicle figure set. Interpret alongside the decision counts and source audit rather than as an observational success rate.

Archived global inventory: {'unresolved_numerical_margin': 24}. All 24 B01–B24 targets remain unresolved; recorded executed physical Hessian count: 0.

## 5. Target-by-target profile atlas (B01–B24)

# 5.01 B01: six-arm profile evidence

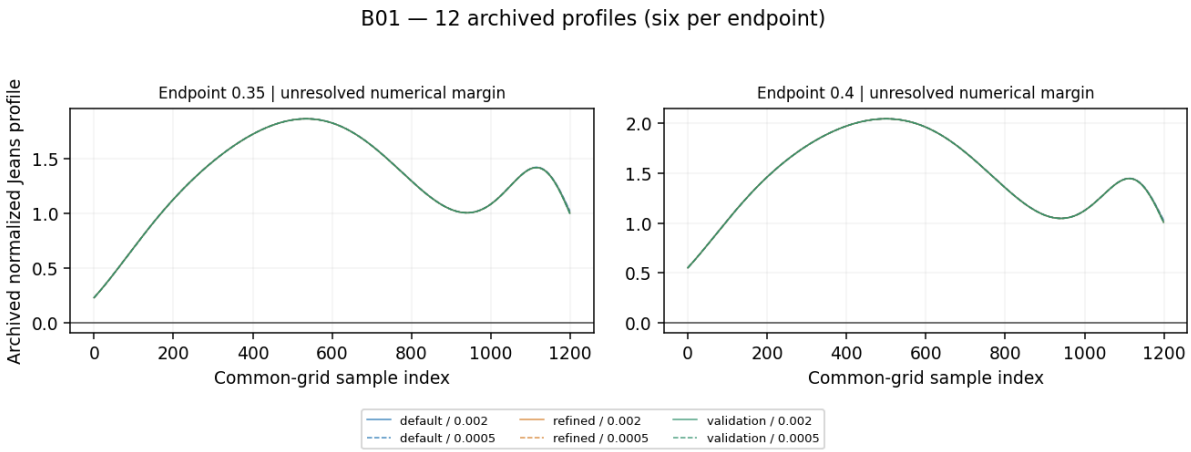



Figure 2. Archived six-arm profile visualization for B01. Compare both endpoint families across radial coordinate; any visual separation must be judged against the registered common-radius and tolerance requirements. Curves are saved exploratory numerical outputs, not observed stellar measurements.

Archived bracket status: unresolved_numerical_margin. Endpoint labels: 0.35 = unresolved_numerical_margin, 0.4 = unresolved_numerical_margin. Six-arm execution flag: True; physical Hessian execution flag: False.

Target-specific archived JSON: UDCT_v7_13_B01_Six_Arm_Profile_Exploratory_20261021_012443_978.json

Recorded keys: classification, target, sha256_verified, six_arms_executed, hessian_executed, endpoint_results, bracket_status, arms, common_grid_warning, elapsed_seconds.

Interpretation: The saved endpoint labels must not be promoted to a confirmed sign boundary unless opposite robust signs are demonstrated at the same shared radii under the registered six-arm tolerance. Apparent negative excursions in the inner native grid may be excluded from the common comparison interval, and uncertainty-envelope overlap may leave the result unresolved. This is a numerical admissibility diagnosis under the chosen Jeans closure, not a dynamical stability proof.

Reproduction trail: read the common-profile NPZ for all six arms; consult the matching target exploratory JSON and the six-arm inventory; recover native-grid minimum evidence from the separate archived audit if available; apply the preregistered decision gate without modifying tolerance after inspecting the plot.

## B01 stress decomposition: source-linked special case

Grid default, beta 0.35: {'lead': 0.6578808776437878, 'radial_derivative': -0.0026059737935458793, 'geometric_stress': -0.814239060567588, 'sum': -0.1589641567173461, 'full': -0.15896415671734707, 'linearity_residual': -9.159339953157541e-16}

Grid default, beta 0.4: {'lead': 1.0507555991201327, 'radial_derivative': -0.0027794874822506532, 'geometric_stress': -0.9697461693639589, 'sum': 0.07822994227392321, 'full': 0.0782299422739232, 'linearity_residual': -1.3877787807814457e-17}

Grid refined, beta 0.35: {'lead': 0.6577987319171928, 'radial_derivative': -0.0026051582819441865, 'geometric_stress': -0.8141253807553865, 'sum': -0.15893180712013788, 'full': -0.1589318071201378, 'linearity_residual': 8.326672684688674e-17}

The geometric stress term dominates the negative inner-grid integrated contribution in the illustrated beta=0.35 case; the radial derivative is comparatively small. This is a decomposition at one setup and must not be generalized to B02–B24 without corresponding decomposition records.

# 5.02 B02: six-arm profile evidence

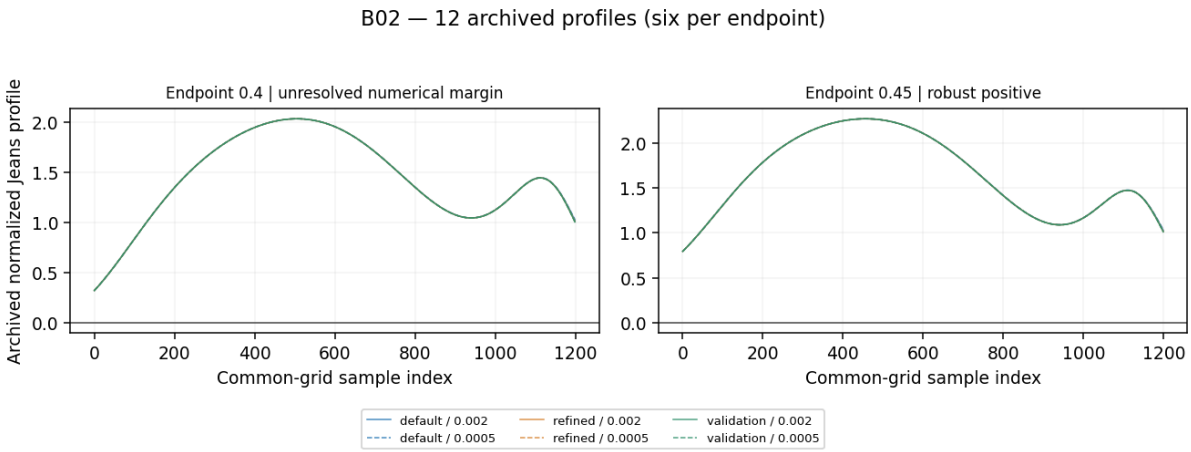



Figure 3. Archived six-arm profile visualization for B02. Compare both endpoint families across radial coordinate; any visual separation must be judged against the registered common-radius and tolerance requirements. Curves are saved exploratory numerical outputs, not observed stellar measurements.

Archived bracket status: unresolved_numerical_margin. Endpoint labels: 0.4 = unresolved_numerical_margin, 0.45 = robust_positive. Six-arm execution flag: True; physical Hessian execution flag: False.

Target-specific archived JSON: UDCT_v7_13_B02_Six_Arm_Profile_Exploratory_20261021_012440_907.json

Recorded keys: classification, target, sha256_verified, six_arms_executed, hessian_executed, endpoint_results, bracket_status, arms, common_grid_warning, elapsed_seconds.

Interpretation: The saved endpoint labels must not be promoted to a confirmed sign boundary unless opposite robust signs are demonstrated at the same shared radii under the registered six-arm tolerance. Apparent negative excursions in the inner native grid may be excluded from the common comparison interval, and uncertainty-envelope overlap may leave the result unresolved. This is a numerical admissibility diagnosis under the chosen Jeans closure, not a dynamical stability proof.

Reproduction trail: read the common-profile NPZ for all six arms; consult the matching target exploratory JSON and the six-arm inventory; recover native-grid minimum evidence from the separate archived audit if available; apply the preregistered decision gate without modifying tolerance after inspecting the plot.

## 5.03 B03: six-arm profile evidence

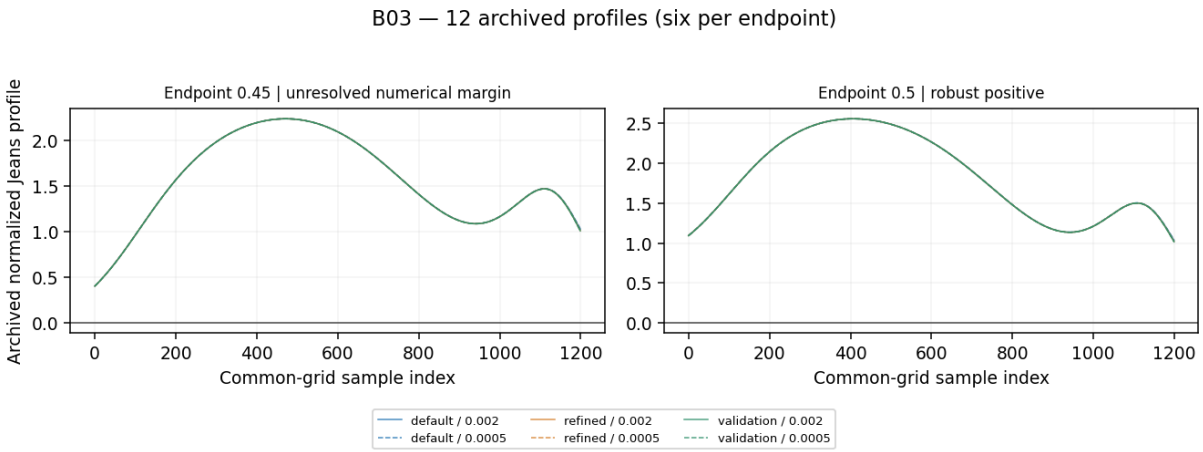



Figure 4. Archived six-arm profile visualization for B03. Compare both endpoint families across radial coordinate; any visual separation must be judged against the registered common-radius and tolerance requirements. Curves are saved exploratory numerical outputs, not observed stellar measurements.

Archived bracket status: unresolved_numerical_margin. Endpoint labels: 0.45 = unresolved_numerical_margin, 0.5 = robust_positive. Six-arm execution flag: True; physical Hessian execution flag: False.

Target-specific archived JSON: UDCT_v7_13_B03_Six_Arm_Profile_Exploratory_20261021_012436_828.json

Recorded keys: classification, target, sha256_verified, six_arms_executed, hessian_executed, endpoint_results, bracket_status, arms, common_grid_warning, elapsed_seconds.

Interpretation: The saved endpoint labels must not be promoted to a confirmed sign boundary unless opposite robust signs are demonstrated at the same shared radii under the registered six-arm tolerance. Apparent negative excursions in the inner native grid may be excluded from the common comparison interval, and uncertainty-envelope overlap may leave the result unresolved. This is a numerical admissibility diagnosis under the chosen Jeans closure, not a dynamical stability proof.

Reproduction trail: read the common-profile NPZ for all six arms; consult the matching target exploratory JSON and the six-arm inventory; recover native-grid minimum evidence from the separate archived audit if available; apply the preregistered decision gate without modifying tolerance after inspecting the plot.

# 5.04 B04: six-arm profile evidence

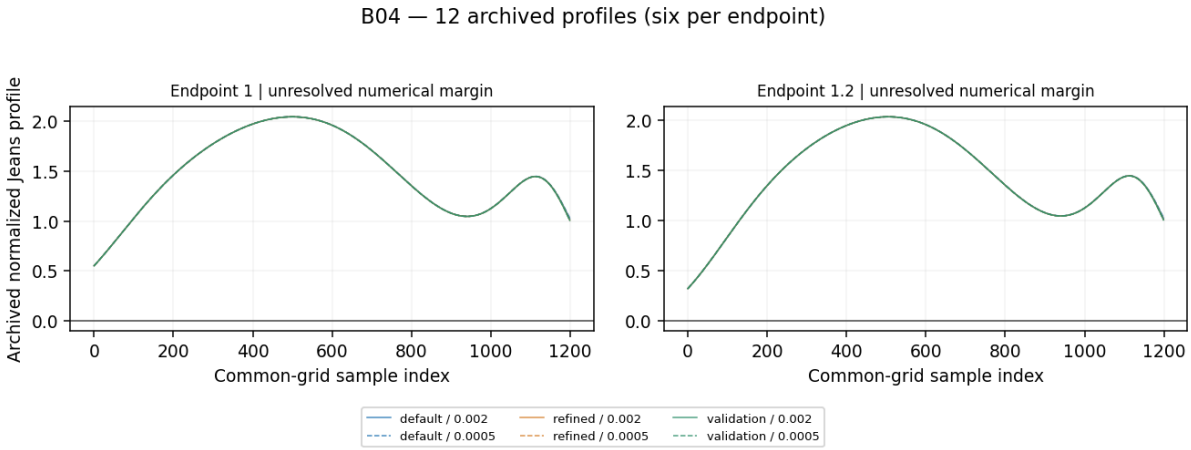



Figure 5. Archived six-arm profile visualization for B04. Compare both endpoint families across radial coordinate; any visual separation must be judged against the registered common-radius and tolerance requirements. Curves are saved exploratory numerical outputs, not observed stellar measurements.

Archived bracket status: unresolved_numerical_margin. Endpoint labels: 1.0 = unresolved_numerical_margin, 1.2 = unresolved_numerical_margin. Six-arm execution flag: True; physical Hessian execution flag: False.

Target-specific archived JSON: UDCT_v7_13_B04_Six_Arm_Profile_Exploratory_20261021_012433_635.json

Recorded keys: classification, target, sha256_verified, six_arms_executed, hessian_executed, endpoint_results, bracket_status, arms, common_grid_warning, elapsed_seconds.

Interpretation: The saved endpoint labels must not be promoted to a confirmed sign boundary unless opposite robust signs are demonstrated at the same shared radii under the registered six-arm tolerance. Apparent negative excursions in the inner native grid may be excluded from the common comparison interval, and uncertainty-envelope overlap may leave the result unresolved. This is a numerical admissibility diagnosis under the chosen Jeans closure, not a dynamical stability proof.

Reproduction trail: read the common-profile NPZ for all six arms; consult the matching target exploratory JSON and the six-arm inventory; recover native-grid minimum evidence from the separate archived audit if available; apply the preregistered decision gate without modifying tolerance after inspecting the plot.

# 5.05 B05: six-arm profile evidence

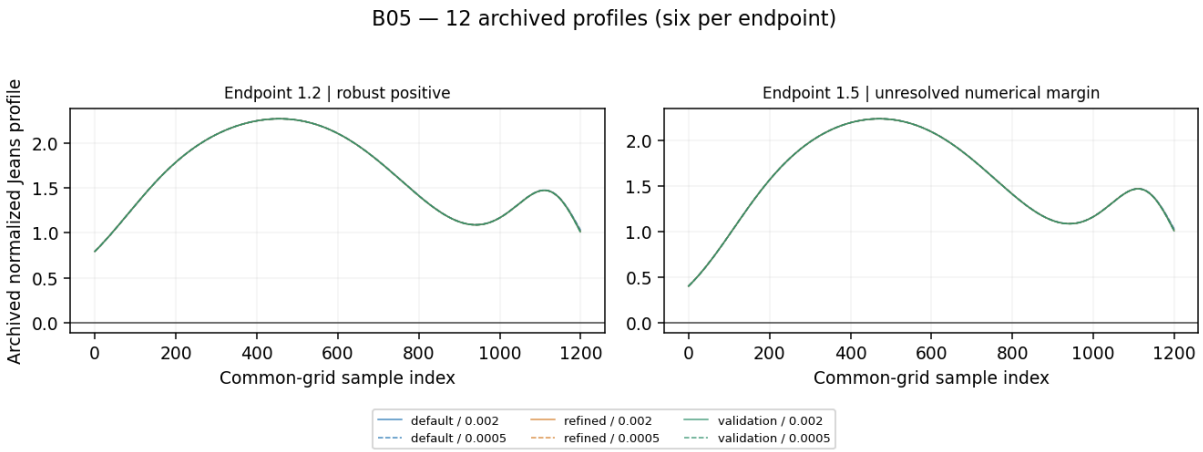



Figure 6. Archived six-arm profile visualization for B05. Compare both endpoint families across radial coordinate; any visual separation must be judged against the registered common-radius and tolerance requirements. Curves are saved exploratory numerical outputs, not observed stellar measurements.

Archived bracket status: unresolved_numerical_margin. Endpoint labels: 1.2 = robust_positive, 1.5 = unresolved_numerical_margin. Six-arm execution flag: True; physical Hessian execution flag: False.

Target-specific archived JSON: UDCT_v7_13_B05_Six_Arm_Profile_Exploratory_20261021_012429_424.json

Recorded keys: classification, target, sha256_verified, six_arms_executed, hessian_executed, endpoint_results, bracket_status, arms, common_grid_warning, elapsed_seconds.

Interpretation: The saved endpoint labels must not be promoted to a confirmed sign boundary unless opposite robust signs are demonstrated at the same shared radii under the registered six-arm tolerance. Apparent negative excursions in the inner native grid may be excluded from the common comparison interval, and uncertainty-envelope overlap may leave the result unresolved. This is a numerical admissibility diagnosis under the chosen Jeans closure, not a dynamical stability proof.

Reproduction trail: read the common-profile NPZ for all six arms; consult the matching target exploratory JSON and the six-arm inventory; recover native-grid minimum evidence from the separate archived audit if available; apply the preregistered decision gate without modifying tolerance after inspecting the plot.

# 5.06 B06: six-arm profile evidence

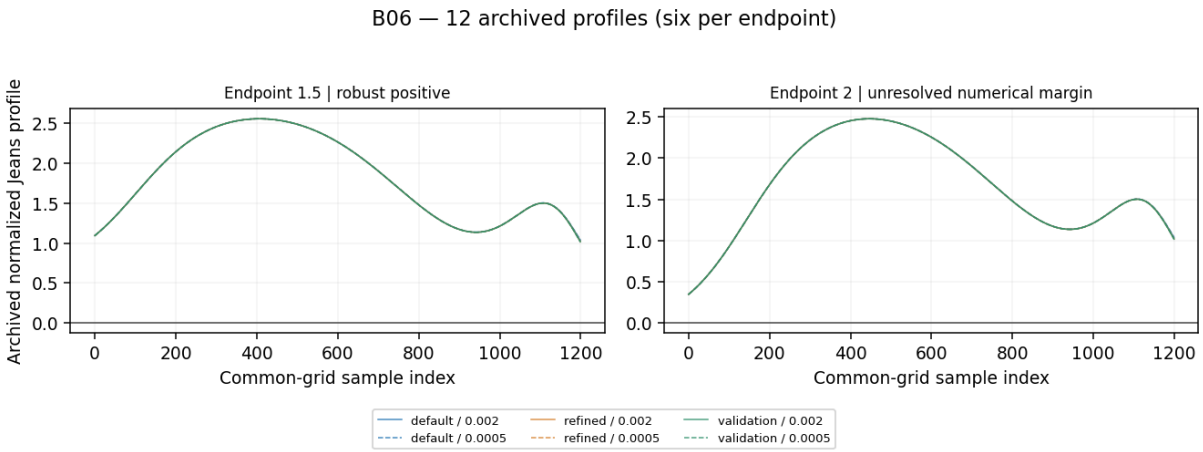



Figure 7. Archived six-arm profile visualization for B06. Compare both endpoint families across radial coordinate; any visual separation must be judged against the registered common-radius and tolerance requirements. Curves are saved exploratory numerical outputs, not observed stellar measurements.

Archived bracket status: unresolved_numerical_margin. Endpoint labels: 1.5 = robust_positive, 2.0 = unresolved_numerical_margin. Six-arm execution flag: True; physical Hessian execution flag: False.

Target-specific archived JSON: UDCT_v7_13_B06_Six_Arm_Profile_Exploratory_20261021_012425_495.json

Recorded keys: classification, target, sha256_verified, six_arms_executed, hessian_executed, endpoint_results, bracket_status, arms, common_grid_warning, elapsed_seconds.

Interpretation: The saved endpoint labels must not be promoted to a confirmed sign boundary unless opposite robust signs are demonstrated at the same shared radii under the registered six-arm tolerance. Apparent negative excursions in the inner native grid may be excluded from the common comparison interval, and uncertainty-envelope overlap may leave the result unresolved. This is a numerical admissibility diagnosis under the chosen Jeans closure, not a dynamical stability proof.

Reproduction trail: read the common-profile NPZ for all six arms; consult the matching target exploratory JSON and the six-arm inventory; recover native-grid minimum evidence from the separate archived audit if available; apply the preregistered decision gate without modifying tolerance after inspecting the plot.

# 5.07 B07: six-arm profile evidence

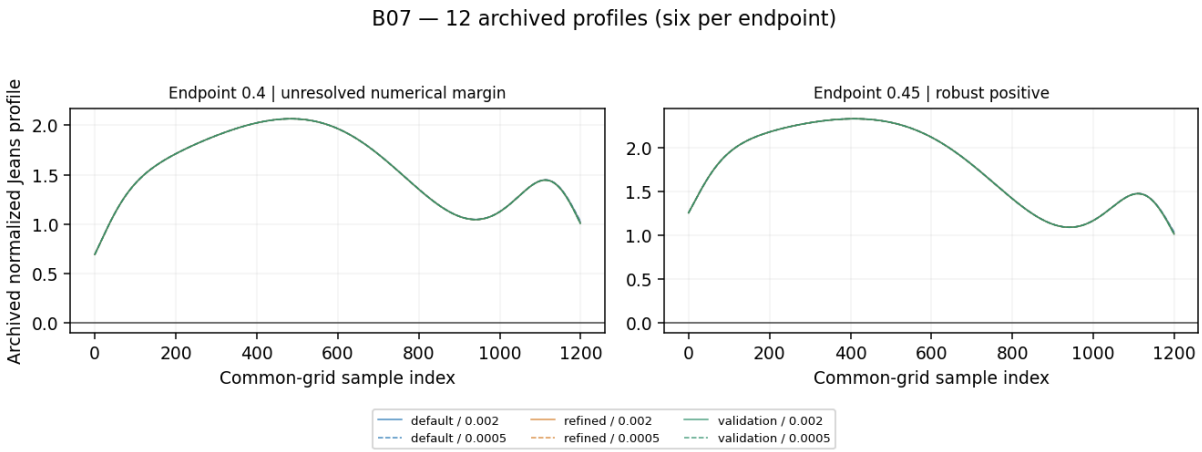



Figure 8. Archived six-arm profile visualization for B07. Compare both endpoint families across radial coordinate; any visual separation must be judged against the registered common-radius and tolerance requirements. Curves are saved exploratory numerical outputs, not observed stellar measurements.

Archived bracket status: unresolved_numerical_margin. Endpoint labels: 0.4 = unresolved_numerical_margin, 0.45 = robust_positive. Six-arm execution flag: True; physical Hessian execution flag: False.

Target-specific archived JSON: UDCT_v7_13_B07_Six_Arm_Profile_Exploratory_20261021_012418_036.json

Recorded keys: classification, target, sha256_verified, six_arms_executed, hessian_executed, endpoint_results, bracket_status, arms, common_grid_warning, elapsed_seconds.

Interpretation: The saved endpoint labels must not be promoted to a confirmed sign boundary unless opposite robust signs are demonstrated at the same shared radii under the registered six-arm tolerance. Apparent negative excursions in the inner native grid may be excluded from the common comparison interval, and uncertainty-envelope overlap may leave the result unresolved. This is a numerical admissibility diagnosis under the chosen Jeans closure, not a dynamical stability proof.

Reproduction trail: read the common-profile NPZ for all six arms; consult the matching target exploratory JSON and the six-arm inventory; recover native-grid minimum evidence from the separate archived audit if available; apply the preregistered decision gate without modifying tolerance after inspecting the plot.

# 5.08 B08: six-arm profile evidence

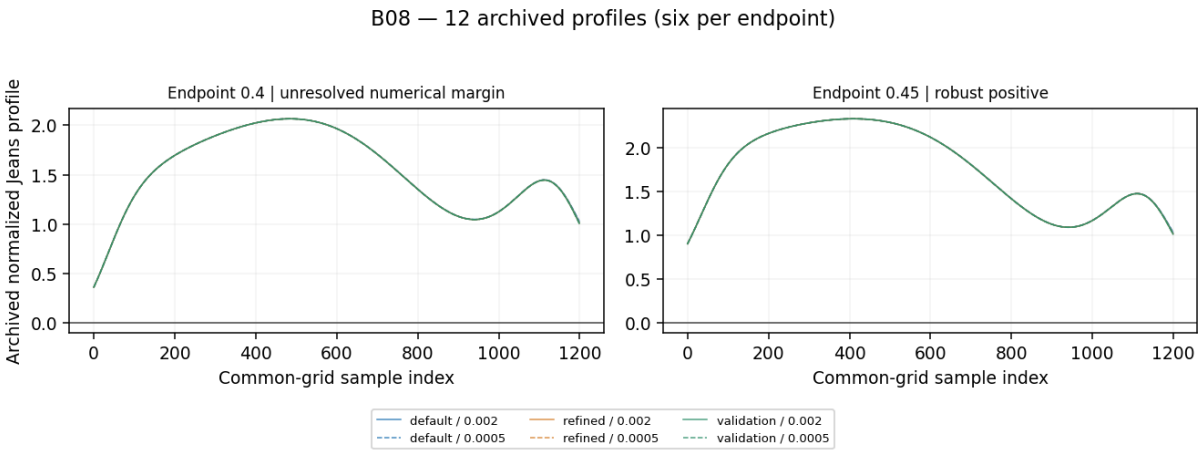



Figure 9. Archived six-arm profile visualization for B08. Compare both endpoint families across radial coordinate; any visual separation must be judged against the registered common-radius and tolerance requirements. Curves are saved exploratory numerical outputs, not observed stellar measurements.

Archived bracket status: unresolved_numerical_margin. Endpoint labels: 0.4 = unresolved_numerical_margin, 0.45 = robust_positive. Six-arm execution flag: True; physical Hessian execution flag: False.

Target-specific archived JSON: UDCT_v7_13_B08_Six_Arm_Profile_Exploratory_20261021_012413_596.json

Recorded keys: classification, target, sha256_verified, six_arms_executed, hessian_executed, endpoint_results, bracket_status, arms, common_grid_warning, elapsed_seconds.

Interpretation: The saved endpoint labels must not be promoted to a confirmed sign boundary unless opposite robust signs are demonstrated at the same shared radii under the registered six-arm tolerance. Apparent negative excursions in the inner native grid may be excluded from the common comparison interval, and uncertainty-envelope overlap may leave the result unresolved. This is a numerical admissibility diagnosis under the chosen Jeans closure, not a dynamical stability proof.

Reproduction trail: read the common-profile NPZ for all six arms; consult the matching target exploratory JSON and the six-arm inventory; recover native-grid minimum evidence from the separate archived audit if available; apply the preregistered decision gate without modifying tolerance after inspecting the plot.

# 5.09 B09: six-arm profile evidence

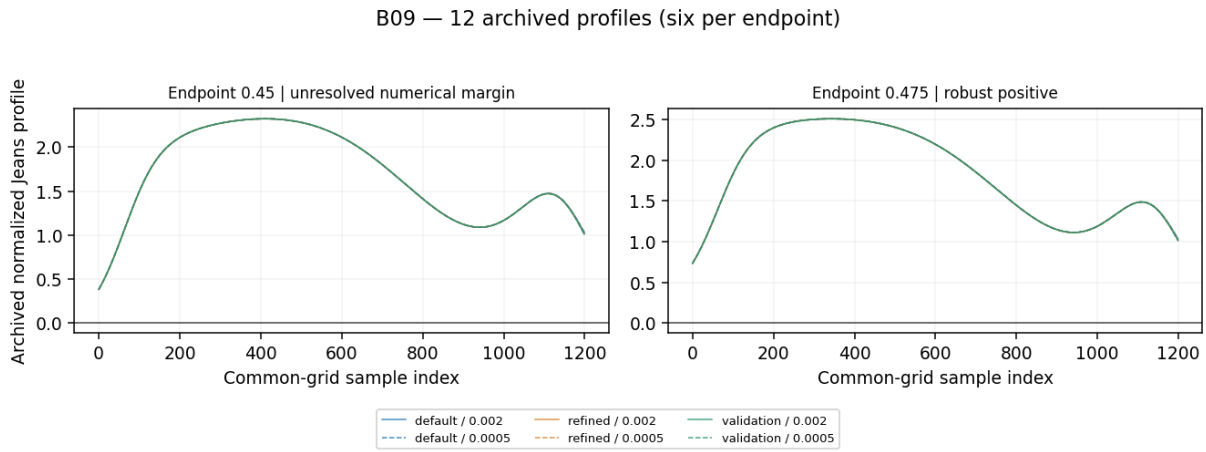



Figure 10. Archived six-arm profile visualization for B09. Compare both endpoint families across radial coordinate; any visual separation must be judged against the registered common-radius and tolerance requirements. Curves are saved exploratory numerical outputs, not observed stellar measurements.

Archived bracket status: unresolved_numerical_margin. Endpoint labels: 0.45 = unresolved_numerical_margin, 0.475 = robust_positive. Six-arm execution flag: True; physical Hessian execution flag: False.

Target-specific archived JSON: UDCT_v7_13_B09_Six_Arm_Profile_Exploratory_20261021_012409_760.json

Recorded keys: classification, target, sha256_verified, six_arms_executed, hessian_executed, endpoint_results, bracket_status, arms, common_grid_warning, elapsed_seconds.

Interpretation: The saved endpoint labels must not be promoted to a confirmed sign boundary unless opposite robust signs are demonstrated at the same shared radii under the registered six-arm tolerance. Apparent negative excursions in the inner native grid may be excluded from the common comparison interval, and uncertainty-envelope overlap may leave the result unresolved. This is a numerical admissibility diagnosis under the chosen Jeans closure, not a dynamical stability proof.

Reproduction trail: read the common-profile NPZ for all six arms; consult the matching target exploratory JSON and the six-arm inventory; recover native-grid minimum evidence from the separate archived audit if available; apply the preregistered decision gate without modifying tolerance after inspecting the plot.

## 5.10 B10: six-arm profile evidence

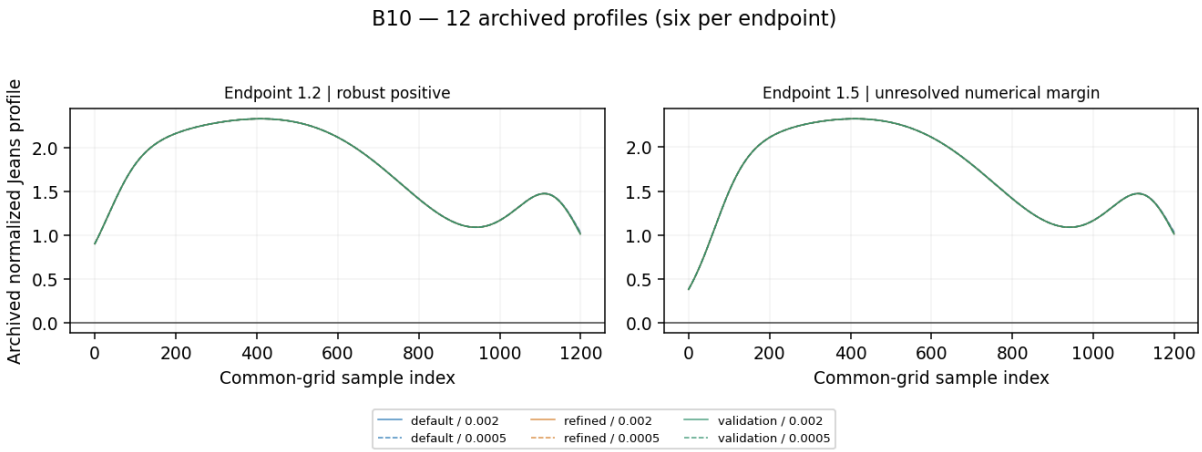



Figure 11. Archived six-arm profile visualization for B10. Compare both endpoint families across radial coordinate; any visual separation must be judged against the registered common-radius and tolerance requirements. Curves are saved exploratory numerical outputs, not observed stellar measurements.

Archived bracket status: unresolved_numerical_margin. Endpoint labels: 1.2 = robust_positive, 1.5 = unresolved_numerical_margin. Six-arm execution flag: True; physical Hessian execution flag: False.

Target-specific archived JSON: UDCT_v7_13_B10_Six_Arm_Profile_Exploratory_20261021_012405_115.json

Recorded keys: classification, target, sha256_verified, six_arms_executed, hessian_executed, endpoint_results, bracket_status, arms, common_grid_warning, elapsed_seconds.

Interpretation: The saved endpoint labels must not be promoted to a confirmed sign boundary unless opposite robust signs are demonstrated at the same shared radii under the registered six-arm tolerance. Apparent negative excursions in the inner native grid may be excluded from the common comparison interval, and uncertainty-envelope overlap may leave the result unresolved. This is a numerical admissibility diagnosis under the chosen Jeans closure, not a dynamical stability proof.

Reproduction trail: read the common-profile NPZ for all six arms; consult the matching target exploratory JSON and the six-arm inventory; recover native-grid minimum evidence from the separate archived audit if available; apply the preregistered decision gate without modifying tolerance after inspecting the plot.

## 5.11 B11: six-arm profile evidence

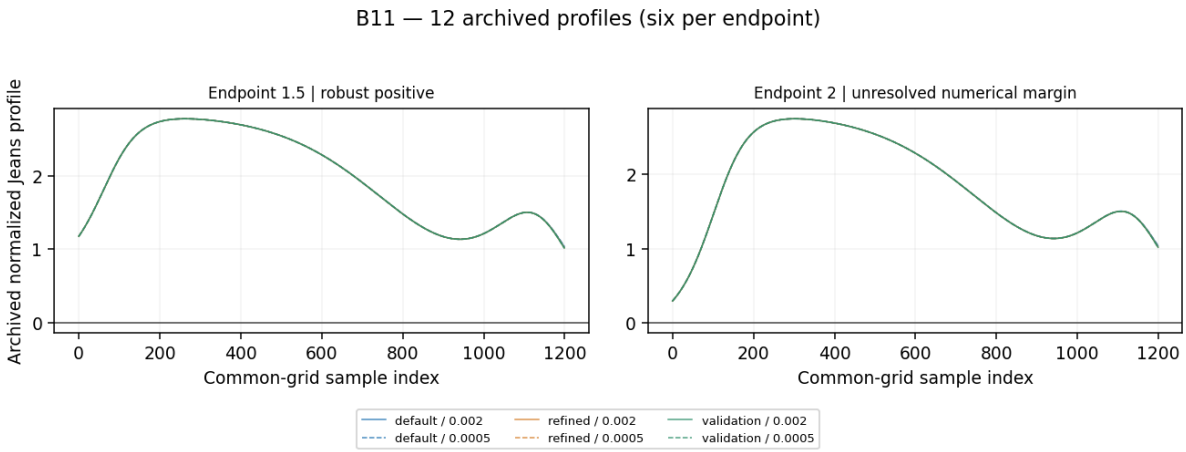



Figure 12. Archived six-arm profile visualization for B11. Compare both endpoint families across radial coordinate; any visual separation must be judged against the registered common-radius and tolerance requirements. Curves are saved exploratory numerical outputs, not observed stellar measurements.

Archived bracket status: unresolved_numerical_margin. Endpoint labels: 1.5 = robust_positive, 2.0 = unresolved_numerical_margin. Six-arm execution flag: True; physical Hessian execution flag: False.

Target-specific archived JSON: UDCT_v7_13_B11_Six_Arm_Profile_Exploratory_20261021_012358_483.json

Recorded keys: classification, target, sha256_verified, six_arms_executed, hessian_executed, endpoint_results, bracket_status, arms, common_grid_warning, elapsed_seconds.

Interpretation: The saved endpoint labels must not be promoted to a confirmed sign boundary unless opposite robust signs are demonstrated at the same shared radii under the registered six-arm tolerance. Apparent negative excursions in the inner native grid may be excluded from the common comparison interval, and uncertainty-envelope overlap may leave the result unresolved. This is a numerical admissibility diagnosis under the chosen Jeans closure, not a dynamical stability proof.

Reproduction trail: read the common-profile NPZ for all six arms; consult the matching target exploratory JSON and the six-arm inventory; recover native-grid minimum evidence from the separate archived audit if available; apply the preregistered decision gate without modifying tolerance after inspecting the plot.

## 5.12 B12: six-arm profile evidence

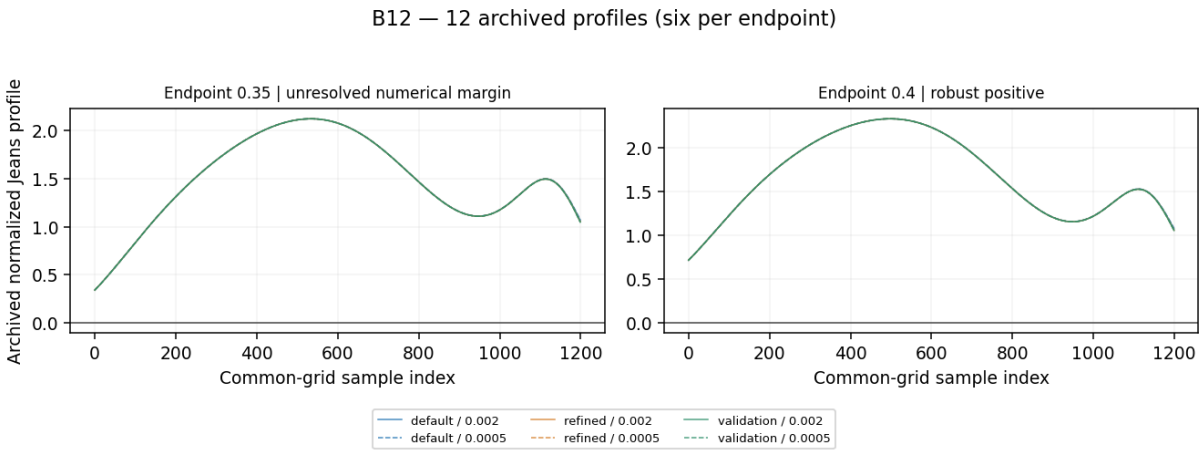



Figure 13. Archived six-arm profile visualization for B12. Compare both endpoint families across radial coordinate; any visual separation must be judged against the registered common-radius and tolerance requirements. Curves are saved exploratory numerical outputs, not observed stellar measurements.

Archived bracket status: unresolved_numerical_margin. Endpoint labels: 0.35 = unresolved_numerical_margin, 0.4 = robust_positive. Six-arm execution flag: True; physical Hessian execution flag: False.

Target-specific archived JSON: UDCT_v7_13_B12_Six_Arm_Profile_Exploratory_20261021_011759_654.json

Recorded keys: classification, target, sha256_verified, six_arms_executed, hessian_executed, endpoint_results, bracket_status, arms, common_grid_warning, elapsed_seconds.

Interpretation: The saved endpoint labels must not be promoted to a confirmed sign boundary unless opposite robust signs are demonstrated at the same shared radii under the registered six-arm tolerance. Apparent negative excursions in the inner native grid may be excluded from the common comparison interval, and uncertainty-envelope overlap may leave the result unresolved. This is a numerical admissibility diagnosis under the chosen Jeans closure, not a dynamical stability proof.

Reproduction trail: read the common-profile NPZ for all six arms; consult the matching target exploratory JSON and the six-arm inventory; recover native-grid minimum evidence from the separate archived audit if available; apply the preregistered decision gate without modifying tolerance after inspecting the plot.

## 5.13 B13: six-arm profile evidence

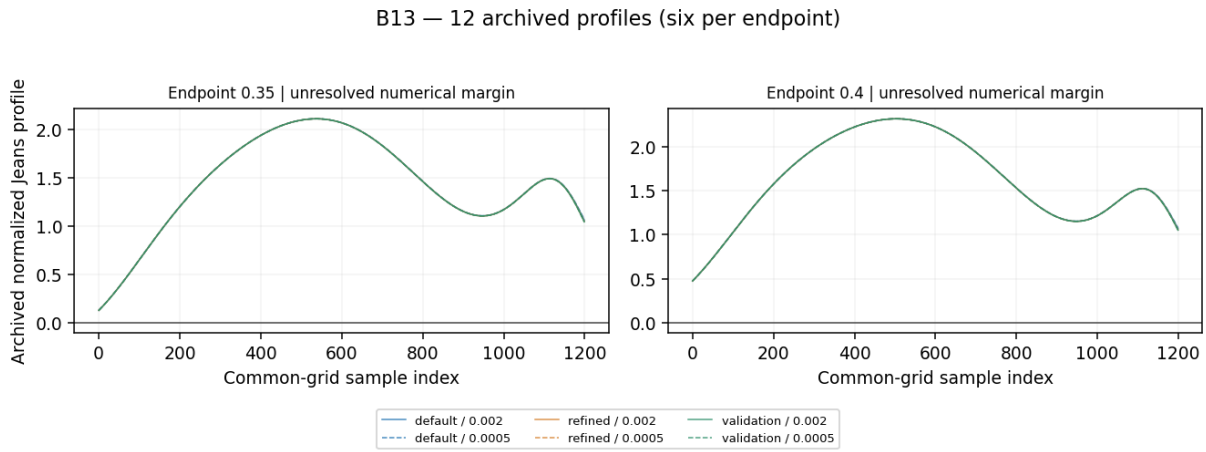



Figure 14. Archived six-arm profile visualization for B13. Compare both endpoint families across radial coordinate; any visual separation must be judged against the registered common-radius and tolerance requirements. Curves are saved exploratory numerical outputs, not observed stellar measurements.

Archived bracket status: unresolved_numerical_margin. Endpoint labels: 0.35 = unresolved_numerical_margin, 0.4 = unresolved_numerical_margin. Six-arm execution flag: True; physical Hessian execution flag: False.

Target-specific archived JSON: UDCT_v7_13_B13_Six_Arm_Profile_Exploratory_20261021_012349_829.json

Recorded keys: classification, target, sha256_verified, six_arms_executed, hessian_executed, endpoint_results, bracket_status, arms, common_grid_warning, elapsed_seconds.

Interpretation: The saved endpoint labels must not be promoted to a confirmed sign boundary unless opposite robust signs are demonstrated at the same shared radii under the registered six-arm tolerance. Apparent negative excursions in the inner native grid may be excluded from the common comparison interval, and uncertainty-envelope overlap may leave the result unresolved. This is a numerical admissibility diagnosis under the chosen Jeans closure, not a dynamical stability proof.

Reproduction trail: read the common-profile NPZ for all six arms; consult the matching target exploratory JSON and the six-arm inventory; recover native-grid minimum evidence from the separate archived audit if available; apply the preregistered decision gate without modifying tolerance after inspecting the plot.

## 5.14 B14: six-arm profile evidence

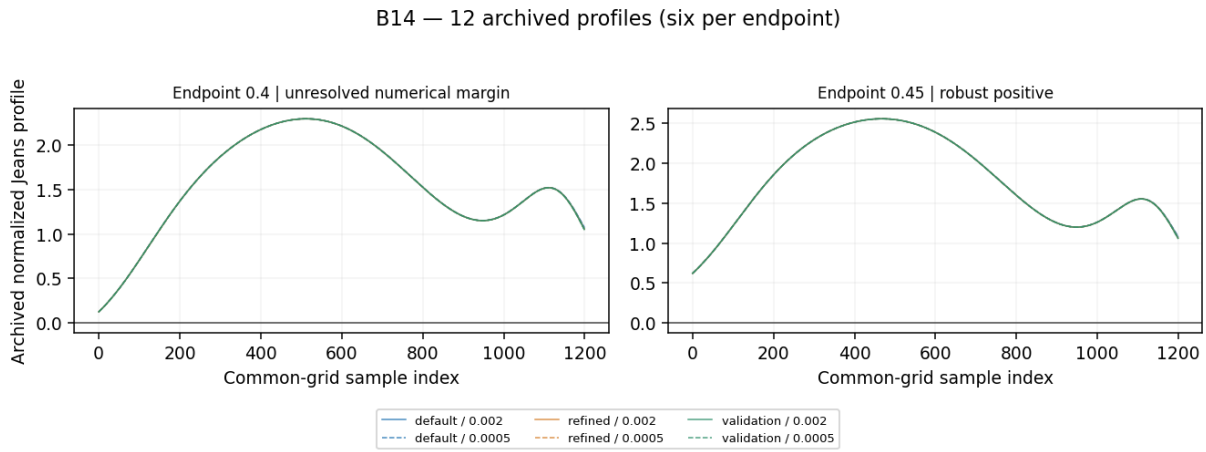



Figure 15. Archived six-arm profile visualization for B14. Compare both endpoint families across radial coordinate; any visual separation must be judged against the registered common-radius and tolerance requirements. Curves are saved exploratory numerical outputs, not observed stellar measurements.

Archived bracket status: unresolved_numerical_margin. Endpoint labels: 0.4 = unresolved_numerical_margin, 0.45 = robust_positive. Six-arm execution flag: True; physical Hessian execution flag: False.

Target-specific archived JSON: UDCT_v7_13_B14_Six_Arm_Profile_Exploratory_20261021_012339_663.json

Recorded keys: classification, target, sha256_verified, six_arms_executed, hessian_executed, endpoint_results, bracket_status, arms, common_grid_warning, elapsed_seconds.

Interpretation: The saved endpoint labels must not be promoted to a confirmed sign boundary unless opposite robust signs are demonstrated at the same shared radii under the registered six-arm tolerance. Apparent negative excursions in the inner native grid may be excluded from the common comparison interval, and uncertainty-envelope overlap may leave the result unresolved. This is a numerical admissibility diagnosis under the chosen Jeans closure, not a dynamical stability proof.

Reproduction trail: read the common-profile NPZ for all six arms; consult the matching target exploratory JSON and the six-arm inventory; recover native-grid minimum evidence from the separate archived audit if available; apply the preregistered decision gate without modifying tolerance after inspecting the plot.

## 5.15 B15: six-arm profile evidence

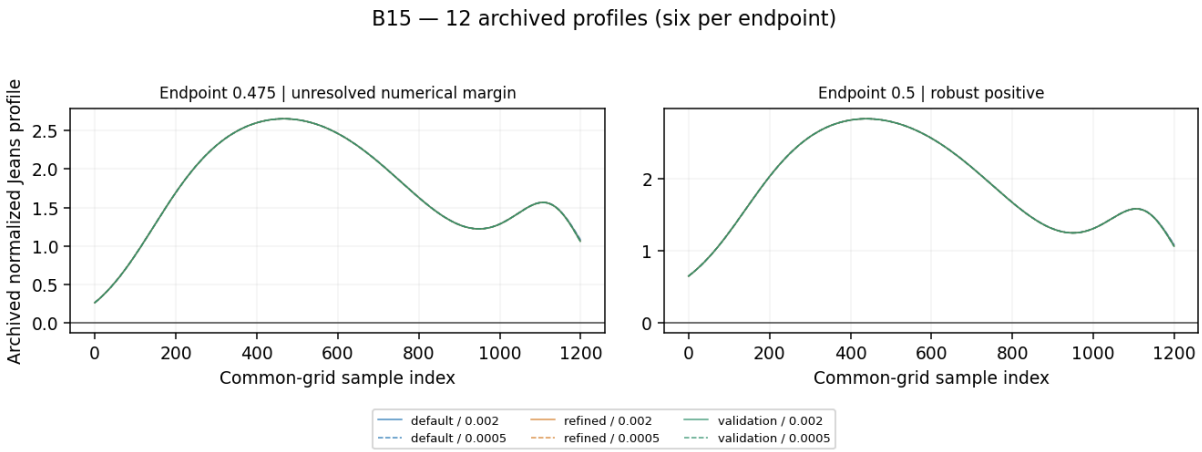



Figure 16. Archived six-arm profile visualization for B15. Compare both endpoint families across radial coordinate; any visual separation must be judged against the registered common-radius and tolerance requirements. Curves are saved exploratory numerical outputs, not observed stellar measurements.

Archived bracket status: unresolved_numerical_margin. Endpoint labels: 0.475 = unresolved_numerical_margin, 0.5 = robust_positive. Six-arm execution flag: True; physical Hessian execution flag: False.

Target-specific archived JSON: UDCT_v7_13_B15_Six_Arm_Profile_Exploratory_20261021_012335_481.json

Recorded keys: classification, target, sha256_verified, six_arms_executed, hessian_executed, endpoint_results, bracket_status, arms, common_grid_warning, elapsed_seconds.

Interpretation: The saved endpoint labels must not be promoted to a confirmed sign boundary unless opposite robust signs are demonstrated at the same shared radii under the registered six-arm tolerance. Apparent negative excursions in the inner native grid may be excluded from the common comparison interval, and uncertainty-envelope overlap may leave the result unresolved. This is a numerical admissibility diagnosis under the chosen Jeans closure, not a dynamical stability proof.

Reproduction trail: read the common-profile NPZ for all six arms; consult the matching target exploratory JSON and the six-arm inventory; recover native-grid minimum evidence from the separate archived audit if available; apply the preregistered decision gate without modifying tolerance after inspecting the plot.

## 5.16 B16: six-arm profile evidence

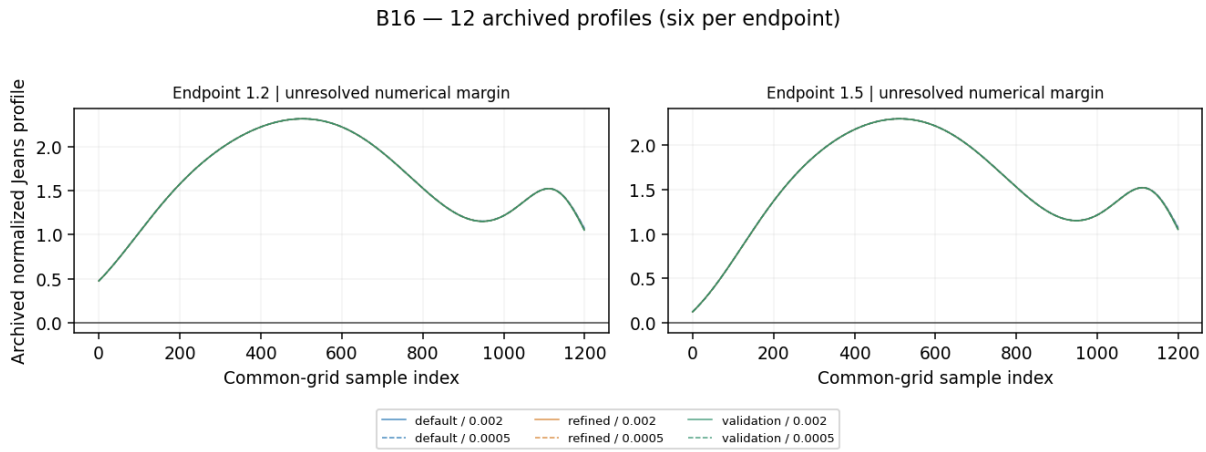



Figure 17. Archived six-arm profile visualization for B16. Compare both endpoint families across radial coordinate; any visual separation must be judged against the registered common-radius and tolerance requirements. Curves are saved exploratory numerical outputs, not observed stellar measurements.

Archived bracket status: unresolved_numerical_margin. Endpoint labels: 1.2 = unresolved_numerical_margin, 1.5 = unresolved_numerical_margin. Six-arm execution flag: True; physical Hessian execution flag: False.

Target-specific archived JSON: UDCT_v7_13_B16_Six_Arm_Profile_Exploratory_20261021_012331_628.json

Recorded keys: classification, target, sha256_verified, six_arms_executed, hessian_executed, endpoint_results, bracket_status, arms, common_grid_warning, elapsed_seconds.

Interpretation: The saved endpoint labels must not be promoted to a confirmed sign boundary unless opposite robust signs are demonstrated at the same shared radii under the registered six-arm tolerance. Apparent negative excursions in the inner native grid may be excluded from the common comparison interval, and uncertainty-envelope overlap may leave the result unresolved. This is a numerical admissibility diagnosis under the chosen Jeans closure, not a dynamical stability proof.

Reproduction trail: read the common-profile NPZ for all six arms; consult the matching target exploratory JSON and the six-arm inventory; recover native-grid minimum evidence from the separate archived audit if available; apply the preregistered decision gate without modifying tolerance after inspecting the plot.

## 5.17 B17: six-arm profile evidence

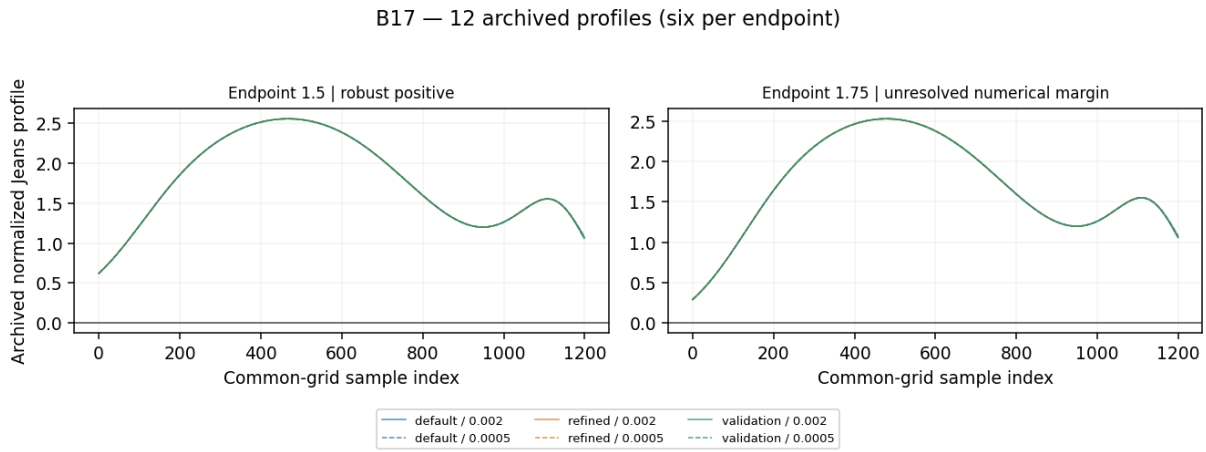



Figure 18. Archived six-arm profile visualization for B17. Compare both endpoint families across radial coordinate; any visual separation must be judged against the registered common-radius and tolerance requirements. Curves are saved exploratory numerical outputs, not observed stellar measurements.

Archived bracket status: unresolved_numerical_margin. Endpoint labels: 1.5 = robust_positive, 1.75 = unresolved_numerical_margin. Six-arm execution flag: True; physical Hessian execution flag: False.

Target-specific archived JSON: UDCT_v7_13_B17_Six_Arm_Profile_Exploratory_20261021_012327_430.json

Recorded keys: classification, target, sha256_verified, six_arms_executed, hessian_executed, endpoint_results, bracket_status, arms, common_grid_warning, elapsed_seconds.

Interpretation: The saved endpoint labels must not be promoted to a confirmed sign boundary unless opposite robust signs are demonstrated at the same shared radii under the registered six-arm tolerance. Apparent negative excursions in the inner native grid may be excluded from the common comparison interval, and uncertainty-envelope overlap may leave the result unresolved. This is a numerical admissibility diagnosis under the chosen Jeans closure, not a dynamical stability proof.

Reproduction trail: read the common-profile NPZ for all six arms; consult the matching target exploratory JSON and the six-arm inventory; recover native-grid minimum evidence from the separate archived audit if available; apply the preregistered decision gate without modifying tolerance after inspecting the plot.

## 5.18 B18: six-arm profile evidence

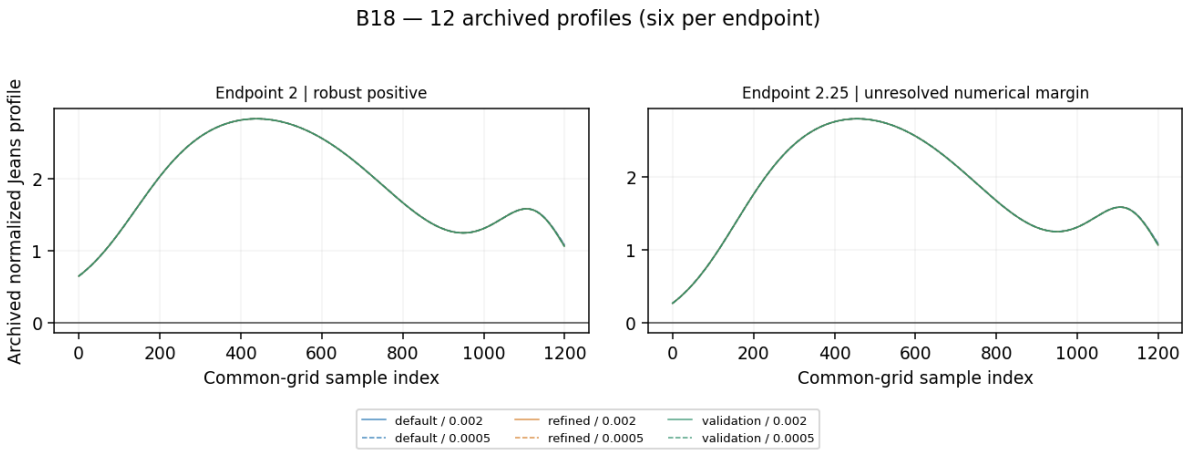



Figure 19. Archived six-arm profile visualization for B18. Compare both endpoint families across radial coordinate; any visual separation must be judged against the registered common-radius and tolerance requirements. Curves are saved exploratory numerical outputs, not observed stellar measurements.

Archived bracket status: unresolved_numerical_margin. Endpoint labels: 2.0 = robust_positive, 2.25 = unresolved_numerical_margin. Six-arm execution flag: True; physical Hessian execution flag: False.

Interpretation: The saved endpoint labels must not be promoted to a confirmed sign boundary unless opposite robust signs are demonstrated at the same shared radii under the registered six-arm tolerance. Apparent negative excursions in the inner native grid may be excluded from the common comparison interval, and uncertainty-envelope overlap may leave the result unresolved. This is a numerical admissibility diagnosis under the chosen Jeans closure, not a dynamical stability proof.

Reproduction trail: read the common-profile NPZ for all six arms; consult the matching target exploratory JSON and the six-arm inventory; recover native-grid minimum evidence from the separate archived audit if available; apply the preregistered decision gate without modifying tolerance after inspecting the plot.

## 5.19 B19: six-arm profile evidence

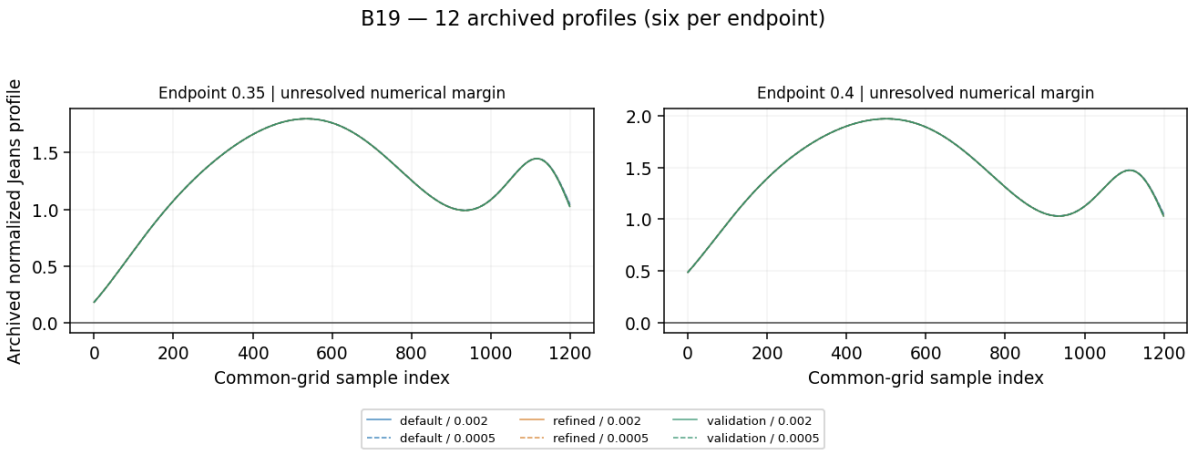



Figure 20. Archived six-arm profile visualization for B19. Compare both endpoint families across radial coordinate; any visual separation must be judged against the registered common-radius and tolerance requirements. Curves are saved exploratory numerical outputs, not observed stellar measurements.

Archived bracket status: unresolved_numerical_margin. Endpoint labels: 0.35 = unresolved_numerical_margin, 0.4 = unresolved_numerical_margin. Six-arm execution flag: True; physical Hessian execution flag: False.

Target-specific archived JSON: UDCT_v7_13_B19_Six_Arm_Profile_Exploratory_20261021_012315_327.json

Recorded keys: classification, target, sha256_verified, six_arms_executed, hessian_executed, endpoint_results, bracket_status, arms, common_grid_warning, elapsed_seconds.

Interpretation: The saved endpoint labels must not be promoted to a confirmed sign boundary unless opposite robust signs are demonstrated at the same shared radii under the registered six-arm tolerance. Apparent negative excursions in the inner native grid may be excluded from the common comparison interval, and uncertainty-envelope overlap may leave the result unresolved. This is a numerical admissibility diagnosis under the chosen Jeans closure, not a dynamical stability proof.

Reproduction trail: read the common-profile NPZ for all six arms; consult the matching target exploratory JSON and the six-arm inventory; recover native-grid minimum evidence from the separate archived audit if available; apply the preregistered decision gate without modifying tolerance after inspecting the plot.

## 5.20 B20: six-arm profile evidence

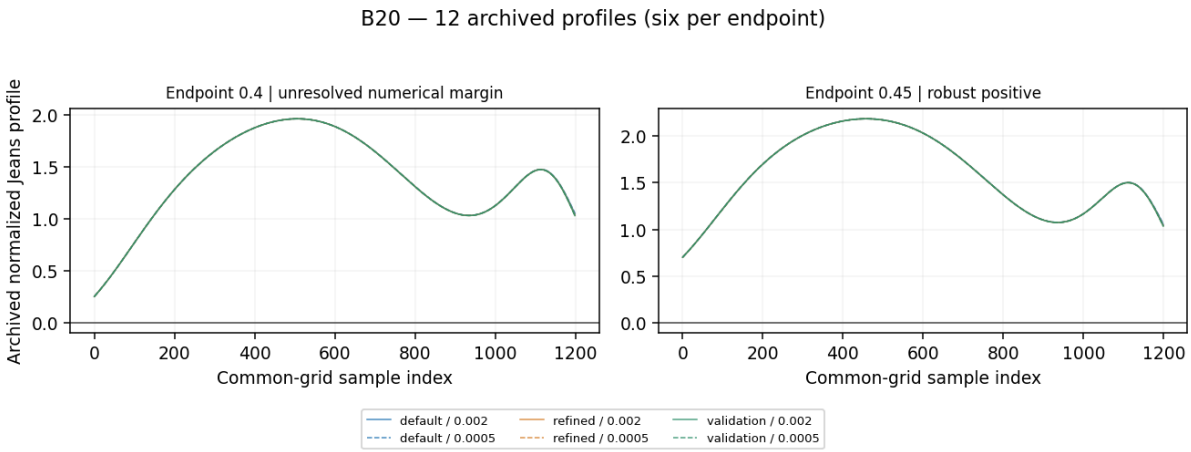



Figure 21. Archived six-arm profile visualization for B20. Compare both endpoint families across radial coordinate; any visual separation must be judged against the registered common-radius and tolerance requirements. Curves are saved exploratory numerical outputs, not observed stellar measurements.

Archived bracket status: unresolved_numerical_margin. Endpoint labels: 0.4 = unresolved_numerical_margin, 0.45 = robust_positive. Six-arm execution flag: True; physical Hessian execution flag: False.

Target-specific archived JSON: UDCT_v7_13_B20_Six_Arm_Profile_Exploratory_20261021_012308_699.json

Recorded keys: classification, target, sha256_verified, six_arms_executed, hessian_executed, endpoint_results, bracket_status, arms, common_grid_warning, elapsed_seconds.

Interpretation: The saved endpoint labels must not be promoted to a confirmed sign boundary unless opposite robust signs are demonstrated at the same shared radii under the registered six-arm tolerance. Apparent negative excursions in the inner native grid may be excluded from the common comparison interval, and uncertainty-envelope overlap may leave the result unresolved. This is a numerical admissibility diagnosis under the chosen Jeans closure, not a dynamical stability proof.

Reproduction trail: read the common-profile NPZ for all six arms; consult the matching target exploratory JSON and the six-arm inventory; recover native-grid minimum evidence from the separate archived audit if available; apply the preregistered decision gate without modifying tolerance after inspecting the plot.

# 5.21 B21: six-arm profile evidence

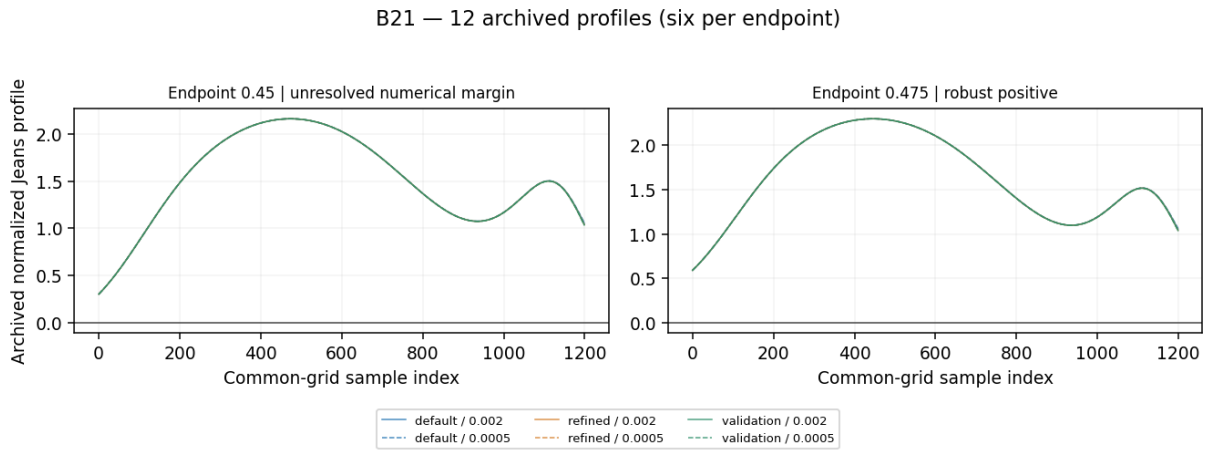



Figure 22. Archived six-arm profile visualization for B21. Compare both endpoint families across radial coordinate; any visual separation must be judged against the registered common-radius and tolerance requirements. Curves are saved exploratory numerical outputs, not observed stellar measurements.

Archived bracket status: unresolved_numerical_margin. Endpoint labels: 0.45 = unresolved_numerical_margin, 0.475 = robust_positive. Six-arm execution flag: True; physical Hessian execution flag: False.

Target-specific archived JSON: UDCT_v7_13_B21_Six_Arm_Profile_Exploratory_20261021_012302_765.json

Recorded keys: classification, target, sha256_verified, six_arms_executed, hessian_executed, endpoint_results, bracket_status, arms, common_grid_warning, elapsed_seconds.

Interpretation: The saved endpoint labels must not be promoted to a confirmed sign boundary unless opposite robust signs are demonstrated at the same shared radii under the registered six-arm tolerance. Apparent negative excursions in the inner native grid may be excluded from the common comparison interval, and uncertainty-envelope overlap may leave the result unresolved. This is a numerical admissibility diagnosis under the chosen Jeans closure, not a dynamical stability proof.

Reproduction trail: read the common-profile NPZ for all six arms; consult the matching target exploratory JSON and the six-arm inventory; recover native-grid minimum evidence from the separate archived audit if available; apply the preregistered decision gate without modifying tolerance after inspecting the plot.

## 5.22 B22: six-arm profile evidence

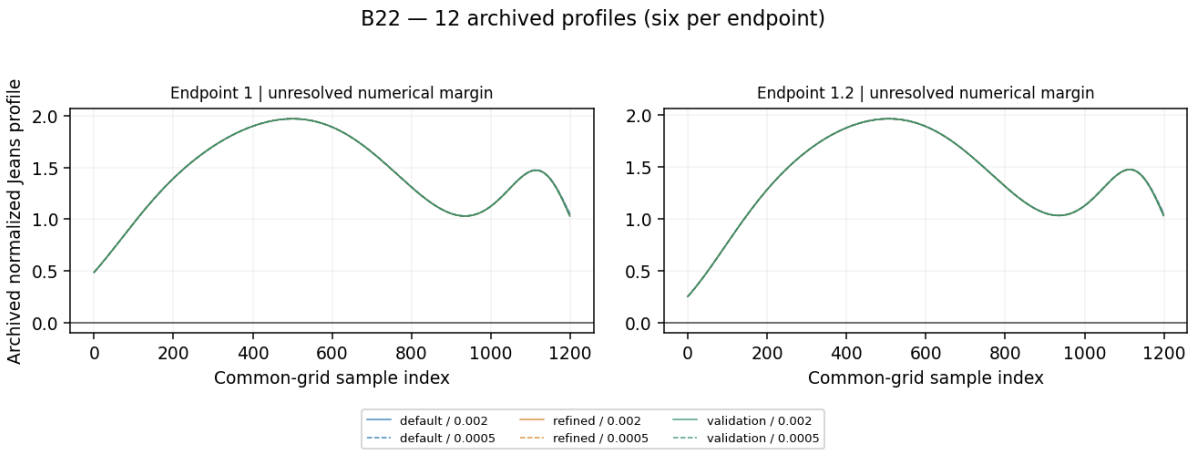



Figure 23. Archived six-arm profile visualization for B22. Compare both endpoint families across radial coordinate; any visual separation must be judged against the registered common-radius and tolerance requirements. Curves are saved exploratory numerical outputs, not observed stellar measurements.

Archived bracket status: unresolved_numerical_margin. Endpoint labels: 1.0 = unresolved_numerical_margin, 1.2 = unresolved_numerical_margin. Six-arm execution flag: True; physical Hessian execution flag: False.

Target-specific archived JSON: UDCT_v7_13_B22_Six_Arm_Profile_Exploratory_20261021_012256_617.json

Recorded keys: classification, target, sha256_verified, six_arms_executed, hessian_executed, endpoint_results, bracket_status, arms, common_grid_warning, elapsed_seconds.

Interpretation: The saved endpoint labels must not be promoted to a confirmed sign boundary unless opposite robust signs are demonstrated at the same shared radii under the registered six-arm tolerance. Apparent negative excursions in the inner native grid may be excluded from the common comparison interval, and uncertainty-envelope overlap may leave the result unresolved. This is a numerical admissibility diagnosis under the chosen Jeans closure, not a dynamical stability proof.

Reproduction trail: read the common-profile NPZ for all six arms; consult the matching target exploratory JSON and the six-arm inventory; recover native-grid minimum evidence from the separate archived audit if available; apply the preregistered decision gate without modifying tolerance after inspecting the plot.

## 5.23 B23: six-arm profile evidence

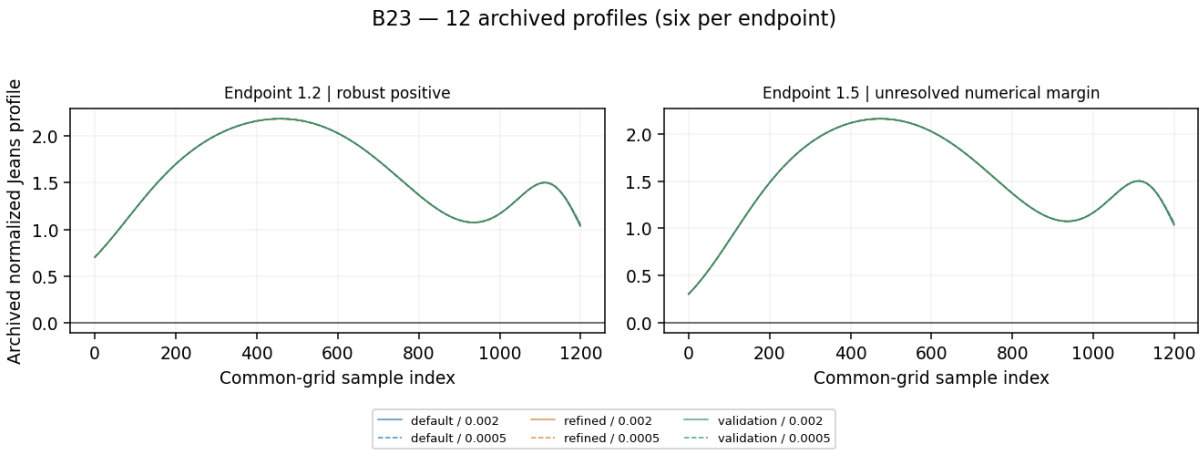



Figure 24. Archived six-arm profile visualization for B23. Compare both endpoint families across radial coordinate; any visual separation must be judged against the registered common-radius and tolerance requirements. Curves are saved exploratory numerical outputs, not observed stellar measurements.

Archived bracket status: unresolved_numerical_margin. Endpoint labels: 1.2 = robust_positive, 1.5 = unresolved_numerical_margin. Six-arm execution flag: True; physical Hessian execution flag: False.

Target-specific archived JSON: UDCT_v7_13_B23_Six_Arm_Profile_Exploratory_20261021_012251_935.json

Recorded keys: classification, target, sha256_verified, six_arms_executed, hessian_executed, endpoint_results, bracket_status, arms, common_grid_warning, elapsed_seconds.

Interpretation: The saved endpoint labels must not be promoted to a confirmed sign boundary unless opposite robust signs are demonstrated at the same shared radii under the registered six-arm tolerance. Apparent negative excursions in the inner native grid may be excluded from the common comparison interval, and uncertainty-envelope overlap may leave the result unresolved. This is a numerical admissibility diagnosis under the chosen Jeans closure, not a dynamical stability proof.

Reproduction trail: read the common-profile NPZ for all six arms; consult the matching target exploratory JSON and the six-arm inventory; recover native-grid minimum evidence from the separate archived audit if available; apply the preregistered decision gate without modifying tolerance after inspecting the plot.

## 5.24 B24: six-arm profile evidence

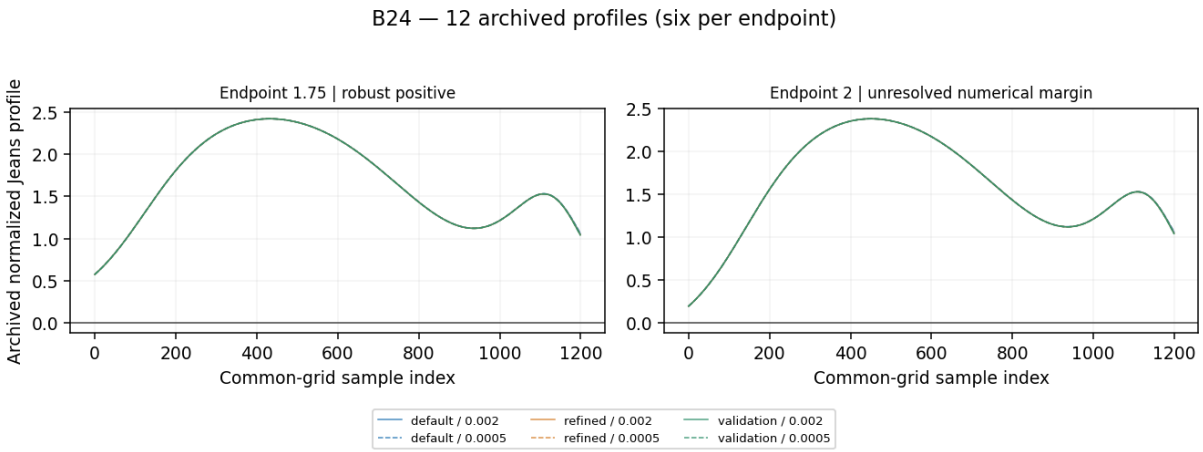



Figure 25. Archived six-arm profile visualization for B24. Compare both endpoint families across radial coordinate; any visual separation must be judged against the registered common-radius and tolerance requirements. Curves are saved exploratory numerical outputs, not observed stellar measurements.

Archived bracket status: unresolved_numerical_margin. Endpoint labels: 1.75 = robust_positive, 2.0 = unresolved_numerical_margin. Six-arm execution flag: True; physical Hessian execution flag: False.

Target-specific archived JSON: UDCT_v7_13_B24_Six_Arm_Profile_Exploratory_20261021_012247_408.json

Recorded keys: classification, target, sha256_verified, six_arms_executed, hessian_executed, endpoint_results, bracket_status, arms, common_grid_warning, elapsed_seconds.

Interpretation: The saved endpoint labels must not be promoted to a confirmed sign boundary unless opposite robust signs are demonstrated at the same shared radii under the registered six-arm tolerance. Apparent negative excursions in the inner native grid may be excluded from the common comparison interval, and uncertainty-envelope overlap may leave the result unresolved. This is a numerical admissibility diagnosis under the chosen Jeans closure, not a dynamical stability proof.

Reproduction trail: read the common-profile NPZ for all six arms; consult the matching target exploratory JSON and the six-arm inventory; recover native-grid minimum evidence from the separate archived audit if available; apply the preregistered decision gate without modifying tolerance after inspecting the plot.

# 6. Source-code architecture and interpretive guide

## Jeans profile production and traceability

6 Python scripts identified in this thematic family. File grouping is descriptive; scripts may overlap families.

UDCT_v7_13_B01_Independent_Jeans_Quadrature_Audit_20261021_012453_346.py

Functions: No top-level def found. Imports: from pathlib import Path; import importlib.util, sys, json, hashlib; import numpy as np. Source length 41 lines.

UDCT_v7_13_B01_Inner_Cutoff_Stability_20261021_011711_520.py

Functions: No top-level def found. Imports: import json,hashlib,importlib.util,sys; from pathlib import Path; import numpy as np. Source length 28 lines.

UDCT_v7_13_B01_Radial_Domain_Audit_20261021_011730_078.py

Functions: load. Imports: import json, hashlib, importlib.util, sys; from pathlib import Path; import numpy as np. Source length 33 lines.

UDCT_v7_13_B01_Radial_Zero_Exploratory_20261021_011719_053.py

Functions: No top-level def found. Imports: import json,hashlib,importlib.util,sys; from pathlib import Path; import numpy as np. Source length 33 lines.

UDCT_v7_13_B02_B24_Full_Profile_Trace_20261021_011741_455.py

Functions: load. Imports: from pathlib import Path; import json,csv,hashlib,importlib.util,sys,time; import numpy as np. Source length 61 lines.

UDCT_v7_13_Green_Cell_Quadrature_Audit_20261021_011804_539.py

Functions: cube_kernel. Imports: import json, hashlib; from pathlib import Path; import numpy as np. Source length 32 lines.

Audit guidance: distinguish manufactured analytical self-tests from physically specified galaxy boundary-value solves; inspect source constants and input provenance; preserve exception paths and failed diagnostics; avoid equating one-method resolution refinement with independent continuum convergence.

## Potential solvers and physical boundaries

18 Python scripts identified in this thematic family. File grouping is descriptive; scripts may overlap families.

UDCT_v7_13_3D_Constitutive_Poisson_Coupling_Exploratory_20261021_012132_569.py

Functions: solve, lap, run. Imports: import sys, json, hashlib, importlib.util; from pathlib import Path; import numpy as np. Source length 70 lines.

UDCT_v7_13_3D_Constitutive_Poisson_Coupling_Factor_Corrected_20261021_012118_509.py

Functions: solve, lap, run. Imports: import sys, json, hashlib, importlib.util; from pathlib import Path; import numpy as np. Source length 70 lines.

UDCT_v7_13_3D_Poisson_Solver_Manufactured_Control_20261021_012151_799.py

Functions: solve_poisson_dirichlet, laplacian_7pt. Imports: from pathlib import Path; import hashlib, json, time; import numpy as np. Source length 53 lines.

UDCT_v7_13_Boundary_Moment_Cancellation_Audit_20261021_011833_901.py

Functions: No top-level def found. Imports: from pathlib import Path; import hashlib, json. Source length 35 lines.

UDCT_v7_13_Direct_Boundary_59cube_Audit_20261021_011808_455.py

Functions: solve_bc. Imports: import json,hashlib; from pathlib import Path; import numpy as np. Source length 46 lines.

UDCT_v7_13_Direct_Boundary_Radial_Audit_20261021_011815_545.py

Functions: solve_dirichlet, calc. Imports: from pathlib import Path; import json,hashlib; import numpy as np. Source length 43 lines.

UDCT_v7_13_Direct_Boundary_Resolution_Audit_20261021_011812_014.py

Functions: solve_bc. Imports: from pathlib import Path; import json,hashlib; import numpy as np. Source length 48 lines.

UDCT_v7_13_Direct_Source_Boundary_Audit_20261021_011820_499.py

Functions: solve_dirichlet, calculate. Imports: import json,hashlib; from pathlib import Path; import numpy as np. Source length 76 lines.

UDCT_v7_13_Fixed_Box_Resolution_Audit_20261021_011901_260.py

Functions: No top-level def found. Imports: import json, hashlib; from pathlib import Path. Source length 22 lines.

UDCT_v7_13_Green_Cell_Quadrature_Audit_20261021_011804_539.py

Functions: cube_kernel. Imports: import json, hashlib; from pathlib import Path; import numpy as np. Source length 32 lines.

UDCT_v7_13_Hydrus_3D_Angular_Newtonian_Gate_20261021_011958_087.py

Functions: No top-level def found. Imports: import json, hashlib; from pathlib import Path; import numpy as np. Source length 40 lines.

UDCT_v7_13_Hydrus_3D_DeltaPotential_Exploratory_20261021_011907_412.py

Functions: solve, lap, run. Imports: import json, hashlib, importlib.util,sys; from pathlib import Path; import numpy as np. Source length 70 lines.

UDCT_v7_13_Hydrus_Box_Boundary_Sensitivity_20261021_011904_569.py

Functions: No top-level def found. Imports: import importlib.util,sys,json,hashlib; from pathlib import Path. Source length 27 lines.

UDCT_v7_13_Hydrus_Common_Frame_Newtonian_Audit_20261021_012020_218.py

Functions: h_in, metrics. Imports: import json, numpy as np; from pathlib import Path. Source length 47 lines.

UDCT_v7_13_Multipole_Boundary_Exploratory_20261021_011827_040.py

Functions: solve_dirichlet, run. Imports: import json,hashlib; from pathlib import Path; import numpy as np. Source length 68 lines.

UDCT_v7_13_Poisson_Sign_Units_Boundary_Gate_20261021_012146_283.py

Functions: solve, lap, run. Imports: import json, math, hashlib; from pathlib import Path; import numpy as np. Source length 61 lines.

UDCT_v7_13_Quadrupole_Boundary_Diagnostic_20261021_011830_763.py

Functions: No top-level def found. Imports: from pathlib import Path; import json,hashlib; import numpy as np. Source length 48 lines.

UDCT_v7_13_Source_Localization_Boundary_Audit_20261021_011840_423.py

Functions: No top-level def found. Imports: import json, hashlib; from pathlib import Path; import numpy as np. Source length 38 lines.

Audit guidance: distinguish manufactured analytical self-tests from physically specified galaxy boundary-value solves; inspect source constants and input provenance; preserve exception paths and failed diagnostics; avoid equating one-method resolution refinement with independent continuum convergence.

## Hessian and constitutive stress

18 Python scripts identified in this thematic family. File grouping is descriptive; scripts may overlap families.

UDCT_v7_13_3D_Constitutive_Poisson_Coupling_Exploratory_20261021_012132_569.py

Functions: solve, lap, run. Imports: import sys, json, hashlib, importlib.util; from pathlib import Path; import numpy as np. Source length 70 lines.

UDCT_v7_13_3D_Constitutive_Poisson_Coupling_Factor_Corrected_20261021_012118_509.py

Functions: solve, lap, run. Imports: import sys, json, hashlib, importlib.util; from pathlib import Path; import numpy as np. Source length 70 lines.

UDCT_v7_13_Action_Variation_Factor_Audit_20261021_012121_374.py

Functions: D, derivs, calc. Imports: from pathlib import Path; import zipfile,hashlib,json,sys,importlib.util; import numpy as np. Source length 51 lines.

UDCT_v7_13_B01_Explicit_Hessian_Ablation_20261021_011656_464.py

Functions: No top-level def found. Imports: from pathlib import Path; import importlib.util,sys,json,hashlib; import numpy as np. Source length 38 lines.

UDCT_v7_13_B01_Hessian_Law_Control_Exploratory_20261021_011702_712.py

Functions: No top-level def found. Imports: from pathlib import Path; import json,hashlib,importlib.util,sys; import numpy as np. Source length 35 lines.

UDCT_v7_13_B01_Stress_Derivative_Decomposition_20261021_011646_125.py

Functions: No top-level def found. Imports: from pathlib import Path; import importlib.util, sys, json, hashlib; import numpy as np. Source length 42 lines.

UDCT_v7_13_Constitutive_3D_Source_Prototype_20261021_012203_512.py

Functions: response, diagnostic. Imports: import sys, json, hashlib, importlib.util; from pathlib import Path; import numpy as np. Source length 54 lines.

UDCT_v7_13_Constitutive_3D_Source_Revalidated_20261021_012140_566.py

Functions: response, diagnostic. Imports: import sys, json, hashlib, importlib.util; from pathlib import Path; import numpy as np. Source length 54 lines.

UDCT_v7_13_Hessian_Equivalent_Density_Audit_20261021_012046_765.py

Functions: main. Imports: import json, math; from pathlib import Path. Source length 31 lines.

UDCT_v7_13_Hessian_External_Field_Audit_20261021_012216_909.py

Functions: No top-level def found. Imports: import hashlib, importlib.util, json, sys; from pathlib import Path; import numpy as np. Source length 39 lines.

UDCT_v7_13_Hessian_Factor_Grid_Convergence_20261021_012100_129.py

Functions: solve, run. Imports: import json,zipfile,hashlib,importlib.util,sys; from pathlib import Path; import numpy as np. Source length 69 lines.

UDCT_v7_13_Hessian_Factor_Stress_Audit_20261021_012105_059.py

Functions: solve, test. Imports: import zipfile,hashlib,json,importlib.util,sys; from pathlib import Path; import numpy as np. Source length 41 lines.

UDCT_v7_13_Hessian_Preflight_and_Estimator_Selftest_20261021_012224_191.py

Functions: hessian. Imports: from pathlib import Path; import hashlib, json, ast; import numpy as np. Source length 57 lines.

UDCT_v7_13_Hessian_Preflight_and_Estimator_Selftest_20261021_012458_489.py

Functions: hessian. Imports: from pathlib import Path; import hashlib, json, ast; import numpy as np. Source length 57 lines.

UDCT_v7_13_Hydrus_Hessian_Increment_Convergence_Audit_20261021_011931_333.py

Functions: No top-level def found. Imports: import json,hashlib,math,statistics; from pathlib import Path. Source length 23 lines.

UDCT_v7_13_Hydrus_Hessian_Orientation_Bounds_20261021_012030_205.py

Functions: No top-level def found. Imports: import json; from pathlib import Path; import numpy as np. Source length 49 lines.

UDCT_v7_13_Hydrus_Hessian_Scale_Comparison_20261021_012041_002.py

Functions: No top-level def found. Imports: import json, math; from pathlib import Path; import numpy as np. Source length 41 lines.

UDCT_v7_13_Hydrus_Plummer_3D_Hessian_Control_20261021_012213_249.py

Functions: phi, exact_hessian, numerical_hessian. Imports: import hashlib, importlib.util, json, sys; from pathlib import Path; import numpy as np. Source length 61 lines.

Audit guidance: distinguish manufactured analytical self-tests from physically specified galaxy boundary-value solves; inspect source constants and input provenance; preserve exception paths and failed diagnostics; avoid equating one-method resolution refinement with independent continuum convergence.

## Numerical convergence and readiness

9 Python scripts identified in this thematic family. File grouping is descriptive; scripts may overlap families.

UDCT_v7_13_Acceleration_Richardson_Diagnostic_20261021_011857_779.py

Functions: No top-level def found. Imports: import json, hashlib; from pathlib import Path. Source length 26 lines.

UDCT_v7_13_B01_B24_Trace_Inventory_20261021_011740_803.py

Functions: No top-level def found. Imports: from pathlib import Path; import json,hashlib,numpy as np. Source length 30 lines.

UDCT_v7_13_Common_Coordinate_Acceleration_Audit_20261021_011847_335.py

Functions: No top-level def found. Imports: import json,hashlib; from pathlib import Path; import numpy as np. Source length 41 lines.

UDCT_v7_13_Confirmatory_Readiness_Audit_20261021_011753_298.py

Functions: No top-level def found. Imports: import json,hashlib,subprocess,sys,pathlib,collections. Source length 40 lines.

UDCT_v7_13_Direct_Boundary_Resolution_Audit_20261021_011812_014.py

Functions: solve_bc. Imports: from pathlib import Path; import json,hashlib; import numpy as np. Source length 48 lines.

UDCT_v7_13_Fixed_Box_Resolution_Audit_20261021_011901_260.py

Functions: No top-level def found. Imports: import json, hashlib; from pathlib import Path. Source length 22 lines.

UDCT_v7_13_Hessian_Preflight_and_Estimator_Selftest_20261021_012224_191.py

Functions: hessian. Imports: from pathlib import Path; import hashlib, json, ast; import numpy as np. Source length 57 lines.

UDCT_v7_13_Hessian_Preflight_and_Estimator_Selftest_20261021_012458_489.py

Functions: hessian. Imports: from pathlib import Path; import hashlib, json, ast; import numpy as np. Source length 57 lines.

UDCT_v7_13_Radial_Acceleration_Convergence_20261021_011853_533.py

Functions: No top-level def found. Imports: import json, hashlib; from pathlib import Path; import numpy as np. Source length 40 lines.

Audit guidance: distinguish manufactured analytical self-tests from physically specified galaxy boundary-value solves; inspect source constants and input provenance; preserve exception paths and failed diagnostics; avoid equating one-method resolution refinement with independent continuum convergence.

# 7. 3-D Hessian gate: evidence and unresolved prerequisites

classification: V7.13_3D_HESSIAN_GATE_AUDIT_NOT_CONFIRMATORY

verified_at_runtime: True

hessian_stencil_selftest: PASS

frozen_hash_verified_in_preflight: True

required_steps_rh: [0.02, 0.01, 0.005]

quadratic_errors: [2.2470914018413168e-13, 1.220357148667972e-12, 2.5508484213787597e-12]

quartic_errors: [0.0004800000000013682, 0.00011999999998257849, 2.9999999336283167e-05]

prior_source_gate: {'3d_total_modified_potential_solver': 'not identified', 'external_newtonian_tidal_hessian': 'identified', 'angle_averaged_radial_jeans': 'identified', 'B01_B24_3d_modified_hessian': 'NOT EVALUATED'}

existing_delta_potential_script: True

existing_delta_potential_result: True

confirmatory_hessian: NOT_EVALUATED

blocking_issues: ['No verified 3D total modified-gravity potential with justified physical boundary conditions', 'Existing delta potential uses finite-box zero Dirichlet boundary and is exploratory', 'Frozen 1D Jeans B01-B24 cannot be relabeled as 3D Hessian test']

next_gate: Construct and independently validate total potential and exterior boundary; only then compute 3x3 Hessian at 0.02,0.01,0.005 rh and apply preregistration gate

The manufactured quadratic and quartic tests are useful finite-difference implementation checks. They do not provide a verified full modified-gravity Hydrus I total potential. The physical gate remains NOT EVALUATED because a validated exterior boundary and converged total-potential Hessian were not established. The required normalized spatial steps are $h/r_h = 0.02, 0.01, 0.005$.

# 8. Numerical provenance and incomplete records

The archive contains 23 unique target-specific exploratory JSON IDs: B18 has no individual target JSON, but B18 common profiles are present in the common NPZ; B19 has two target JSON files. This is an archival completeness issue, not grounds to omit either target. Seven native-F endpoint entries were absent from the earlier audit and remain conservatively unresolved (B04, B06, B11, B18 twice, B22, B24).

File timestamps in names and ZIP member timestamps cannot prove exact scientific execution order, public availability of amended preregistration criteria, or independence of calculations. No retrospective confirmatory claim is made.

# 9. Scientific interpretation and falsification boundaries

The important outcome is an unresolved strict-sign-boundary search under the preserved gate, not proof of universal positive admissibility, physical instability, or rejection of MOND. A physically meaningful comparison requires a correctly specified 3-D total potential, justified exterior boundary, convergence tests, tracer distribution function admissibility, and eventually direct comparison with individual stellar velocities and uncertainties. A negative calculated Jeans moment is a model-closure diagnostic, not an observed negative dispersion.

# 10. Reproducibility protocol

• Preserve the original 7.13 Hessian.zip bytes and publish SHA-256 checksums.

• Keep the amended preregistration package and exact frozen criteria alongside the results; verify amendment-publication chronology before confirmatory language.

• Run code in a documented Python environment and capture stdout, stderr, exit status and hashes.

• Regenerate each six-arm profile from the NPZ, then independently recalculate E and tau.

- Treat missing native-grid minima as missing rather than impute favorable signs.
- Run independent 3-D physical boundary and total-potential Hessian checks before any Hessian validation claim.
- Separate observationally informed model inputs from direct individual-star likelihood fitting.

# Appendix A. Full archived file manifest

UDCT_v7_13_3D_Constitutive_Poisson_Coupling_Exploratory_20261021_012132_569.py | 3965 bytes | sha256 9117e6f13b30b97180e1…

UDCT_v7_13_3D_Constitutive_Poisson_Coupling_Factor_Corrected_20261021_012116_770.json | 4188 bytes | sha256 ac6c9896a88ccbe707b3…

UDCT_v7_13_3D_Constitutive_Poisson_Coupling_Factor_Corrected_20261021_012118_509.py | 4091 bytes | sha256 732cf09e9eb5bc001451…

UDCT_v7_13_3D_Hessian_Gate_Audit_20261020_20261021_011634_216.json | 1302 bytes | sha256 58e33822cab3b8154862…

UDCT_v7_13_3D_Poisson_Manufactured_Control_20261021_012151_118.json | 1779 bytes | sha256 c8c70349d11345f358d5…

UDCT_v7_13_3D_Poisson_Solver_Manufactured_Control_20261021_012151_799.py | 3212 bytes | sha256 b96bdabce1bbd8d51d55…

UDCT_v7_13_3D_Potential_Implementation_Gate_20261021_012207_049.json | 1991 bytes | sha256 3a2f42a372d8c9f20cf6…

UDCT_v7_13_3D_Potential_Implementation_Gate_20261021_012207_715.py | 2006 bytes | sha256 1d2b75de19a8dd99cf2e…

UDCT_v7_13_Acceleration_Richardson_Diagnostic_20261021_011856_979.json | 2545 bytes | sha256 05e5d672799274a77d8f…

UDCT_v7_13_Acceleration_Richardson_Diagnostic_20261021_011857_779.py | 1977 bytes | sha256 41ddb033101b49f1fc86…

UDCT_v7_13_Action_Variation_Factor_Audit_20261021_012119_839.json | 1330 bytes | sha256 b2c5440345862c153e11…

UDCT_v7_13_Action_Variation_Factor_Audit_20261021_012121_374.py | 3025 bytes | sha256 9ea62ea5348764013bbc…

UDCT_v7_13_B01_B24_All_Common_Profiles_20261021_011738_243.npz | 2661383 bytes | sha256 a6a040bd7987f99b2f69…

UDCT_v7_13_B01_B24_Six_Arm_Inventory_20261020_20261021_011759_252.json | 8745 bytes | sha256 2691edabb8b69a49f3da…

UDCT_v7_13_B01_B24_Trace_Inventory_20261021_011739_280.json | 3994 bytes | sha256 100d324e9d8648a68b3f…

UDCT_v7_13_B01_B24_Trace_Inventory_20261021_011740_803.py | 2092 bytes | sha256 f5e1a15d44d7f0952728…

UDCT_v7_13_B01_Explicit_Hessian_Ablation_20261021_011652_218.json | 3213 bytes | sha256 f65d3f213a9b51e6ce91…

UDCT_v7_13_B01_Explicit_Hessian_Ablation_20261021_011656_464.py | 3003 bytes | sha256 b7779b7678a52369d343…

UDCT_v7_13_B01_Full_Common_Profiles_20261021_011746_763.npz | 110342 bytes | sha256 12cf1f5265f125a8dd07…

UDCT_v7_13_B01_Full_Profile_Trace_20261021_011747_575.json | 4804 bytes | sha256 70a7e9cbd11a23d2d029…

UDCT_v7_13_B01_Hessian_Law_Control_Exploratory_20261021_011700_929.json | 5661 bytes | sha256 876f17e32e02fe32096f…

UDCT_v7_13_B01_Hessian_Law_Control_Exploratory_20261021_011702_712.py | 2648 bytes | sha256 2b8083fd691f1e39a7df…

UDCT_v7_13_B01_Independent_Jeans_Quadrature_Audit_20261021_012453_346.py | 2409 bytes | sha256 22ffabd0e7434d3b05e7…

UDCT_v7_13_B01_Inner_Cutoff_Stability_20261021_011709_985.json | 2654 bytes | sha256 44fbac8e50d4d3823dab…

UDCT_v7_13_B01_Inner_Cutoff_Stability_20261021_011711_520.py | 2112 bytes | sha256 96937328f38b24cfe7c9…

UDCT_v7_13_B01_Quadrature_Audit_20261021_012454_325.json | 2747 bytes | sha256 131723fee2d6bc914bf4…

UDCT_v7_13_B01_Radial_Domain_Audit_20261021_011729_130.json | 2866 bytes | sha256 411b227f891161576d9e…

UDCT_v7_13_B01_Radial_Domain_Audit_20261021_011730_078.py | 2119 bytes | sha256 17f6a90bef9f02a945e1…

UDCT_v7_13_B01_Radial_Zero_Exploratory_20261021_011717_972.json | 3093 bytes | sha256 6ce205f6c41fe0931e8c…

UDCT_v7_13_B01_Radial_Zero_Exploratory_20261021_011719_053.py | 2632 bytes | sha256 9cdc1cf246ad68a68b98…

UDCT_v7_13_B01_Six_Arm_Profile_Exploratory_20261021_012443_978.json | 3256 bytes | sha256 a17c57eeca8bb1762b7b…

UDCT_v7_13_B01_Stress_Derivative_Decomposition_20261021_011643_661.json | 7355 bytes | sha256 dcc79cae4c8c19002e3d…

UDCT_v7_13_B01_Stress_Derivative_Decomposition_20261021_011646_125.py | 3727 bytes | sha256 5c0a5d8a00ae9374bf88…

UDCT_v7_13_B01_validation_grid_exploratory_20261021_012447_920.json | 1246 bytes | sha256 6e4a2f01f9c881b89c6b…

UDCT_v7_13_B02_B24_Full_Profile_Trace_20261021_011741_455.py | 5067 bytes | sha256 9a521ddee5a211759b53…

UDCT_v7_13_B02_Six_Arm_Profile_Exploratory_20261021_012440_907.json | 3230 bytes | sha256 ca7122df533d58b9f612…

UDCT_v7_13_B03_Six_Arm_Profile_Exploratory_20261021_012436_828.json | 3236 bytes | sha256 089d98253c58a70efe3f…

UDCT_v7_13_B04_Six_Arm_Profile_Exploratory_20261021_012433_635.json | 3492 bytes | sha256 f009dec73106ce24a31e…

UDCT_v7_13_B05_Six_Arm_Profile_Exploratory_20261021_012429_424.json | 3501 bytes | sha256 15a658d28edf6e7a76e2…

UDCT_v7_13_B06_Six_Arm_Profile_Exploratory_20261021_012425_495.json | 3480 bytes | sha256 407c381cbc2358872a85…

UDCT_v7_13_B07_Six_Arm_Profile_Exploratory_20261021_012418_036.json | 3211 bytes | sha256 9e376b4e7cb3d553012a…

UDCT_v7_13_B08_Six_Arm_Profile_Exploratory_20261021_012413_596.json | 3236 bytes | sha256 2f1ff5d693a93790f0e8…

UDCT_v7_13_B09_Six_Arm_Profile_Exploratory_20261021_012409_760.json | 3248 bytes | sha256 f04ef7a87a6ac72d00e9…

UDCT_v7_13_B10_Six_Arm_Profile_Exploratory_20261021_012405_115.json | 3246 bytes | sha256 326ce8d33916d37535a3…

UDCT_v7_13_B11_Six_Arm_Profile_Exploratory_20261021_012358_483.json | 3247 bytes | sha256 5b3b247ad4d17c92687a…

UDCT_v7_13_B12_Six_Arm_Profile_Exploratory_20261021_011759_654.json | 3243 bytes | sha256 8da915f3044b2f142aaa…

UDCT_v7_13_B13_Six_Arm_Profile_Exploratory_20261021_012349_829.json | 3276 bytes | sha256 1285098b8ab4b2804b6d…

UDCT_v7_13_B14_Six_Arm_Profile_Exploratory_20261021_012339_663.json | 3238 bytes | sha256 2de7a9844738bbf7c05b…

UDCT_v7_13_B15_Six_Arm_Profile_Exploratory_20261021_012335_481.json | 3242 bytes | sha256 9f53aba24413edca1a5f…

UDCT_v7_13_B16_Six_Arm_Profile_Exploratory_20261021_012331_628.json | 3289 bytes | sha256 0a0584538782ec683fd8…

UDCT_v7_13_B17_Six_Arm_Profile_Exploratory_20261021_012327_430.json | 3263 bytes | sha256 2fce5b4a0ec159f59af5…

UDCT_v7_13_B19_Six_Arm_Profile_Exploratory_20261021_012314_716.json | 3259 bytes | sha256 732de77b0bfa2c404949…

UDCT_v7_13_B19_Six_Arm_Profile_Exploratory_20261021_012315_327.json | 3259 bytes | sha256 732de77b0bfa2c404949…

UDCT_v7_13_B20_Six_Arm_Profile_Exploratory_20261021_012308_699.json | 3239 bytes | sha256 2131b8633287e71dfb75…

UDCT_v7_13_B21_Six_Arm_Profile_Exploratory_20261021_012302_765.json | 3250 bytes | sha256 9bd3837c3f50b26ef748…

UDCT_v7_13_B22_Six_Arm_Profile_Exploratory_20261021_012256_617.json | 3284 bytes | sha256 b4e851c61cf76b3a5d07…

UDCT_v7_13_B23_Six_Arm_Profile_Exploratory_20261021_012251_935.json | 3253 bytes | sha256 f7b4ff42c03d34d7c6ae…

UDCT_v7_13_B24_Six_Arm_Profile_Exploratory_20261021_012247_408.json | 3261 bytes | sha256 732e8d0bb2e49c49ef9b…

UDCT_v7_13_Boundary_Moment_Cancellation_Audit_20261021_011833_079.json | 2685 bytes | sha256 8965715d7e5d8efd85ce…

UDCT_v7_13_Boundary_Moment_Cancellation_Audit_20261021_011833_901.py | 2223 bytes | sha256 955e301b4c8c1b2a1b77…

UDCT_v7_13_Common_Coordinate_Acceleration_Audit_20261021_011846_682.json | 8588 bytes | sha256 601014a62975d6bf1d5b…

UDCT_v7_13_Common_Coordinate_Acceleration_Audit_20261021_011847_335.py | 2582 bytes | sha256 8a86040ef901bc557465…

UDCT_v7_13_Confirmatory_Readiness_Audit_20261021_011752_281.json | 11735 bytes | sha256 24dd0f4de408f944aa94…

UDCT_v7_13_Confirmatory_Readiness_Audit_20261021_011753_298.py | 3060 bytes | sha256 6d68e443ce0e0e71ec1d…

UDCT_v7_13_Constitutive_3D_Source_Prototype_20261021_012202_683.json | 3245 bytes | sha256 6d2cebbeac02d320ea88…

UDCT_v7_13_Constitutive_3D_Source_Prototype_20261021_012203_512.py | 3166 bytes | sha256 c8cc277b301d2acf3e0c…

UDCT_v7_13_Constitutive_3D_Source_Revalidated_20261021_012139_451.json | 3245 bytes | sha256 6d2cebbeac02d320ea88…

UDCT_v7_13_Constitutive_3D_Source_Revalidated_20261021_012140_566.py | 3178 bytes | sha256 dd2f014916449be1905e…

UDCT_v7_13_Direct_Boundary_59cube_Audit_20261021_011807_885.json | 3394 bytes | sha256 ab681dc93f5bc78e3bd4…

UDCT_v7_13_Direct_Boundary_59cube_Audit_20261021_011808_455.py | 3073 bytes | sha256 6c7a6730538b61b2ebff…

UDCT_v7_13_Direct_Boundary_Radial_Audit_20261021_011814_868.json | 6697 bytes | sha256 ab3a7b7540ee005d87a6…

UDCT_v7_13_Direct_Boundary_Radial_Audit_20261021_011815_545.py | 2529 bytes | sha256 1b7382db45a68bc844de…

UDCT_v7_13_Direct_Boundary_Resolution_Audit_20261021_011811_208.json | 6652 bytes | sha256 5532afc962e4ea471fe8…

UDCT_v7_13_Direct_Boundary_Resolution_Audit_20261021_011812_014.py | 2772 bytes | sha256 42a43e28ff2b965e22eb…

UDCT_v7_13_Direct_Source_Boundary_Audit_20261021_011819_759.json | 5345 bytes | sha256 959fa9edb6fdb86f7112…

UDCT_v7_13_Direct_Source_Boundary_Audit_20261021_011820_499.py | 4232 bytes | sha256 7e37682cc0de3e6428f4…

UDCT_v7_13_Fixed_Box_Resolution_Audit_20261021_011900_790.json | 3702 bytes | sha256 a8ffdac950935416d2a8…

UDCT_v7_13_Fixed_Box_Resolution_Audit_20261021_011901_260.py | 1521 bytes | sha256 1dd084d6a784cd74537f…

UDCT_v7_13_Frozen_v710_Selftest_Log_20261021_012141_009.txt | 5006 bytes | sha256 81ad969b2baed784b091…

UDCT_v7_13_Green_Cell_Quadrature_Audit_20261021_011803_844.json | 2941 bytes | sha256 8f01142a521291d9fa61…

UDCT_v7_13_Green_Cell_Quadrature_Audit_20261021_011804_539.py | 2278 bytes | sha256 2e4d2036a8453e8b0e80…

UDCT_v7_13_Hessian_Equivalent_Density_Audit_20261021_012045_768.json | 1023 bytes | sha256 28e12730d8053233162b…

UDCT_v7_13_Hessian_Equivalent_Density_Audit_20261021_012046_765.py | 1430 bytes | sha256 10072e73fbd6217c98ca…

UDCT_v7_13_Hessian_External_Field_Audit_20261021_012216_909.py | 2809 bytes | sha256 27f5c1e9cea2895a36e2…

UDCT_v7_13_Hessian_Factor_Grid_Convergence_20261021_012058_970.json | 3769 bytes | sha256 627c6eb174a7fb74fdf9…

UDCT_v7_13_Hessian_Factor_Grid_Convergence_20261021_012100_129.py | 3729 bytes | sha256 67be43fcea2bc12dffa2…

UDCT_v7_13_Hessian_Factor_Stress_Audit_20261021_012103_946.json | 7243 bytes | sha256 2685714a837c18603510…

UDCT_v7_13_Hessian_Factor_Stress_Audit_20261021_012105_059.py | 3177 bytes | sha256 620479a54966e61192f9…

UDCT_v7_13_Hessian_Preflight_and_Estimator_Selftest_20261021_012224_191.py | 3508 bytes | sha256 7fe6f26020246587fe89…

UDCT_v7_13_Hessian_Preflight_and_Estimator_Selftest_20261021_012458_489.py | 3508 bytes | sha256 7fe6f26020246587fe89…

UDCT_v7_13_Hessian_Preflight_Results_20261021_012224_972.json | 2903 bytes | sha256 a51a12c506e0d28aed97…

UDCT_v7_13_Hessian_Preflight_Results_20261021_012459_097.json | 2903 bytes | sha256 a51a12c506e0d28aed97…

UDCT_v7_13_Hydrus_3D_Angular_Newtonian_Gate_20261021_011957_586.json | 4162 bytes | sha256 0d08c38fcb7d0ce76cd4…

UDCT_v7_13_Hydrus_3D_Angular_Newtonian_Gate_20261021_011958_087.py | 2852 bytes | sha256 9e7369841bb9d940ba3e…

UDCT_v7_13_Hydrus_3D_DeltaPotential_Exploratory_20261021_011907_010.json | 14237 bytes | sha256 8a20e3595f07751e845d…

UDCT_v7_13_Hydrus_3D_DeltaPotential_Exploratory_20261021_011907_412.py | 4784 bytes | sha256 531e28f3e5bae1643c19…

UDCT_v7_13_Hydrus_Box_Boundary_Sensitivity_20261021_011903_745.json | 6971 bytes | sha256 e1f01205e55409c8be47…

UDCT_v7_13_Hydrus_Box_Boundary_Sensitivity_20261021_011904_569.py | 1632 bytes | sha256 071209f8821298647e50…

UDCT_v7_13_Hydrus_Common_Frame_Newtonian_Audit_20261021_012019_022.json | 2709 bytes | sha256 6da6601d0eaa93658e11…

UDCT_v7_13_Hydrus_Common_Frame_Newtonian_Audit_20261021_012020_218.py | 3842 bytes | sha256 d4fceefd7ae6cae91fcf…

UDCT_v7_13_Hydrus_Ell_Sensitivity_Local_Source_20261021_011947_819.json | 53561 bytes | sha256 689ee7d3e66936e9affa…

UDCT_v7_13_Hydrus_Ell_Sensitivity_Local_Source_20261021_011948_903.py | 4025 bytes | sha256 3e61d6103c36d6589995…

UDCT_v7_13_Hydrus_External_Field_3D_Input_Gate_20261021_012001_726.json | 7096 bytes | sha256 2ff93e4f6ef151207377…

UDCT_v7_13_Hydrus_External_Field_3D_Input_Gate_20261021_012002_349.py | 3904 bytes | sha256 fedde7a28704bee3fa1d…

UDCT_v7_13_Hydrus_External_Field_vs_Tide_Audit_20261021_012006_151.json | 4782 bytes | sha256 55812e53be4426453e69…

UDCT_v7_13_Hydrus_External_Field_vs_Tide_Audit_20261021_012006_844.py | 3312 bytes | sha256 1ddc7e6528796aa9e150…

UDCT_v7_13_Hydrus_External_Grid_Source_Audit_20261021_011910_184.json | 28294 bytes | sha256 a93107db65093a704732…

UDCT_v7_13_Hydrus_External_Grid_Source_Audit_20261021_011910_302.py | 3795 bytes | sha256 7ae7ffa8b81b9af1f337…

UDCT_v7_13_Hydrus_External_Newtonian_Hessian_Exploratory_20261021_012217_257.json | 3088 bytes | sha256 90477b73dcf6314332ff…

UDCT_v7_13_Hydrus_Hessian_Increment_Convergence_Audit_20261021_011930_841.json | 56075 bytes | sha256 f0eccb1ffe86451c7d67…

UDCT_v7_13_Hydrus_Hessian_Increment_Convergence_Audit_20261021_011931_333.py | 2464 bytes | sha256 63b8b67f3b157e491144…

UDCT_v7_13_Hydrus_Hessian_Increment_Convergence_Audit_summary_20261021_011929_452.txt | 9838 bytes | sha256 96cad32e79621f5ec467…

UDCT_v7_13_Hydrus_Hessian_Orientation_Bounds_20261021_012028_843.json | 1393 bytes | sha256 3135bb219884652fa2a2…

UDCT_v7_13_Hydrus_Hessian_Orientation_Bounds_20261021_012030_205.py | 2908 bytes | sha256 4b73bad127b4dd5e8819…

UDCT_v7_13_Hydrus_Hessian_Scale_Comparison_20261021_012039_950.json | 1588 bytes | sha256 9978aca7790c0b36973e…

UDCT_v7_13_Hydrus_Hessian_Scale_Comparison_20261021_012041_002.py | 2696 bytes | sha256 61ac3eef2ed4bc660e96…

UDCT_v7_13_Hydrus_Local_3D_Source_Gate_20261021_011952_641.py | 4288 bytes | sha256 46a8009600d0f7549a61…

UDCT_v7_13_Hydrus_Local_3D_Source_Gate_20261021_011953_357.json | 11339 bytes | sha256 a6787659fe17752a2961…

UDCT_v7_13_Hydrus_Plummer_3D_Hessian_Control_20261021_012212_626.json | 2761 bytes | sha256 d9f1262bb2be1b3dcd35…

UDCT_v7_13_Hydrus_Plummer_3D_Hessian_Control_20261021_012213_249.py | 4021 bytes | sha256 3fba5c142eb20d289fac…

UDCT_v7_13_Hydrus_Radial_Tidal_Scan_20261021_012010_487.json | 6542 bytes | sha256 d82f67c0c4c139fceb8d…

UDCT_v7_13_Hydrus_Radial_Tidal_Scan_20261021_012010_956.py | 2274 bytes | sha256 0a906597563a4a12fed9…

UDCT_v7_13_Independent_Endpoint_Decision_Audit_20261021_011733_110.json | 23588 bytes | sha256 ae99e13b92e51d8986b8…

UDCT_v7_13_Independent_Endpoint_Decision_Audit_20261021_011734_252.py | 3716 bytes | sha256 ec902e4fd983c973352a…

UDCT_v7_13_Joint_Grid_Box_StopGate_Audit_20261021_011843_265.json | 4029 bytes | sha256 cbcae90fb8ab6e4f635a…

UDCT_v7_13_Joint_Grid_Box_StopGate_Audit_20261021_011843_977.py | 2929 bytes | sha256 6a563d967c40878d19dd…

UDCT_v7_13_Multipole_Boundary_Exploratory_20261021_011826_685.json | 4127 bytes | sha256 ad6685e9c0338866bdf9…

UDCT_v7_13_Multipole_Boundary_Exploratory_20261021_011827_040.py | 3576 bytes | sha256 25e4ed1340291a0335e1…

UDCT_v7_13_Poisson_Sign_Units_Boundary_Gate_20261021_012145_753.json | 3342 bytes | sha256 68d990d05e5b2917728d…

UDCT_v7_13_Poisson_Sign_Units_Boundary_Gate_20261021_012146_283.py | 3454 bytes | sha256 7b310af88ad1323c1277…

UDCT_v7_13_Quadrupole_Boundary_Diagnostic_20261021_011829_919.json | 3693 bytes | sha256 f17772db2bac1568279b…

UDCT_v7_13_Quadrupole_Boundary_Diagnostic_20261021_011830_763.py | 2644 bytes | sha256 7904cffc875d250c06dd…

UDCT_v7_13_Radial_Acceleration_Convergence_20261021_011853_198.json | 11662 bytes | sha256 bc0bd2cdae2d4153dcf6…

UDCT_v7_13_Radial_Acceleration_Convergence_20261021_011853_533.py | 2819 bytes | sha256 bb8281c2c7dea363b304…

UDCT_v7_13_Radial_Box_Size_Audit_20261021_011848_100.json | 6037 bytes | sha256 6ae3e56ae6bd4eb69851…

UDCT_v7_13_Radial_Box_Size_Audit_20261021_011848_697.py | 2587 bytes | sha256 bc23e7865073da8001e0…

UDCT_v7_13_Source_Localization_Boundary_Audit_20261021_011839_531.json | 2784 bytes | sha256 c6558c5c2d5b8b9c3177…

UDCT_v7_13_Source_Localization_Boundary_Audit_20261021_011840_423.py | 2635 bytes | sha256 c2292a794d7ed00bd214…

v713_b01_six_arm_profile_audit_20261021_012444_551.py | 3168 bytes | sha256 e17af1355b0f01bc4196…

v713_b02_six_arm_profile_audit_20261021_012441_190.py | 3169 bytes | sha256 6acbcc845b912aff0c3f…

v713_b03_six_arm_profile_audit_20261021_012437_272.py | 3169 bytes | sha256 85b1ac132c0de0ab448a…

v713_b04_six_arm_profile_audit_20261021_012434_168.py | 3201 bytes | sha256 5efbf820496e151281b4…

v713_b05_six_arm_profile_audit_20261021_012430_260.py | 3201 bytes | sha256 b6567789b3bb36f36a03…

v713_b07_six_arm_profile_audit_20261021_012418_439.py | 3209 bytes | sha256 f81f02bc7a61c25fb562…

v713_b08_six_arm_profile_audit_20261021_012414_332.py | 3247 bytes | sha256 eba91e72b893a70da198…

v713_b09_six_arm_profile_audit_20261021_012410_509.py | 3247 bytes | sha256 28d9a5de9d2e6556435c…

v713_b10_six_arm_profile_audit_20261021_012405_948.py | 3255 bytes | sha256 7122406b4a868b67fb2f…

v713_b11_six_arm_profile_audit_20261021_012359_216.py | 3255 bytes | sha256 c14ddf61a266c47a4c8a…

v713_b12_six_arm_profile_audit_20261021_011800_309.py | 3251 bytes | sha256 a9d3b181a2bdfe33f0ce…

v713_b13_six_arm_profile_audit_20261021_012350_682.py | 3251 bytes | sha256 c5027120d2e01596e58a…

v713_b14_six_arm_profile_audit_20261021_012340_232.py | 3251 bytes | sha256 5750cf8c95bf0c9322d6…

v713_b15_six_arm_profile_audit_20261021_012336_457.py | 3251 bytes | sha256 055883ebd1d7c8630343…

v713_b16_six_arm_profile_audit_20261021_012332_233.py | 3255 bytes | sha256 1d6c4e8982b3347ec456…

v713_b17_six_arm_profile_audit_20261021_012328_469.py | 3256 bytes | sha256 a74f21a3b9df309491b5…

v713_b19_six_arm_profile_audit_20261021_012316_290.py | 3256 bytes | sha256 873edd23f52b73579507…

v713_b20_six_arm_profile_audit_20261021_012309_515.py | 3256 bytes | sha256 71c57c3aea0d208b2669…

v713_b21_six_arm_profile_audit_20261021_012303_573.py | 3256 bytes | sha256 b22c4baf0f9d4e8711da…

v713_b22_six_arm_profile_audit_20261021_012257_380.py | 3253 bytes | sha256 ef2dc2de328d1e0a448f…

v713_b23_six_arm_profile_audit_20261021_012252_402.py | 3254 bytes | sha256 87ec05d7cb1501da14b8…

v713_b24_six_arm_profile_audit_20261021_012248_516.py | 3253 bytes | sha256 56f6a447378f7e8cd51b…

v713_validate_b01_20261021_012448_352.py | 2020 bytes | sha256 0bb7e1d64c3a275d942b…

UDCT_v7_13_3D_Constitutive_Poisson_Coupling_Exploratory_20261021_012131_309.json | 4088 bytes | sha256 8badf231e2df0258e902…

# Appendix B. Selected source excerpts and function entry points

## UDCT_v7_13_3D_Potential_Implementation_Gate_20261021_012207_715.py

Source excerpt: first 45 lines. Full original is in the companion ZIP.

```
#!/usr/bin/env python3

"""Read-only, reproducible source-level implementation gate; not a galaxy simulation."""

import ast,hashlib,json,pathlib,datetime

root=pathlib.Path('/mnt/data/v713_hessian_stage')

files=['UDCT_v7_06_Hessian_Term_Sign_Audit_reproduce.py','UDCT_v7_10_Postregistration_run.py','UDCT_v7_11_Preregistratio
n_Anisotro

result={'audit_type':'source-level implementation inventory (not numerical Hessian
validation)','files':{},'criteria':{'3d_total_m

for name in files:

p=root/name; s=p.read_text(); tree=ast.parse(s); funcs=[x.name for x in ast.walk(tree) if
isinstance(x,(ast.FunctionDef,ast.Async

result['files'][name]={'sha256':hashlib.sha256(p.read_bytes()).hexdigest(),'functions_of_interest':[f for f in funcs if
any(k in

result['specific_findings']={'v706_galactic_field':'returns external visible-mass gradient and Hessian at Hydrus I
location','v710

out=pathlib.Path('/mnt/data/UDCT_v7_13_3D_Potential_Implementation_Gate.json');out.write_text(json.dumps(result,indent=2
,ensure_as
```

## UDCT_v7_13_B01_Independent_Jeans_Quadrature_Audit_20261021_012453_346.py

Source excerpt: first 45 lines. Full original is in the companion ZIP.

```
#!/usr/bin/env python3

"""Exploratory independent integration audit for B01. Not a six-arm field validation."""

from pathlib import Path

import importlib.util, sys, json, hashlib

import numpy as np

from scipy.integrate import cumulative_simpson

ROOT=Path('/mnt/data/v713_post_work')

plan=json.loads((ROOT/'preregistration_plan.json').read_text())

for name,digest in plan['frozen_hashes'].items():

assert hashlib.sha256((ROOT/name).read_bytes()).hexdigest()==digest,name

spec=importlib.util.spec_from_file_location('v712frozen',ROOT/'UDCT_v7_12_Preregistration_Jeans_Transition.py')

v=importlib.util.module_from_spec(spec);sys.modules[spec.name]=v;spec.loader.exec_module(v)

m=v.load_frozen(); b=m.load_baseline(); obj=b.DWARFS['Hydrus I']

results=[]

for gridname in ('default','refined'):

grid=b.make_grid(gridname);e,h=b.V706.galactic_field(obj,grid)[:2]

for inner in (.002,.0005):

st=b.model_state(obj,'Tapered cusp',b.mesh('Tapered cusp',grid,rmin=inner),e,h,grid,b.BASE_LAW,1.,0.)

_,force=b.V706.work_from_state(st)

r,rho=st['r'],st['rho'];vref2=float(np.interp(1.,r/st['rh'],st['gn']))*st['rh']

for beta in (.35,.40):

p_trap,_,q=m.solve(r,rho,force,beta,0.)

# Independent integration rule (composite Simpson on nonuniform radial mesh).

y=q*rho*force;x=r
```

```
cumulative=cumulative_simpson(y,x=x,initial=0.)
integral=cumulative[-1]-cumulative
p_simp=integral/q
mask=(r/st['rh']<=10.)&(rho>1e-280)
trap=p_trap[mask]/rho[mask]/vref2
simp=p_simp[mask]/rho[mask]/vref2
results.append({'grid':gridname,'inner_rh':inner,'beta':beta,
'trapezoid_min_normalized':float(trap.min()),
'simpson_min_normalized':float(simp.min()),
'max_abs_normalized_profile_difference':float(np.max(abs(trap-simp))),
'trapezoid_sign':int(np.sign(trap.min())),
'simpson_sign':int(np.sign(simp.min()))})
report={'status':'EXPLORATORY quadrature-only check; not independent gravitational field or six-arm validation',
'frozen_hashes_verified':True,'target':'B01','hessian_performed':False,'six_arm_validation_performed':False,'results':re
su
path=Path('/mnt/data/UDCT_v7_13_B01_Quadrature_Audit.json')
path.write_text(json.dumps(report,indent=2)+'\n')
print(json.dumps(report,indent=2))
```

## UDCT_v7_13_B01_Stress_Derivative_Decomposition_20261021_011646_125.py

Source excerpt: first 45 lines. Full original is in the companion ZIP.

```
#!/usr/bin/env python3
"""Exploratory decomposition of explicit GQUMOND stress force in frozen B01.
NOT a new theory, preregistered confirmation, or physical Hessian measurement.
"""
from pathlib import Path
import importlib.util, sys, json, hashlib
import numpy as np
ROOT=Path('/mnt/data/UDCT_v7_13_Confirmatory_Gate_Staging')
plan=json.loads((ROOT/'preregistration_plan.json').read_text())
hashes={name:hashlib.sha256((ROOT/name).read_bytes()).hexdigest() for name in plan['frozen_hashes']}
assert all(hashes[n]==v for n,v in plan['frozen_hashes'].items())
spec=importlib.util.spec_from_file_location('v712_stress_split',ROOT/'UDCT_v7_12_Preregistration_Jeans_Transition.py')
v=importlib.util.module_from_spec(spec);sys.modules[spec.name]=v;spec.loader.exec_module(v)
m=v.load_frozen();b=m.load_baseline();obj=b.DWARFS['Hydrus I']
rows=[]
for grid_name in ('default','refined'):
grid=b.make_grid(grid_name);e,h=b.V706.galactic_field(obj,grid)[:2]
ratios=b.mesh('Tapered cusp',grid,rmin=.0005)
st=b.model_state(obj,'Tapered cusp',ratios,e,h,grid,b.BASE_LAW,1.,0.)
r=st['r'];x=r/st['rh'];rho=st['rho'];vref=float(np.interp(1.,x,st['gn']))*st['rh']
lead=st['lead'];pnn=st['pnn'];trp=st['trp']
grad=-.5*np.gradient(pnn,r,edge_order=2)
geom=-.5*(3*pnn-trp)/r
forces={'screened_lead':lead,'lead_plus_radial_derivative':lead+grad,'lead_plus_geometric_stress':lead+geom,'full':lead+
grad+geom
for beta in (.35,.40):
```

```
vals={}

for label,force in forces.items():

p,_,_=m.solve(r,rho,force,beta)

F=np.divide(p,rho*vref,out=np.full_like(p,np.nan),where=rho>1e-280)

mask=(x>=.0005)&(x<=.002)&np.isfinite(F)

roots=[]

for i in np.flatnonzero((F[:-1]*F[1:]<0)&(x[:-1]<.002)&np.isfinite(F[:-1])&np.isfinite(F[1:])):

t=-F[i]/(F[i+1]-F[i]);roots.append(float(np.exp(np.log(x[i])+t*(np.log(x[i+1])-np.log(x[i])))))

vals[label]={'F_inner':float(F[0]),'F_min_inner':float(np.min(F[mask])),'negative_inner_samples':int(np.sum(F[mask]<0)),
'zero_r

# Because Jeans solve is linear in force at fixed rho,beta, calculate contributions relative to lead.

F0=vals['screened_lead']['F_inner'];Fg=vals['lead_plus_radial_derivative']['F_inner'];Ft=vals['lead_plus_geometric_stres
s']['F_i

rows.append({'grid':grid_name,'beta':beta,'force_at_inner_over_gn':{'lead':float(lead[0]/st['gn'][0]),'radial_derivative
':float(

print('completed',grid_name,flush=True)

out={'status':'EXPLORATORY_STRESS_FORCE_DECOMPOSITION','verified_frozen_hashes':hashes,'setup':'Hydrus I B01 BASE law,
ell=1 pc, t

path=Path('/mnt/data/UDCT_v7_13_B01_Stress_Derivative_Decomposition.json');path.write_text(json.dumps(out,indent=2)+'\n'
)

for row in rows:print(row['grid'],row['beta'],row['F_inner_contributions'],{k:v['zero_rh'] for k,v in
row['variants'].items()})

print(path)
```

## UDCT_v7_13_B02_B24_Full_Profile_Trace_20261021_011741_455.py

Source excerpt: first 45 lines. Full original is in the companion ZIP.

```
#!/usr/bin/env python3

"""B02-B24 exploratory six-arm full-profile archival. Frozen physics; no 3D Hessian."""

from pathlib import Path

import json,csv,hashlib,importlib.util,sys,time

import numpy as np

R=Path('/mnt/data/UDCT_v7_13_Confirmatory_Gate_Staging')

OUT=Path('/mnt/data')

plan=json.loads((R/'preregistration_plan.json').read_text())

for name,digest in plan['frozen_hashes'].items():

assert hashlib.sha256((R/name).read_bytes()).hexdigest()==digest,name

def load(path,name):

spec=importlib.util.spec_from_file_location(name,path);mod=importlib.util.module_from_spec(spec);sys.modules[name]=mod;s
pec.lo

old=load(R/'UDCT_v7_12_Preregistration_Jeans_Transition.py','v712_batch_trace')

proto=load(Path('/mnt/data/v713_b07_run/UDCT_v7_13_Six_Arm_Decision_Prototype.py'),'v713_batch_proto')

m=old.load_frozen();b=m.load_baseline();obj=b.DWARFS['Hydrus I']

with (R/'adaptive_brackets.csv').open() as f: targets=list(csv.DictReader(f))

assert len(targets)==24

laws={law.name:law for law in (b.BASE_LAW,)+b.PART_A_LAWS}

configs=[('default',b.make_grid('default')),('refined',b.make_grid('refined')),('validation',b.Grid(radial=1440,polar=96
,azimuth=1

# Cache the force for each law, ell, resolution, and inner radius across targets.

cache={};profiles={};results=[];t0=time.monotonic()
```

```
start=int(sys.argv[1]) if len(sys.argv)>1 else 2
stop=int(sys.argv[2]) if len(sys.argv)>2 else 24
for index in range(start,stop+1):
target=targets[index-1]
tid=f'B{index:02d}'; axis=target['axis'];law=target['law'];
fixed=float(target['fixed_coordinate']);lo=float(target['lo']);hi=
assert axis in ('beta','ell_pc') and law in laws
endpoints=[lo,hi]; arms={p:{} for p in endpoints}; logs=[]
for gridname,grid in configs:
e,h=b.V706.galactic_field(obj,grid)[:2]
for inner in (.002,.0005):
for p in endpoints:
ell=fixed if axis=='beta' else p
beta=p if axis=='beta' else fixed
cachekey=(law,ell,gridname,inner)
if cachekey not in cache:
st=b.model_state(obj,'Tapered cusp',b.mesh('Tapered cusp',grid,rmin=inner),e,h,grid,laws[law],ell,0.)
_,force=b.V706.work_from_state(st)
_,ref=b.V706.aperture_jeans(st,force,obj.re_pc)
pressure,_,_=m.solve(st['r'],st['rho'],force,0.)
sigma=np.divide(pressure,st['rho'],out=np.zeros_like(pressure),where=st['rho']>1e-280)
vref2=float(np.interp(1.,st['r']/st['rh'],st['gn']))*st['rh']
gate=float(np.max(abs(sigma-ref))/vref2)
assert np.isfinite(gate) and gate<1e-10,(tid,cachekey,gate)
cache[cachekey]=(st,force,gate)
```

## UDCT_v7_13_Hessian_Preflight_and_Estimator_Selftest_20261021_012224_191.py

Source excerpt: first 45 lines. Full original is in the companion ZIP.

```
#!/usr/bin/env python3
"""Non-confirmatory preflight: frozen-source audit and synthetic 3-D Hessian estimator tests.
Does NOT calculate a Hydrus I total-potential Hessian or any B01-B24 Hessian outcome.
"""
from pathlib import Path
import hashlib, json, ast
import numpy as np
ROOT=Path('/mnt/data/v713_b07_run')
PLAN=json.loads((ROOT/'preregistration_plan.json').read_text())
verified={name: hashlib.sha256((ROOT/name).read_bytes()).hexdigest()==digest for name,digest in
PLAN['frozen_hashes'].items()}
assert all(verified.values())
cp=PLAN['complete_protocol']
steps=cp['hessian_steps_rh']
assert steps==[0.02,0.01,0.005]
# Generic central-difference tensor, not a galaxy solver.
def hessian(phi,x,h):
x=np.array(x,dtype=float); out=np.zeros((3,3)); f0=phi(x)
for i in range(3):
```

```
ei=np.eye(3)[i]*h
out[i,i]=(phi(x+ei)-2*f0+phi(x-ei))/h**2
for j in range(i):
ej=np.eye(3)[j]*h
out[i,j]=out[j,i]=(phi(x+ei+ej)-phi(x+ei-ej)-phi(x-ei+ej)+phi(x-ei-ej))/(4*h*h)
return out
A=np.array([[2.,0.3,-0.2],[0.3,-1.,0.4],[-0.2,0.4,3.]])
x=np.array([0.1,-0.2,0.3]); rh=1.
quad=lambda y: 0.5*y@A@y + np.array([0.4,0.1,-0.5])@y+0.7
quad_errors=[float(np.max(abs(hessian(quad,x,h*rh)-A))) for h in steps]
assert max(quad_errors)<1e-10
# Nonquadratic test has nonzero finite-difference truncation error; verify order.
quart=lambda y: 0.5*y@A@y + 0.6*y[0]**4+0.4*y[1]**4+0.2*y[2]**4
exact=A+np.diag([7.2*x[0]**2,4.8*x[1]**2,2.4*x[2]**2])
quart_errors=[float(np.max(abs(hessian(quart,x,h)-exact))) for h in steps]
assert quart_errors[0]>quart_errors[1]>quart_errors[2]
# Static source API check: no function returns Phi_total(x,y,z) for full modified gravity.
modules={}
for name in
['UDCT_v7_06_Hessian_Term_Sign_Audit_reproduce.py','UDCT_v7_10_Postregistration_run.py','UDCT_v7_12_Preregistration_Je
tree=ast.parse((ROOT/name).read_text()); modules[name]=[n.name for n in tree.body if
isinstance(n,(ast.FunctionDef,ast.AsyncFu
report={
'status':'PRELIMINARY PREFLIGHT ONLY — NOT GALAXY HESSIAN EXECUTION',
'frozen_sha256_verified':all(verified.values()),'frozen_hash_checks':verified,
'hessian_steps_rh':steps,
'synthetic_quadratic_max_component_errors':quad_errors,
'synthetic_quartic_max_component_errors':quart_errors,
'synthetic_estimator_selftest':'PASS',
```

## UDCT_v7_13_Hessian_Preflight_and_Estimator_Selftest_20261021_012458_489.py

Source excerpt: first 45 lines. Full original is in the companion ZIP.

```
#!/usr/bin/env python3
"""Non-confirmatory preflight: frozen-source audit and synthetic 3-D Hessian estimator tests.
Does NOT calculate a Hydrus I total-potential Hessian or any B01-B24 Hessian outcome.
"""
from pathlib import Path
import hashlib, json, ast
import numpy as np
ROOT=Path('/mnt/data/v713_b07_run')
PLAN=json.loads((ROOT/'preregistration_plan.json').read_text())
verified={name: hashlib.sha256((ROOT/name).read_bytes()).hexdigest()==digest for name,digest in
PLAN['frozen_hashes'].items()}
assert all(verified.values())
cp=PLAN['complete_protocol']
steps=cp['hessian_steps_rh']
assert steps==[0.02,0.01,0.005]
```

```
# Generic central-difference tensor, not a galaxy solver.
def hessian(phi,x,h):
x=np.array(x,dtype=float); out=np.zeros((3,3)); f0=phi(x)
for i in range(3):
ei=np.eye(3)[i]*h
out[i,i]=(phi(x+ei)-2*f0+phi(x-ei))/h**2
for j in range(i):
ej=np.eye(3)[j]*h
out[i,j]=out[j,i]=(phi(x+ei+ej)-phi(x+ei-ej)-phi(x-ei+ej)+phi(x-ei-ej))/(4*h*h)
return out
A=np.array([[2.,0.3,-0.2],[0.3,-1.,0.4],[-0.2,0.4,3.]])
x=np.array([0.1,-0.2,0.3]); rh=1.
quad=lambda y: 0.5*y@A@y + np.array([0.4,0.1,-0.5])@y+0.7
quad_errors=[float(np.max(abs(hessian(quad,x,h*rh)-A))) for h in steps]
assert max(quad_errors)<1e-10
# Nonquadratic test has nonzero finite-difference truncation error; verify order.
quart=lambda y: 0.5*y@A@y + 0.6*y[0]**4+0.4*y[1]**4+0.2*y[2]**4
exact=A+np.diag([7.2*x[0]**2,4.8*x[1]**2,2.4*x[2]**2])
quart_errors=[float(np.max(abs(hessian(quart,x,h)-exact))) for h in steps]
assert quart_errors[0]>quart_errors[1]>quart_errors[2]
# Static source API check: no function returns Phi_total(x,y,z) for full modified gravity.
modules={}
for name in
['UDCT_v7_06_Hessian_Term_Sign_Audit_reproduce.py','UDCT_v7_10_Postregistration_run.py','UDCT_v7_12_Preregistration_Je
tree=ast.parse((ROOT/name).read_text()); modules[name]=[n.name for n in tree.body if
isinstance(n,(ast.FunctionDef,ast.AsyncFu
report={
'status':'PRELIMINARY PREFLIGHT ONLY — NOT GALAXY HESSIAN EXECUTION',
'frozen_sha256_verified':all(verified.values()),'frozen_hash_checks':verified,
'hessian_steps_rh':steps,
'synthetic_quadratic_max_component_errors':quad_errors,
'synthetic_quartic_max_component_errors':quart_errors,
'synthetic_estimator_selftest':'PASS',
```

## References and archived lineage

UDCT v7.13 preregistration: https://doi.org/10.5281/zenodo.23342971

UDCT v7.12 post-registration: https://doi.org/10.5281/zenodo.23296444

UDCT v7.11 post-registration: https://doi.org/10.5281/zenodo.23263099

UDCT v7.10 post-registration: https://doi.org/10.5281/zenodo.23233837

UDCT v7.09 post-registration: https://doi.org/10.5281/zenodo.23130843

Primary reproducibility source: user-supplied 7.13 Hessian.zip (unchanged).

PART II

# Technical Supplement

## Full Mathematical & Code Companion

114 pages | mathematical details, code analysis and audit records

# UDCT v7.13 — Mathematical and Source-Code Companion

A comprehensive, source-linked technical record of 24 six-arm Jeans sign-boundary targets and 77 archived Python programs

Won Shik Paik | 22 October 2026 | Retrospective exploratory documentation

Companion to UDCT v7.13 Comprehensive Technical Monograph. Archive: 7.13 Hessian.zip, 162 members. No fresh galaxy computation is asserted. Source quotations are extracted directly from the archive. This work does not establish a physical 3-D Hessian or a validated strict sign bracket.

# Reading map and mathematical conventions

The report is organized by concepts and code dependencies, not file timestamps. Part I describes the Jeans integration and the strict sign gate. Part II presents 24 target-by-target numerical records and figures. Part III annotates every archived Python program. Part IV gives the evidence map, open issues, and an exact file manifest.

## Spherical Jeans equation

d(nu sigma_r^2)/dr + (2 beta/r) nu sigma_r^2 = -nu g_r. Here nu is tracer density, beta is velocity anisotropy and g_r is the inward acceleration magnitude under the adopted sign convention. Integrating-factor solutions are boundary-dependent; positive radial second moment is a necessary, not sufficient, physical-admissibility condition.

## Anisotropy definition

beta = 1 - (sigma_theta^2 + sigma_phi^2)/(2 sigma_r^2). Under spherical symmetry with equal tangential components this reduces to 1 - sigma_t^2/sigma_r^2. A fixed positive beta near a regular center requires particular care with phase-space admissibility.

## Full Hessian

H_ij = partial_i partial_j Phi_total. Its trace is the Laplacian of the total gravitational potential; eigenvalues describe local principal curvatures. A radial second derivative alone does not specify the full tensor. Hessian tests on manufactured potentials check implementation, not a physical galaxy solution.

## B01 stress-force decomposition

g_full = g_lead - (1/2) d p_nn/dr - (1/2)(3p_nn - tr p)/r. This is an exploratory archived formula; terms and units must be checked against source normalization before cross-target comparison.

## Strict six-arm decision

E is the largest pointwise difference between six common-domain profiles; tau=max(1e-8,5E). F is the smallest of six common-domain and six native-domain minima. Positive: F>tau. Negative: F<-tau and all six profiles are below -tau at a shared sampled radius. Otherwise unresolved.

## Limits of observational inference

Hydrus I observationally informed inputs are not a direct statistical fit to individual stellar line-of-sight velocities. Numerical positivity does not establish a positive distribution function or dynamical stability.

# Part I — Derivation and audit methodology

## Integrating factor

Let y(r)=nu(r) sigma_r^2(r). The differential equation y′ + 2 beta(r)y/r = -nu(r)g_r(r) has integrating factor I(r)=exp[integral 2 beta(r)/r dr]. Therefore (Iy)′=-I nu g_r. A boundary condition at an outer radius R gives y(r)=[I(R)y(R)+integral_r^R I(s)nu(s)g_r(s) ds]/I(r). This identity is conditional on the acceleration sign convention and boundary definition.

Audit questions: Which file defines this quantity? Is the boundary condition explicit? Is the numerical domain common across arms? Does the saved output contain sufficient data for independent recomputation? Which claims are implementation-only versus physical?

## Constant-beta special case

For beta constant, I(r)=r^(2 beta). The formal solution is sigma_r^2(r)=r^(-2 beta)/nu(r) [R^(2 beta)nu(R)sigma_r^2(R)+integral_r^R s^(2 beta)nu(s)g_r(s) ds]. This expression is not evidence that the numerical archive used an identical outer boundary; consult each solver implementation.

Audit questions: Which file defines this quantity? Is the boundary condition explicit? Is the numerical domain common across arms? Does the saved output contain sufficient data for independent recomputation? Which claims are implementation-only versus physical?

## Why negative values matter

For an ordinary velocity distribution with nonnegative weights, sigma_r^2 is nonnegative. A computed negative radial second moment therefore fails that candidate interpretation. It may reflect the closure, the force law, a boundary, or numerical discretization. It does not mean negative variance was observed.

Audit questions: Which file defines this quantity? Is the boundary condition explicit? Is the numerical domain common across arms? Does the saved output contain sufficient data for independent recomputation? Which claims are implementation-only versus physical?

## Radial-domain mismatch

Some arms integrate inward to 0.0005 r_h, while the comparison domain begins at 0.002 r_h. A negative minimum attained only inside the excluded common-radius interval cannot be a common-radius negative witness, even if F includes that native minimum.

Audit questions: Which file defines this quantity? Is the boundary condition explicit? Is the numerical domain common across arms? Does the saved output contain sufficient data for independent recomputation? Which claims are implementation-only versus physical?

## Resolution checks

Three resolutions (360 x 24 x 36), (720 x 48 x 72), (1440 x 96 x 144) are paired with two inner cutoffs. Six arms are related implementations, not six independent mathematical solvers. Agreement at sampled radii can miss a narrow unsampled excursion.

Audit questions: Which file defines this quantity? Is the boundary condition explicit? Is the numerical domain common across arms? Does the saved output contain sufficient data for independent recomputation? Which claims are implementation-only versus physical?

## Potential boundary conditions

A 3-D Poisson or nonlinear constitutive solve requires a physical exterior boundary. A manufactured polynomial or periodic field with an exact solution is useful for code testing but does not supply the exterior field of Hydrus I.

Audit questions: Which file defines this quantity? Is the boundary condition explicit? Is the numerical domain common across arms? Does the saved output contain sufficient data for independent recomputation? Which claims are implementation-only versus physical?

## Discretization and Hessian

For a smooth Cartesian potential, centered second differences approximate H_ii with O(h^2) truncation under regularity assumptions. Mixed derivatives require consistent cross stencils and boundary treatment. Numerical Hessian convergence must be evaluated on the same physically solved total potential.

Audit questions: Which file defines this quantity? Is the boundary condition explicit? Is the numerical domain common across arms? Does the saved output contain sufficient data for independent recomputation? Which claims are implementation-only versus physical?

## Preregistration gate

The amended preregistration criteria are a reference specification. The public availability of the amendment before each exploratory run is not independently established. Consequently these outputs are retrospective exploratory evidence, not demonstrated preregistered confirmation.

Audit questions: Which file defines this quantity? Is the boundary condition explicit? Is the numerical domain common across arms? Does the saved output contain sufficient data for independent recomputation? Which claims are implementation-only versus physical?

# Part II — B01–B24 numerical dossiers

## Target B01: preserved six-arm sign-boundary record

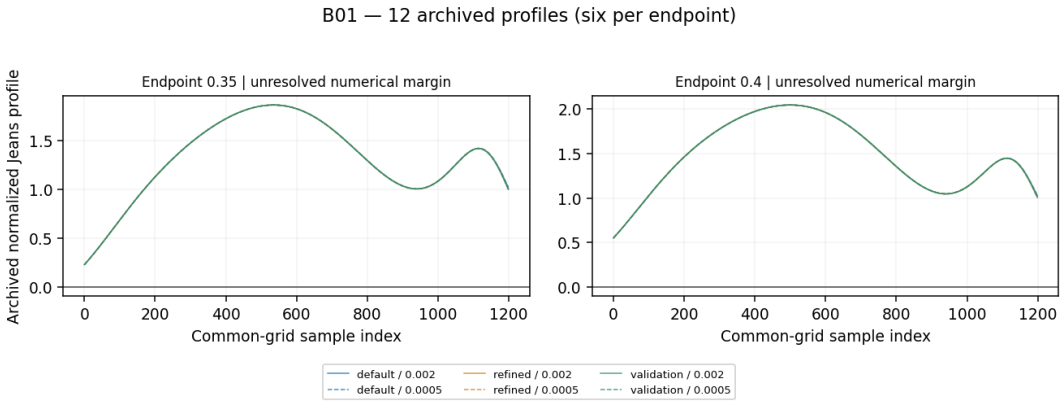



Figure: archived six-arm profiles. Vertical coordinate is the saved normalized Jeans profile; plotted lines must not be interpreted as observed stellar velocities. See the companion source archive for exact normalization.

Source JSON: UDCT_v7_13_B01_Six_Arm_Profile_Exploratory_20261021_012443_978.json

Endpoint beta=0.35: recorded status=unresolved_numerical_margin; F=-0.15896415671734707; E=0.023564479964140572; tau=0.11782239982070286; common_negative=False.

Native minima by arm: +0.231174, +0.231222, -0.158964, -0.158932, +0.231234, -0.158924

Common minima by arm: +0.231174, +0.231222, +0.231148, +0.231212, +0.231234, +0.231231

Endpoint beta=0.4: recorded status=unresolved_numerical_margin; F=0.07822994227392319; E=0.023432939893254234; tau=0.11716469946627117; common_negative=False.

Native minima by arm: +0.552167, +0.552182, +0.0782299, +0.0782327, +0.552186, +0.0782333

Common minima by arm: +0.552167, +0.552182, +0.552151, +0.552174, +0.552186, +0.552184

Preserved bracket decision: unresolved_numerical_margin. Physical Hessian executed: False. No strict transition coordinate is confirmed for this target.

Interpretation: An endpoint labeled robust-positive does not form a validated sign bracket without an opposite robust-negative endpoint. A negative native-grid minimum can coexist with a positive common-domain minimum. Examine the shared-radius requirement before treating any inner dip as a physical sign transition.

Reproduction and limitations: retain all six arms, original native cutoffs, common comparison radius, numerical tolerance and the archived hashes. Inspect B01-specific force decomposition only for B01; do not extrapolate its component attribution to this target without a matching decomposition calculation.

# Target B02: preserved six-arm sign-boundary record

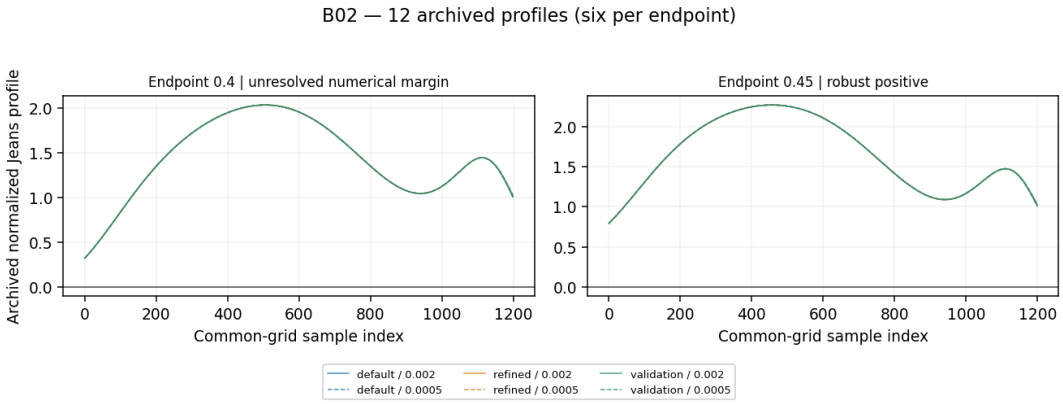



Figure: archived six-arm profiles. Vertical coordinate is the saved normalized Jeans profile; plotted lines must not be interpreted as observed stellar velocities. See the companion source archive for exact normalization.

Source JSON: UDCT_v7_13_B02_Six_Arm_Profile_Exploratory_20261021_012440_907.json

Endpoint beta=0.4: recorded status=unresolved_numerical_margin; F=-0.09215470903181372; E=0.023432939995744473; tau=0.11716469997872236; common_negative=False.

Native minima by arm: +0.322363, +0.3224, -0.0921547, -0.0921321, +0.322409, -0.0921265

Common minima by arm: +0.322363, +0.3224, +0.322344, +0.322393, +0.322409, +0.322408

Endpoint beta=0.45: recorded status=robust_positive; F=0.32118872762518696; E=0.02330145144799811; tau=0.11650725723999056; common_negative=False.

Native minima by arm: +0.794044, +0.794034, +0.321224, +0.321196, +0.794032, +0.321189

Common minima by arm: +0.794044, +0.794034, +0.79404, +0.79403, +0.794032, +0.794031

Preserved bracket decision: unresolved_numerical_margin. Physical Hessian executed: False. No strict transition coordinate is confirmed for this target.

Interpretation: An endpoint labeled robust-positive does not form a validated sign bracket without an opposite robust-negative endpoint. A negative native-grid minimum can coexist with a positive common-domain minimum. Examine the shared-radius requirement before treating any inner dip as a physical sign transition.

Reproduction and limitations: retain all six arms, original native cutoffs, common comparison radius, numerical tolerance and the archived hashes. Inspect B01-specific force decomposition only for B01; do not extrapolate its component attribution to this target without a matching decomposition calculation.

# Target B03: preserved six-arm sign-boundary record

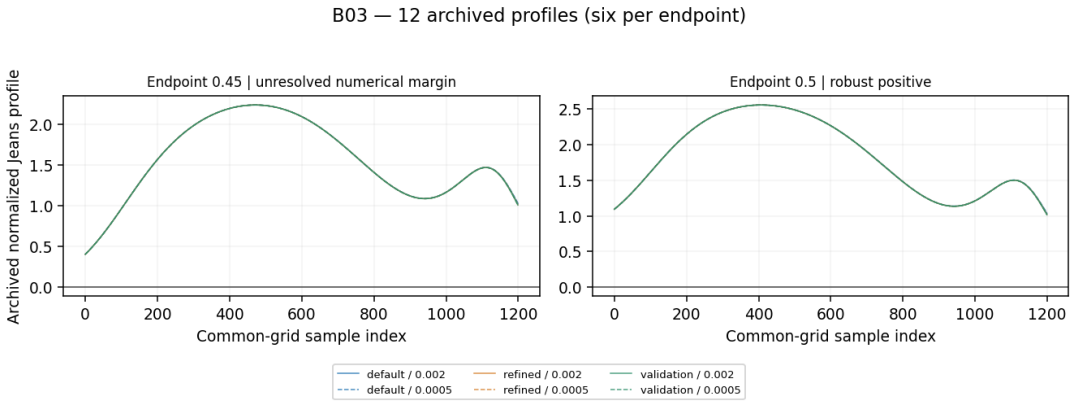



Figure: archived six-arm profiles. Vertical coordinate is the saved normalized Jeans profile; plotted lines must not be interpreted as observed stellar velocities. See the companion source archive for exact normalization.

Source JSON: UDCT_v7_13_B03_Six_Arm_Profile_Exploratory_20261021_012436_828.json

Endpoint beta=0.45: recorded status=unresolved_numerical_margin; F=-0.0013594551349353924; E=0.02330145163775388; tau=0.11650725818876939; common_negative=False.

Native minima by arm: +0.404283, +0.404307, -0.00135946, -0.00134871, +0.404313, -0.00134628

Common minima by arm: +0.404283, +0.404307, +0.404272, +0.404304, +0.404313, +0.404313

Endpoint beta=0.5: recorded status=robust_positive; F=0.701149902656224; E=0.023170016332308085; tau=0.11585008166154043; common_negative=False.

Native minima by arm: +1.0578, +1.02563, +0.701244, +0.701169, +1.021, +0.70115

Common minima by arm: +1.03371, +1.0196, +1.03926, +1.021, +1.01609, +1.01644

Preserved bracket decision: unresolved_numerical_margin. Physical Hessian executed: False. No strict transition coordinate is confirmed for this target.

Interpretation: An endpoint labeled robust-positive does not form a validated sign bracket without an opposite robust-negative endpoint. A negative native-grid minimum can coexist with a positive common-domain minimum. Examine the shared-radius requirement before treating any inner dip as a physical sign transition.

Reproduction and limitations: retain all six arms, original native cutoffs, common comparison radius, numerical tolerance and the archived hashes. Inspect B01-specific force decomposition only for B01; do not extrapolate its component attribution to this target without a matching decomposition calculation.

# Target B04: preserved six-arm sign-boundary record

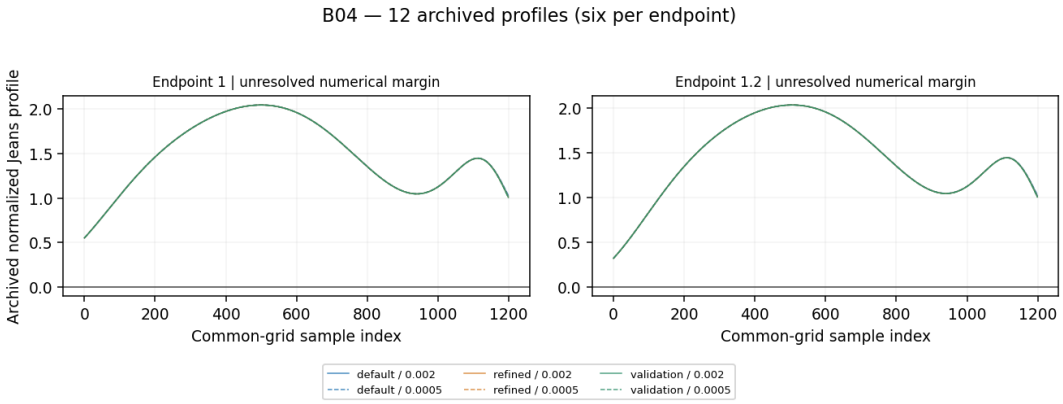



Figure: archived six-arm profiles. Vertical coordinate is the saved normalized Jeans profile; plotted lines must not be interpreted as observed stellar velocities. See the companion source archive for exact normalization.

Source JSON: UDCT_v7_13_B04_Six_Arm_Profile_Exploratory_20261021_012433_635.json

Endpoint beta=1.0: recorded status=unresolved_numerical_margin; F=0.07822994227392319; E=0.023432939893254234; tau=0.11716469946627117; common_negative=False.

Native minima by arm: +0.552167, +0.552182, +0.0782299, +0.0782327, +0.552186, +0.0782333

Common minima by arm: +0.552167, +0.552182, +0.552151, +0.552174, +0.552186, +0.552184

Endpoint beta=1.2: recorded status=unresolved_numerical_margin; F=-0.09215470903181372; E=0.023432939995744473; tau=0.11716469997872236; common_negative=False.

Native minima by arm: +0.322363, +0.3224, -0.0921547, -0.0921321, +0.322409, -0.0921265

Common minima by arm: +0.322363, +0.3224, +0.322344, +0.322393, +0.322409, +0.322408

Preserved bracket decision: unresolved_numerical_margin. Physical Hessian executed: False. No strict transition coordinate is confirmed for this target.

Interpretation: An endpoint labeled robust-positive does not form a validated sign bracket without an opposite robust-negative endpoint. A negative native-grid minimum can coexist with a positive common-domain minimum. Examine the shared-radius requirement before treating any inner dip as a physical sign transition.

Reproduction and limitations: retain all six arms, original native cutoffs, common comparison radius, numerical tolerance and the archived hashes. Inspect B01-specific force decomposition only for B01; do not extrapolate its component attribution to this target without a matching decomposition calculation.

# Target B05: preserved six-arm sign-boundary record

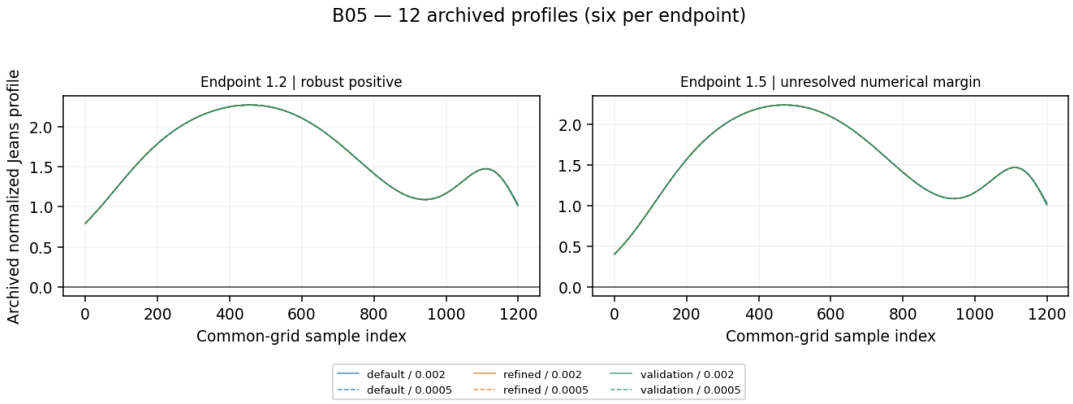



Figure: archived six-arm profiles. Vertical coordinate is the saved normalized Jeans profile; plotted lines must not be interpreted as observed stellar velocities. See the companion source archive for exact normalization.

Source JSON: UDCT_v7_13_B05_Six_Arm_Profile_Exploratory_20261021_012429_424.json

Endpoint beta=1.2: recorded status=robust_positive; F=0.32118872762518696; E=0.02330145144799811; tau=0.11650725723999056; common_negative=False.

Native minima by arm: +0.794044, +0.794034, +0.321224, +0.321196, +0.794032, +0.321189

Common minima by arm: +0.794044, +0.794034, +0.79404, +0.79403, +0.794032, +0.794031

Endpoint beta=1.5: recorded status=unresolved_numerical_margin; F=-0.0013594551349353924; E=0.02330145163775388; tau=0.11650725818876939; common_negative=False.

Native minima by arm: +0.404283, +0.404307, -0.00135946, -0.00134871, +0.404313, -0.00134628

Common minima by arm: +0.404283, +0.404307, +0.404272, +0.404304, +0.404313, +0.404313

Preserved bracket decision: unresolved_numerical_margin. Physical Hessian executed: False. No strict transition coordinate is confirmed for this target.

Interpretation: An endpoint labeled robust-positive does not form a validated sign bracket without an opposite robust-negative endpoint. A negative native-grid minimum can coexist with a positive common-domain minimum. Examine the shared-radius requirement before treating any inner dip as a physical sign transition.

Reproduction and limitations: retain all six arms, original native cutoffs, common comparison radius, numerical tolerance and the archived hashes. Inspect B01-specific force decomposition only for B01; do not extrapolate its component attribution to this target without a matching decomposition calculation.

# Target B06: preserved six-arm sign-boundary record

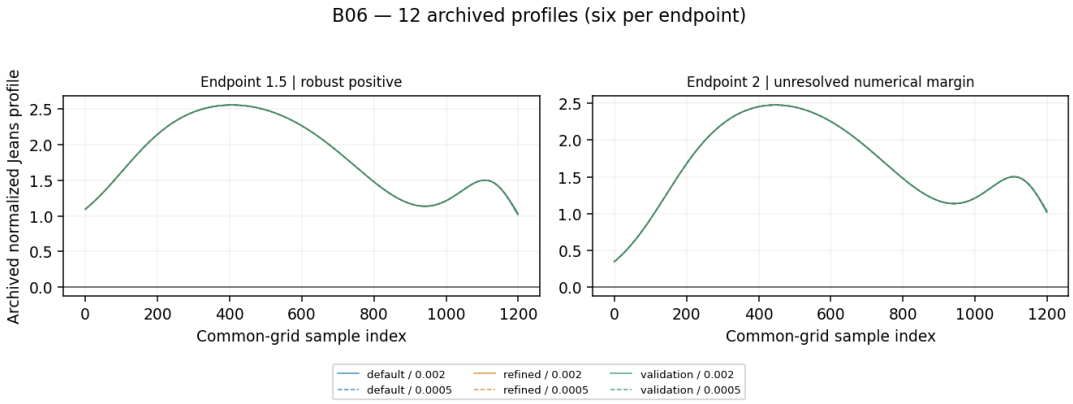



Figure: archived six-arm profiles. Vertical coordinate is the saved normalized Jeans profile; plotted lines must not be interpreted as observed stellar velocities. See the companion source archive for exact normalization.

Source JSON: UDCT_v7_13_B06_Six_Arm_Profile_Exploratory_20261021_012425_495.json

Endpoint beta=1.5: recorded status=robust_positive; F=0.701149902656224; E=0.023170016332308085; tau=0.11585008166154043; common_negative=False.

Native minima by arm: +1.0578, +1.02563, +0.701244, +0.701169, +1.021, +0.70115

Common minima by arm: +1.03371, +1.0196, +1.03926, +1.021, +1.01609, +1.01644

Endpoint beta=2.0: recorded status=unresolved_numerical_margin; F=-0.005163225053161814; E=0.023170016744638033; tau=0.11585008372319017; common_negative=False.

Native minima by arm: +0.34693, +0.346951, -0.00516323, -0.0051507, +0.346956, -0.00514809

Common minima by arm: +0.34693, +0.346951, +0.346925, +0.346952, +0.346956, +0.346957

Preserved bracket decision: unresolved_numerical_margin. Physical Hessian executed: False. No strict transition coordinate is confirmed for this target.

Interpretation: An endpoint labeled robust-positive does not form a validated sign bracket without an opposite robust-negative endpoint. A negative native-grid minimum can coexist with a positive common-domain minimum. Examine the shared-radius requirement before treating any inner dip as a physical sign transition.

Reproduction and limitations: retain all six arms, original native cutoffs, common comparison radius, numerical tolerance and the archived hashes. Inspect B01-specific force decomposition only for B01; do not extrapolate its component attribution to this target without a matching decomposition calculation.

# Target B07: preserved six-arm sign-boundary record

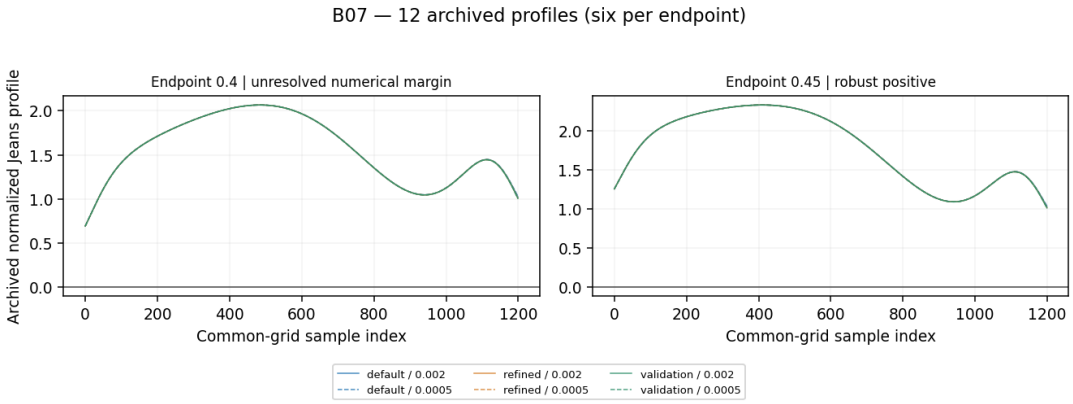



Figure: archived six-arm profiles. Vertical coordinate is the saved normalized Jeans profile; plotted lines must not be interpreted as observed stellar velocities. See the companion source archive for exact normalization.

Source JSON: UDCT_v7_13_B07_Six_Arm_Profile_Exploratory_20261021_012418_036.json

Endpoint beta=0.4: recorded status=unresolved_numerical_margin; F=-0.0473392896872525; E=0.02343293966031168; tau=0.1171646983015584; common_negative=False.

Native minima by arm: +0.692759, +0.692853, -0.0473393, -0.0473298, +0.692877, -0.0473276

Common minima by arm: +0.692759, +0.692853, +0.69269, +0.692824, +0.692877, +0.69287

Endpoint beta=0.45: recorded status=robust_positive; F=0.4583884832113066; E=0.023301451110646187; tau=0.11650725555323094; common_negative=False.

Native minima by arm: +1.05238, +1.02047, +0.458455, +0.458402, +1.01584, +0.458388

Common minima by arm: +1.02873, +1.01455, +1.03431, +1.01596, +1.01101, +1.01136

Preserved bracket decision: unresolved_numerical_margin. Physical Hessian executed: False. No strict transition coordinate is confirmed for this target.

Interpretation: An endpoint labeled robust-positive does not form a validated sign bracket without an opposite robust-negative endpoint. A negative native-grid minimum can coexist with a positive common-domain minimum. Examine the shared-radius requirement before treating any inner dip as a physical sign transition.

Reproduction and limitations: retain all six arms, original native cutoffs, common comparison radius, numerical tolerance and the archived hashes. Inspect B01-specific force decomposition only for B01; do not extrapolate its component attribution to this target without a matching decomposition calculation.

# Target B08: preserved six-arm sign-boundary record

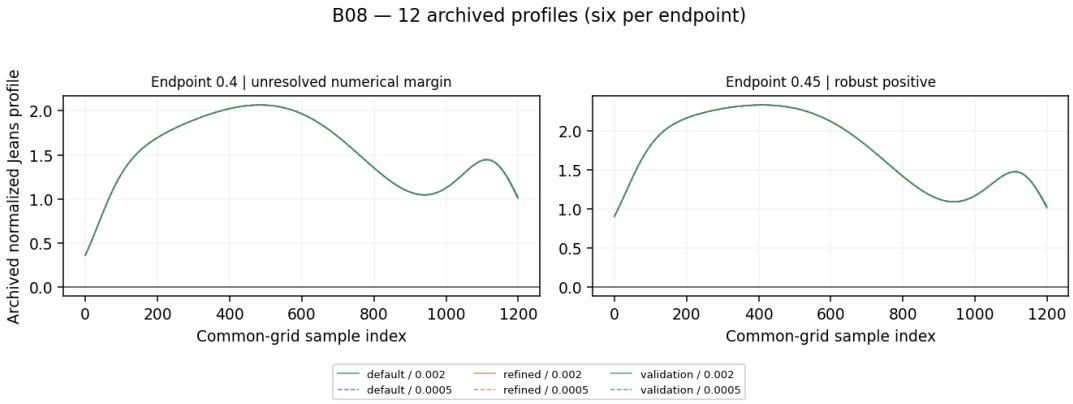



Figure: archived six-arm profiles. Vertical coordinate is the saved normalized Jeans profile; plotted lines must not be interpreted as observed stellar velocities. See the companion source archive for exact normalization.

Source JSON: UDCT_v7_13_B08_Six_Arm_Profile_Exploratory_20261021_012413_596.json

Endpoint beta=0.4: recorded status=unresolved_numerical_margin; F=-0.2332146101142309; E=0.02343293966031168; tau=0.1171646983015584; common_negative=False.

Native minima by arm: +0.36132, +0.361415, -0.233215, -0.233183, +0.361439, -0.233176

Common minima by arm: +0.36132, +0.361415, +0.361287, +0.361409, +0.361439, +0.361438

Endpoint beta=0.45: recorded status=robust_positive; F=0.2217400111790492; E=0.023301451110646187; tau=0.11650725555323094; common_negative=False.

Native minima by arm: +0.900992, +0.90103, +0.221771, +0.221746, +0.90104, +0.22174

Common minima by arm: +0.900992, +0.90103, +0.900976, +0.901026, +0.90104, +0.901039

Preserved bracket decision: unresolved_numerical_margin. Physical Hessian executed: False. No strict transition coordinate is confirmed for this target.

Interpretation: An endpoint labeled robust-positive does not form a validated sign bracket without an opposite robust-negative endpoint. A negative native-grid minimum can coexist with a positive common-domain minimum. Examine the shared-radius requirement before treating any inner dip as a physical sign transition.

Reproduction and limitations: retain all six arms, original native cutoffs, common comparison radius, numerical tolerance and the archived hashes. Inspect B01-specific force decomposition only for B01; do not extrapolate its component attribution to this target without a matching decomposition calculation.

# Target B09: preserved six-arm sign-boundary record

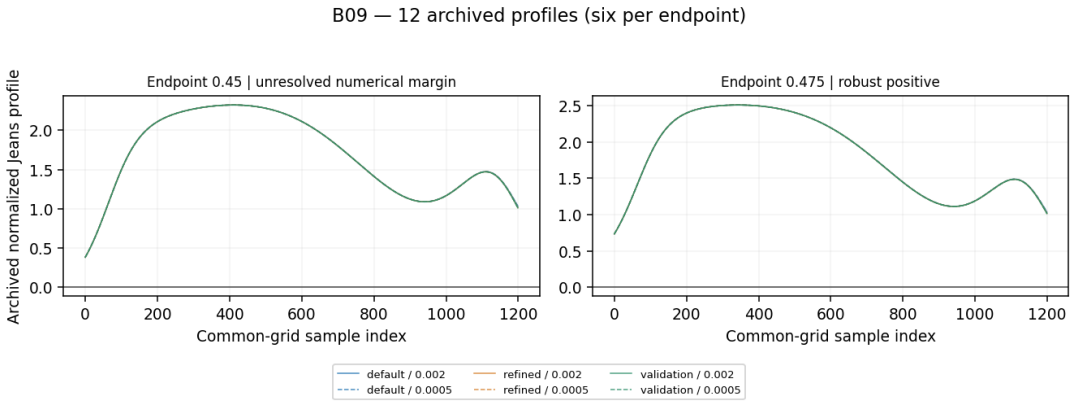



Figure: archived six-arm profiles. Vertical coordinate is the saved normalized Jeans profile; plotted lines must not be interpreted as observed stellar velocities. See the companion source archive for exact normalization.

Source JSON: UDCT_v7_13_B09_Six_Arm_Profile_Exploratory_20261021_012409_760.json

Endpoint beta=0.45: recorded status=unresolved_numerical_margin; F=-0.11918527818758426; E=0.02330145111064663; tau=0.11650725555323316; common_negative=False.

Native minima by arm: +0.381731, +0.381774, -0.119185, -0.119169, +0.381785, -0.119165

Common minima by arm: +0.381731, +0.381774, +0.381743, +0.38179, +0.381785, +0.381789

Endpoint beta=0.475: recorded status=robust_positive; F=0.20644945302517392; E=0.023235726692406367; tau=0.11617863346203183; common_negative=False.

Native minima by arm: +0.733047, +0.733054, +0.206479, +0.206456, +0.733056, +0.206449

Common minima by arm: +0.733047, +0.733054, +0.733071, +0.733072, +0.733056, +0.733061

Preserved bracket decision: unresolved_numerical_margin. Physical Hessian executed: False. No strict transition coordinate is confirmed for this target.

Interpretation: An endpoint labeled robust-positive does not form a validated sign bracket without an opposite robust-negative endpoint. A negative native-grid minimum can coexist with a positive common-domain minimum. Examine the shared-radius requirement before treating any inner dip as a physical sign transition.

Reproduction and limitations: retain all six arms, original native cutoffs, common comparison radius, numerical tolerance and the archived hashes. Inspect B01-specific force decomposition only for B01; do not extrapolate its component attribution to this target without a matching decomposition calculation.

# Target B10: preserved six-arm sign-boundary record

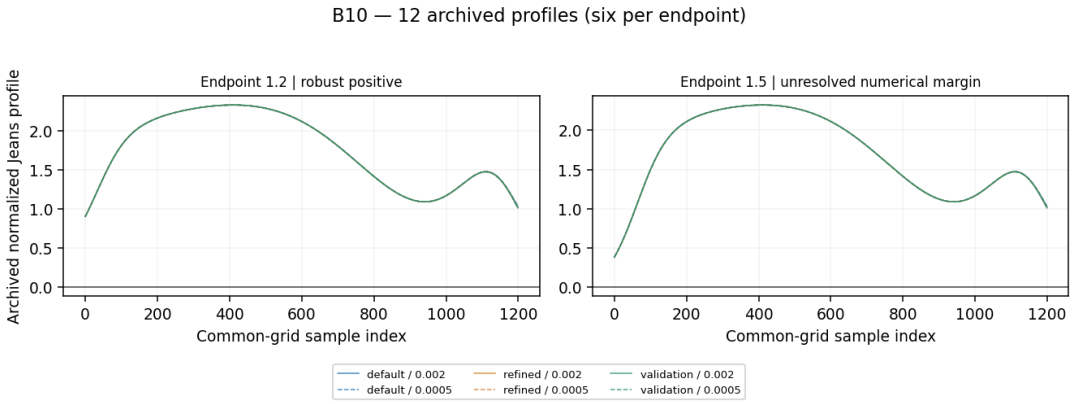



Figure: archived six-arm profiles. Vertical coordinate is the saved normalized Jeans profile; plotted lines must not be interpreted as observed stellar velocities. See the companion source archive for exact normalization.

Source JSON: UDCT_v7_13_B10_Six_Arm_Profile_Exploratory_20261021_012405_115.json

Endpoint beta=1.2: recorded status=robust_positive; F=0.2217400111790492; E=0.023301451110646187; tau=0.11650725555323094; common_negative=False.

Native minima by arm: +0.900992, +0.90103, +0.221771, +0.221746, +0.90104, +0.22174

Common minima by arm: +0.900992, +0.90103, +0.900976, +0.901026, +0.90104, +0.901039

Endpoint beta=1.5: recorded status=unresolved_numerical_margin; F=-0.11918527818758426; E=0.02330145111064663; tau=0.11650725555323316; common_negative=False.

Native minima by arm: +0.381731, +0.381774, -0.119185, -0.119169, +0.381785, -0.119165

Common minima by arm: +0.381731, +0.381774, +0.381743, +0.38179, +0.381785, +0.381789

Preserved bracket decision: unresolved_numerical_margin. Physical Hessian executed: False. No strict transition coordinate is confirmed for this target.

Interpretation: An endpoint labeled robust-positive does not form a validated sign bracket without an opposite robust-negative endpoint. A negative native-grid minimum can coexist with a positive common-domain minimum. Examine the shared-radius requirement before treating any inner dip as a physical sign transition.

Reproduction and limitations: retain all six arms, original native cutoffs, common comparison radius, numerical tolerance and the archived hashes. Inspect B01-specific force decomposition only for B01; do not extrapolate its component attribution to this target without a matching decomposition calculation.

# Target B11: preserved six-arm sign-boundary record

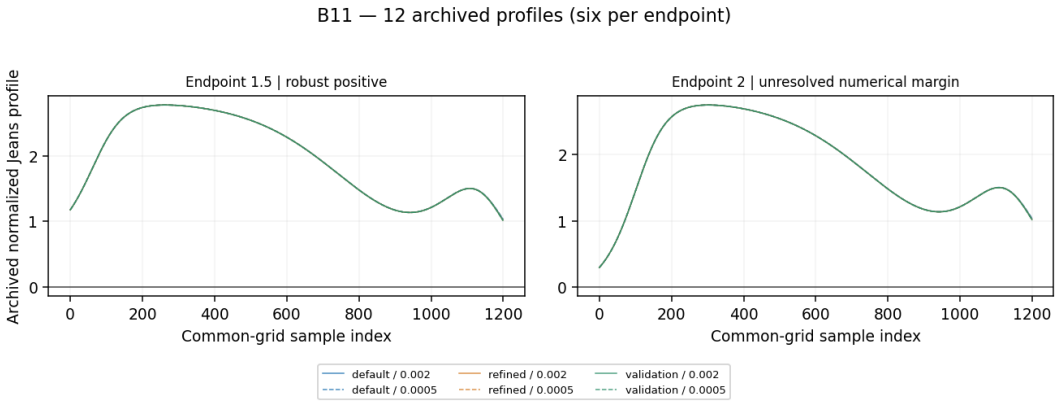



Figure: archived six-arm profiles. Vertical coordinate is the saved normalized Jeans profile; plotted lines must not be interpreted as observed stellar velocities. See the companion source archive for exact normalization.

Source JSON: UDCT_v7_13_B11_Six_Arm_Profile_Exploratory_20261021_012358_483.json

Endpoint beta=1.5: recorded status=robust_positive; F=0.669705601514931; E=0.023170015802179922; tau=0.11585007901089961; common_negative=False.

Native minima by arm: +1.0578, +1.02563, +0.669806, +0.669726, +1.021, +0.669706

Common minima by arm: +1.03371, +1.0196, +1.03926, +1.021, +1.01609, +1.01644

Endpoint beta=2.0: recorded status=unresolved_numerical_margin; F=-0.054078276043354453; E=0.02317001580217859; tau=0.11585007901089295; common_negative=False.

Native minima by arm: +0.295728, +0.295739, -0.0540783, -0.0540691, +0.295741, -0.0540673

Common minima by arm: +0.295728, +0.295739, +0.29576, +0.295762, +0.295741, +0.295748

Preserved bracket decision: unresolved_numerical_margin. Physical Hessian executed: False. No strict transition coordinate is confirmed for this target.

Interpretation: An endpoint labeled robust-positive does not form a validated sign bracket without an opposite robust-negative endpoint. A negative native-grid minimum can coexist with a positive common-domain minimum. Examine the shared-radius requirement before treating any inner dip as a physical sign transition.

Reproduction and limitations: retain all six arms, original native cutoffs, common comparison radius, numerical tolerance and the archived hashes. Inspect B01-specific force decomposition only for B01; do not extrapolate its component attribution to this target without a matching decomposition calculation.

# Target B12: preserved six-arm sign-boundary record

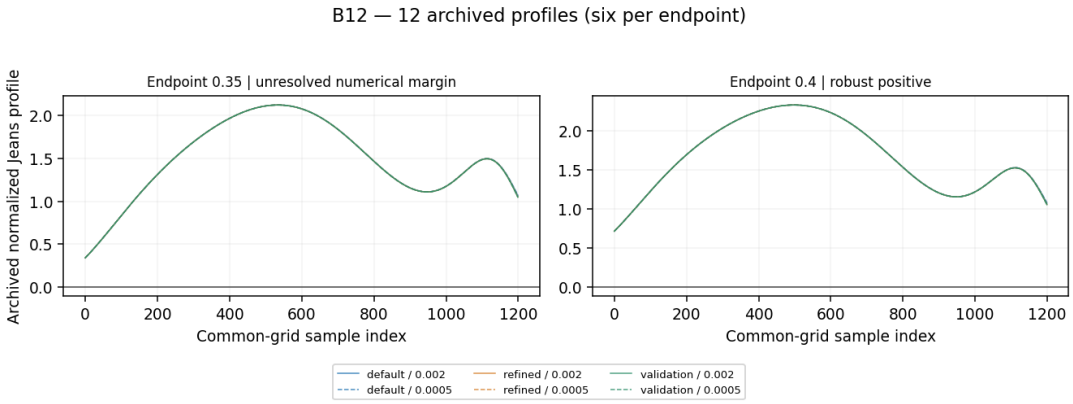



Figure: archived six-arm profiles. Vertical coordinate is the saved normalized Jeans profile; plotted lines must not be interpreted as observed stellar velocities. See the companion source archive for exact normalization.

Source JSON: UDCT_v7_13_B12_Six_Arm_Profile_Exploratory_20261021_011759_654.json

Endpoint beta=0.35: recorded status=unresolved_numerical_margin; F=-0.10384987833308063; E=0.024670061263517784; tau=0.12335030631758892; common_negative=False.

Native minima by arm: +0.341076, +0.341116, -0.10385, -0.103822, +0.341126, -0.103816

Common minima by arm: +0.341076, +0.341116, +0.341052, +0.341106, +0.341126, +0.341124

Endpoint beta=0.4: recorded status=robust_positive; F=0.18176516848138335; E=0.02453242532931421; tau=0.12266212664657106; common_negative=False.

Native minima by arm: +0.71388, +0.713882, +0.181775, +0.181767, +0.713882, +0.181765

Common minima by arm: +0.71388, +0.713882, +0.713867, +0.713874, +0.713882, +0.71388

Preserved bracket decision: unresolved_numerical_margin. Physical Hessian executed: False. No strict transition coordinate is confirmed for this target.

Interpretation: An endpoint labeled robust-positive does not form a validated sign bracket without an opposite robust-negative endpoint. A negative native-grid minimum can coexist with a positive common-domain minimum. Examine the shared-radius requirement before treating any inner dip as a physical sign transition.

Reproduction and limitations: retain all six arms, original native cutoffs, common comparison radius, numerical tolerance and the archived hashes. Inspect B01-specific force decomposition only for B01; do not extrapolate its component attribution to this target without a matching decomposition calculation.

# Target B13: preserved six-arm sign-boundary record

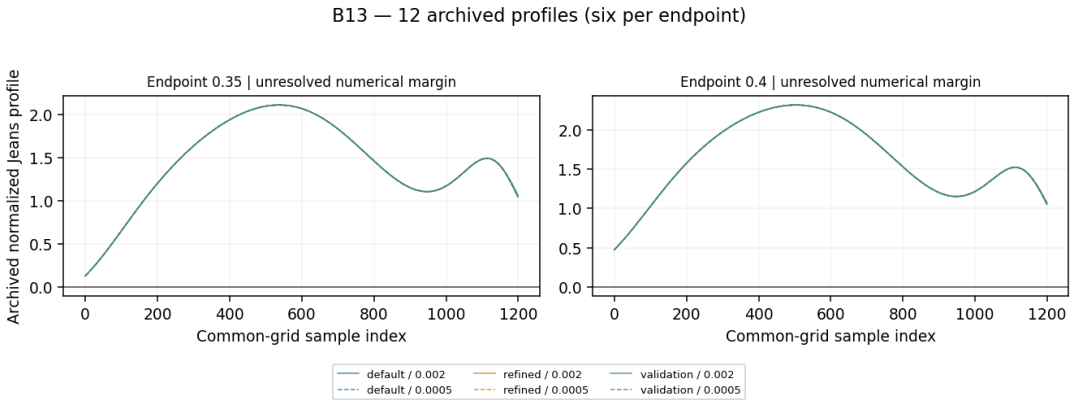



Figure: archived six-arm profiles. Vertical coordinate is the saved normalized Jeans profile; plotted lines must not be interpreted as observed stellar velocities. See the companion source archive for exact normalization.

Source JSON: UDCT_v7_13_B13_Six_Arm_Profile_Exploratory_20261021_012349_829.json

Endpoint beta=0.35: recorded status=unresolved_numerical_margin; F=-0.24142505176116819; E=0.024670061367916274; tau=0.12335030683958137; common_negative=False.

Native minima by arm: +0.130197, +0.130258, -0.241425, -0.241382, +0.130273, -0.241371

Common minima by arm: +0.130197, +0.130258, +0.13017, +0.13025, +0.130273, +0.130271

Endpoint beta=0.4: recorded status=unresolved_numerical_margin; F=0.002628403096054326; E=0.024532425434311556; tau=0.12266212717155778; common_negative=False.

Native minima by arm: +0.475852, +0.475878, +0.0026284, +0.00264129, +0.475884, +0.00264436

Common minima by arm: +0.475852, +0.475878, +0.475835, +0.475871, +0.475884, +0.475882

Preserved bracket decision: unresolved_numerical_margin. Physical Hessian executed: False. No strict transition coordinate is confirmed for this target.

Interpretation: An endpoint labeled robust-positive does not form a validated sign bracket without an opposite robust-negative endpoint. A negative native-grid minimum can coexist with a positive common-domain minimum. Examine the shared-radius requirement before treating any inner dip as a physical sign transition.

Reproduction and limitations: retain all six arms, original native cutoffs, common comparison radius, numerical tolerance and the archived hashes. Inspect B01-specific force decomposition only for B01; do not extrapolate its component attribution to this target without a matching decomposition calculation.

# Target B14: preserved six-arm sign-boundary record

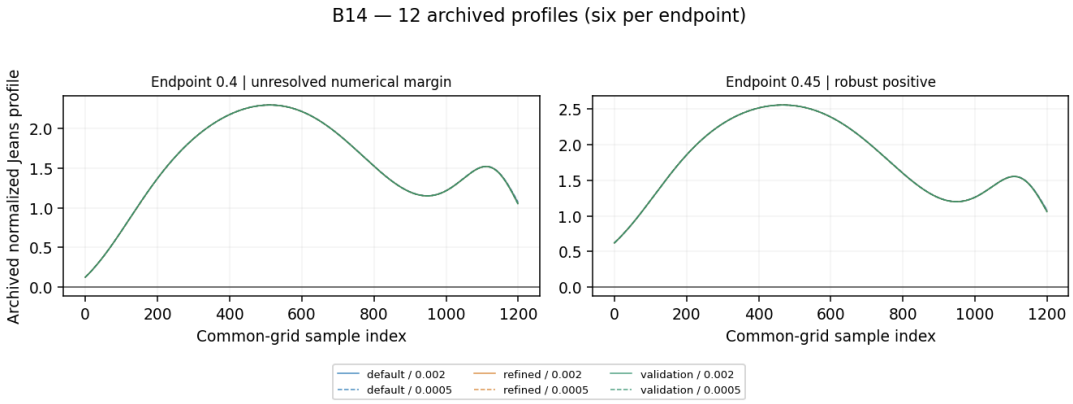



Figure: archived six-arm profiles. Vertical coordinate is the saved normalized Jeans profile; plotted lines must not be interpreted as observed stellar velocities. See the companion source archive for exact normalization.

Source JSON: UDCT_v7_13_B14_Six_Arm_Profile_Exploratory_20261021_012339_663.json

Endpoint beta=0.4: recorded status=unresolved_numerical_margin; F=-0.25052056811595563; E=0.024532425627597165; tau=0.12266212813798583; common_negative=False.

Native minima by arm: +0.125183, +0.12524, -0.250521, -0.250477, +0.125254, -0.250467

Common minima by arm: +0.125183, +0.12524, +0.125161, +0.125235, +0.125254, +0.125253

Endpoint beta=0.45: recorded status=robust_positive; F=0.1597301142625548; E=0.02439484364549349; tau=0.12197421822746746; common_negative=False.

Native minima by arm: +0.6203, +0.620307, +0.159739, +0.159732, +0.620309, +0.15973

Common minima by arm: +0.6203, +0.620307, +0.620294, +0.620305, +0.620309, +0.620309

Preserved bracket decision: unresolved_numerical_margin. Physical Hessian executed: False. No strict transition coordinate is confirmed for this target.

Interpretation: An endpoint labeled robust-positive does not form a validated sign bracket without an opposite robust-negative endpoint. A negative native-grid minimum can coexist with a positive common-domain minimum. Examine the shared-radius requirement before treating any inner dip as a physical sign transition.

Reproduction and limitations: retain all six arms, original native cutoffs, common comparison radius, numerical tolerance and the archived hashes. Inspect B01-specific force decomposition only for B01; do not extrapolate its component attribution to this target without a matching decomposition calculation.

# Target B15: preserved six-arm sign-boundary record

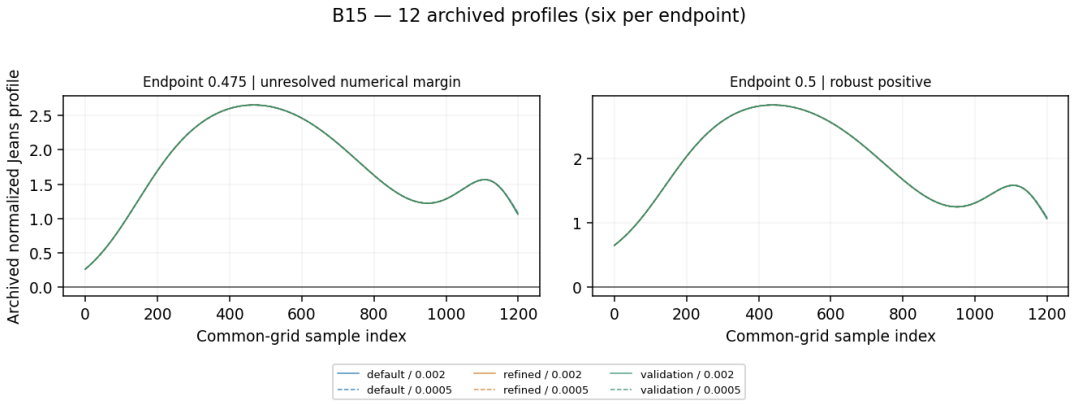



Figure: archived six-arm profiles. Vertical coordinate is the saved normalized Jeans profile; plotted lines must not be interpreted as observed stellar velocities. See the companion source archive for exact normalization.

Source JSON: UDCT_v7_13_B15_Six_Arm_Profile_Exploratory_20261021_012335_481.json

Endpoint beta=0.475: recorded status=unresolved_numerical_margin; F=-0.12063240180997307; E=0.024326073867530162; tau=0.12163036933765081; common_negative=False.

Native minima by arm: +0.263042, +0.263077, -0.120632, -0.120605, +0.263085, -0.120598

Common minima by arm: +0.263042, +0.263077, +0.263032, +0.263076, +0.263085, +0.263085

Endpoint beta=0.5: recorded status=robust_positive; F=0.2634101876061834; E=0.024257317834583292; tau=0.12128658917291646; common_negative=False.

Native minima by arm: +0.649596, +0.649593, +0.263434, +0.263415, +0.649592, +0.26341

Common minima by arm: +0.649596, +0.649593, +0.6496, +0.649595, +0.649592, +0.649593

Preserved bracket decision: unresolved_numerical_margin. Physical Hessian executed: False. No strict transition coordinate is confirmed for this target.

Interpretation: An endpoint labeled robust-positive does not form a validated sign bracket without an opposite robust-negative endpoint. A negative native-grid minimum can coexist with a positive common-domain minimum. Examine the shared-radius requirement before treating any inner dip as a physical sign transition.

Reproduction and limitations: retain all six arms, original native cutoffs, common comparison radius, numerical tolerance and the archived hashes. Inspect B01-specific force decomposition only for B01; do not extrapolate its component attribution to this target without a matching decomposition calculation.

# Target B16: preserved six-arm sign-boundary record

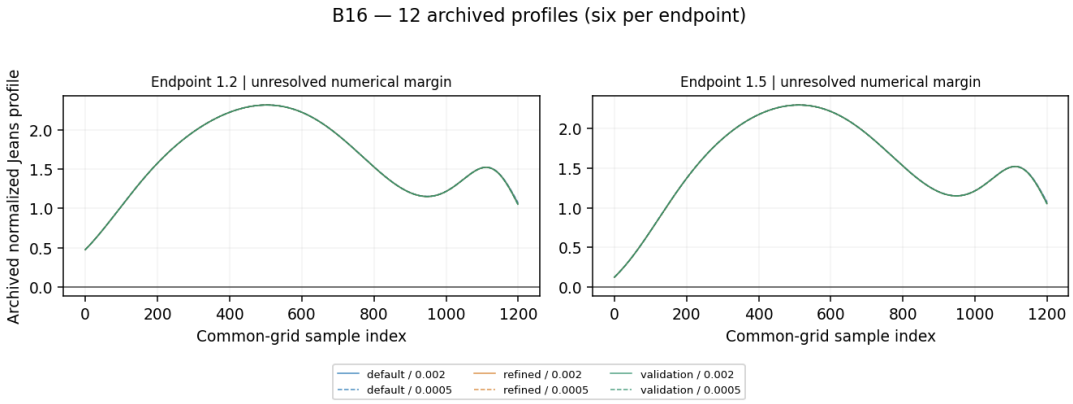



Figure: archived six-arm profiles. Vertical coordinate is the saved normalized Jeans profile; plotted lines must not be interpreted as observed stellar velocities. See the companion source archive for exact normalization.

Source JSON: UDCT_v7_13_B16_Six_Arm_Profile_Exploratory_20261021_012331_628.json

Endpoint beta=1.2: recorded status=unresolved_numerical_margin; F=0.002628403096054326; E=0.024532425434311556; tau=0.12266212717155778; common_negative=False.

Native minima by arm: +0.475852, +0.475878, +0.0026284, +0.00264129, +0.475884, +0.00264436

Common minima by arm: +0.475852, +0.475878, +0.475835, +0.475871, +0.475884, +0.475882

Endpoint beta=1.5: recorded status=unresolved_numerical_margin; F=-0.25052056811595563; E=0.024532425627597165; tau=0.12266212813798583; common_negative=False.

Native minima by arm: +0.125183, +0.12524, -0.250521, -0.250477, +0.125254, -0.250467

Common minima by arm: +0.125183, +0.12524, +0.125161, +0.125235, +0.125254, +0.125253

Preserved bracket decision: unresolved_numerical_margin. Physical Hessian executed: False. No strict transition coordinate is confirmed for this target.

Interpretation: An endpoint labeled robust-positive does not form a validated sign bracket without an opposite robust-negative endpoint. A negative native-grid minimum can coexist with a positive common-domain minimum. Examine the shared-radius requirement before treating any inner dip as a physical sign transition.

Reproduction and limitations: retain all six arms, original native cutoffs, common comparison radius, numerical tolerance and the archived hashes. Inspect B01-specific force decomposition only for B01; do not extrapolate its component attribution to this target without a matching decomposition calculation.

# Target B17: preserved six-arm sign-boundary record

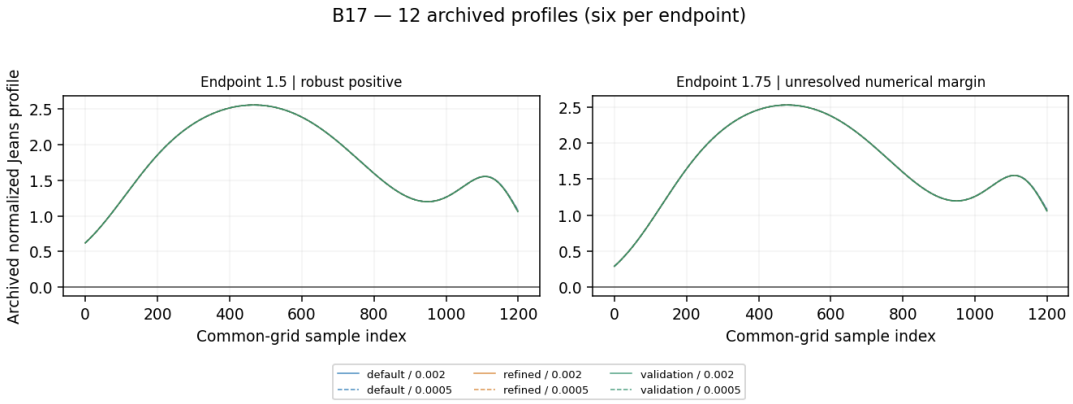



Figure: archived six-arm profiles. Vertical coordinate is the saved normalized Jeans profile; plotted lines must not be interpreted as observed stellar velocities. See the companion source archive for exact normalization.

Source JSON: UDCT_v7_13_B17_Six_Arm_Profile_Exploratory_20261021_012327_430.json

Endpoint beta=1.5: recorded status=robust_positive; F=0.1597301142625548; E=0.02439484364549349; tau=0.12197421822746746; common_negative=False.

Native minima by arm: +0.6203, +0.620307, +0.159739, +0.159732, +0.620309, +0.15973

Common minima by arm: +0.6203, +0.620307, +0.620294, +0.620305, +0.620309, +0.620309

Endpoint beta=1.75: recorded status=unresolved_numerical_margin; F=-0.11235709231890448; E=0.024394843840484848; tau=0.12197421920242424; common_negative=False.

Native minima by arm: +0.289717, +0.289752, -0.112357, -0.112331, +0.289761, -0.112325

Common minima by arm: +0.289717, +0.289752, +0.289704, +0.28975, +0.289761, +0.289761

Preserved bracket decision: unresolved_numerical_margin. Physical Hessian executed: False. No strict transition coordinate is confirmed for this target.

Interpretation: An endpoint labeled robust-positive does not form a validated sign bracket without an opposite robust-negative endpoint. A negative native-grid minimum can coexist with a positive common-domain minimum. Examine the shared-radius requirement before treating any inner dip as a physical sign transition.

Reproduction and limitations: retain all six arms, original native cutoffs, common comparison radius, numerical tolerance and the archived hashes. Inspect B01-specific force decomposition only for B01; do not extrapolate its component attribution to this target without a matching decomposition calculation.

# Target B18: preserved six-arm sign-boundary record

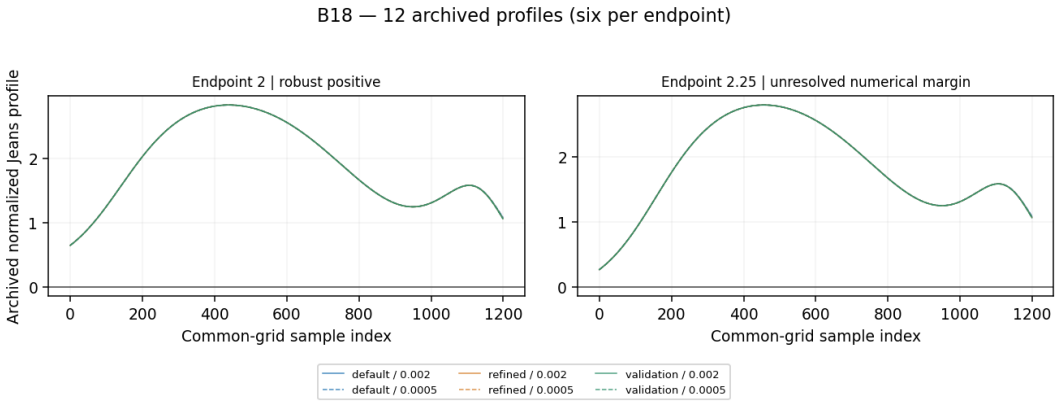



Figure: archived six-arm profiles. Vertical coordinate is the saved normalized Jeans profile; plotted lines must not be interpreted as observed stellar velocities. See the companion source archive for exact normalization.

No target-specific exploratory JSON found. Refer to the common NPZ and inventory; absence is a provenance limitation, not evidence of a missing target profile.

Preserved bracket decision: unresolved_numerical_margin. Physical Hessian executed: False. No strict transition coordinate is confirmed for this target.

Interpretation: An endpoint labeled robust-positive does not form a validated sign bracket without an opposite robust-negative endpoint. A negative native-grid minimum can coexist with a positive common-domain minimum. Examine the shared-radius requirement before treating any inner dip as a physical sign transition.

Reproduction and limitations: retain all six arms, original native cutoffs, common comparison radius, numerical tolerance and the archived hashes. Inspect B01-specific force decomposition only for B01; do not extrapolate its component attribution to this target without a matching decomposition calculation.

# Target B19: preserved six-arm sign-boundary record

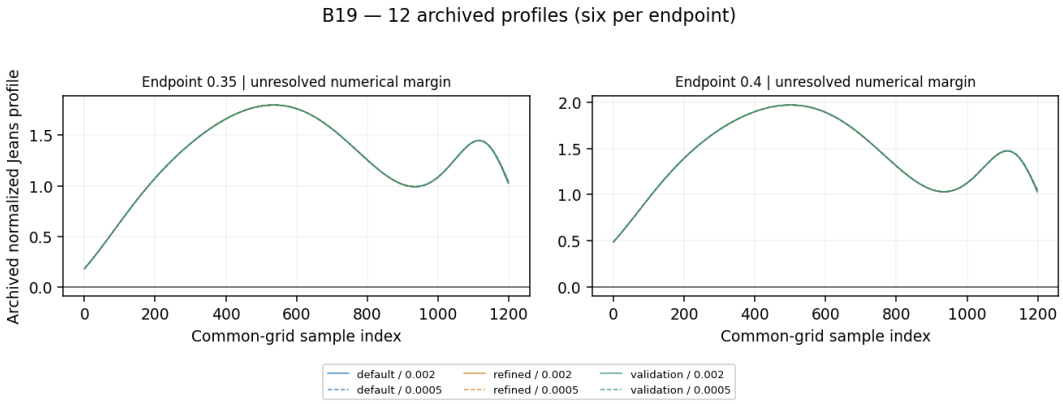



Figure: archived six-arm profiles. Vertical coordinate is the saved normalized Jeans profile; plotted lines must not be interpreted as observed stellar velocities. See the companion source archive for exact normalization.

Source JSON: UDCT_v7_13_B19_Six_Arm_Profile_Exploratory_20261021_012315_327.json

Endpoint beta=0.35: recorded status=unresolved_numerical_margin; F=-0.20180829293742844; E=0.0241172585976972; tau=0.120586292988486; common_negative=False.

Native minima by arm: +0.182872, +0.182926, -0.201808, -0.20177, +0.182939, -0.20176

Common minima by arm: +0.182872, +0.182926, +0.182843, +0.182916, +0.182939, +0.182937

Endpoint beta=0.4: recorded status=unresolved_numerical_margin; F=0.018325326585618824; E=0.02398258325495073; tau=0.11991291627475364; common_negative=False.

Native minima by arm: +0.488588, +0.488611, +0.0183253, +0.0183361, +0.488617, +0.0183387

Common minima by arm: +0.488588, +0.488611, +0.488569, +0.488602, +0.488617, +0.488615

Preserved bracket decision: unresolved_numerical_margin. Physical Hessian executed: False. No strict transition coordinate is confirmed for this target.

Interpretation: An endpoint labeled robust-positive does not form a validated sign bracket without an opposite robust-negative endpoint. A negative native-grid minimum can coexist with a positive common-domain minimum. Examine the shared-radius requirement before treating any inner dip as a physical sign transition.

Reproduction and limitations: retain all six arms, original native cutoffs, common comparison radius, numerical tolerance and the archived hashes. Inspect B01-specific force decomposition only for B01; do not extrapolate its component attribution to this target without a matching decomposition calculation.

# Target B20: preserved six-arm sign-boundary record

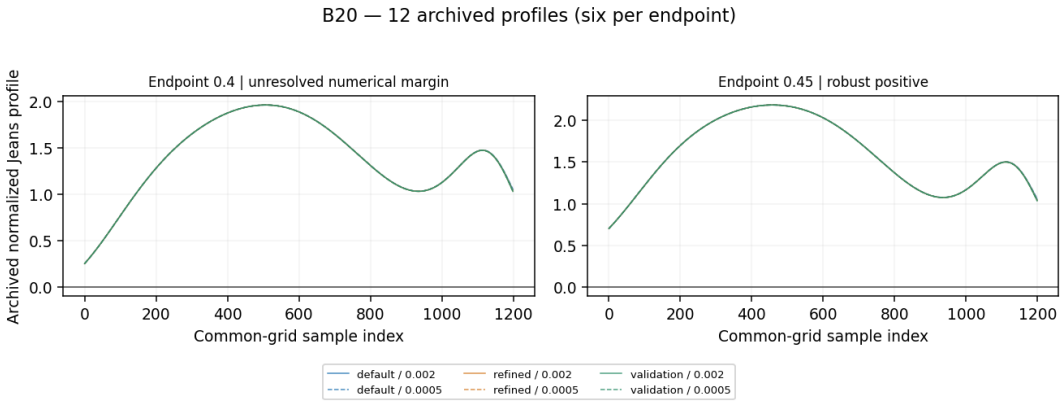



Figure: archived six-arm profiles. Vertical coordinate is the saved normalized Jeans profile; plotted lines must not be interpreted as observed stellar velocities. See the companion source archive for exact normalization.

Source JSON: UDCT_v7_13_B20_Six_Arm_Profile_Exploratory_20261021_012308_699.json

Endpoint beta=0.4: recorded status=unresolved_numerical_margin; F=-0.15644075535465266; E=0.02398258335901371; tau=0.11991291679506855; common_negative=False.

Native minima by arm: +0.25396, +0.254006, -0.156441, -0.15641, +0.254018, -0.156402

Common minima by arm: +0.25396, +0.254006, +0.253937, +0.253999, +0.254018, +0.254016

Endpoint beta=0.45: recorded status=robust_positive; F=0.22841709585952097; E=0.0238479607409674; tau=0.119239803704837; common_negative=False.

Native minima by arm: +0.702713, +0.702714, +0.228438, +0.228421, +0.702714, +0.228417

Common minima by arm: +0.702713, +0.702714, +0.702705, +0.702709, +0.702714, +0.702713

Preserved bracket decision: unresolved_numerical_margin. Physical Hessian executed: False. No strict transition coordinate is confirmed for this target.

Interpretation: An endpoint labeled robust-positive does not form a validated sign bracket without an opposite robust-negative endpoint. A negative native-grid minimum can coexist with a positive common-domain minimum. Examine the shared-radius requirement before treating any inner dip as a physical sign transition.

Reproduction and limitations: retain all six arms, original native cutoffs, common comparison radius, numerical tolerance and the archived hashes. Inspect B01-specific force decomposition only for B01; do not extrapolate its component attribution to this target without a matching decomposition calculation.

# Target B21: preserved six-arm sign-boundary record

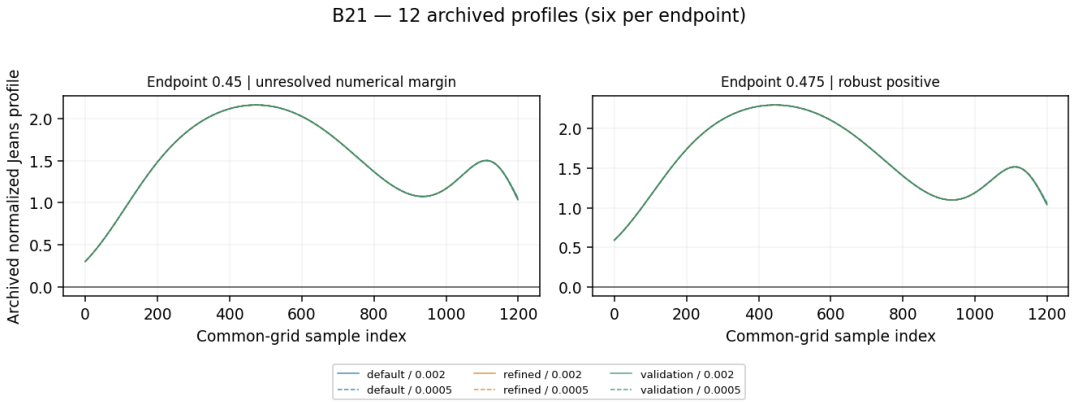



Figure: archived six-arm profiles. Vertical coordinate is the saved normalized Jeans profile; plotted lines must not be interpreted as observed stellar velocities. See the companion source archive for exact normalization.

Source JSON: UDCT_v7_13_B21_Six_Arm_Profile_Exploratory_20261021_012302_765.json

Endpoint beta=0.45: recorded status=unresolved_numerical_margin; F=-0.10201579413937799; E=0.023847960933631507; tau=0.11923980466815753; common_negative=False.

Native minima by arm: +0.304504, +0.30454, -0.102016, -0.101992, +0.304549, -0.101987

Common minima by arm: +0.304504, +0.30454, +0.304488, +0.304536, +0.304549, +0.304548

Endpoint beta=0.475: recorded status=robust_positive; F=0.1655493421340387; E=0.02378066994508843; tau=0.11890334972544214; common_negative=False.

Native minima by arm: +0.591093, +0.591101, +0.165561, +0.165552, +0.591103, +0.165549

Common minima by arm: +0.591093, +0.591101, +0.591088, +0.591099, +0.591103, +0.591103

Preserved bracket decision: unresolved_numerical_margin. Physical Hessian executed: False. No strict transition coordinate is confirmed for this target.

Interpretation: An endpoint labeled robust-positive does not form a validated sign bracket without an opposite robust-negative endpoint. A negative native-grid minimum can coexist with a positive common-domain minimum. Examine the shared-radius requirement before treating any inner dip as a physical sign transition.

Reproduction and limitations: retain all six arms, original native cutoffs, common comparison radius, numerical tolerance and the archived hashes. Inspect B01-specific force decomposition only for B01; do not extrapolate its component attribution to this target without a matching decomposition calculation.

# Target B22: preserved six-arm sign-boundary record

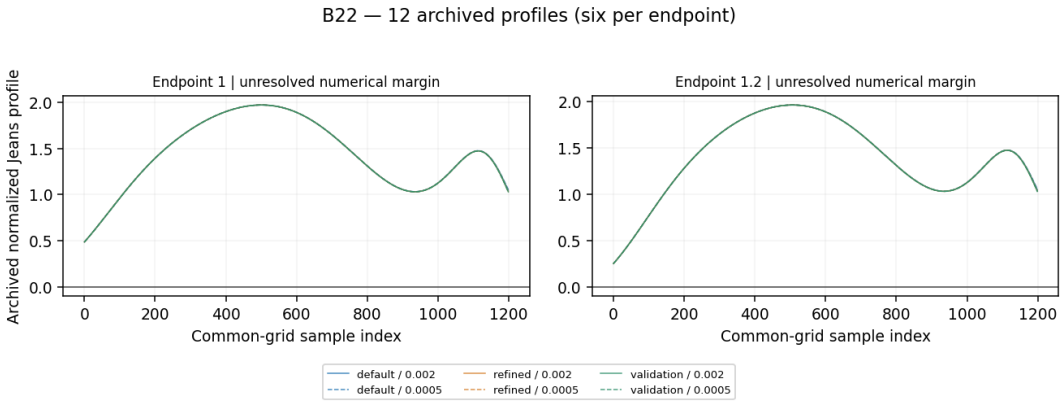



Figure: archived six-arm profiles. Vertical coordinate is the saved normalized Jeans profile; plotted lines must not be interpreted as observed stellar velocities. See the companion source archive for exact normalization.

Source JSON: UDCT_v7_13_B22_Six_Arm_Profile_Exploratory_20261021_012256_617.json

Endpoint beta=1.0: recorded status=unresolved_numerical_margin; F=0.018325326585618824; E=0.02398258325495073; tau=0.11991291627475364; common_negative=False.

Native minima by arm: +0.488588, +0.488611, +0.0183253, +0.0183361, +0.488617, +0.0183387

Common minima by arm: +0.488588, +0.488611, +0.488569, +0.488602, +0.488617, +0.488615

Endpoint beta=1.2: recorded status=unresolved_numerical_margin; F=-0.15644075535465266; E=0.02398258335901371; tau=0.11991291679506855; common_negative=False.

Native minima by arm: +0.25396, +0.254006, -0.156441, -0.15641, +0.254018, -0.156402

Common minima by arm: +0.25396, +0.254006, +0.253937, +0.253999, +0.254018, +0.254016

Preserved bracket decision: unresolved_numerical_margin. Physical Hessian executed: False. No strict transition coordinate is confirmed for this target.

Interpretation: An endpoint labeled robust-positive does not form a validated sign bracket without an opposite robust-negative endpoint. A negative native-grid minimum can coexist with a positive common-domain minimum. Examine the shared-radius requirement before treating any inner dip as a physical sign transition.

Reproduction and limitations: retain all six arms, original native cutoffs, common comparison radius, numerical tolerance and the archived hashes. Inspect B01-specific force decomposition only for B01; do not extrapolate its component attribution to this target without a matching decomposition calculation.

# Target B23: preserved six-arm sign-boundary record

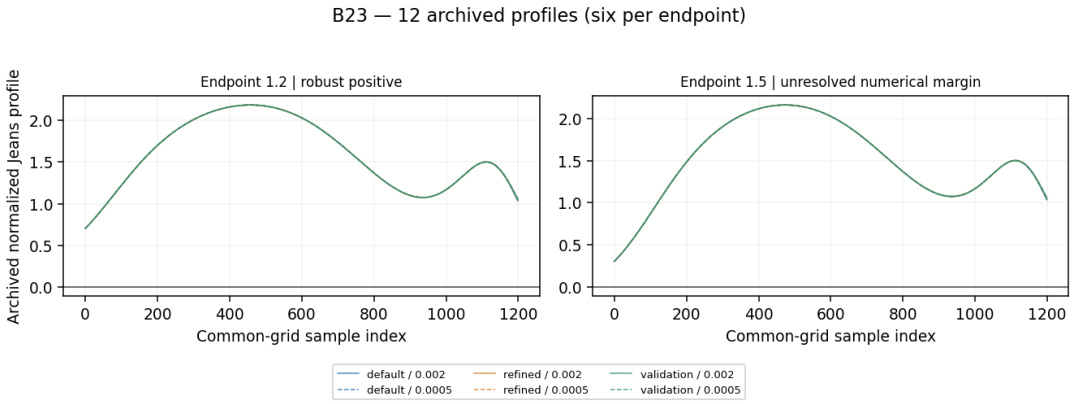



Figure: archived six-arm profiles. Vertical coordinate is the saved normalized Jeans profile; plotted lines must not be interpreted as observed stellar velocities. See the companion source archive for exact normalization.

Source JSON: UDCT_v7_13_B23_Six_Arm_Profile_Exploratory_20261021_012251_935.json

Endpoint beta=1.2: recorded status=robust_positive; F=0.22841709585952097; E=0.0238479607409674; tau=0.119239803704837; common_negative=False.

Native minima by arm: +0.702713, +0.702714, +0.228438, +0.228421, +0.702714, +0.228417

Common minima by arm: +0.702713, +0.702714, +0.702705, +0.702709, +0.702714, +0.702713

Endpoint beta=1.5: recorded status=unresolved_numerical_margin; F=-0.10201579413937799; E=0.023847960933631507; tau=0.11923980466815753; common_negative=False.

Native minima by arm: +0.304504, +0.30454, -0.102016, -0.101992, +0.304549, -0.101987

Common minima by arm: +0.304504, +0.30454, +0.304488, +0.304536, +0.304549, +0.304548

Preserved bracket decision: unresolved_numerical_margin. Physical Hessian executed: False. No strict transition coordinate is confirmed for this target.

Interpretation: An endpoint labeled robust-positive does not form a validated sign bracket without an opposite robust-negative endpoint. A negative native-grid minimum can coexist with a positive common-domain minimum. Examine the shared-radius requirement before treating any inner dip as a physical sign transition.

Reproduction and limitations: retain all six arms, original native cutoffs, common comparison radius, numerical tolerance and the archived hashes. Inspect B01-specific force decomposition only for B01; do not extrapolate its component attribution to this target without a matching decomposition calculation.

# Target B24: preserved six-arm sign-boundary record

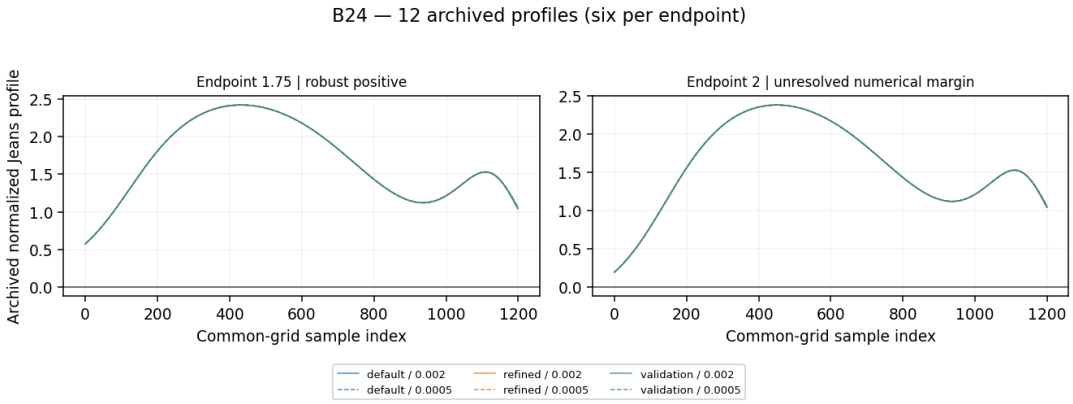



Figure: archived six-arm profiles. Vertical coordinate is the saved normalized Jeans profile; plotted lines must not be interpreted as observed stellar velocities. See the companion source archive for exact normalization.

Source JSON: UDCT_v7_13_B24_Six_Arm_Profile_Exploratory_20261021_012247_408.json

Endpoint beta=1.75: recorded status=robust_positive; F=0.18763224078453997; E=0.023713392994738403; tau=0.11856696497369201; common_negative=False.

Native minima by arm: +0.574759, +0.574765, +0.187647, +0.187636, +0.574766, +0.187632

Common minima by arm: +0.574759, +0.574765, +0.574758, +0.574765, +0.574766, +0.574766

Endpoint beta=2.0: recorded status=unresolved_numerical_margin; F=-0.1710332280963257; E=0.023713393219015; tau=0.118566966095075; common_negative=False.

Native minima by arm: +0.195413, +0.195452, -0.171033, -0.171, +0.195461, -0.170993

Common minima by arm: +0.195413, +0.195452, +0.195402, +0.19545, +0.195461, +0.195461

Preserved bracket decision: unresolved_numerical_margin. Physical Hessian executed: False. No strict transition coordinate is confirmed for this target.

Interpretation: An endpoint labeled robust-positive does not form a validated sign bracket without an opposite robust-negative endpoint. A negative native-grid minimum can coexist with a positive common-domain minimum. Examine the shared-radius requirement before treating any inner dip as a physical sign transition.

Reproduction and limitations: retain all six arms, original native cutoffs, common comparison radius, numerical tolerance and the archived hashes. Inspect B01-specific force decomposition only for B01; do not extrapolate its component attribution to this target without a matching decomposition calculation.

# Part III — Annotated Python source atlas

Each of the 77 archived Python programs receives its own page. The purpose is to preserve exact code entry points, imports, executable blocks, documented intent and algorithmic audit questions. Short literal source excerpts are not substitutes for the full ZIP; the original script remains the authoritative executable artifact.

# Source 01/77 — UDCT_v7_13_3D_Constitutive_Poisson_Coupling_Exploratory

Full archive filename: UDCT_v7_13_3D_Constitutive_Poisson_Coupling_Exploratory_20261021_012132_569.py

Source length: 70 lines; bytes: 3965; SHA-256: 9117e6f13b30b97180e1d93797d797425ff1435ec9e310373c05ce45b1eb28bd.

Functions: solve(rhs,h):); lap(u,h):); run(n,law,ell):).

Classes: none. Imports: import sys, json, hashlib, importlib.util; from pathlib import Path; import numpy as np; from scipy.fft import dstn,idstn.

Opening source documentation: Manufactured Newtonian field -> frozen constitutive source -> 3D Dirichlet solve. EXPLORATORY numerical integration test only; NOT a galaxy field or 24-bracket validation.

Static implementation signals (presence in source, not proof of execution): linear algebra, JSON output, internal assertion, hash verification, solver path.

Source excerpt (first 23 physical lines; whitespace and line order retained):

```
 1 #!/usr/bin/env python3
 2 """Manufactured Newtonian field -> frozen constitutive source -> 3D Dirichlet solve.
 3 EXPLORATORY numerical integration test only; NOT a galaxy field or 24-bracket validation.
 4 """
 5 import sys, json, hashlib, importlib.util
 6 from pathlib import Path
 7 import numpy as np
 8 from scipy.fft import dstn,idstn
 9 R=Path('/mnt/data/v713_frozen_source_recovered')
10 P=R/'UDCT_v7_10_Postregistration_run.py'
11 spec=importlib.util.spec_from_file_location('udct_frozen_coupling',P)
12 m=importlib.util.module_from_spec(spec);sys.modules[spec.name]=m;spec.loader.exec_module(m)
13 # Fixed manufactured non-isotropic Newtonian potential psi = g0.x + 1/2 x.H.x
14 H=np.diag([1.1e-30,1.6e-30,2.3e-30]);g0=np.array([1.4e-11,-1.e-11,.7e-11]);L=1.2*m.PC
15
16 def solve(rhs,h):
17  n=rhs.shape[0]; k=np.arange(1,n+1)
18  ev=2*(np.cos(np.pi*k/(n+1))-1)/h**2
19  return idstn(dstn(rhs,type=1,norm='ortho')/(ev[:,None,None]+ev[None,:,None]+ev[None,None,:]),type=1
20
21 def lap(u,h):
22  p=np.pad(u,1)
23  return (p[2:,1:-1,1:-1]+p[:-2,1:-1,1:-1]+p[1:-1,2:,1:-1]+p[1:-1,:-2,1:-1]+p[1:-1,1:-1,2:]+p[1:-1,1:
```

Audit interpretation: Identify the exact inputs, units, normalization and frozen constants in the full script. Determine whether this file computes a new physical profile, reads prior profiles, or only checks a manufactured numerical identity. Inspect the handling of exceptions, finite values, boundaries and convergence; a successful process exit alone is not a scientific validation.

Reproduction note: compare any outputs with the companion JSON and/or NPZ by exact archive name and digest. The filename timestamp is metadata, not proof of run chronology. No independent execution of this script is claimed in this companion.

# Source 02/77 — UDCT_v7_13_3D_Constitutive_Poisson_Coupling_Factor_Corrected

Full archive filename: UDCT_v7_13_3D_Constitutive_Poisson_Coupling_Factor_Corrected_20261021_012118_509.py

Source length: 70 lines; bytes: 4091; SHA-256: 732cf09e9eb5bc001451804d025dec25471e24af90df8abc06e602cb1d0fed87.

Functions: solve(rhs,h):); lap(u,h):); run(n,law,ell):).

Classes: none. Imports: import sys, json, hashlib, importlib.util; from pathlib import Path; import numpy as np; from scipy.fft import dstn,idstn.

Opening source documentation: Manufactured Newtonian field -> frozen constitutive source -> 3D Dirichlet solve. EXPLORATORY numerical integration test only; NOT a galaxy field or 24-bracket validation.

Static implementation signals (presence in source, not proof of execution): linear algebra, JSON output, internal assertion, hash verification, solver path.

Source excerpt (first 23 physical lines; whitespace and line order retained):

```
 1 #!/usr/bin/env python3
 2 """Manufactured Newtonian field -> frozen constitutive source -> 3D Dirichlet solve.
 3 EXPLORATORY numerical integration test only; NOT a galaxy field or 24-bracket validation.
 4 """
 5 import sys, json, hashlib, importlib.util
 6 from pathlib import Path
 7 import numpy as np
 8 from scipy.fft import dstn,idstn
 9 R=Path('/mnt/data/v713_action_audit')
10 P=R/'UDCT_v7_10_Postregistration_run.py'
11 spec=importlib.util.spec_from_file_location('udct_frozen_coupling',P)
12 m=importlib.util.module_from_spec(spec);sys.modules[spec.name]=m;spec.loader.exec_module(m)
13 # Fixed manufactured non-isotropic Newtonian potential psi = g0.x + 1/2 x.H.x
14 H=np.diag([1.1e-30,1.6e-30,2.3e-30]);g0=np.array([1.4e-11,-1.e-11,.7e-11]);L=1.2*m.PC
15
16 def solve(rhs,h):
17  n=rhs.shape[0]; k=np.arange(1,n+1)
18  ev=2*(np.cos(np.pi*k/(n+1))-1)/h**2
19  return idstn(dstn(rhs,type=1,norm='ortho')/(ev[:,None,None]+ev[None,:,None]+ev[None,None,:]),type=1
20
21 def lap(u,h):
22  p=np.pad(u,1)
23  return (p[2:,1:-1,1:-1]+p[:-2,1:-1,1:-1]+p[1:-1,2:,1:-1]+p[1:-1,:-2,1:-1]+p[1:-1,1:-1,2:]+p[1:-1,1:
```

Audit interpretation: Identify the exact inputs, units, normalization and frozen constants in the full script. Determine whether this file computes a new physical profile, reads prior profiles, or only checks a manufactured numerical identity. Inspect the handling of exceptions, finite values, boundaries and convergence; a successful process exit alone is not a scientific validation.

Reproduction note: compare any outputs with the companion JSON and/or NPZ by exact archive name and digest. The filename timestamp is metadata, not proof of run chronology. No independent execution of this script is claimed in this companion.

# Source 03/77 — UDCT_v7_13_3D_Poisson_Solver_Manufactured_Control

Full archive filename: UDCT_v7_13_3D_Poisson_Solver_Manufactured_Control_20261021_012151_799.py

Source length: 53 lines; bytes: 3212; SHA-256: b96bdabce1bbd8d51d5509391ba7b979fdaca16c53a05577f2dceb851b660d34.

Functions: solve_poisson_dirichlet(rhs, spacing):); laplacian_7pt(u,h):).

Classes: none. Imports: from pathlib import Path; import hashlib, json, time; import numpy as np; from scipy.fft import dstn, idstn.

Opening source documentation: Exploratory 3D Dirichlet Poisson solver verification; NOT a GQUMOND galaxy solution.

Static implementation signals (presence in source, not proof of execution): linear algebra, JSON output, internal assertion, hash verification, solver path.

Source excerpt (first 23 physical lines; whitespace and line order retained):

```
 1 #!/usr/bin/env python3
 2 """Exploratory 3D Dirichlet Poisson solver verification; NOT a GQUMOND galaxy solution."""
 3 from pathlib import Path
 4 import hashlib, json, time
 5 import numpy as np
 6 from scipy.fft import dstn, idstn
 7 ROOT=Path('/mnt/data/v713_3d_solver_stage')
 8 expected={}
 9 for line in (ROOT/'SHA256SUMS.txt').read_text().splitlines():
10     h,n=line.split(maxsplit=1); expected[n]=h
11 for name,h in expected.items():
12     actual=hashlib.sha256((ROOT/name).read_bytes()).hexdigest()
13     assert actual==h, f'Frozen file hash mismatch: {name}'
14 
15 def solve_poisson_dirichlet(rhs, spacing):
16     # Discrete 7-point Laplacian with zero Dirichlet boundary values.
17     n=rhs.shape[0]
18     assert rhs.shape==(n,n,n)
19     k=np.arange(1,n+1)
20     eigen1=2*(np.cos(np.pi*k/(n+1))-1)/spacing**2
21     eig=eigen1[:,None,None]+eigen1[None,:,None]+eigen1[None,None,:]
22     transformed=dstn(rhs,type=1,norm='ortho')
23     return idstn(transformed/eig,type=1,norm='ortho')
```

Audit interpretation: Identify the exact inputs, units, normalization and frozen constants in the full script. Determine whether this file computes a new physical profile, reads prior profiles, or only checks a manufactured numerical identity. Inspect the handling of exceptions, finite values, boundaries and convergence; a successful process exit alone is not a scientific validation.

Reproduction note: compare any outputs with the companion JSON and/or NPZ by exact archive name and digest. The filename timestamp is metadata, not proof of run chronology. No independent execution of this script is claimed in this companion.

# Source 04/77 — UDCT_v7_13_3D_Potential_Implementation_Gate

Full archive filename: UDCT_v7_13_3D_Potential_Implementation_Gate_20261021_012207_715.py

Source length: 11 lines; bytes: 2006; SHA-256: 1d2b75de19a8dd99cf2ebfbeb12edb46afba4b5d25eee7162f4886233c435948.

Functions: No named function definitions found.

Classes: none. Imports: import ast,hashlib,json,pathlib,datetime.

Opening source documentation: Read-only, reproducible source-level implementation gate; not a galaxy simulation.

Static implementation signals (presence in source, not proof of execution): JSON output, hash verification, solver path.

Source excerpt (first 23 physical lines; whitespace and line order retained):

```
 1 #!/usr/bin/env python3
 2 """Read-only, reproducible source-level implementation gate; not a galaxy simulation."""
 3 import ast,hashlib,json,pathlib,datetime
 4 root=pathlib.Path('/mnt/data/v713_hessian_stage')
 5 files=['UDCT_v7_06_Hessian_Term_Sign_Audit_reproduce.py','UDCT_v7_10_Postregistration_run.py','UDCT_
 6 result={'audit_type':'source-level implementation inventory (not numerical Hessian validation)','fil
 7 for name in files:
 8  p=root/name; s=p.read_text(); tree=ast.parse(s); funcs=[x.name for x in ast.walk(tree) if isinstanc
 9  result['files'][name]={'sha256':hashlib.sha256(p.read_bytes()).hexdigest(),'functions_of_interest':
10 result['specific_findings']={'v706_galactic_field':'returns external visible-mass gradient and Hessi
11 out=pathlib.Path('/mnt/data/UDCT_v7_13_3D_Potential_Implementation_Gate.json');out.write_text(json.d
```

Audit interpretation: Identify the exact inputs, units, normalization and frozen constants in the full script. Determine whether this file computes a new physical profile, reads prior profiles, or only checks a manufactured numerical identity. Inspect the handling of exceptions, finite values, boundaries and convergence; a successful process exit alone is not a scientific validation.

Reproduction note: compare any outputs with the companion JSON and/or NPZ by exact archive name and digest. The filename timestamp is metadata, not proof of run chronology. No independent execution of this script is claimed in this companion.

# Source 05/77 — UDCT_v7_13_Acceleration_Richardson_Diagnostic

Full archive filename: UDCT_v7_13_Acceleration_Richardson_Diagnostic_20261021_011857_779.py

Source length: 26 lines; bytes: 1977; SHA-256: 41ddb033101b49f1fc869662b30eef789cf88f5fe21a07bd89fa4aa04994397b.

Functions: extrap(i,j):return (x[i]*y[j]-x[j]*y[i])/(x[i]-x[j])).

Classes: none. Imports: import json, hashlib; from pathlib import Path.

Opening source documentation: Exploratory three-grid h^2 extrapolation of EXISTING fixed-box results. Not a new 3D solve; not a preregistered B01-B24 decision.

Static implementation signals (presence in source, not proof of execution): JSON output, internal assertion, hash verification, solver path.

Source excerpt (first 23 physical lines; whitespace and line order retained):

```
 1 #!/usr/bin/env python3
 2 """Exploratory three-grid h^2 extrapolation of EXISTING fixed-box results.
 3 Not a new 3D solve; not a preregistered B01-B24 decision.
 4 """
 5 import json, hashlib
 6 from pathlib import Path
 7 src=Path(__file__).with_name('UDCT_v7_13_Fixed_Box_Resolution_Audit.json')
 8 raw=src.read_bytes(); data=json.loads(raw)
 9 results=[]
10 for law in ('A-Qs','A-f1/2'):
11     records=sorted((r for r in data['results'] if r['law']==law),key=lambda r:r['spacing_over_rh'],r
12     assert [r['n'] for r in records]==[23,35,47]
13     fields={}
14     for key in ('center_delta_phi_over_a0_rh','center_delta_acceleration_over_a0'):
15         x=[r['spacing_over_rh']**2 for r in records]
16         y=[r[key] for r in records]
17         def extrap(i,j):return (x[i]*y[j]-x[j]*y[i])/(x[i]-x[j])
18         coarse=extrap(0,1); fine=extrap(1,2)
19         fields[key]={'samples':y,'h2_extrap_23_35':coarse,'h2_extrap_35_47':fine,'pair_extrap_disagr
20     results.append({'law':law,'ell_pc':10,'box_side_rh':6,'metrics':fields})
21 out={'status':'EXPLORATORY_POSTPROCESS_ONLY','input_file':src.name,'input_sha256':hashlib.sha256(raw
22 path=Path(__file__).with_suffix('.json');path.write_text(json.dumps(out,indent=2)+'\n')
23 for r in results:
```

Audit interpretation: Identify the exact inputs, units, normalization and frozen constants in the full script. Determine whether this file computes a new physical profile, reads prior profiles, or only checks a manufactured numerical identity. Inspect the handling of exceptions, finite values, boundaries and convergence; a successful process exit alone is not a scientific validation.

Reproduction note: compare any outputs with the companion JSON and/or NPZ by exact archive name and digest. The filename timestamp is metadata, not proof of run chronology. No independent execution of this script is claimed in this companion.

# Source 06/77 — UDCT_v7_13_Action_Variation_Factor_Audit

Full archive filename: UDCT_v7_13_Action_Variation_Factor_Audit_20261021_012121_374.py

Source length: 51 lines; bytes: 3025; SHA-256: 9ea62ea5348764013bbc86d7c00c5a48d9f93fa0f6ed27bf30dfb9bc50f56ab3.

Functions: D(a,i):return (np.roll(a,-1,axis=i)-np.roll(a,1,axis=i))/); derivs(a):); calc(a,law):).

Classes: none. Imports: from pathlib import Path; import zipfile,hashlib,json,sys,importlib.util; import numpy as np.

Opening source documentation: Exploratory discrete action variation audit, not a galaxy calculation.

Static implementation signals (presence in source, not proof of execution): linear algebra, JSON output, internal assertion, hash verification.

Source excerpt (first 23 physical lines; whitespace and line order retained):

```
 1 #!/usr/bin/env python3
 2 """Exploratory discrete action variation audit, not a galaxy calculation."""
 3 from pathlib import Path
 4 import zipfile,hashlib,json,sys,importlib.util
 5 import numpy as np
 6 ZIP=Path('/mnt/data/UDCT_v7_13_Preregistration_Complete_Updated.zip')
 7 R=Path('/mnt/data/v713_action_audit');R.mkdir(exist_ok=True)
 8 with zipfile.ZipFile(ZIP) as z:
 9  plan=json.loads(z.read('preregistration_plan.json'))
10  for name,digest in plan['frozen_hashes'].items():
11   data=z.read(name);assert hashlib.sha256(data).hexdigest()==digest,name
12   (R/name).write_bytes(data)
13 spec=importlib.util.spec_from_file_location('udct_v710_action',R/'UDCT_v7_10_Postregistration_run.py
14 m=importlib.util.module_from_spec(spec);sys.modules[spec.name]=m;spec.loader.exec_module(m)
15 rng=np.random.default_rng(713)
16 n=14;L=1.0;h=L/n
17 x=np.arange(n)*2*np.pi/n
18 X,Y,Z=np.meshgrid(x,x,x,indexing='ij')
19 base=0.16*np.sin(X)+0.11*np.cos(Y+Z)+0.08*np.sin(X+2*Y-Z)
20 pert=rng.normal(size=base.shape);pert/=np.linalg.norm(pert)
21 external=np.array([1.1,-0.7,0.5])
22 def D(a,i):return (np.roll(a,-1,axis=i)-np.roll(a,1,axis=i))/(2*h)
23 def derivs(a):
```

Audit interpretation: Identify the exact inputs, units, normalization and frozen constants in the full script. Determine whether this file computes a new physical profile, reads prior profiles, or only checks a manufactured numerical identity. Inspect the handling of exceptions, finite values, boundaries and convergence; a successful process exit alone is not a scientific validation.

Reproduction note: compare any outputs with the companion JSON and/or NPZ by exact archive name and digest. The filename timestamp is metadata, not proof of run chronology. No independent execution of this script is claimed in this companion.

# Source 07/77 — UDCT_v7_13_B01_B24_Trace_Inventory

Full archive filename: UDCT_v7_13_B01_B24_Trace_Inventory_20261021_011740_803.py

Source length: 30 lines; bytes: 2092; SHA-256: f5e1a15d44d7f0952728eff1daaaee8dd056865bb8de46b3738120123d7a7016.

Functions: No named function definitions found.

Classes: none. Imports: from pathlib import Path; import json,hashlib,numpy as np.

Static implementation signals (presence in source, not proof of execution): JSON output, internal assertion, hash verification, NPZ input, solver path.

Source excerpt (first 23 physical lines; whitespace and line order retained):

```
 1 from pathlib import Path
 2 import json,hashlib,numpy as np
 3 R=Path('/mnt/data')
 4 ranges=[(2,3),(4,4),(5,6),(7,9),(10,10),(11,12),(13,15),(16,17),(18,18),(19,20),(21,22),(23,24)]
 5 records=[json.loads((R/'UDCT_v7_13_B01_Full_Profile_Trace.json').read_text())]
 6 profiles={}
 7 with np.load(R/'UDCT_v7_13_B01_Full_Common_Profiles.npz') as z:
 8  for key in z.files: profiles['B01|'+key]=z[key]
 9 for a,b in ranges:
10  stem=f'UDCT_v7_13_B{a:02d}_B{b:02d}_Full'
11  obj=json.loads((R/(stem+'_Profile_Trace.json')).read_text())
12  assert obj['profile_count']==12*(b-a+1)
13  archive=R/(stem+'_Common_Profiles.npz')
14  assert hashlib.sha256(archive.read_bytes()).hexdigest()==obj['profile_archive_sha256']
15  records+=obj['targets']
16  with np.load(archive) as z:
17   for key in z.files:
18    assert key not in profiles
19    profiles[key]=z[key]
20 assert len(records)==24 and len(profiles)==288
21 for i,rec in enumerate(records,1):
22  if i==1: assert rec['target']=='B01'
23  else: assert rec['target']==f'B{i:02d}'
```

Audit interpretation: Identify the exact inputs, units, normalization and frozen constants in the full script. Determine whether this file computes a new physical profile, reads prior profiles, or only checks a manufactured numerical identity. Inspect the handling of exceptions, finite values, boundaries and convergence; a successful process exit alone is not a scientific validation.

Reproduction note: compare any outputs with the companion JSON and/or NPZ by exact archive name and digest. The filename timestamp is metadata, not proof of run chronology. No independent execution of this script is claimed in this companion.

# Source 08/77 — UDCT_v7_13_B01_Explicit_Hessian_Ablation

Full archive filename: UDCT_v7_13_B01_Explicit_Hessian_Ablation_20261021_011656_464.py

Source length: 38 lines; bytes: 3003; SHA-256: b7779b7678a52369d3435a4566b54b81df24c98ea698c03bf573d606d67ed508.

Functions: No named function definitions found.

Classes: none. Imports: from pathlib import Path; import importlib.util,sys,json,hashlib; import numpy as np.

Opening source documentation: Exploratory explicit stress-divergence ablation, holding nu_eff and density fixed. Not a variationally consistent independent gravity law or preregistered Hessian validation.

Static implementation signals (presence in source, not proof of execution): JSON output, internal assertion, hash verification, solver path.

Source excerpt (first 23 physical lines; whitespace and line order retained):

```
 1 #!/usr/bin/env python3
 2 """Exploratory explicit stress-divergence ablation, holding nu_eff and density fixed.
 3 Not a variationally consistent independent gravity law or preregistered Hessian validation.
 4 """
 5 from pathlib import Path
 6 import importlib.util,sys,json,hashlib
 7 import numpy as np
 8 ROOT=Path('/mnt/data/UDCT_v7_13_Confirmatory_Gate_Staging')
 9 plan=json.loads((ROOT/'preregistration_plan.json').read_text())
10 for name,digest in plan['frozen_hashes'].items():
11     assert hashlib.sha256((ROOT/name).read_bytes()).hexdigest()==digest,name
12 spec=importlib.util.spec_from_file_location('v712_ablation',ROOT/'UDCT_v7_12_Preregistration_Jeans_T
13 v=importlib.util.module_from_spec(spec);sys.modules[spec.name]=v;spec.loader.exec_module(v)
14 m=v.load_frozen(); b=m.load_baseline(); obj=b.DWARFS['Hydrus I']
15 rows=[]
16 for grid_name in ('default','refined'):
17     grid=b.make_grid(grid_name); e,h=b.V706.galactic_field(obj,grid)[:2]
18     ratios=b.mesh('Tapered cusp',grid,rmin=.0005)
19     st=b.model_state(obj,'Tapered cusp',ratios,e,h,grid,b.BASE_LAW,1.,0.)
20     r=st['r']; x=r/st['rh'];rho=st['rho']; vref=float(np.interp(1.,x,st['gn']))*st['rh']
21     full=b.V706.work_from_state(st)[1]
22     noexplicit=st['lead'].copy()
23     hess=full-noexplicit
```

Audit interpretation: Identify the exact inputs, units, normalization and frozen constants in the full script. Determine whether this file computes a new physical profile, reads prior profiles, or only checks a manufactured numerical identity. Inspect the handling of exceptions, finite values, boundaries and convergence; a successful process exit alone is not a scientific validation.

Reproduction note: compare any outputs with the companion JSON and/or NPZ by exact archive name and digest. The filename timestamp is metadata, not proof of run chronology. No independent execution of this script is claimed in this companion.

# Source 09/77 — UDCT_v7_13_B01_Hessian_Law_Control_Exploratory

Full archive filename: UDCT_v7_13_B01_Hessian_Law_Control_Exploratory_20261021_011702_712.py

Source length: 35 lines; bytes: 2648; SHA-256: 2b8083fd691f1e39a7dfce89d36ff2b5c983af7be59f1558a9d66262143f2cf2.

Functions: No named function definitions found.

Classes: none. Imports: from pathlib import Path; import json,hashlib,importlib.util,sys; import numpy as np.

Opening source documentation: B01 fixed-physics GQUMOND versus QUMOND-control radial Jeans zero diagnostic. Exploratory only; no 3D Hessian isolation, no preregistered boundary decision.

Static implementation signals (presence in source, not proof of execution): JSON output, internal assertion, hash verification, solver path.

Source excerpt (first 23 physical lines; whitespace and line order retained):

```
 1 #!/usr/bin/env python3
 2 """B01 fixed-physics GQUMOND versus QUMOND-control radial Jeans zero diagnostic.
 3 Exploratory only; no 3D Hessian isolation, no preregistered boundary decision.
 4 """
 5 from pathlib import Path
 6 import json,hashlib,importlib.util,sys
 7 import numpy as np
 8 R=Path('/mnt/data/UDCT_v7_13_Confirmatory_Gate_Staging')
 9 plan=json.loads((R/'preregistration_plan.json').read_text())
10 for name,digest in plan['frozen_hashes'].items():
11     assert hashlib.sha256((R/name).read_bytes()).hexdigest()==digest,name
12 spec=importlib.util.spec_from_file_location('v712_hessiancontrol',R/'UDCT_v7_12_Preregistration_Jean
13 v=importlib.util.module_from_spec(spec);sys.modules[spec.name]=v;spec.loader.exec_module(v)
14 m=v.load_frozen();b=m.load_baseline();obj=b.DWARFS['Hydrus I']
15 rows=[]
16 for grid_name in ('default','refined'):
17     grid=b.make_grid(grid_name);e,h=b.V706.galactic_field(obj,grid)[:2]
18     ratios=b.mesh('Tapered cusp',grid,rmin=.0005)
19     for label,law,ell in [('BASE_GQ',b.BASE_LAW,1.),('BASE_QUMOND',b.BASE_LAW,None),('A_fhalf_GQ',b.
20         st=b.model_state(obj,'Tapered cusp',ratios,e,h,grid,law,ell,0.)
21         _,force=b.V706.work_from_state(st)
22         x=st['r']/st['rh'];rho=st['rho'];vref=float(np.interp(1.,x,st['gn']))*st['rh']
23         for beta in (.35,.40):
```

Audit interpretation: Identify the exact inputs, units, normalization and frozen constants in the full script. Determine whether this file computes a new physical profile, reads prior profiles, or only checks a manufactured numerical identity. Inspect the handling of exceptions, finite values, boundaries and convergence; a successful process exit alone is not a scientific validation.

Reproduction note: compare any outputs with the companion JSON and/or NPZ by exact archive name and digest. The filename timestamp is metadata, not proof of run chronology. No independent execution of this script is claimed in this companion.

# Source 10/77 — UDCT_v7_13_B01_Independent_Jeans_Quadrature_Audit

Full archive filename: UDCT_v7_13_B01_Independent_Jeans_Quadrature_Audit_20261021_012453_346.py

Source length: 41 lines; bytes: 2409; SHA-256: 22ffabd0e7434d3b05e715214bfdbaa72356e942b10d3b6b24c1866032b81f06.

Functions: No named function definitions found.

Classes: none. Imports: from pathlib import Path; import importlib.util, sys, json, hashlib; import numpy as np; from scipy.integrate import cumulative_simpson.

Opening source documentation: Exploratory independent integration audit for B01. Not a six-arm field validation.

Static implementation signals (presence in source, not proof of execution): JSON output, internal assertion, hash verification, solver path.

Source excerpt (first 23 physical lines; whitespace and line order retained):

```
 1 #!/usr/bin/env python3
 2 """Exploratory independent integration audit for B01. Not a six-arm field validation."""
 3 from pathlib import Path
 4 import importlib.util, sys, json, hashlib
 5 import numpy as np
 6 from scipy.integrate import cumulative_simpson
 7 ROOT=Path('/mnt/data/v713_post_work')
 8 plan=json.loads((ROOT/'preregistration_plan.json').read_text())
 9 for name,digest in plan['frozen_hashes'].items():
10     assert hashlib.sha256((ROOT/name).read_bytes()).hexdigest()==digest,name
11 spec=importlib.util.spec_from_file_location('v712frozen',ROOT/'UDCT_v7_12_Preregistration_Jeans_Tran
12 v=importlib.util.module_from_spec(spec);sys.modules[spec.name]=v;spec.loader.exec_module(v)
13 m=v.load_frozen(); b=m.load_baseline(); obj=b.DWARFS['Hydrus I']
14 results=[]
15 for gridname in ('default','refined'):
16     grid=b.make_grid(gridname);e,h=b.V706.galactic_field(obj,grid)[:2]
17     for inner in (.002,.0005):
18         st=b.model_state(obj,'Tapered cusp',b.mesh('Tapered cusp',grid,rmin=inner),e,h,grid,b.BASE_L
19         _,force=b.V706.work_from_state(st)
20         r,rho=st['r'],st['rho'];vref2=float(np.interp(1.,r/st['rh'],st['gn']))*st['rh']
21         for beta in (.35,.40):
22             p_trap,_,q=m.solve(r,rho,force,beta,0.)
23             # Independent integration rule (composite Simpson on nonuniform radial mesh).
```

Audit interpretation: Identify the exact inputs, units, normalization and frozen constants in the full script. Determine whether this file computes a new physical profile, reads prior profiles, or only checks a manufactured numerical identity. Inspect the handling of exceptions, finite values, boundaries and convergence; a successful process exit alone is not a scientific validation.

Reproduction note: compare any outputs with the companion JSON and/or NPZ by exact archive name and digest. The filename timestamp is metadata, not proof of run chronology. No independent execution of this script is claimed in this companion.

# Source 11/77 — UDCT_v7_13_B01_Inner_Cutoff_Stability

Full archive filename: UDCT_v7_13_B01_Inner_Cutoff_Stability_20261021_011711_520.py

Source length: 28 lines; bytes: 2112; SHA-256: 96937328f38b24cfe7c922c3c33cc703868035ba0dfa5041263e53ebe46c478b.

Functions: No named function definitions found.

Classes: none. Imports: import json,hashlib,importlib.util,sys; from pathlib import Path; import numpy as np.

Opening source documentation: Exploratory B01 radial zero sensitivity to inner cutoff, frozen v7.12 physics.

Static implementation signals (presence in source, not proof of execution): JSON output, internal assertion, hash verification, solver path.

Source excerpt (first 23 physical lines; whitespace and line order retained):

```
 1 #!/usr/bin/env python3
 2 """Exploratory B01 radial zero sensitivity to inner cutoff, frozen v7.12 physics."""
 3 import json,hashlib,importlib.util,sys
 4 from pathlib import Path
 5 import numpy as np
 6 R=Path('/mnt/data/UDCT_v7_13_Confirmatory_Gate_Staging')
 7 plan=json.loads((R/'preregistration_plan.json').read_text())
 8 for name,digest in plan['frozen_hashes'].items():
 9  assert hashlib.sha256((R/name).read_bytes()).hexdigest()==digest,name
10 spec=importlib.util.spec_from_file_location('v712_cutoff_audit',R/'UDCT_v7_12_Preregistration_Jeans_
11 m=importlib.util.module_from_spec(spec);sys.modules[spec.name]=m;spec.loader.exec_module(m)
12 old=m.load_frozen();b=old.load_baseline();obj=b.DWARFS['Hydrus I']
13 rows=[]
14 for grid_name in ('default','refined'):
15  grid=b.make_grid(grid_name);e,h=b.V706.galactic_field(obj,grid)[:2]
16  for inner in (.00025,.0005,.00075,.001):
17   st=b.model_state(obj,'Tapered cusp',b.mesh('Tapered cusp',grid,rmin=inner),e,h,grid,b.BASE_LAW,1.0
18   _,force=b.V706.work_from_state(st);x=st['r']/st['rh']
19   pressure,_,_=old.solve(st['r'],st['rho'],force,.35)
20   f=pressure/st['rho']/(float(np.interp(1.,x,st['gn']))*st['rh'])
21   idx=np.flatnonzero((f[:-1]*f[1:]<0)&(x[:-1]<.002))
22   roots=[]
23   for i in idx:
```

Audit interpretation: Identify the exact inputs, units, normalization and frozen constants in the full script. Determine whether this file computes a new physical profile, reads prior profiles, or only checks a manufactured numerical identity. Inspect the handling of exceptions, finite values, boundaries and convergence; a successful process exit alone is not a scientific validation.

Reproduction note: compare any outputs with the companion JSON and/or NPZ by exact archive name and digest. The filename timestamp is metadata, not proof of run chronology. No independent execution of this script is claimed in this companion.

# Source 12/77 — UDCT_v7_13_B01_Radial_Domain_Audit

Full archive filename: UDCT_v7_13_B01_Radial_Domain_Audit_20261021_011730_078.py

Source length: 33 lines; bytes: 2119; SHA-256: 17f6a90bef9f02a945e19d3750ad75edcaab0ce34ec0bae8949b89314dff80bd.

Functions: load(path,name):).

Classes: none. Imports: import json, hashlib, importlib.util, sys; from pathlib import Path; import numpy as np.

Opening source documentation: Audit B01 native-vs-common minimum locations without modifying frozen physics.

Static implementation signals (presence in source, not proof of execution): JSON output, internal assertion, hash verification, solver path.

Source excerpt (first 23 physical lines; whitespace and line order retained):

```
 1 #!/usr/bin/env python3
 2 """Audit B01 native-vs-common minimum locations without modifying frozen physics."""
 3 import json, hashlib, importlib.util, sys
 4 from pathlib import Path
 5 import numpy as np
 6 R=Path('/mnt/data/UDCT_v7_13_Confirmatory_Gate_Staging')
 7 plan=json.loads((R/'preregistration_plan.json').read_text())
 8 for name,sha in plan['frozen_hashes'].items():
 9     assert hashlib.sha256((R/name).read_bytes()).hexdigest()==sha,name
10
11 def load(path,name):
12     spec=importlib.util.spec_from_file_location(name,path)
13     mod=importlib.util.module_from_spec(spec);sys.modules[name]=mod;spec.loader.exec_module(mod)
14     return mod
15 old=load(R/'UDCT_v7_12_Preregistration_Jeans_Transition.py','v712_domain')
16 m=old.load_frozen(); b=m.load_baseline(); obj=b.DWARFS['Hydrus I']
17 common=np.geomspace(.002,10.,1200)
18 rows=[]
19 for grid_name in ['default','refined']:
20     grid=b.make_grid(grid_name)
21     e,h=b.V706.galactic_field(obj,grid)[:2]
22     for inner in [.002,.0005]:
23         st=b.model_state(obj,'Tapered cusp',b.mesh('Tapered cusp',grid,rmin=inner),e,h,grid,b.BASE_L
```

Audit interpretation: Identify the exact inputs, units, normalization and frozen constants in the full script. Determine whether this file computes a new physical profile, reads prior profiles, or only checks a manufactured numerical identity. Inspect the handling of exceptions, finite values, boundaries and convergence; a successful process exit alone is not a scientific validation.

Reproduction note: compare any outputs with the companion JSON and/or NPZ by exact archive name and digest. The filename timestamp is metadata, not proof of run chronology. No independent execution of this script is claimed in this companion.

# Source 13/77 — UDCT_v7_13_B01_Radial_Zero_Exploratory

Full archive filename: UDCT_v7_13_B01_Radial_Zero_Exploratory_20261021_011719_053.py

Source length: 33 lines; bytes: 2632; SHA-256: 9cdc1cf246ad68a68b988c985bfa97fd9b8031c0ee92d56496616a6b7eaf5948.

Functions: No named function definitions found.

Classes: none. Imports: import json,hashlib,importlib.util,sys; from pathlib import Path; import numpy as np.

Opening source documentation: B01 radial sign-change exploration; frozen physics, no altered prereg criteria.

Static implementation signals (presence in source, not proof of execution): JSON output, internal assertion, hash verification, solver path.

Source excerpt (first 23 physical lines; whitespace and line order retained):

```
 1 #!/usr/bin/env python3
 2 """B01 radial sign-change exploration; frozen physics, no altered prereg criteria."""
 3 import json,hashlib,importlib.util,sys
 4 from pathlib import Path
 5 import numpy as np
 6 R=Path('/mnt/data/UDCT_v7_13_Confirmatory_Gate_Staging')
 7 plan=json.loads((R/'preregistration_plan.json').read_text())
 8 for name,digest in plan['frozen_hashes'].items():
 9     assert hashlib.sha256((R/name).read_bytes()).hexdigest()==digest,name
10 spec=importlib.util.spec_from_file_location('udct712_radialzero',R/'UDCT_v7_12_Preregistration_Jeans
11 old=importlib.util.module_from_spec(spec);sys.modules[spec.name]=old;spec.loader.exec_module(old)
12 m=old.load_frozen(); b=m.load_baseline(); obj=b.DWARFS['Hydrus I']
13 rows=[]
14 for grid_name in ('default','refined','validation'):
15     grid=b.make_grid(grid_name) if grid_name!='validation' else b.Grid(radial=1440,polar=96,azimuth=
16     e,h=b.V706.galactic_field(obj,grid)[:2]
17     st=b.model_state(obj,'Tapered cusp',b.mesh('Tapered cusp',grid,rmin=.0005),e,h,grid,b.BASE_LAW,1
18     _,force=b.V706.work_from_state(st)
19     x=st['r']/st['rh']
20     for beta in (.35,.40):
21         pressure,_,_=m.solve(st['r'],st['rho'],force,beta)
22         f=pressure/st['rho']/(float(np.interp(1.,x,st['gn']))*st['rh'])
23         inds=np.flatnonzero((f[:-1]*f[1:]<0)&(x[:-1]<.002))
```

Audit interpretation: Identify the exact inputs, units, normalization and frozen constants in the full script. Determine whether this file computes a new physical profile, reads prior profiles, or only checks a manufactured numerical identity. Inspect the handling of exceptions, finite values, boundaries and convergence; a successful process exit alone is not a scientific validation.

Reproduction note: compare any outputs with the companion JSON and/or NPZ by exact archive name and digest. The filename timestamp is metadata, not proof of run chronology. No independent execution of this script is claimed in this companion.

# Source 14/77 — UDCT_v7_13_B01_Stress_Derivative_Decomposition

Full archive filename: UDCT_v7_13_B01_Stress_Derivative_Decomposition_20261021_011646_125.py

Source length: 42 lines; bytes: 3727; SHA-256: 5c0a5d8a00ae9374bf8830181f99c1a9377ff4aa5a6d53eddd05273536d67ed6.

Functions: No named function definitions found.

Classes: none. Imports: from pathlib import Path; import importlib.util, sys, json, hashlib; import numpy as np.

Opening source documentation: Exploratory decomposition of explicit GQUMOND stress force in frozen B01. NOT a new theory, preregistered confirmation, or physical Hessian measurement.

Static implementation signals (presence in source, not proof of execution): numerical derivative, JSON output, internal assertion, hash verification, solver path.

Source excerpt (first 23 physical lines; whitespace and line order retained):

```
#!/usr/bin/env python3
"""Exploratory decomposition of explicit GQUMOND stress force in frozen B01.
NOT a new theory, preregistered confirmation, or physical Hessian measurement.
"""
from pathlib import Path
import importlib.util, sys, json, hashlib
import numpy as np
ROOT=Path('/mnt/data/UDCT_v7_13_Confirmatory_Gate_Staging')
plan=json.loads((ROOT/'preregistration_plan.json').read_text())
hashes={name:hashlib.sha256((ROOT/name).read_bytes()).hexdigest() for name in plan['frozen_hashes']}
assert all(hashes[n]==v for n,v in plan['frozen_hashes'].items())
spec=importlib.util.spec_from_file_location('v712_stress_split',ROOT/'UDCT_v7_12_Preregistration_Jea
v=importlib.util.module_from_spec(spec);sys.modules[spec.name]=v;spec.loader.exec_module(v)
m=v.load_frozen();b=m.load_baseline();obj=b.DWARFS['Hydrus I']
rows=[]
for grid_name in ('default','refined'):
 grid=b.make_grid(grid_name);e,h=b.V706.galactic_field(obj,grid)[:2]
 ratios=b.mesh('Tapered cusp',grid,rmin=.0005)
 st=b.model_state(obj,'Tapered cusp',ratios,e,h,grid,b.BASE_LAW,1.,0.)
 r=st['r'];x=r/st['rh'];rho=st['rho'];vref=float(np.interp(1.,x,st['gn']))*st['rh']
 lead=st['lead'];pnn=st['pnn'];trp=st['trp']
 grad=-.5*np.gradient(pnn,r,edge_order=2)
 geom=-.5*(3*pnn-trp)/r
```

Audit interpretation: Identify the exact inputs, units, normalization and frozen constants in the full script. Determine whether this file computes a new physical profile, reads prior profiles, or only checks a manufactured numerical identity. Inspect the handling of exceptions, finite values, boundaries and convergence; a successful process exit alone is not a scientific validation.

Reproduction note: compare any outputs with the companion JSON and/or NPZ by exact archive name and digest. The filename timestamp is metadata, not proof of run chronology. No independent execution of this script is claimed in this companion.

# Source 15/77 — UDCT_v7_13_B02_B24_Full_Profile_Trace

Full archive filename: UDCT_v7_13_B02_B24_Full_Profile_Trace_20261021_011741_455.py

Source length: 61 lines; bytes: 5067; SHA-256: 9a521ddee5a211759b5353628f6dfc2988ba38322655cefb3856271e7754acb5.

Functions: load(path,name):).

Classes: none. Imports: from pathlib import Path; import json,csv,hashlib,importlib.util,sys,time; import numpy as np.

Opening source documentation: B02-B24 exploratory six-arm full-profile archival. Frozen physics; no 3D Hessian.

Static implementation signals (presence in source, not proof of execution): JSON output, internal assertion, hash verification, solver path.

Source excerpt (first 23 physical lines; whitespace and line order retained):

```
 1 #!/usr/bin/env python3
 2 """B02-B24 exploratory six-arm full-profile archival. Frozen physics; no 3D Hessian."""
 3 from pathlib import Path
 4 import json,csv,hashlib,importlib.util,sys,time
 5 import numpy as np
 6 R=Path('/mnt/data/UDCT_v7_13_Confirmatory_Gate_Staging')
 7 OUT=Path('/mnt/data')
 8 plan=json.loads((R/'preregistration_plan.json').read_text())
 9 for name,digest in plan['frozen_hashes'].items():
10     assert hashlib.sha256((R/name).read_bytes()).hexdigest()==digest,name
11 def load(path,name):
12     spec=importlib.util.spec_from_file_location(name,path);mod=importlib.util.module_from_spec(spec)
13 old=load(R/'UDCT_v7_12_Preregistration_Jeans_Transition.py','v712_batch_trace')
14 proto=load(Path('/mnt/data/v713_b07_run/UDCT_v7_13_Six_Arm_Decision_Prototype.py'),'v713_batch_proto
15 m=old.load_frozen();b=m.load_baseline();obj=b.DWARFS['Hydrus I']
16 with (R/'adaptive_brackets.csv').open() as f: targets=list(csv.DictReader(f))
17 assert len(targets)==24
18 laws={law.name:law for law in (b.BASE_LAW,)+b.PART_A_LAWS}
19 configs=[('default',b.make_grid('default')),('refined',b.make_grid('refined')),('validation',b.Grid(
20 # Cache the force for each law, ell, resolution, and inner radius across targets.
21 cache={};profiles={};results=[];t0=time.monotonic()
22 start=int(sys.argv[1]) if len(sys.argv)>1 else 2
23 stop=int(sys.argv[2]) if len(sys.argv)>2 else 24
```

Audit interpretation: Identify the exact inputs, units, normalization and frozen constants in the full script. Determine whether this file computes a new physical profile, reads prior profiles, or only checks a manufactured numerical identity. Inspect the handling of exceptions, finite values, boundaries and convergence; a successful process exit alone is not a scientific validation.

Reproduction note: compare any outputs with the companion JSON and/or NPZ by exact archive name and digest. The filename timestamp is metadata, not proof of run chronology. No independent execution of this script is claimed in this companion.

# Source 16/77 — UDCT_v7_13_Boundary_Moment_Cancellation_Audit

Full archive filename: UDCT_v7_13_Boundary_Moment_Cancellation_Audit_20261021_011833_901.py

Source length: 35 lines; bytes: 2223; SHA-256: 955e301b4c8c1b2a1b77c309bd5bc34227414b1306572a27f15584cf8515ee61.

Functions: No named function definitions found.

Classes: none. Imports: from pathlib import Path; import hashlib, json.

Opening source documentation: Reproducible secondary audit of frozen exploratory source-localization results. No new 3D solve and no preregistered confirmatory test.

Static implementation signals (presence in source, not proof of execution): JSON output, internal assertion, hash verification, solver path.

Source excerpt (first 23 physical lines; whitespace and line order retained):

```
#!/usr/bin/env python3
"""Reproducible secondary audit of frozen exploratory source-localization results.
No new 3D solve and no preregistered confirmatory test.
"""
from pathlib import Path
import hashlib, json
p=Path(__file__).with_name('UDCT_v7_13_Source_Localization_Boundary_Audit.json')
raw=p.read_bytes(); src=json.loads(raw)
rows=[]
for x in src['results']:
    a=x['absolute_source_integral']; signed=x['signed_source_integral']
    assert a>0 and abs(signed)<=a*(1+1e-12)
    rows.append({
      'model':x['law'], 'box_side_rh':round(x['box_side_over_rh']),
      'signed_source_over_abs_source':signed/a,
      'opposing_sign_cancellation_fraction':1-abs(signed)/a,
      'outer_abs_source_fraction':x['outer_shell_abs_fraction'],
      'dipole_norm_over_abs_source_rh':x['dipole_moment_norm_over_abs_integral_rh'],
      'outer_rms_over_inner_rms':x['outer_shell_rms_to_inner_rms']})
by={(r['model'],r['box_side_rh']):r for r in rows}
comparisons=[]
for model in sorted(set(r['model'] for r in rows)):
    a,b=by[(model,6)],by[(model,8)]
```

Audit interpretation: Identify the exact inputs, units, normalization and frozen constants in the full script. Determine whether this file computes a new physical profile, reads prior profiles, or only checks a manufactured numerical identity. Inspect the handling of exceptions, finite values, boundaries and convergence; a successful process exit alone is not a scientific validation.

Reproduction note: compare any outputs with the companion JSON and/or NPZ by exact archive name and digest. The filename timestamp is metadata, not proof of run chronology. No independent execution of this script is claimed in this companion.

# Source 17/77 — UDCT_v7_13_Common_Coordinate_Acceleration_Audit

Full archive filename: UDCT_v7_13_Common_Coordinate_Acceleration_Audit_20261021_011847_335.py

Source length: 41 lines; bytes: 2582; SHA-256: 8a86040ef901bc557465cfcec1d36b2e89683c41cfc3df64d2da4b4ad57ab0d4.

Functions: grad_at(ix):).

Classes: none. Imports: import json,hashlib; from pathlib import Path; import numpy as np.

Opening source documentation: Exploratory common-coordinate interpolation, finite-box delta-potential; NOT prereg B01-B24.

Static implementation signals (presence in source, not proof of execution): linear algebra, JSON output, internal assertion, hash verification.

Source excerpt (first 23 physical lines; whitespace and line order retained):

```
#!/usr/bin/env python3
"""Exploratory common-coordinate interpolation, finite-box delta-potential; NOT prereg B01-B24."""
import json,hashlib
from pathlib import Path
import numpy as np
root=Path('/mnt/data'); src=root/'UDCT_v7_13_Hydrus_3D_DeltaPotential_Exploratory.py'
s=src.read_text().split("out={'status'")[0]
needle=" return {'n':n,'spacing_over_rh':h/rh"
assert needle in s
replacement='''
 samples=[]
 for target in (0.5,1.,2.):
  position=target*rh/h+mid
  lo=int(np.floor(position));hi=lo+1;alpha=position-lo
  assert lo>=1 and hi+1<n
  def grad_at(ix):
   return np.array([(pot[ix+1,mid,mid]-pot[ix-1,mid,mid]),(pot[ix,mid+1,mid]-pot[ix,mid-1,mid]),(pot
  v=(1-alpha)*grad_at(lo)+alpha*grad_at(hi)
  samples.append({'target_radius_rh':target,'fractional_grid_index':float(position),'interpolated_de
 return {'n':n,'law':law.name,'ell_pc':ellpc,'spacing_over_rh':float(h/rh),'samples':samples,'relati
'''
s=s.replace(needle,replacement+' # unused return stub: '+needle)
ns={'__file__':str(src)}
```

Audit interpretation: Identify the exact inputs, units, normalization and frozen constants in the full script. Determine whether this file computes a new physical profile, reads prior profiles, or only checks a manufactured numerical identity. Inspect the handling of exceptions, finite values, boundaries and convergence; a successful process exit alone is not a scientific validation.

Reproduction note: compare any outputs with the companion JSON and/or NPZ by exact archive name and digest. The filename timestamp is metadata, not proof of run chronology. No independent execution of this script is claimed in this companion.

# Source 18/77 — UDCT_v7_13_Confirmatory_Readiness_Audit

Full archive filename: UDCT_v7_13_Confirmatory_Readiness_Audit_20261021_011753_298.py

Source length: 40 lines; bytes: 3060; SHA-256: 6d68e443ce0e0e71ec1d190d50b76bedc1c394acf2edd0747f61373d90915b67.

Functions: No named function definitions found.

Classes: none. Imports: import json,hashlib,subprocess,sys,pathlib,collections.

Opening source documentation: Read-only readiness audit. Does NOT certify v7.13 scientific results.

Static implementation signals (presence in source, not proof of execution): JSON output, hash verification, solver path.

Source excerpt (first 23 physical lines; whitespace and line order retained):

```
 1 #!/usr/bin/env python3
 2 """Read-only readiness audit. Does NOT certify v7.13 scientific results."""
 3 import json,hashlib,subprocess,sys,pathlib,collections
 4 R=pathlib.Path(__file__).resolve().parent
 5 P=R/'UDCT_v7_13_Confirmatory_Gate_Staging'
 6 plan=json.loads((P/'preregistration_plan.json').read_text())
 7 checks=[]
 8 for name,digest in plan['frozen_hashes'].items():
 9  p=P/name
10  checks.append({'file':name,'exists':p.exists(),'hash_matches':p.exists() and hashlib.sha256(p.read_
11 selftests={}
12 for filename in ('UDCT_v7_13_Protocol_Audit.py','UDCT_v7_13_Preregistration_Preflight.py'):
13  r=subprocess.run([sys.executable,str(P/filename),'--selftest'],capture_output=True,text=True,cwd=P)
14  selftests[filename]={'passed':r.returncode==0,'output':(r.stdout+r.stderr).strip()}
15 r=subprocess.run([sys.executable,str(R/'v713_b07_run/UDCT_v7_13_Six_Arm_Decision_Prototype.py'),'--s
16 selftests['Six_Arm_Decision_Prototype']={'passed':r.returncode==0,'output':(r.stdout+r.stderr).strip
17 rows=[]
18 for i in range(1,25):
19  p=R/f'UDCT_v7_13_B{i:02d}_Six_Arm_Profile_Exploratory.json'
20  if not p.exists(): rows.append({'id':f'B{i:02d}','present':False});continue
21  d=json.loads(p.read_text());ends=d.get('endpoint_results',{})
22  rows.append({'id':f'B{i:02d}','present':True,'sha256':hashlib.sha256(p.read_bytes()).hexdigest(),
23  'six_arms_executed_claim':d.get('six_arms_executed'),'status':d.get('bracket_status'),
```

Audit interpretation: Identify the exact inputs, units, normalization and frozen constants in the full script. Determine whether this file computes a new physical profile, reads prior profiles, or only checks a manufactured numerical identity. Inspect the handling of exceptions, finite values, boundaries and convergence; a successful process exit alone is not a scientific validation.

Reproduction note: compare any outputs with the companion JSON and/or NPZ by exact archive name and digest. The filename timestamp is metadata, not proof of run chronology. No independent execution of this script is claimed in this companion.

# Source 19/77 — UDCT_v7_13_Constitutive_3D_Source_Prototype

Full archive filename: UDCT_v7_13_Constitutive_3D_Source_Prototype_20261021_012203_512.py

Source length: 54 lines; bytes: 3166; SHA-256: c8cc277b301d2acf3e0c7c3889bd59f24ff8957987e9bbb612a7bb17087c2d8c.

Functions: response(x,law,ell):); diagnostic(h,law,ell):).

Classes: none. Imports: import sys, json, hashlib, importlib.util; from pathlib import Path; import numpy as np.

Opening source documentation: Exploratory 3-D constitutive derivative diagnostic; NOT a solved field equation.

Static implementation signals (presence in source, not proof of execution): JSON output, internal assertion, hash verification, solver path.

Source excerpt (first 23 physical lines; whitespace and line order retained):

```
 1 #!/usr/bin/env python3
 2 """Exploratory 3-D constitutive derivative diagnostic; NOT a solved field equation."""
 3 import sys, json, hashlib, importlib.util
 4 from pathlib import Path
 5 import numpy as np
 6 ROOT=Path('/mnt/data/v713_hessian_stage')
 7 P=ROOT/'UDCT_v7_10_Postregistration_run.py'
 8 spec=importlib.util.spec_from_file_location('v710_source_diagnostic',P)
 9 m=importlib.util.module_from_spec(spec);sys.modules[spec.name]=m;spec.loader.exec_module(m)
10 # Quadratic manufactured Newtonian potential: constant non-isotropic Hessian.
11 # Inputs are dimensionless with physical gradient and Hessian magnitudes.
12 H=np.diag([1.1e-30,1.6e-30,2.3e-30])
13 g0=np.array([1.4e-11,-1.0e-11,0.7e-11])
14 x0=np.array([1.2e18,-0.8e18,0.4e18])
15 L=1.2*m.PC
16
17 def response(x,law,ell):
18  g=g0+H@x
19  nu,wh=m.gq_kernel(np.array(np.dot(g,g)),np.array(np.sum(H*H)),ell,law)
20  return float(nu)*g, float(wh)*H
21
22 def diagnostic(h,law,ell):
23  # div(nu grad psi) and double-div(wh Hess psi), independently approximated.
```

Audit interpretation: Identify the exact inputs, units, normalization and frozen constants in the full script. Determine whether this file computes a new physical profile, reads prior profiles, or only checks a manufactured numerical identity. Inspect the handling of exceptions, finite values, boundaries and convergence; a successful process exit alone is not a scientific validation.

Reproduction note: compare any outputs with the companion JSON and/or NPZ by exact archive name and digest. The filename timestamp is metadata, not proof of run chronology. No independent execution of this script is claimed in this companion.

# Source 20/77 — UDCT_v7_13_Constitutive_3D_Source_Revalidated

Full archive filename: UDCT_v7_13_Constitutive_3D_Source_Revalidated_20261021_012140_566.py

Source length: 54 lines; bytes: 3178; SHA-256: dd2f014916449be1905e3c5152a3250257c35119a05f53e516e1f3ab6eb176e1.

Functions: response(x,law,ell):); diagnostic(h,law,ell):).

Classes: none. Imports: import sys, json, hashlib, importlib.util; from pathlib import Path; import numpy as np.

Opening source documentation: Exploratory 3-D constitutive derivative diagnostic; NOT a solved field equation.

Static implementation signals (presence in source, not proof of execution): JSON output, internal assertion, hash verification, solver path.

Source excerpt (first 23 physical lines; whitespace and line order retained):

```
 1 #!/usr/bin/env python3
 2 """Exploratory 3-D constitutive derivative diagnostic; NOT a solved field equation."""
 3 import sys, json, hashlib, importlib.util
 4 from pathlib import Path
 5 import numpy as np
 6 ROOT=Path('/mnt/data/v713_frozen_source_recovered')
 7 P=ROOT/'UDCT_v7_10_Postregistration_run.py'
 8 spec=importlib.util.spec_from_file_location('v710_source_diagnostic',P)
 9 m=importlib.util.module_from_spec(spec);sys.modules[spec.name]=m;spec.loader.exec_module(m)
10 # Quadratic manufactured Newtonian potential: constant non-isotropic Hessian.
11 # Inputs are dimensionless with physical gradient and Hessian magnitudes.
12 H=np.diag([1.1e-30,1.6e-30,2.3e-30])
13 g0=np.array([1.4e-11,-1.0e-11,0.7e-11])
14 x0=np.array([1.2e18,-0.8e18,0.4e18])
15 L=1.2*m.PC
16
17 def response(x,law,ell):
18  g=g0+H@x
19  nu,wh=m.gq_kernel(np.array(np.dot(g,g)),np.array(np.sum(H*H)),ell,law)
20  return float(nu)*g, float(wh)*H
21
22 def diagnostic(h,law,ell):
23  # div(nu grad psi) and double-div(wh Hess psi), independently approximated.
```

Audit interpretation: Identify the exact inputs, units, normalization and frozen constants in the full script. Determine whether this file computes a new physical profile, reads prior profiles, or only checks a manufactured numerical identity. Inspect the handling of exceptions, finite values, boundaries and convergence; a successful process exit alone is not a scientific validation.

Reproduction note: compare any outputs with the companion JSON and/or NPZ by exact archive name and digest. The filename timestamp is metadata, not proof of run chronology. No independent execution of this script is claimed in this companion.

# Source 21/77 — UDCT_v7_13_Direct_Boundary_59cube_Audit

Full archive filename: UDCT_v7_13_Direct_Boundary_59cube_Audit_20261021_011808_455.py

Source length: 46 lines; bytes: 3073; SHA-256: 6c7a6730538b61b2ebff91dc9cd2562c27208664e81c9df07acf763ae9267b2e.

Functions: solve_bc(rhs,h):).

Classes: none. Imports: import json,hashlib; from pathlib import Path; import numpy as np; from scipy.signal import fftconvolve.

Opening source documentation: Exploratory 59^3 resolution extension; NOT preregistration or observational validation.

Static implementation signals (presence in source, not proof of execution): linear algebra, JSON output, hash verification, solver path.

Source excerpt (first 23 physical lines; whitespace and line order retained):

```
 1 #!/usr/bin/env python3
 2 """Exploratory 59^3 resolution extension; NOT preregistration or observational validation."""
 3 import json,hashlib
 4 from pathlib import Path
 5 import numpy as np
 6 from scipy.signal import fftconvolve
 7 R=Path('/mnt/data')
 8 source=R/'UDCT_v7_13_Hydrus_3D_DeltaPotential_Exploratory.py'
 9 prior=R/'UDCT_v7_13_Direct_Boundary_Resolution_Audit.json'
10 s=source.read_text().split("out={'status'")[0].replace(' pot=solve(rhs,h)', ' return rhs,h')
11 ns={'__file__':str(source)};exec(compile(s,str(source),'exec'),ns)
12 m=ns['m'];rh=ns['rh'];ns['run'].__globals__['L']=6*rh
13
14 def solve_bc(rhs,h):
15  n=rhs.shape[0];N=n+2
16  padded=np.pad(rhs,1)
17  offs=np.arange(-(N-1),N,dtype=float)*h
18  d2=offs[:,None,None]**2+offs[None,:,None]**2+offs[None,None,:]**2
19  ker=np.zeros_like(d2);mask=d2>0;ker[mask]=-h**3/(4*np.pi*np.sqrt(d2[mask]))
20  full=fftconvolve(padded,ker,mode='same')
21  rhs_bc=rhs.copy();hh=h*h
22  for ax in range(3):
23   low=np.take(full,0,axis=ax)[1:-1,1:-1];high=np.take(full,-1,axis=ax)[1:-1,1:-1]
```

Audit interpretation: Identify the exact inputs, units, normalization and frozen constants in the full script. Determine whether this file computes a new physical profile, reads prior profiles, or only checks a manufactured numerical identity. Inspect the handling of exceptions, finite values, boundaries and convergence; a successful process exit alone is not a scientific validation.

Reproduction note: compare any outputs with the companion JSON and/or NPZ by exact archive name and digest. The filename timestamp is metadata, not proof of run chronology. No independent execution of this script is claimed in this companion.

# Source 22/77 — UDCT_v7_13_Direct_Boundary_Radial_Audit

Full archive filename: UDCT_v7_13_Direct_Boundary_Radial_Audit_20261021_011815_545.py

Source length: 43 lines; bytes: 2529; SHA-256: 1b7382db45a68bc844deac62546f231b44027a2e414064269a9b4469b31c11bf.

Functions: solve_dirichlet(rhs,h,faces):); calc(lawname,side,n):); grad(phi):return np.array([(phi[ix+1,mid,mid]-phi[ix-1,mid,m).

Classes: none. Imports: from pathlib import Path; import json,hashlib; import numpy as np; from scipy.signal import fftconvolve.

Opening source documentation: Exploratory same-coordinate radial acceleration under zero vs in-box Green boundary.

Static implementation signals (presence in source, not proof of execution): linear algebra, JSON output, hash verification, solver path.

Source excerpt (first 23 physical lines; whitespace and line order retained):

```
 1 #!/usr/bin/env python3
 2 """Exploratory same-coordinate radial acceleration under zero vs in-box Green boundary."""
 3 from pathlib import Path
 4 import json,hashlib
 5 import numpy as np
 6 from scipy.signal import fftconvolve
 7 R=Path('/mnt/data')
 8 source=R/'UDCT_v7_13_Hydrus_3D_DeltaPotential_Exploratory.py'
 9 s=source.read_text();s=s[:s.index("out={'status'")];s=s.replace(' pot=solve(rhs,h)', ' return rhs,h'
10 ns={'__file__':str(source)};exec(compile(s,str(source),'exec'),ns)
11 m=ns['m'];rh=ns['rh']
12 def solve_dirichlet(rhs,h,faces):
13  v=rhs.copy();hh=h*h
14  for ax,(low,high) in enumerate(faces):
15   if ax==0:v[0,:,:]-=low/hh;v[-1,:,:]-=high/hh
16   elif ax==1:v[:,0,:]-=low/hh;v[:,-1,:]-=high/hh
17   else:v[:,:,0]-=low/hh;v[:,:,-1]-=high/hh
18  return ns['solve'](v,h)
19 def calc(lawname,side,n):
20  ns['run'].__globals__['L']=side*rh
21  law=next(z for z in m.PART_A_LAWS if z.name==lawname)
22  rhs,h=ns['run'](n,law,10.)
23  N=n+2;padded=np.pad(rhs,1)
```

Audit interpretation: Identify the exact inputs, units, normalization and frozen constants in the full script. Determine whether this file computes a new physical profile, reads prior profiles, or only checks a manufactured numerical identity. Inspect the handling of exceptions, finite values, boundaries and convergence; a successful process exit alone is not a scientific validation.

Reproduction note: compare any outputs with the companion JSON and/or NPZ by exact archive name and digest. The filename timestamp is metadata, not proof of run chronology. No independent execution of this script is claimed in this companion.

# Source 23/77 — UDCT_v7_13_Direct_Boundary_Resolution_Audit

Full archive filename: UDCT_v7_13_Direct_Boundary_Resolution_Audit_20261021_011812_014.py

Source length: 48 lines; bytes: 2772; SHA-256: 42a43e28ff2b965e22eb0817dba90f3429b014d02396d9970d2b2d1a4ea98e2c.

Functions: solve_bc(rhs,h):).

Classes: none. Imports: from pathlib import Path; import json,hashlib; import numpy as np; from scipy.signal import fftconvolve.

Opening source documentation: Exploratory fixed-box resolution audit with in-box Green boundary (not prereg).

Static implementation signals (presence in source, not proof of execution): linear algebra, JSON output, hash verification, solver path.

Source excerpt (first 23 physical lines; whitespace and line order retained):

```
 1 #!/usr/bin/env python3
 2 """Exploratory fixed-box resolution audit with in-box Green boundary (not prereg)."""
 3 from pathlib import Path
 4 import json,hashlib
 5 import numpy as np
 6 from scipy.signal import fftconvolve
 7 R=Path('/mnt/data')
 8 source=R/'UDCT_v7_13_Hydrus_3D_DeltaPotential_Exploratory.py'
 9 s=source.read_text();s=s[:s.index("out={'status'")];s=s.replace(' pot=solve(rhs,h)', ' return rhs,h'
10 ns={'__file__':str(source)};exec(compile(s,str(source),'exec'),ns)
11 m=ns['m'];rh=ns['rh'];ns['run'].__globals__['L']=6*rh
12
13 def solve_bc(rhs,h):
14  n=rhs.shape[0];N=n+2
15  padded=np.pad(rhs,1)
16  offs=np.arange(-(N-1),N,dtype=float)*h
17  d2=offs[:,None,None]**2+offs[None,:,None]**2+offs[None,None,:]**2
18  ker=np.zeros_like(d2);mask=d2>0;ker[mask]=-h**3/(4*np.pi*np.sqrt(d2[mask]))
19  full=fftconvolve(padded,ker,mode='same')
20  rhs_bc=rhs.copy();hh=h*h
21  for ax in range(3):
22   low=np.take(full,0,axis=ax)[1:-1,1:-1];high=np.take(full,-1,axis=ax)[1:-1,1:-1]
23   if ax==0:rhs_bc[0,:,:]-=low/hh;rhs_bc[-1,:,:]-=high/hh
```

Audit interpretation: Identify the exact inputs, units, normalization and frozen constants in the full script. Determine whether this file computes a new physical profile, reads prior profiles, or only checks a manufactured numerical identity. Inspect the handling of exceptions, finite values, boundaries and convergence; a successful process exit alone is not a scientific validation.

Reproduction note: compare any outputs with the companion JSON and/or NPZ by exact archive name and digest. The filename timestamp is metadata, not proof of run chronology. No independent execution of this script is claimed in this companion.

# Source 24/77 — UDCT_v7_13_Direct_Source_Boundary_Audit

Full archive filename: UDCT_v7_13_Direct_Source_Boundary_Audit_20261021_011820_499.py

Source length: 76 lines; bytes: 4232; SHA-256: 7e37682cc0de3e6428f4bebae2178fd252d14d678fe7de645a4ef59ca3fcd6e1.

Functions: solve_dirichlet(rhs,h,faces):); calculate(lawname,side,n):); multipole(pos,degree):); faces_for(degree):).

Classes: none. Imports: import json,hashlib; from pathlib import Path; import numpy as np; from scipy.signal import fftconvolve.

Opening source documentation: Exploratory: compare direct in-box source boundary to multipoles/zero boundary. Outside-box source omitted; not a physical completion or preregistered B01-B24 test.

Static implementation signals (presence in source, not proof of execution): linear algebra, JSON output, hash verification, solver path.

Source excerpt (first 23 physical lines; whitespace and line order retained):

```
#!/usr/bin/env python3
"""Exploratory: compare direct in-box source boundary to multipoles/zero boundary.
Outside-box source omitted; not a physical completion or preregistered B01-B24 test.
"""
import json,hashlib
from pathlib import Path
import numpy as np
from scipy.signal import fftconvolve
R=Path('/mnt/data')
source=R/'UDCT_v7_13_Hydrus_3D_DeltaPotential_Exploratory.py'
s=source.read_text();s=s[:s.index("out={'status'")];s=s.replace(' pot=solve(rhs,h)', ' return rhs,h'
ns={'__file__':str(source)};exec(compile(s,str(source),'exec'),ns)
m=ns['m'];rh=ns['rh']

def solve_dirichlet(rhs,h,faces):
 v=rhs.copy(); hh=h*h
 for axis,(low,high) in enumerate(faces):
  if axis==0:v[0,:,:]-=low/hh;v[-1,:,:]-=high/hh
  elif axis==1:v[:,0,:]-=low/hh;v[:,-1,:]-=high/hh
  else:v[:,:,0]-=low/hh;v[:,:,-1]-=high/hh
 return ns['solve'](v,h)

def calculate(lawname,side,n):
```

Audit interpretation: Identify the exact inputs, units, normalization and frozen constants in the full script. Determine whether this file computes a new physical profile, reads prior profiles, or only checks a manufactured numerical identity. Inspect the handling of exceptions, finite values, boundaries and convergence; a successful process exit alone is not a scientific validation.

Reproduction note: compare any outputs with the companion JSON and/or NPZ by exact archive name and digest. The filename timestamp is metadata, not proof of run chronology. No independent execution of this script is claimed in this companion.

# Source 25/77 — UDCT_v7_13_Fixed_Box_Resolution_Audit

Full archive filename: UDCT_v7_13_Fixed_Box_Resolution_Audit_20261021_011901_260.py

Source length: 22 lines; bytes: 1521; SHA-256: 1dd084d6a784cd74537f79ec374d2731d40c489046fb682b629176c10a02f5b7.

Functions: No named function definitions found.

Classes: none. Imports: import json, hashlib; from pathlib import Path.

Opening source documentation: Exploratory fixed-box resolution audit, NOT physical boundary validation.

Static implementation signals (presence in source, not proof of execution): JSON output, hash verification, solver path.

Source excerpt (first 23 physical lines; whitespace and line order retained):

```
 1 #!/usr/bin/env python3
 2 """Exploratory fixed-box resolution audit, NOT physical boundary validation."""
 3 import json, hashlib
 4 from pathlib import Path
 5 P=Path('/mnt/data/UDCT_v7_13_Hydrus_3D_DeltaPotential_Exploratory.py')
 6 s=P.read_text(); s=s[:s.index("out={'status'")]
 7 ns={'__file__':str(P)}
 8 exec(compile(s,str(P),'exec'),ns)
 9 rows=[]
10 for lawname in ('A-Qs','A-f1/2'):
11  law=next(x for x in ns['m'].PART_A_LAWS if x.name==lawname)
12  for n in (23,35,47):
13   r=ns['run'](n,law,10.)
14   rows.append(r)
15   print(lawname,n,r['center_delta_phi_over_a0_rh'],r['center_delta_acceleration_over_a0'],flush=True
16 summary={}
17 for lawname in ('A-Qs','A-f1/2'):
18  v=[x for x in rows if x['law']==lawname]
19  summary[lawname]={"phi_relative_change_35_to_47":(v[2]['center_delta_phi_over_a0_rh']-v[1]['center_
20 out={'status':'EXPLORATORY_NOT_PREREG_NOT_PHYSICAL_BOUNDARY','method':'fixed cube side 6 rh; n=23,35
21 q=Path('/mnt/data/UDCT_v7_13_Fixed_Box_Resolution_Audit.json');q.write_text(json.dumps(out,indent=2)
22 print('saved',q)
```

Audit interpretation: Identify the exact inputs, units, normalization and frozen constants in the full script. Determine whether this file computes a new physical profile, reads prior profiles, or only checks a manufactured numerical identity. Inspect the handling of exceptions, finite values, boundaries and convergence; a successful process exit alone is not a scientific validation.

Reproduction note: compare any outputs with the companion JSON and/or NPZ by exact archive name and digest. The filename timestamp is metadata, not proof of run chronology. No independent execution of this script is claimed in this companion.

# Source 26/77 — UDCT_v7_13_Green_Cell_Quadrature_Audit

Full archive filename: UDCT_v7_13_Green_Cell_Quadrature_Audit_20261021_011804_539.py

Source length: 32 lines; bytes: 2278; SHA-256: 2e4d2036a8453e8b0e803f20e04053f7fd1773ba909928b50878a60fc8e87c63.

Functions: cube_kernel(distance_in_h, order):.

Classes: none. Imports: import json, hashlib; from pathlib import Path; import numpy as np; from numpy.polynomial.legendre import leggauss.

Opening source documentation: Standalone local Green kernel cell quadrature audit; NOT a full Hydrus solver. Compare midpoint approximation of 1/r over a cubic source cell against tensor Gauss quadrature. Boundary nodes of existing solver are >= one grid spacing from source-cell centers.

Static implementation signals (presence in source, not proof of execution): linear algebra, JSON output, hash verification, solver path.

Source excerpt (first 23 physical lines; whitespace and line order retained):

```
 1 #!/usr/bin/env python3
 2 """Standalone local Green kernel cell quadrature audit; NOT a full Hydrus solver.
 3 Compare midpoint approximation of 1/r over a cubic source cell against tensor Gauss quadrature.
 4 Boundary nodes of existing solver are >= one grid spacing from source-cell centers.
 5 """
 6 import json, hashlib
 7 from pathlib import Path
 8 import numpy as np
 9 from numpy.polynomial.legendre import leggauss
10
11 def cube_kernel(distance_in_h, order):
12     x,w=leggauss(order)
13     x=x/2; w=w/2
14     xx,yy,zz=np.meshgrid(x,x,x,indexing='ij')
15     ww=w[:,None,None]*w[None,:,None]*w[None,None,:]
16     r=np.sqrt((distance_in_h[0]-xx)**2+(distance_in_h[1]-yy)**2+(distance_in_h[2]-zz)**2)
17     return float(np.sum(ww/r))
18
19 cases={'face_nearest':(1.,0.,0.),'face_diagonal':(1.,1.,0.),'face_3d_diagonal':(1.,1.,1.),'face_2h':
20 rows=[]
21 for name,coords in cases.items():
22     vals={str(n):cube_kernel(coords,n) for n in (2,4,8,12,20)}
23     point=1/np.linalg.norm(coords)
```

Audit interpretation: Identify the exact inputs, units, normalization and frozen constants in the full script. Determine whether this file computes a new physical profile, reads prior profiles, or only checks a manufactured numerical identity. Inspect the handling of exceptions, finite values, boundaries and convergence; a successful process exit alone is not a scientific validation.

Reproduction note: compare any outputs with the companion JSON and/or NPZ by exact archive name and digest. The filename timestamp is metadata, not proof of run chronology. No independent execution of this script is claimed in this companion.

# Source 27/77 — UDCT_v7_13_Hessian_Equivalent_Density_Audit

Full archive filename: UDCT_v7_13_Hessian_Equivalent_Density_Audit_20261021_012046_765.py

Source length: 31 lines; bytes: 1430; SHA-256: 10072e73fbd6217c98ca935f3c9c00b08448860ce5f0310417a469b1bcc13e8e.

Functions: main():).

Classes: none. Imports: import json, math; from pathlib import Path.

Opening source documentation: Reproduce equivalent Newtonian densities of the manufactured quadratic field. This is not a Hydrus I observation, simulation, or GQUMOND inference.

Static implementation signals (presence in source, not proof of execution): JSON output, entry-point guard.

Source excerpt (first 23 physical lines; whitespace and line order retained):

```
#!/usr/bin/env python3
"""Reproduce equivalent Newtonian densities of the manufactured quadratic field.
This is not a Hydrus I observation, simulation, or GQUMOND inference.
"""
import json, math
from pathlib import Path
G_SI=6.67430e-11
PC_M=3.085677581491367e16
MSUN_KG=1.98847e30
BASE_H_DIAG_S2=[1.1e-30,1.6e-30,2.3e-30]

def main():
    rows=[]
    for factor in (1,10,100):
        trace=sum(BASE_H_DIAG_S2)*factor
        rho_si=trace/(4*math.pi*G_SI)
        rho_msun_pc3=rho_si*PC_M**3/MSUN_KG
        rows.append({'hessian_multiplier':factor,'trace_H_s_minus_2':trace,
                     'rho_equivalent_kg_m_minus_3':rho_si,
                     'rho_equivalent_Msun_pc_minus_3':rho_msun_pc3})
    result={'title':'UDCT v7.13 manufactured Hessian equivalent-density audit',
            'scope':'Algebraic Newtonian consistency check only; NOT measured Hydrus I density',
            'source_field':'H=diag(1.1,1.6,2.3) * 1e-30 s^-2 * multiplier',
```

Audit interpretation: Identify the exact inputs, units, normalization and frozen constants in the full script. Determine whether this file computes a new physical profile, reads prior profiles, or only checks a manufactured numerical identity. Inspect the handling of exceptions, finite values, boundaries and convergence; a successful process exit alone is not a scientific validation.

Reproduction note: compare any outputs with the companion JSON and/or NPZ by exact archive name and digest. The filename timestamp is metadata, not proof of run chronology. No independent execution of this script is claimed in this companion.

# Source 28/77 — UDCT_v7_13_Hessian_External_Field_Audit

Full archive filename: UDCT_v7_13_Hessian_External_Field_Audit_20261021_012216_909.py

Source length: 39 lines; bytes: 2809; SHA-256: 27f5c1e9cea2895a36e28d1fea721fdfca3013e86f20b2d6e38683ea43ed05d5.

Functions: No named function definitions found.

Classes: none. Imports: import hashlib, importlib.util, json, sys; from pathlib import Path; import numpy as np.

Opening source documentation: Exploratory REAL Hydrus I external Newtonian tidal Hessian audit. NOT a total modified-gravity Hessian, NOT GQUMOND validation.

Static implementation signals (presence in source, not proof of execution): linear algebra, JSON output, internal assertion, hash verification, solver path.

Source excerpt (first 23 physical lines; whitespace and line order retained):

```
 1 #!/usr/bin/env python3
 2 """Exploratory REAL Hydrus I external Newtonian tidal Hessian audit.
 3 NOT a total modified-gravity Hessian, NOT GQUMOND validation.
 4 """
 5 import hashlib, importlib.util, json, sys
 6 from pathlib import Path
 7 import numpy as np
 8 ROOT=Path('/mnt/data/v713_hessian_stage')
 9 P=Path('/mnt/data/UDCT_v7_13_Preregistration_Updated/preregistration_plan.json')
10 plan=json.loads(P.read_text())
11 checks={}
12 for name,sha in plan['frozen_hashes'].items():
13     file=ROOT/name
14     if not file.exists(): file=Path('/mnt/data/v713_b07_run')/name
15     checks[name]=file.exists() and hashlib.sha256(file.read_bytes()).hexdigest()==sha
16 assert all(checks.values()),checks
17 spec=importlib.util.spec_from_file_location('frozen_v706',ROOT/'UDCT_v7_06_Hessian_Term_Sign_Audit_r
18 m=importlib.util.module_from_spec(spec);sys.modules[spec.name]=m;spec.loader.exec_module(m)
19 obj=m.DWARFS['Hydrus I']
20 # Grid radial/polar/azimuth not used by galactic_field; only disk quadrature changes.
21 grids=[('default',m.Grid(disk_radial=120,disk_azimuth=64,disk_height=12)),('refined',m.Grid(disk_rad
22 records=[]
23 for name,g in grids:
```

Audit interpretation: Identify the exact inputs, units, normalization and frozen constants in the full script. Determine whether this file computes a new physical profile, reads prior profiles, or only checks a manufactured numerical identity. Inspect the handling of exceptions, finite values, boundaries and convergence; a successful process exit alone is not a scientific validation.

Reproduction note: compare any outputs with the companion JSON and/or NPZ by exact archive name and digest. The filename timestamp is metadata, not proof of run chronology. No independent execution of this script is claimed in this companion.

# Source 29/77 — UDCT_v7_13_Hessian_Factor_Grid_Convergence

Full archive filename: UDCT_v7_13_Hessian_Factor_Grid_Convergence_20261021_012100_129.py

Source length: 69 lines; bytes: 3729; SHA-256: 67be43fcea2bc12dffa2fc4754b9b040de2f5fac5fcaeb41880b8b0415188331.

Functions: solve(rhs,h):); run(n,law,ell_pc,hfactor):).

Classes: none. Imports: import json,zipfile,hashlib,importlib.util,sys; from pathlib import Path; import numpy as np; from scipy.fft import dstn,idstn.

Opening source documentation: Exploratory grid convergence of factor-correction sensitivity; not Hydrus I.

Static implementation signals (presence in source, not proof of execution): linear algebra, JSON output, internal assertion, hash verification, solver path.

Source excerpt (first 23 physical lines; whitespace and line order retained):

```
 1 #!/usr/bin/env python3
 2 """Exploratory grid convergence of factor-correction sensitivity; not Hydrus I."""
 3 import json,zipfile,hashlib,importlib.util,sys
 4 from pathlib import Path
 5 import numpy as np
 6 from scipy.fft import dstn,idstn
 7 Z=Path('/mnt/data/UDCT_v7_13_Preregistration_Complete_Updated.zip')
 8 with zipfile.ZipFile(Z) as z:
 9  plan=json.loads(z.read('preregistration_plan.json'))
10  for name,sha in plan['frozen_hashes'].items():
11   assert hashlib.sha256(z.read(name)).hexdigest()==sha,name
12  src=z.read('UDCT_v7_10_Postregistration_run.py');base=z.read('UDCT_v7_06_Hessian_Term_Sign_Audit_re
13 R=Path('/mnt/data/v713_grid_convergence');R.mkdir(exist_ok=True)
14 P=R/'UDCT_v7_10_Postregistration_run.py';P.write_bytes(src)
15 (R/'UDCT_v7_06_Hessian_Term_Sign_Audit_reproduce.py').write_bytes(base)
16 spec=importlib.util.spec_from_file_location('v710_grid_audit',P)
17 m=importlib.util.module_from_spec(spec);sys.modules[spec.name]=m;spec.loader.exec_module(m)
18 L=1.2*m.PC
19
20 def solve(rhs,h):
21  n=rhs.shape[0];k=np.arange(1,n+1);ev=2*(np.cos(np.pi*k/(n+1))-1)/h**2
22  return idstn(dstn(rhs,type=1,norm='ortho')/(ev[:,None,None]+ev[None,:,None]+ev[None,None,:]),type=1
23
```

Audit interpretation: Identify the exact inputs, units, normalization and frozen constants in the full script. Determine whether this file computes a new physical profile, reads prior profiles, or only checks a manufactured numerical identity. Inspect the handling of exceptions, finite values, boundaries and convergence; a successful process exit alone is not a scientific validation.

Reproduction note: compare any outputs with the companion JSON and/or NPZ by exact archive name and digest. The filename timestamp is metadata, not proof of run chronology. No independent execution of this script is claimed in this companion.

# Source 30/77 — UDCT_v7_13_Hessian_Factor_Stress_Audit

Full archive filename: UDCT_v7_13_Hessian_Factor_Stress_Audit_20261021_012105_059.py

Source length: 41 lines; bytes: 3177; SHA-256: 620479a54966e61192f96f2323836d3483dd42a97f33905ab02a017429c0da5b.

Functions: solve(rhs,h):); test(n,law,ell_pc,Hfactor):).

Classes: none. Imports: import zipfile,hashlib,json,importlib.util,sys; from pathlib import Path; import numpy as np; from scipy.fft import dstn,idstn.

Opening source documentation: Exploratory Hessian-factor stress test; manufactured 3D quadratic Newtonian field, not Hydrus I.

Static implementation signals (presence in source, not proof of execution): linear algebra, JSON output, internal assertion, hash verification, solver path.

Source excerpt (first 23 physical lines; whitespace and line order retained):

```
 1 #!/usr/bin/env python3
 2 """Exploratory Hessian-factor stress test; manufactured 3D quadratic Newtonian field, not Hydrus I."
 3 import zipfile,hashlib,json,importlib.util,sys
 4 from pathlib import Path
 5 import numpy as np
 6 from scipy.fft import dstn,idstn
 7 Z=Path('/mnt/data/UDCT_v7_13_Preregistration_Complete_Updated.zip')
 8 with zipfile.ZipFile(Z) as z:
 9  plan=json.loads(z.read('preregistration_plan.json'))
10  for name,sha in plan['frozen_hashes'].items():assert hashlib.sha256(z.read(name)).hexdigest()==sha
11  source=z.read('UDCT_v7_10_Postregistration_run.py')
12  v706=z.read('UDCT_v7_06_Hessian_Term_Sign_Audit_reproduce.py')
13 P=Path('/mnt/data/v713_action_audit/UDCT_v7_10_Postregistration_run.py');P.parent.mkdir(exist_ok=Tru
14 spec=importlib.util.spec_from_file_location('v710_stress',P);m=importlib.util.module_from_spec(spec)
15 def solve(rhs,h):
16  n=rhs.shape[0];k=np.arange(1,n+1);v=2*(np.cos(np.pi*k/(n+1))-1)/h**2
17  return idstn(dstn(rhs,type=1,norm='ortho')/(v[:,None,None]+v[None,:,None]+v[None,None,:]),type=1,no
18 def test(n,law,ell_pc,Hfactor):
19  L=1.2*m.PC;h=L/(n+1);q=(np.arange(-1,n+3)*h-L/2)
20  x,y,z=np.meshgrid(q,q,q,indexing='ij')
21  H=np.diag(np.array([1.1,1.6,2.3])*1e-30*Hfactor);g0=np.array([1.4,-1.,.7])*1e-11
22  g=np.stack([g0[0]+H[0,0]*x,g0[1]+H[1,1]*y,g0[2]+H[2,2]*z]);g2=np.sum(g*g,axis=0)
23  nu,wh=m.gq_kernel(g2,np.array(np.sum(H*H)),ell_pc*m.PC,law)
```

Audit interpretation: Identify the exact inputs, units, normalization and frozen constants in the full script. Determine whether this file computes a new physical profile, reads prior profiles, or only checks a manufactured numerical identity. Inspect the handling of exceptions, finite values, boundaries and convergence; a successful process exit alone is not a scientific validation.

Reproduction note: compare any outputs with the companion JSON and/or NPZ by exact archive name and digest. The filename timestamp is metadata, not proof of run chronology. No independent execution of this script is claimed in this companion.

# Source 31/77 — UDCT_v7_13_Hessian_Preflight_and_Estimator_Selftest

Full archive filename: UDCT_v7_13_Hessian_Preflight_and_Estimator_Selftest_20261021_012224_191.py

Source length: 57 lines; bytes: 3508; SHA-256: 7fe6f26020246587fe891b641df164a8bf775fb3fea463484f3e58f0245b6977.

Functions: hessian(phi,x,h):.

Classes: none. Imports: from pathlib import Path; import hashlib, json, ast; import numpy as np.

Opening source documentation: Non-confirmatory preflight: frozen-source audit and synthetic 3-D Hessian estimator tests. Does NOT calculate a Hydrus I total-potential Hessian or any B01-B24 Hessian outcome.

Static implementation signals (presence in source, not proof of execution): JSON output, internal assertion, hash verification, solver path.

Source excerpt (first 23 physical lines; whitespace and line order retained):

```
#!/usr/bin/env python3
"""Non-confirmatory preflight: frozen-source audit and synthetic 3-D Hessian estimator tests.
Does NOT calculate a Hydrus I total-potential Hessian or any B01-B24 Hessian outcome.
"""
from pathlib import Path
import hashlib, json, ast
import numpy as np
ROOT=Path('/mnt/data/v713_b07_run')
PLAN=json.loads((ROOT/'preregistration_plan.json').read_text())
verified={name: hashlib.sha256((ROOT/name).read_bytes()).hexdigest()==digest for name,digest in PLAN
assert all(verified.values())
cp=PLAN['complete_protocol']
steps=cp['hessian_steps_rh']
assert steps==[0.02,0.01,0.005]
# Generic central-difference tensor, not a galaxy solver.
def hessian(phi,x,h):
    x=np.array(x,dtype=float); out=np.zeros((3,3)); f0=phi(x)
    for i in range(3):
        ei=np.eye(3)[i]*h
        out[i,i]=(phi(x+ei)-2*f0+phi(x-ei))/h**2
        for j in range(i):
            ej=np.eye(3)[j]*h
            out[i,j]=out[j,i]=(phi(x+ei+ej)-phi(x+ei-ej)-phi(x-ei+ej)+phi(x-ei-ej))/(4*h*h)
```

Audit interpretation: Identify the exact inputs, units, normalization and frozen constants in the full script. Determine whether this file computes a new physical profile, reads prior profiles, or only checks a manufactured numerical identity. Inspect the handling of exceptions, finite values, boundaries and convergence; a successful process exit alone is not a scientific validation.

Reproduction note: compare any outputs with the companion JSON and/or NPZ by exact archive name and digest. The filename timestamp is metadata, not proof of run chronology. No independent execution of this script is claimed in this companion.

# Source 32/77 — UDCT_v7_13_Hessian_Preflight_and_Estimator_Selftest

Full archive filename: UDCT_v7_13_Hessian_Preflight_and_Estimator_Selftest_20261021_012458_489.py

Source length: 57 lines; bytes: 3508; SHA-256: 7fe6f26020246587fe891b641df164a8bf775fb3fea463484f3e58f0245b6977.

Functions: hessian(phi,x,h):.

Classes: none. Imports: from pathlib import Path; import hashlib, json, ast; import numpy as np.

Opening source documentation: Non-confirmatory preflight: frozen-source audit and synthetic 3-D Hessian estimator tests. Does NOT calculate a Hydrus I total-potential Hessian or any B01-B24 Hessian outcome.

Static implementation signals (presence in source, not proof of execution): JSON output, internal assertion, hash verification, solver path.

Source excerpt (first 23 physical lines; whitespace and line order retained):

```
#!/usr/bin/env python3
"""Non-confirmatory preflight: frozen-source audit and synthetic 3-D Hessian estimator tests.
Does NOT calculate a Hydrus I total-potential Hessian or any B01-B24 Hessian outcome.
"""
from pathlib import Path
import hashlib, json, ast
import numpy as np
ROOT=Path('/mnt/data/v713_b07_run')
PLAN=json.loads((ROOT/'preregistration_plan.json').read_text())
verified={name: hashlib.sha256((ROOT/name).read_bytes()).hexdigest()==digest for name,digest in PLAN
assert all(verified.values())
cp=PLAN['complete_protocol']
steps=cp['hessian_steps_rh']
assert steps==[0.02,0.01,0.005]
# Generic central-difference tensor, not a galaxy solver.
def hessian(phi,x,h):
    x=np.array(x,dtype=float); out=np.zeros((3,3)); f0=phi(x)
    for i in range(3):
        ei=np.eye(3)[i]*h
        out[i,i]=(phi(x+ei)-2*f0+phi(x-ei))/h**2
        for j in range(i):
            ej=np.eye(3)[j]*h
            out[i,j]=out[j,i]=(phi(x+ei+ej)-phi(x+ei-ej)-phi(x-ei+ej)+phi(x-ei-ej))/(4*h*h)
```

Audit interpretation: Identify the exact inputs, units, normalization and frozen constants in the full script. Determine whether this file computes a new physical profile, reads prior profiles, or only checks a manufactured numerical identity. Inspect the handling of exceptions, finite values, boundaries and convergence; a successful process exit alone is not a scientific validation.

Reproduction note: compare any outputs with the companion JSON and/or NPZ by exact archive name and digest. The filename timestamp is metadata, not proof of run chronology. No independent execution of this script is claimed in this companion.

# Source 33/77 — UDCT_v7_13_Hydrus_3D_Angular_Newtonian_Gate

Full archive filename: UDCT_v7_13_Hydrus_3D_Angular_Newtonian_Gate_20261021_011958_087.py

Source length: 40 lines; bytes: 2852; SHA-256: 9e7369841bb9d940ba3e835fd98e9c02e1eb265b3d773374db9c47cec2c9c359.

Functions: No named function definitions found.

Classes: none. Imports: import json, hashlib; from pathlib import Path; import numpy as np.

Opening source documentation: Reproducible 3D directional NEWTONIAN input audit, NOT GQUMOND/Jeans.

Static implementation signals (presence in source, not proof of execution): linear algebra, JSON output, internal assertion, hash verification, solver path.

Source excerpt (first 23 physical lines; whitespace and line order retained):

```
 1 #!/usr/bin/env python3
 2 """Reproducible 3D directional NEWTONIAN input audit, NOT GQUMOND/Jeans."""
 3 import json, hashlib
 4 from pathlib import Path
 5 import numpy as np
 6 P=Path(__file__).resolve().parent
 7 names=['UDCT_v7_13_Hydrus_Plummer_3D_Hessian_Control.json','UDCT_v7_13_Hydrus_External_Newtonian_Hes
 8 I,E=[json.loads((P/n).read_text()) for n in names]
 9 G=6.67430e-11; pc=3.085677581491367e16; a0=1.2e-10
10 M=float(I['mass_kg']); b=float(I['b_pc'])*pc; rh=float(I['rh_pc'])*pc
11 T=np.array(E['grid_results'][-1]['hessian_s_minus2'],float)
12 g0=np.array(E['grid_results'][-1]['gradient_m_s2'],float)
13 assert np.allclose(T,T.T,rtol=0,atol=1e-43)
14 N=4096; k=np.arange(N); zz=1-2*(k+.5)/N; th=k*np.pi*(3-np.sqrt(5)); u=np.stack([np.sqrt(1-zz**2)*np.
15 rows=[]
16 for factor in [0.5,1,2,3]:
17  r=factor*rh; x=r*u
18  gi=G*M*x/(r*r+b*b)**1.5
19  tide=x@T.T
20  gt=gi+tide+g0
21  Hinner=G*M/(r*r+b*b)**1.5*np.eye(3)[None,:,:]-3*G*M/(r*r+b*b)**2.5*x[:,:,None]*x[:,None,:]
22  Htot=Hinner+T
23  norm=lambda a:np.linalg.norm(a,axis=-1)
```

Audit interpretation: Identify the exact inputs, units, normalization and frozen constants in the full script. Determine whether this file computes a new physical profile, reads prior profiles, or only checks a manufactured numerical identity. Inspect the handling of exceptions, finite values, boundaries and convergence; a successful process exit alone is not a scientific validation.

Reproduction note: compare any outputs with the companion JSON and/or NPZ by exact archive name and digest. The filename timestamp is metadata, not proof of run chronology. No independent execution of this script is claimed in this companion.

# Source 34/77 — UDCT_v7_13_Hydrus_3D_DeltaPotential_Exploratory

Full archive filename: UDCT_v7_13_Hydrus_3D_DeltaPotential_Exploratory_20261021_011907_412.py

Source length: 70 lines; bytes: 4784; SHA-256: 531e28f3e5bae1643c19c4f8e8b43c0e0c933f873c101dcfbb3cf5921ce8dc80.

Functions: solve(rhs,h):); lap(u,h):); run(n,law,ellpc):); shift(i,k):); sl(offset):return tuple(slice(2+d,2+d+n) for d in offset)).

Classes: none. Imports: import json, hashlib, importlib.util,sys; from pathlib import Path; import numpy as np; from scipy.fft import dstn,idstn.

Opening source documentation: Exploratory 3D delta-potential: frozen v7.10 GQUMOND minus QUMOND. Zero Dirichlet delta potential on finite cube: NOT physical boundary / not prereg test.

Static implementation signals (presence in source, not proof of execution): linear algebra, JSON output, internal assertion, hash verification, solver path.

Source excerpt (first 23 physical lines; whitespace and line order retained):

```
 1 #!/usr/bin/env python3
 2 """Exploratory 3D delta-potential: frozen v7.10 GQUMOND minus QUMOND.
 3 Zero Dirichlet delta potential on finite cube: NOT physical boundary / not prereg test.
 4 """
 5 import json, hashlib, importlib.util,sys
 6 from pathlib import Path
 7 import numpy as np
 8 from scipy.fft import dstn,idstn
 9 R=Path('/mnt/data');P=R/'UDCT_v7_10_Postregistration_run.py';D=R/'UDCT_v7_06_Hessian_Term_Sign_Audit
10 sha=lambda p:hashlib.sha256(p.read_bytes()).hexdigest()
11 assert sha(D)=='a64440e1350ae8bf10eb3188ec34829b6d586abf275de5cdc841a50ad3478284'
12 assert sha(P)=='a4d1a07ebcaa06348be3022d0f1f6ca463347e00194f7fde59f2bd9dbcd521b7'
13 spec=importlib.util.spec_from_file_location('udct_frozen_delta',P);m=importlib.util.module_from_spec
14 obj=m.DWARFS['Hydrus I'];M=m.ML*obj.luminosity*m.MSUN;b=obj.re_pc*m.PC;rh=b/np.sqrt(2**(2/3)-1)
15 # Use default external grid, as previously tested against smoke
16 extgrid=m.Grid(radial=180,polar=16,azimuth=24,disk_radial=120,disk_azimuth=64,disk_height=12)
17 e,H,_=m.V706.galactic_field(obj,extgrid)
18 L=6*rh
19
20 def solve(rhs,h):
21  n=rhs.shape[0];k=np.arange(1,n+1);ev=2*(np.cos(np.pi*k/(n+1))-1)/h**2
22  return idstn(dstn(rhs,type=1,norm='ortho')/(ev[:,None,None]+ev[None,:,None]+ev[None,None,:]),type=1
23
```

Audit interpretation: Identify the exact inputs, units, normalization and frozen constants in the full script. Determine whether this file computes a new physical profile, reads prior profiles, or only checks a manufactured numerical identity. Inspect the handling of exceptions, finite values, boundaries and convergence; a successful process exit alone is not a scientific validation.

Reproduction note: compare any outputs with the companion JSON and/or NPZ by exact archive name and digest. The filename timestamp is metadata, not proof of run chronology. No independent execution of this script is claimed in this companion.

# Source 35/77 — UDCT_v7_13_Hydrus_Box_Boundary_Sensitivity

Full archive filename: UDCT_v7_13_Hydrus_Box_Boundary_Sensitivity_20261021_011904_569.py

Source length: 27 lines; bytes: 1632; SHA-256: 071209f8821298647e50e18c0f0cc5977376dd5a51f8f90df4c301bb5fb42c5d.

Functions: No named function definitions found.

Classes: none. Imports: import importlib.util,sys,json,hashlib; from pathlib import Path.

Opening source documentation: Exploratory finite-box sensitivity; reuses frozen v7.10 source and prior 3D solver. This is NOT a physical exterior boundary solution or registered B01-B24 validation.

Static implementation signals (presence in source, not proof of execution): JSON output, hash verification, solver path.

Source excerpt (first 23 physical lines; whitespace and line order retained):

```
#!/usr/bin/env python3
"""Exploratory finite-box sensitivity; reuses frozen v7.10 source and prior 3D solver.
This is NOT a physical exterior boundary solution or registered B01-B24 validation.
"""
import importlib.util,sys,json,hashlib
from pathlib import Path
p=Path('/mnt/data/UDCT_v7_13_Hydrus_3D_DeltaPotential_Exploratory.py')
src=p.read_text(); src=src[:src.index("out={'status'")]
# Import definitions without executing the previous campaign.
ns={'__file__':str(p)}
exec(compile(src,str(p),'exec'),ns)
m=ns['m'];rh=ns['rh'];R=Path('/mnt/data')
rows=[]
# Fixed spacing = rh/6, box side = 4,6,8 rh, odd n centers exactly at origin
for lawname in ('A-Qs','A-f1/2'):
 law=next(z for z in m.PART_A_LAWS if z.name==lawname)
 for ell in (1.,10.):
  for box in (4.,6.,8.):
   n=int(6*box)-1
   ns['L']=box*rh
   # run function resolves global namespace ns
   r=ns['run'](n,law,ell)
   r['box_side_over_rh']=box
```

Audit interpretation: Identify the exact inputs, units, normalization and frozen constants in the full script. Determine whether this file computes a new physical profile, reads prior profiles, or only checks a manufactured numerical identity. Inspect the handling of exceptions, finite values, boundaries and convergence; a successful process exit alone is not a scientific validation.

Reproduction note: compare any outputs with the companion JSON and/or NPZ by exact archive name and digest. The filename timestamp is metadata, not proof of run chronology. No independent execution of this script is claimed in this companion.

# Source 36/77 — UDCT_v7_13_Hydrus_Common_Frame_Newtonian_Audit

Full archive filename: UDCT_v7_13_Hydrus_Common_Frame_Newtonian_Audit_20261021_012020_218.py

Source length: 47 lines; bytes: 3842; SHA-256: d4fceefd7ae6cae91fcfdb7db7b3e06db445e891571e20aadb7a9fe6fa2de9ce.

Functions: h_in(n):return A*np.eye(3)-B*np.outer(n,n)); metrics(n):).

Classes: none. Imports: import json, numpy as np; from pathlib import Path.

Opening source documentation: Common Galactocentric frame Newtonian Hessian audit. Not a GQUMOND field solve. At Plummer radius r_h, evaluate stellar Hessian for radial direction pointing from Hydrus I towards Galactic center; also sample sphere for directional sensitivity. The external Hessian is held spatially constant (tidal approximation).

Static implementation signals (presence in source, not proof of execution): linear algebra, JSON output, internal assertion, solver path.

Source excerpt (first 23 physical lines; whitespace and line order retained):

```
#!/usr/bin/env python3
"""Common Galactocentric frame Newtonian Hessian audit. Not a GQUMOND field solve.
At Plummer radius r_h, evaluate stellar Hessian for radial direction pointing
from Hydrus I towards Galactic center; also sample sphere for directional sensitivity.
The external Hessian is held spatially constant (tidal approximation).
"""
import json, numpy as np
from pathlib import Path
P=Path(__file__).resolve().parent
I=json.loads((P/'UDCT_v7_13_Hydrus_Plummer_3D_Hessian_Control.json').read_text())
E=json.loads((P/'UDCT_v7_13_Hydrus_External_Newtonian_Hessian_Exploratory.json').read_text())
G=6.67430e-11; pc=3.085677581491367e16
M=I['mass_kg']; b=I['b_pc']*pc; r=I['rh_pc']*pc
Hext=np.array(E['grid_results'][-1]['hessian_s_minus2'])
R=np.array(E['hydrus_galactocentric_kpc'],float)
n=-R/np.linalg.norm(R)
# Hessian Phi=-GM/sqrt(r²+b²): GM/(r²+b²)^3/2 I - 3 GM r²/(r²+b²)^5/2 n n^T
A=G*M/(r*r+b*b)**1.5
B=3*G*M*r*r/(r*r+b*b)**2.5
def h_in(n):return A*np.eye(3)-B*np.outer(n,n)
def metrics(n):
 h=h_in(n); tot=h+Hext
 return {'internal_norm_s_minus2':float(np.linalg.norm(h)), 'total_norm_s_minus2':float(np.linalg.no
```

Audit interpretation: Identify the exact inputs, units, normalization and frozen constants in the full script. Determine whether this file computes a new physical profile, reads prior profiles, or only checks a manufactured numerical identity. Inspect the handling of exceptions, finite values, boundaries and convergence; a successful process exit alone is not a scientific validation.

Reproduction note: compare any outputs with the companion JSON and/or NPZ by exact archive name and digest. The filename timestamp is metadata, not proof of run chronology. No independent execution of this script is claimed in this companion.

# Source 37/77 — UDCT_v7_13_Hydrus_Ell_Sensitivity_Local_Source

Full archive filename: UDCT_v7_13_Hydrus_Ell_Sensitivity_Local_Source_20261021_011948_903.py

Source length: 65 lines; bytes: 4025; SHA-256: 3e61d6103c36d65899958f9d5b4e28331a68b972e8fa28e16f19613f559890be.

Functions: field(x):); constit(x,law,ell):); source(x,law,ell,step):).

Classes: none. Imports: import json,hashlib,sys,importlib.util; from pathlib import Path; import numpy as np.

Opening source documentation: Exploratory local 3D source diagnostic using frozen v7.10 law; NOT full 3D solution.

Static implementation signals (presence in source, not proof of execution): linear algebra, JSON output, internal assertion, hash verification, solver path.

Source excerpt (first 23 physical lines; whitespace and line order retained):

```
 1 #!/usr/bin/env python3
 2 """Exploratory local 3D source diagnostic using frozen v7.10 law; NOT full 3D solution."""
 3 import json,hashlib,sys,importlib.util
 4 from pathlib import Path
 5 import numpy as np
 6 ROOT=Path('/mnt/data')
 7 P=ROOT/'UDCT_v7_10_Postregistration_run.py'
 8 D=ROOT/'UDCT_v7_06_Hessian_Term_Sign_Audit_reproduce.py'
 9 assert hashlib.sha256(D.read_bytes()).hexdigest()=='a64440e1350ae8bf10eb3188ec34829b6d586abf275de5cd
10 spec=importlib.util.spec_from_file_location('frozen_udct_v710',P)
11 m=importlib.util.module_from_spec(spec);sys.modules[spec.name]=m;spec.loader.exec_module(m)
12 assert hashlib.sha256(P.read_bytes()).hexdigest()=='a4d1a07ebcaa06348be3022d0f1f6ca463347e00194f7fde
13 obj=m.DWARFS['Hydrus I']; grid=m.Grid(radial=180,polar=16,azimuth=24,disk_radial=40,disk_azimuth=24,
14 e,H,_=m.V706.galactic_field(obj,grid)
15 M=m.ML*obj.luminosity*m.MSUN;b=obj.re_pc*m.PC;rh=b/np.sqrt(2**(2/3)-1)
16 # Newtonian Plummer internal gradient and Hessian; Galactic Taylor field about dwarf center.
17 def field(x):
18  r2=float(x@x);q=r2+b*b
19  g=m.G*M*x/q**1.5+e+H@x
20  h=m.G*M*(np.eye(3)/q**1.5-3*np.outer(x,x)/q**2.5)+H
21  return g,h
22
23 def constit(x,law,ell):
```

Audit interpretation: Identify the exact inputs, units, normalization and frozen constants in the full script. Determine whether this file computes a new physical profile, reads prior profiles, or only checks a manufactured numerical identity. Inspect the handling of exceptions, finite values, boundaries and convergence; a successful process exit alone is not a scientific validation.

Reproduction note: compare any outputs with the companion JSON and/or NPZ by exact archive name and digest. The filename timestamp is metadata, not proof of run chronology. No independent execution of this script is claimed in this companion.

# Source 38/77 — UDCT_v7_13_Hydrus_External_Field_3D_Input_Gate

Full archive filename: UDCT_v7_13_Hydrus_External_Field_3D_Input_Gate_20261021_012002_349.py

Source length: 47 lines; bytes: 3904; SHA-256: fedde7a28704bee3fa1d26c771c79926864e680427959bbdf06de697618be7aa.

Functions: No named function definitions found.

Classes: none. Imports: import json, hashlib; from pathlib import Path; import numpy as np.

Opening source documentation: Exploratory 3D Newtonian external-field input gate; NOT a GQUMOND solver.

Static implementation signals (presence in source, not proof of execution): linear algebra, JSON output, internal assertion, hash verification, solver path.

Source excerpt (first 23 physical lines; whitespace and line order retained):

```
 1 #!/usr/bin/env python3
 2 """Exploratory 3D Newtonian external-field input gate; NOT a GQUMOND solver."""
 3 import json, hashlib
 4 from pathlib import Path
 5 import numpy as np
 6 P=Path(__file__).resolve().parent
 7 files=['UDCT_v7_13_Hydrus_Plummer_3D_Hessian_Control.json','UDCT_v7_13_Hydrus_External_Newtonian_Hes
 8 I,E=[json.loads((P/f).read_text()) for f in files]
 9 G=6.67430e-11; pc=3.085677581491367e16; a0=1.2e-10
10 M=float(I['mass_kg']); b=float(I['b_pc'])*pc; rh=float(I['rh_pc'])*pc
11 T=np.array(E['grid_results'][-1]['hessian_s_minus2'],float)
12 g0=np.array(E['grid_results'][-1]['gradient_m_s2'],float)
13 R=np.array(E['hydrus_galactocentric_kpc'],float); n=-R/np.linalg.norm(R)
14 assert np.allclose(T,T.T,atol=1e-43,rtol=0)
15 assert abs(np.trace(T))<1e-10*np.linalg.norm(T)
16 # Affine Taylor approximation to the EXTERNAL gradient, around the dwarf center.
17 # The stellar Plummer gradient is evaluated analytically in the SAME frame.
18 rows=[]
19 for direction,label in [(n,'toward_MW_center'),(-n,'away_from_MW_center'),(np.array([0.,0.,0.]),'cen
20  for scale in ([0.] if label=='center' else [0.5,1.,2.,3.]):
21   x=scale*rh*direction; r=np.linalg.norm(x)
22   gin=G*M*x/(r*r+b*b)**1.5
23   gout=g0+T@x
```

Audit interpretation: Identify the exact inputs, units, normalization and frozen constants in the full script. Determine whether this file computes a new physical profile, reads prior profiles, or only checks a manufactured numerical identity. Inspect the handling of exceptions, finite values, boundaries and convergence; a successful process exit alone is not a scientific validation.

Reproduction note: compare any outputs with the companion JSON and/or NPZ by exact archive name and digest. The filename timestamp is metadata, not proof of run chronology. No independent execution of this script is claimed in this companion.

# Source 39/77 — UDCT_v7_13_Hydrus_External_Field_vs_Tide_Audit

Full archive filename: UDCT_v7_13_Hydrus_External_Field_vs_Tide_Audit_20261021_012006_844.py

Source length: 35 lines; bytes: 3312; SHA-256: 1ddc7e6528796aa9e150734ed16aeceae0579448a18d3909e6589259718f0885.

Functions: No named function definitions found.

Classes: none. Imports: import json, hashlib; from pathlib import Path; import numpy as np.

Opening source documentation: Exploratory Newtonian Hydrus I: distinguish external uniform acceleration from tidal Hessian. Not a GQUMOND field solve, Jeans fit, or observational stellar velocity fit.

Static implementation signals (presence in source, not proof of execution): linear algebra, JSON output, internal assertion, hash verification, solver path.

Source excerpt (first 23 physical lines; whitespace and line order retained):

```
 1 #!/usr/bin/env python3
 2 """Exploratory Newtonian Hydrus I: distinguish external uniform acceleration from tidal Hessian.
 3 Not a GQUMOND field solve, Jeans fit, or observational stellar velocity fit.
 4 """
 5 import json, hashlib
 6 from pathlib import Path
 7 import numpy as np
 8 P=Path(__file__).resolve().parent
 9 files=['UDCT_v7_13_Hydrus_Plummer_3D_Hessian_Control.json','UDCT_v7_13_Hydrus_External_Newtonian_Hes
10 I,E=[json.loads((P/f).read_text()) for f in files]
11 G=6.67430e-11; pc=3.085677581491367e16
12 M=float(I['mass_kg']); b=float(I['b_pc'])*pc; rh=float(I['rh_pc'])*pc
13 T=np.array(E['grid_results'][-1]['hessian_s_minus2'],float)
14 gext=np.array(E['grid_results'][-1]['gradient_m_s2'],float)
15 R=np.array(E['hydrus_galactocentric_kpc'],float); n=-R/np.linalg.norm(R)
16 assert np.allclose(T,T.T,rtol=0,atol=1e-43)
17 assert abs(np.trace(T))<1e-9*np.linalg.norm(T)
18 rows=[]
19 for f in [0.25,0.5,1,1.5,2,3,5]:
20  r=f*rh
21  # Internal Plummer acceleration magnitude, external uniform field, and differential tidal accelerat
22  gint=G*M*r/(r*r+b*b)**1.5
23  tidal=np.linalg.norm(T@(r*n))
```

Audit interpretation: Identify the exact inputs, units, normalization and frozen constants in the full script. Determine whether this file computes a new physical profile, reads prior profiles, or only checks a manufactured numerical identity. Inspect the handling of exceptions, finite values, boundaries and convergence; a successful process exit alone is not a scientific validation.

Reproduction note: compare any outputs with the companion JSON and/or NPZ by exact archive name and digest. The filename timestamp is metadata, not proof of run chronology. No independent execution of this script is claimed in this companion.

# Source 40/77 — UDCT_v7_13_Hydrus_External_Grid_Source_Audit

Full archive filename: UDCT_v7_13_Hydrus_External_Grid_Source_Audit_20261021_011910_302.py

Source length: 58 lines; bytes: 3795; SHA-256: 7ae7ffa8b81b9af1f337b993a18aef040283909cc8fc4050f29c78f15d7cc376.

Functions: calc(e,H,x,law,ell,h):); c(y):).

Classes: none. Imports: import hashlib, importlib.util, json, sys; from pathlib import Path; import numpy as np.

Opening source documentation: Posthoc exploratory: Galactic quadrature grid sensitivity of local 3D GQUMOND source. Uses frozen v7.10/v7.06 laws, not the registered six-arm Jeans validation.

Static implementation signals (presence in source, not proof of execution): linear algebra, JSON output, internal assertion, hash verification, solver path.

Source excerpt (first 23 physical lines; whitespace and line order retained):

```
 1 #!/usr/bin/env python3
 2 """Posthoc exploratory: Galactic quadrature grid sensitivity of local 3D GQUMOND source.
 3 Uses frozen v7.10/v7.06 laws, not the registered six-arm Jeans validation.
 4 """
 5 import hashlib, importlib.util, json, sys
 6 from pathlib import Path
 7 import numpy as np
 8 ROOT=Path(__file__).resolve().parent
 9 P=ROOT/'UDCT_v7_10_Postregistration_run.py';D=ROOT/'UDCT_v7_06_Hessian_Term_Sign_Audit_reproduce.py'
10 assert hashlib.sha256(P.read_bytes()).hexdigest()=='a4d1a07ebcaa06348be3022d0f1f6ca463347e00194f7fde
11 assert hashlib.sha256(D.read_bytes()).hexdigest()=='a64440e1350ae8bf10eb3188ec34829b6d586abf275de5cd
12 spec=importlib.util.spec_from_file_location('v710_grid_audit',P);m=importlib.util.module_from_spec(s
13 obj=m.DWARFS['Hydrus I'];M=m.ML*obj.luminosity*m.MSUN;b=obj.re_pc*m.PC;rh=b/np.sqrt(2**(2/3)-1)
14 direction=-m.V706.galactocentric_position(obj);direction/=np.linalg.norm(direction)
15
16 def calc(e,H,x,law,ell,h):
17     def c(y):
18         r2=y@y;q=r2+b*b
19         g=m.G*M*y/q**1.5+e+H@y
20         t=m.G*M*(np.eye(3)/q**1.5-3*np.outer(y,y)/q**2.5)+H
21         nu,w=m.gq_kernel(np.array([g@g]),np.array([np.sum(t*t)]),ell,law)
22         return float(nu[0])*g,float(w[0])*t
23     eye=np.eye(3);f0,p0=c(x);div=0.;dd=0.
```

Audit interpretation: Identify the exact inputs, units, normalization and frozen constants in the full script. Determine whether this file computes a new physical profile, reads prior profiles, or only checks a manufactured numerical identity. Inspect the handling of exceptions, finite values, boundaries and convergence; a successful process exit alone is not a scientific validation.

Reproduction note: compare any outputs with the companion JSON and/or NPZ by exact archive name and digest. The filename timestamp is metadata, not proof of run chronology. No independent execution of this script is claimed in this companion.

# Source 41/77 — UDCT_v7_13_Hydrus_Hessian_Increment_Convergence_Audit

Full archive filename: UDCT_v7_13_Hydrus_Hessian_Increment_Convergence_Audit_20261021_011931_333.py

Source length: 23 lines; bytes: 2464; SHA-256: 63b8b67f3b157e491144d995514b449237df31af3acc72b2b83b6374ae34159a.

Functions: No named function definitions found.

Classes: none. Imports: import json,hashlib,math,statistics; from pathlib import Path.

Opening source documentation: Post-hoc exploratory convergence audit of archived v7.13 local Hessian source increments. No rerun of frozen laws, no 3D Poisson solution, no preregistered inference.

Static implementation signals (presence in source, not proof of execution): JSON output, hash verification.

Source excerpt (first 23 physical lines; whitespace and line order retained):

```
#!/usr/bin/env python3
"""Post-hoc exploratory convergence audit of archived v7.13 local Hessian source increments.
No rerun of frozen laws, no 3D Poisson solution, no preregistered inference.
"""
import json,hashlib,math,statistics
from pathlib import Path
src=Path(__file__).with_name('UDCT_v7_13_Hydrus_Ell_Sensitivity_Local_Source.json')
dat=json.loads(src.read_text()); output=[]
for x in dat['rows']:
    a,b,c=map(float,x['hessian_by_step']) # h = .08,.04,.02 rh
    d1=b-a;d2=c-b
    order=math.log(abs(d1/d2),2) if d1*d2>0 and abs(d2)>0 else None
    extrap=c+(c-b)/3 # formal O(h^2) Richardson
    estimated=abs(extrap-c)
    relative=estimated/max(abs(extrap),1e-300)
    output.append(dict(radius_rh=x['r_over_rh'],law=x['law'],ell_pc=x['ell_pc'],hessian_increments=[
summary={}
for law in sorted(set(x['law'] for x in output)):
    q=[x for x in output if x['law']==law]
    summary[law]=dict(n=len(q),median_relative_error=statistics.median(x['estimated_relative_discret
selected=[x for x in output if x['radius_rh']==2 and x['ell_pc'] in (1,3,10)]
result=dict(classification='POSTHOC_EXPLORATORY_NUMERICAL_AUDIT_NOT_CONFIRMATORY',input_file=src.nam
out=src.with_name('UDCT_v7_13_Hydrus_Hessian_Increment_Convergence_Audit.json');out.write_text(json.
```

Audit interpretation: Identify the exact inputs, units, normalization and frozen constants in the full script. Determine whether this file computes a new physical profile, reads prior profiles, or only checks a manufactured numerical identity. Inspect the handling of exceptions, finite values, boundaries and convergence; a successful process exit alone is not a scientific validation.

Reproduction note: compare any outputs with the companion JSON and/or NPZ by exact archive name and digest. The filename timestamp is metadata, not proof of run chronology. No independent execution of this script is claimed in this companion.

# Source 42/77 — UDCT_v7_13_Hydrus_Hessian_Orientation_Bounds

Full archive filename: UDCT_v7_13_Hydrus_Hessian_Orientation_Bounds_20261021_012030_205.py

Source length: 49 lines; bytes: 2908; SHA-256: 4b73bad127b4dd5e88193de646e5a333df48487303335c93518d125deebb3164.

Functions: No named function definitions found.

Classes: none. Imports: import json; from pathlib import Path; import numpy as np.

Opening source documentation: Orientation-independent bounds and illustrative rotations of two existing Newtonian Hessian controls. This is NOT physical alignment, modified gravity, Jeans fitting, or new observation.

Static implementation signals (presence in source, not proof of execution): linear algebra, JSON output, internal assertion, solver path.

Source excerpt (first 23 physical lines; whitespace and line order retained):

```
 1 #!/usr/bin/env python3
 2 """Orientation-independent bounds and illustrative rotations of two existing Newtonian Hessian contr
 3 This is NOT physical alignment, modified gravity, Jeans fitting, or new observation.
 4 """
 5 import json
 6 from pathlib import Path
 7 import numpy as np
 8 root=Path(__file__).resolve().parent
 9 internal=json.loads((root/'UDCT_v7_13_Hydrus_Plummer_3D_Hessian_Control.json').read_text())
10 external=json.loads((root/'UDCT_v7_13_Hydrus_External_Newtonian_Hessian_Exploratory.json').read_text
11 A=np.array(internal['analytic_hessian_s_minus2'],float)
12 B=np.array(external['grid_results'][-1]['hessian_s_minus2'],float)
13 scale=1e32
14 a=A*scale;b=B*scale
15 na=np.linalg.norm(a);nb=np.linalg.norm(b)
16 wa=np.linalg.eigvalsh(a);wb=np.linalg.eigvalsh(b)
17 # Trace(A RBRt) optimized by rearrangement inequality; trace of product, not sum
18 # Norm squared of A+RBRt = norm(A)^2 + norm(B)^2 + 2 Tr(A RBRt)
19 inner_min=float(np.dot(wa,wb[::-1]));inner_max=float(np.dot(wa,wb))
20 min_norm=float(np.sqrt(max(0,na**2+nb**2+2*inner_min)))
21 max_norm=float(np.sqrt(max(0,na**2+nb**2+2*inner_max)))
22 # Illustrative random SO(3) rotations only, fixed seed, not inferred actual orientation
23 rng=np.random.default_rng(713)
```

Audit interpretation: Identify the exact inputs, units, normalization and frozen constants in the full script. Determine whether this file computes a new physical profile, reads prior profiles, or only checks a manufactured numerical identity. Inspect the handling of exceptions, finite values, boundaries and convergence; a successful process exit alone is not a scientific validation.

Reproduction note: compare any outputs with the companion JSON and/or NPZ by exact archive name and digest. The filename timestamp is metadata, not proof of run chronology. No independent execution of this script is claimed in this companion.

# Source 43/77 — UDCT_v7_13_Hydrus_Hessian_Scale_Comparison

Full archive filename: UDCT_v7_13_Hydrus_Hessian_Scale_Comparison_20261021_012041_002.py

Source length: 41 lines; bytes: 2696; SHA-256: 61ac3eef2ed4bc660e96188b26c5cee1fc8cd6375fc60454d0a8b255fc05d6ec.

Functions: No named function definitions found.

Classes: none. Imports: import json, math; from pathlib import Path; import numpy as np.

Opening source documentation: Reproduce a comparison of two PREVIOUSLY computed Newtonian Hessian controls. No 3D modified gravity solution, no new observations, no coordinate-frame alignment.

Static implementation signals (presence in source, not proof of execution): linear algebra, JSON output, internal assertion, solver path.

Source excerpt (first 23 physical lines; whitespace and line order retained):

```
 1 #!/usr/bin/env python3
 2 """Reproduce a comparison of two PREVIOUSLY computed Newtonian Hessian controls.
 3 No 3D modified gravity solution, no new observations, no coordinate-frame alignment.
 4 """
 5 import json, math
 6 from pathlib import Path
 7 import numpy as np
 8 ROOT=Path(__file__).resolve().parent
 9 internal=json.loads((ROOT/'UDCT_v7_13_Hydrus_Plummer_3D_Hessian_Control.json').read_text())
10 external=json.loads((ROOT/'UDCT_v7_13_Hydrus_External_Newtonian_Hessian_Exploratory.json').read_text
11 H_in=np.asarray(internal['analytic_hessian_s_minus2'],dtype=float)
12 H_ext=np.asarray(external['grid_results'][-1]['hessian_s_minus2'],dtype=float)
13 G=6.67430e-11; PC=3.085677581491367e16; MSUN=1.98847e30
14 norm=lambda h:float(np.linalg.norm(h,'fro'))
15 scale=PC**3/(4*math.pi*G*MSUN)
16 result={
17  'classification':'EXPLORATORY comparison of existing Newtonian controls; NOT a new modified-gravity
18  'input_files':[ 'UDCT_v7_13_Hydrus_Plummer_3D_Hessian_Control.json', 'UDCT_v7_13_Hydrus_External_Ne
19  'internal_evaluation':'off-axis at r=r_h; baryonic Plummer analytic Hessian',
20  'external_evaluation':'Milky Way external Newtonian Hessian at modeled Hydrus I location',
21  'coordinate_warning':'Tensor components are NOT established in a common coordinate frame. Do not ad
22  'internal_frobenius_s_minus2':norm(H_in),
23  'external_frobenius_s_minus2':norm(H_ext),
```

Audit interpretation: Identify the exact inputs, units, normalization and frozen constants in the full script. Determine whether this file computes a new physical profile, reads prior profiles, or only checks a manufactured numerical identity. Inspect the handling of exceptions, finite values, boundaries and convergence; a successful process exit alone is not a scientific validation.

Reproduction note: compare any outputs with the companion JSON and/or NPZ by exact archive name and digest. The filename timestamp is metadata, not proof of run chronology. No independent execution of this script is claimed in this companion.

# Source 44/77 — UDCT_v7_13_Hydrus_Local_3D_Source_Gate

Full archive filename: UDCT_v7_13_Hydrus_Local_3D_Source_Gate_20261021_011952_641.py

Source length: 66 lines; bytes: 4288; SHA-256: 46a8009600d0f7549a61d93d3fb905d0966bf59252ced1e0cdff3f13e385f27a.

Functions: field(x):); constit(x,law,ell):); source(x,law,ell,step):).

Classes: none. Imports: import json,hashlib,sys,importlib.util; from pathlib import Path; import numpy as np.

Opening source documentation: Exploratory local 3D source diagnostic using frozen v7.10 law; NOT full 3D solution.

Static implementation signals (presence in source, not proof of execution): linear algebra, JSON output, internal assertion, hash verification, solver path.

Source excerpt (first 23 physical lines; whitespace and line order retained):

```
 1 #!/usr/bin/env python3
 2 """Exploratory local 3D source diagnostic using frozen v7.10 law; NOT full 3D solution."""
 3 import json,hashlib,sys,importlib.util
 4 from pathlib import Path
 5 import numpy as np
 6 ROOT=Path('/mnt/data')
 7 P=ROOT/'UDCT_v7_10_Postregistration_run.py'
 8 D=ROOT/'UDCT_v7_06_Hessian_Term_Sign_Audit_reproduce.py'
 9 assert hashlib.sha256(D.read_bytes()).hexdigest()=='a64440e1350ae8bf10eb3188ec34829b6d586abf275de5cd
10 spec=importlib.util.spec_from_file_location('frozen_udct_v710',P)
11 m=importlib.util.module_from_spec(spec);sys.modules[spec.name]=m;spec.loader.exec_module(m)
12 assert hashlib.sha256(P.read_bytes()).hexdigest()=='a4d1a07ebcaa06348be3022d0f1f6ca463347e00194f7fde
13 obj=m.DWARFS['Hydrus I']; grid=m.Grid(radial=180,polar=16,azimuth=24,disk_radial=40,disk_azimuth=24,
14 e,H,_=m.V706.galactic_field(obj,grid)
15 M=m.ML*obj.luminosity*m.MSUN;b=obj.re_pc*m.PC;rh=b/np.sqrt(2**(2/3)-1)
16 # Newtonian Plummer internal gradient and Hessian; Galactic Taylor field about dwarf center.
17 def field(x):
18  r2=float(x@x);q=r2+b*b
19  g=m.G*M*x/q**1.5+e+H@x
20  h=m.G*M*(np.eye(3)/q**1.5-3*np.outer(x,x)/q**2.5)+H
21  return g,h
22
23 def constit(x,law,ell):
```

Audit interpretation: Identify the exact inputs, units, normalization and frozen constants in the full script. Determine whether this file computes a new physical profile, reads prior profiles, or only checks a manufactured numerical identity. Inspect the handling of exceptions, finite values, boundaries and convergence; a successful process exit alone is not a scientific validation.

Reproduction note: compare any outputs with the companion JSON and/or NPZ by exact archive name and digest. The filename timestamp is metadata, not proof of run chronology. No independent execution of this script is claimed in this companion.

# Source 45/77 — UDCT_v7_13_Hydrus_Plummer_3D_Hessian_Control

Full archive filename: UDCT_v7_13_Hydrus_Plummer_3D_Hessian_Control_20261021_012213_249.py

Source length: 61 lines; bytes: 4021; SHA-256: 3fba5c142eb20d289fac71889344a5033d31fb9693f24ae60348fa2a57c2bec6.

Functions: phi(x): return -m.G*mass/np.sqrt(np.dot(x,x)+b*b)); exact_hessian(x):); numerical_hessian(x,h):).

Classes: none. Imports: import hashlib, importlib.util, json, sys; from pathlib import Path; import numpy as np.

Opening source documentation: Exploratory analytic 3D baryonic Plummer control for Hydrus I. NOT a full modified gravity solution, NOT any B01-B24 Hessian result.

Static implementation signals (presence in source, not proof of execution): linear algebra, JSON output, internal assertion, hash verification, solver path.

Source excerpt (first 23 physical lines; whitespace and line order retained):

```
 1 #!/usr/bin/env python3
 2 """Exploratory analytic 3D baryonic Plummer control for Hydrus I.
 3 NOT a full modified gravity solution, NOT any B01-B24 Hessian result.
 4 """
 5 import hashlib, importlib.util, json, sys
 6 from pathlib import Path
 7 import numpy as np
 8 root=Path('/mnt/data/v713_hessian_stage')
 9 plan=json.loads(Path('/mnt/data/UDCT_v7_13_Preregistration_Updated/preregistration_plan.json').read_
10 verified={}
11 for name,digest in plan['frozen_hashes'].items():
12     path=root/name
13     if not path.exists(): path=Path('/mnt/data/v713_b07_run')/name
14     verified[name]=path.exists() and hashlib.sha256(path.read_bytes()).hexdigest()==digest
15 assert all(verified.values())
16 spec=importlib.util.spec_from_file_location('v706_plummer_control',root/'UDCT_v7_06_Hessian_Term_Sig
17 m=importlib.util.module_from_spec(spec);sys.modules[spec.name]=m;spec.loader.exec_module(m)
18 obj=m.DWARFS['Hydrus I']; mass=m.ML*obj.luminosity*m.MSUN; b=obj.re_pc*m.PC
19 rh=b/np.sqrt(2**(2/3)-1)
20 # Explicit analytic 3-D Newtonian Plummer potential and its exact Hessian.
21 def phi(x): return -m.G*mass/np.sqrt(np.dot(x,x)+b*b)
22 def exact_hessian(x):
23     d=np.dot(x,x)+b*b
```

Audit interpretation: Identify the exact inputs, units, normalization and frozen constants in the full script. Determine whether this file computes a new physical profile, reads prior profiles, or only checks a manufactured numerical identity. Inspect the handling of exceptions, finite values, boundaries and convergence; a successful process exit alone is not a scientific validation.

Reproduction note: compare any outputs with the companion JSON and/or NPZ by exact archive name and digest. The filename timestamp is metadata, not proof of run chronology. No independent execution of this script is claimed in this companion.

# Source 46/77 — UDCT_v7_13_Hydrus_Radial_Tidal_Scan

Full archive filename: UDCT_v7_13_Hydrus_Radial_Tidal_Scan_20261021_012010_956.py

Source length: 23 lines; bytes: 2274; SHA-256: 0a906597563a4a12fed95c16f13df2071080fc74c87e4dc4d9484536d7433cef.

Functions: No named function definitions found.

Classes: none. Imports: import json; from pathlib import Path; import numpy as np.

Opening source documentation: Exploratory Newtonian radial tidal scan; not a modified-gravity solve.

Static implementation signals (presence in source, not proof of execution): linear algebra, JSON output, internal assertion, solver path.

Source excerpt (first 23 physical lines; whitespace and line order retained):

```
 1 #!/usr/bin/env python3
 2 """Exploratory Newtonian radial tidal scan; not a modified-gravity solve."""
 3 import json
 4 from pathlib import Path
 5 import numpy as np
 6 P=Path(__file__).resolve().parent
 7 I=json.loads((P/'UDCT_v7_13_Hydrus_Plummer_3D_Hessian_Control.json').read_text())
 8 E=json.loads((P/'UDCT_v7_13_Hydrus_External_Newtonian_Hessian_Exploratory.json').read_text())
 9 G=6.67430e-11; pc=3.085677581491367e16
10 M=I['mass_kg']; b=I['b_pc']*pc; rh=I['rh_pc']*pc
11 T=np.array(E['grid_results'][-1]['hessian_s_minus2'],float)
12 R=np.array(E['hydrus_galactocentric_kpc'],float); n=-R/np.linalg.norm(R)
13 rows=[]
14 for f in [0,0.25,0.5,1,1.5,2,3,5,10]:
15  r=f*rh; A=G*M/(r*r+b*b)**1.5; B=3*G*M*r*r/(r*r+b*b)**2.5
16  H=A*np.eye(3)-B*np.outer(n,n); S=H+T
17  rows.append({'r_over_rh':f,'internal_norm_s_minus2':float(np.linalg.norm(H)), 'external_norm_s_minu
18 output={'classification':'EXPLORATORY Newtonian Plummer + constant Milky Way tidal Hessian radial sc
19 assert all(np.isfinite(v) for row in rows for k,v in row.items() if isinstance(v,float))
20 assert max(abs(x['trace_combined_s_minus2']-x['trace_internal_s_minus2']) for x in rows)<1e-42
21 out=P/'UDCT_v7_13_Hydrus_Radial_Tidal_Scan.json';out.write_text(json.dumps(output,indent=2)+'\n')
22 for r in rows:print(f"r/rh={r['r_over_rh']:>5}: ext/int={r['external_over_internal']:.4f}, total/int
23 print('saved',out)
```

Audit interpretation: Identify the exact inputs, units, normalization and frozen constants in the full script. Determine whether this file computes a new physical profile, reads prior profiles, or only checks a manufactured numerical identity. Inspect the handling of exceptions, finite values, boundaries and convergence; a successful process exit alone is not a scientific validation.

Reproduction note: compare any outputs with the companion JSON and/or NPZ by exact archive name and digest. The filename timestamp is metadata, not proof of run chronology. No independent execution of this script is claimed in this companion.

# Source 47/77 — UDCT_v7_13_Independent_Endpoint_Decision_Audit

Full archive filename: UDCT_v7_13_Independent_Endpoint_Decision_Audit_20261021_011734_252.py

Source length: 51 lines; bytes: 3716; SHA-256: ec902e4fd983c973352a92cc0d09d857e2a4aee5ddbc8043b8078fac667a50fd.

Functions: No named function definitions found.

Classes: none. Imports: import json,hashlib; from pathlib import Path; import numpy as np.

Opening source documentation: Independent read-only audit of archived v7.13 six-arm endpoint profiles. No galaxy field solve; no new physical results; not a confirmatory solver validation.

Static implementation signals (presence in source, not proof of execution): JSON output, internal assertion, hash verification, NPZ input, solver path.

Source excerpt (first 23 physical lines; whitespace and line order retained):

```
 1 #!/usr/bin/env python3
 2 """Independent read-only audit of archived v7.13 six-arm endpoint profiles.
 3 No galaxy field solve; no new physical results; not a confirmatory solver validation.
 4 """
 5 import json,hashlib
 6 from pathlib import Path
 7 import numpy as np
 8 R=Path(__file__).resolve().parent
 9 src=R/'UDCT_v7_13_B01_B24_All_Common_Profiles.npz'
10 inv=json.loads((R/'UDCT_v7_13_B01_B24_Trace_Inventory.json').read_text())
11 assert hashlib.sha256(src.read_bytes()).hexdigest()==inv['full_profiles_sha256']
12 arm_names=['default|0.002','refined|0.002','default|0.0005','refined|0.0005','validation|0.002','val
13 raw=[json.loads((R/'UDCT_v7_13_B01_Full_Profile_Trace.json').read_text())]
14 for a,b in [(2,3),(4,4),(5,6),(7,9),(10,10),(11,12),(13,15),(16,17),(18,18),(19,20),(21,22),(23,24)]
15  d=json.loads((R/f'UDCT_v7_13_B{a:02d}_B{b:02d}_Full_Profile_Trace.json').read_text())
16  raw.extend(d['targets'])
17 rows=[];exceptions=[]
18 with np.load(src) as archive:
19  for i,rec in enumerate(raw,1):
20   target=f'B{i:02d}';assert rec['target']==target
21   endpoint_results=[]
22   for p in rec['endpoints']:
23    vals=[];native=[]
```

Audit interpretation: Identify the exact inputs, units, normalization and frozen constants in the full script. Determine whether this file computes a new physical profile, reads prior profiles, or only checks a manufactured numerical identity. Inspect the handling of exceptions, finite values, boundaries and convergence; a successful process exit alone is not a scientific validation.

Reproduction note: compare any outputs with the companion JSON and/or NPZ by exact archive name and digest. The filename timestamp is metadata, not proof of run chronology. No independent execution of this script is claimed in this companion.

# Source 48/77 — UDCT_v7_13_Joint_Grid_Box_StopGate_Audit

Full archive filename: UDCT_v7_13_Joint_Grid_Box_StopGate_Audit_20261021_011843_977.py

Source length: 36 lines; bytes: 2929; SHA-256: 6a563d967c40878d19dd73c0f278826d117ca6dd506b0eeaf9af918cbf386a75.

Functions: No named function definitions found.

Classes: none. Imports: from pathlib import Path; import hashlib,json,numpy as np.

Opening source documentation: Independent post-processing of EXISTING exploratory Hydrus I delta-acceleration runs. Not a preregistered six-arm Jeans or physical boundary validation.

Static implementation signals (presence in source, not proof of execution): linear algebra, JSON output, internal assertion, hash verification, solver path.

Source excerpt (first 23 physical lines; whitespace and line order retained):

```
#!/usr/bin/env python3
"""Independent post-processing of EXISTING exploratory Hydrus I delta-acceleration runs.
Not a preregistered six-arm Jeans or physical boundary validation.
"""
from pathlib import Path
import hashlib,json,numpy as np
root=Path(__file__).resolve().parent
files=['UDCT_v7_13_Radial_Acceleration_Convergence.json','UDCT_v7_13_Radial_Box_Size_Audit.json']
raw=[(root/f).read_bytes() for f in files]
res,box=[json.loads(x) for x in raw]
assert res['status'].startswith('EXPLORATORY') and box['status'].startswith('EXPLORATORY')
rows=[]
for law in ['A-Qs','A-f1/2']:
 rr=sorted((r for r in res['results'] if r['law']==law),key=lambda x:x['n'])
 bb=sorted((r for r in box['results'] if r['law']==law),key=lambda x:x['box_side_over_rh'])
 assert [r['n'] for r in rr]==[35,47,59]
 assert [r['box_side_over_rh'] for r in bb]==[6,8]
 for k,rad in enumerate([0.5,1.,2.]):
  vv=[np.array(r['radial_samples'][k]['delta_acceleration_vector_over_a0']) for r in rr]
  ww=[np.array(r['samples'][k]['interpolated_delta_gradient_vector_over_a0']) for r in bb]
  assert all(abs(r['radial_samples'][k]['actual_radius_rh']-rad)<1e-12 for r in rr)
  assert np.allclose(vv[1],ww[0],rtol=1e-8,atol=1e-12), (law,rad,'source mismatch')
  fine=float(100*np.linalg.norm(vv[2]-vv[1])/np.linalg.norm(vv[2]))
```

Audit interpretation: Identify the exact inputs, units, normalization and frozen constants in the full script. Determine whether this file computes a new physical profile, reads prior profiles, or only checks a manufactured numerical identity. Inspect the handling of exceptions, finite values, boundaries and convergence; a successful process exit alone is not a scientific validation.

Reproduction note: compare any outputs with the companion JSON and/or NPZ by exact archive name and digest. The filename timestamp is metadata, not proof of run chronology. No independent execution of this script is claimed in this companion.

# Source 49/77 — UDCT_v7_13_Multipole_Boundary_Exploratory

Full archive filename: UDCT_v7_13_Multipole_Boundary_Exploratory_20261021_011827_040.py

Source length: 68 lines; bytes: 3576; SHA-256: 25e4ed1340291a0335e1816797986940cfb5e1634d3019f374e34bccbe7c25ed.

Functions: solve_dirichlet(rhs,h,bfaces):); run(lawname,side,n):); boundary_val(pos,degree):); faces(degree):).

Classes: none. Imports: import json,hashlib; from pathlib import Path; import numpy as np; from scipy.fft import dstn,idstn.

Opening source documentation: Exploratory truncated-source multipole boundary vs zero Dirichlet. This is NOT an exterior-complete physical boundary and NOT v7.13 prereg execution.

Static implementation signals (presence in source, not proof of execution): linear algebra, JSON output, hash verification, solver path.

Source excerpt (first 23 physical lines; whitespace and line order retained):

```
 1 #!/usr/bin/env python3
 2 """Exploratory truncated-source multipole boundary vs zero Dirichlet.
 3 This is NOT an exterior-complete physical boundary and NOT v7.13 prereg execution.
 4 """
 5 import json,hashlib
 6 from pathlib import Path
 7 import numpy as np
 8 from scipy.fft import dstn,idstn
 9 R=Path('/mnt/data'); source=R/'UDCT_v7_13_Hydrus_3D_DeltaPotential_Exploratory.py'
10 s=source.read_text(); s=s[:s.index("out={'status'")]
11 s=s.replace(' pot=solve(rhs,h)', ' return rhs,h')
12 ns={'__file__':str(source)};exec(compile(s,str(source),'exec'),ns)
13 rh=ns['rh'];m=ns['m'];base_L=ns['L']
14
15 def solve_dirichlet(rhs,h,bfaces):
16  n=rhs.shape[0]; v=rhs.copy(); hh=h*h
17  # faces each is a 2D (n,n) array of boundary values
18  for ax in range(3):
19   low,high=bfaces[ax]
20   if ax==0: v[0,:,:]-=low/hh;v[-1,:,:]-=high/hh
21   if ax==1: v[:,0,:]-=low/hh;v[:,-1,:]-=high/hh
22   if ax==2: v[:,:,0]-=low/hh;v[:,:,-1]-=high/hh
23  return ns['solve'](v,h)
```

Audit interpretation: Identify the exact inputs, units, normalization and frozen constants in the full script. Determine whether this file computes a new physical profile, reads prior profiles, or only checks a manufactured numerical identity. Inspect the handling of exceptions, finite values, boundaries and convergence; a successful process exit alone is not a scientific validation.

Reproduction note: compare any outputs with the companion JSON and/or NPZ by exact archive name and digest. The filename timestamp is metadata, not proof of run chronology. No independent execution of this script is claimed in this companion.

# Source 50/77 — UDCT_v7_13_Poisson_Sign_Units_Boundary_Gate

Full archive filename: UDCT_v7_13_Poisson_Sign_Units_Boundary_Gate_20261021_012146_283.py

Source length: 61 lines; bytes: 3454; SHA-256: 7b310af88ad1323c1277f7457c8894a2c4f43507b75553b07160a4e100f6e3e7.

Functions: solve(rhs, h):); lap(u,h):); run(n,nu):).

Classes: none. Imports: import json, math, hashlib; from pathlib import Path; import numpy as np; from scipy.fft import dstn, idstn.

Opening source documentation: Independent analytic Plummer Poisson sign/boundary test; NOT a GQUMOND solver.

Static implementation signals (presence in source, not proof of execution): linear algebra, JSON output, internal assertion, hash verification, solver path.

Source excerpt (first 23 physical lines; whitespace and line order retained):

```
 1 #!/usr/bin/env python3
 2 """Independent analytic Plummer Poisson sign/boundary test; NOT a GQUMOND solver."""
 3 import json, math, hashlib
 4 from pathlib import Path
 5 import numpy as np
 6 from scipy.fft import dstn, idstn
 7
 8 def solve(rhs, h):
 9  n=rhs.shape[0]; k=np.arange(1,n+1)
10  eig=2*(np.cos(np.pi*k/(n+1))-1)/h**2
11  return idstn(dstn(rhs,type=1,norm='ortho')/(eig[:,None,None]+eig[None,:,None]+eig[None,None,:]),typ
12
13 def lap(u,h):
14  a=np.pad(u,1)
15  return (a[2:,1:-1,1:-1]+a[:-2,1:-1,1:-1]+a[1:-1,2:,1:-1]+a[1:-1,:-2,1:-1]+a[1:-1,1:-1,2:]+a[1:-1,1:
16
17 def run(n,nu):
18  # Nondimensional GM=b=1, box [-4,4]^3. Exact Plummer Phi=-nu/sqrt(r^2+1)
19  box=4.; h=2*box/(n+1)
20  t=-box+h*np.arange(1,n+1)
21  x=t[:,None,None];y=t[None,:,None];z=t[None,None,:]
22  r2=x*x+y*y+z*z
23  exact=-nu/np.sqrt(1+r2)
```

Audit interpretation: Identify the exact inputs, units, normalization and frozen constants in the full script. Determine whether this file computes a new physical profile, reads prior profiles, or only checks a manufactured numerical identity. Inspect the handling of exceptions, finite values, boundaries and convergence; a successful process exit alone is not a scientific validation.

Reproduction note: compare any outputs with the companion JSON and/or NPZ by exact archive name and digest. The filename timestamp is metadata, not proof of run chronology. No independent execution of this script is claimed in this companion.

# Source 51/77 — UDCT_v7_13_Quadrupole_Boundary_Diagnostic

Full archive filename: UDCT_v7_13_Quadrupole_Boundary_Diagnostic_20261021_011830_763.py

Source length: 48 lines; bytes: 2644; SHA-256: 7904cffc875d250c06dde5124f538a7925d69f00a8b33128ae0d344d1f9b4909.

Functions: No named function definitions found.

Classes: none. Imports: from pathlib import Path; import json,hashlib; import numpy as np.

Opening source documentation: Exploratory quadrupole moment audit of frozen 3D delta-Poisson source. No new Poisson solve, no physical asymptotic boundary, no prereg B01-B24 execution.

Static implementation signals (presence in source, not proof of execution): linear algebra, JSON output, hash verification, solver path.

Source excerpt (first 23 physical lines; whitespace and line order retained):

```
 1 #!/usr/bin/env python3
 2 """Exploratory quadrupole moment audit of frozen 3D delta-Poisson source.
 3 No new Poisson solve, no physical asymptotic boundary, no prereg B01-B24 execution.
 4 """
 5 from pathlib import Path
 6 import json,hashlib
 7 import numpy as np
 8 root=Path(__file__).resolve().parent
 9 src=root/'UDCT_v7_13_Hydrus_3D_DeltaPotential_Exploratory.py'
10 script=src.read_text().split("out={'status'")[0]
11 start=script.index(' pot=solve(rhs,h)')
12 end=script.index('\nout={',start) if '\nout={' in script[start:] else len(script)
13 script=script[:start]+''' axis=(np.arange(n)-(n-1)/2)*h
14  xx,yy,zz=np.meshgrid(axis,axis,axis,indexing='ij')
15  coords=(xx,yy,zz)
16  r2p=xx*xx+yy*yy+zz*zz
17  dv=h**3
18  absint=float(np.abs(rhs).sum()*dv)
19  quad=np.array([[float(np.sum(rhs*(3*coords[i]*coords[j]-(r2p if i==j else 0)))*dv) for j in range(3
20  # Quadrupole tensor normalized by absolute source integral and rh^2
21  qnorm=quad/(absint*rh**2)
22  outer=r2p >= (0.8*L/2)**2
23  outerq=np.array([[float(np.sum((rhs*(3*coords[i]*coords[j]-(r2p if i==j else 0)))[outer])*dv) for j
```

Audit interpretation: Identify the exact inputs, units, normalization and frozen constants in the full script. Determine whether this file computes a new physical profile, reads prior profiles, or only checks a manufactured numerical identity. Inspect the handling of exceptions, finite values, boundaries and convergence; a successful process exit alone is not a scientific validation.

Reproduction note: compare any outputs with the companion JSON and/or NPZ by exact archive name and digest. The filename timestamp is metadata, not proof of run chronology. No independent execution of this script is claimed in this companion.

# Source 52/77 — UDCT_v7_13_Radial_Acceleration_Convergence

Full archive filename: UDCT_v7_13_Radial_Acceleration_Convergence_20261021_011853_533.py

Source length: 40 lines; bytes: 2819; SHA-256: bb8281c2c7dea363b30401d1a92ea6d795bba27f92b62b4e20e87e7f98cb34ac.

Functions: No named function definitions found.

Classes: none. Imports: import json, hashlib; from pathlib import Path; import numpy as np.

Opening source documentation: Exploratory radial vector acceleration audit; not preregistered B01-B24. Uses frozen v7.10 source and finite zero-Dirichlet delta-potential box.

Static implementation signals (presence in source, not proof of execution): linear algebra, JSON output, internal assertion, hash verification, solver path.

Source excerpt (first 23 physical lines; whitespace and line order retained):

```
 1 #!/usr/bin/env python3
 2 """Exploratory radial vector acceleration audit; not preregistered B01-B24.
 3 Uses frozen v7.10 source and finite zero-Dirichlet delta-potential box.
 4 """
 5 import json, hashlib
 6 from pathlib import Path
 7 import numpy as np
 8 root=Path('/mnt/data')
 9 original=root/'UDCT_v7_13_Hydrus_3D_DeltaPotential_Exploratory.py'
10 s=original.read_text()
11 s=s[:s.index("out={'status'")]
12 old=" return {'n':n,'spacing_over_rh':h/rh"
13 new=""" samples=[]
14  for target in (0.5,1.,2.):
15   ix=int(round(mid+target*rh/h))
16   if ix+1>=n: raise ValueError('sample outside grid')
17   vec=np.array([(pot[ix+1,mid,mid]-pot[ix-1,mid,mid]),(pot[ix,mid+1,mid]-pot[ix,mid-1,mid]),(pot[ix,
18   samples.append({'requested_radius_rh':target,'actual_radius_rh':(ix-mid)*h/rh,'delta_acceleration_
19  return {'radial_samples':samples,'n':n,'spacing_over_rh':h/rh"""
20 assert old in s
21 s=s.replace(old,new)
22 ns={'__file__':str(original)}
23 exec(compile(s,str(original),'exec'),ns)
```

Audit interpretation: Identify the exact inputs, units, normalization and frozen constants in the full script. Determine whether this file computes a new physical profile, reads prior profiles, or only checks a manufactured numerical identity. Inspect the handling of exceptions, finite values, boundaries and convergence; a successful process exit alone is not a scientific validation.

Reproduction note: compare any outputs with the companion JSON and/or NPZ by exact archive name and digest. The filename timestamp is metadata, not proof of run chronology. No independent execution of this script is claimed in this companion.

# Source 53/77 — UDCT_v7_13_Radial_Box_Size_Audit

Full archive filename: UDCT_v7_13_Radial_Box_Size_Audit_20261021_011848_697.py

Source length: 43 lines; bytes: 2587; SHA-256: bc23e7865073da8001e02aa969e4c404a66255d4a31b982d6116fde3a2fabea2.

Functions: grad_at(ix):).

Classes: none. Imports: import json,hashlib; from pathlib import Path; import numpy as np.

Opening source documentation: Exploratory common-coordinate interpolation, finite-box delta-potential; NOT prereg B01-B24.

Static implementation signals (presence in source, not proof of execution): linear algebra, JSON output, internal assertion, hash verification.

Source excerpt (first 23 physical lines; whitespace and line order retained):

```
#!/usr/bin/env python3
"""Exploratory common-coordinate interpolation, finite-box delta-potential; NOT prereg B01-B24."""
import json,hashlib
from pathlib import Path
import numpy as np
root=Path('/mnt/data'); src=root/'UDCT_v7_13_Hydrus_3D_DeltaPotential_Exploratory.py'
s=src.read_text().split("out={'status'")[0]
needle=" return {'n':n,'spacing_over_rh':h/rh"
assert needle in s
replacement='''
 samples=[]
 for target in (0.5,1.,2.):
  position=target*rh/h+mid
  lo=int(np.floor(position));hi=lo+1;alpha=position-lo
  assert lo>=1 and hi+1<n
  def grad_at(ix):
   return np.array([(pot[ix+1,mid,mid]-pot[ix-1,mid,mid]),(pot[ix,mid+1,mid]-pot[ix,mid-1,mid]),(pot
  v=(1-alpha)*grad_at(lo)+alpha*grad_at(hi)
  samples.append({'target_radius_rh':target,'fractional_grid_index':float(position),'interpolated_de
 return {'n':n,'law':law.name,'ell_pc':ellpc,'spacing_over_rh':float(h/rh),'samples':samples,'relati
'''
s=s.replace(needle,replacement+' # unused return stub: '+needle)
ns={'__file__':str(src)}
```

Audit interpretation: Identify the exact inputs, units, normalization and frozen constants in the full script. Determine whether this file computes a new physical profile, reads prior profiles, or only checks a manufactured numerical identity. Inspect the handling of exceptions, finite values, boundaries and convergence; a successful process exit alone is not a scientific validation.

Reproduction note: compare any outputs with the companion JSON and/or NPZ by exact archive name and digest. The filename timestamp is metadata, not proof of run chronology. No independent execution of this script is claimed in this companion.

# Source 54/77 — UDCT_v7_13_Source_Localization_Boundary_Audit

Full archive filename: UDCT_v7_13_Source_Localization_Boundary_Audit_20261021_011840_423.py

Source length: 38 lines; bytes: 2635; SHA-256: c2292a794d7ed00bd214dffad7206311ffb87c268d15166b9f39dc7b11e6765d.

Functions: No named function definitions found.

Classes: none. Imports: import json, hashlib; from pathlib import Path; import numpy as np.

Opening source documentation: Exploratory boundary/source-localization audit of frozen 3D delta-source. No preregistered B01-B24 result, no physical boundary validation.

Static implementation signals (presence in source, not proof of execution): linear algebra, JSON output, hash verification, solver path.

Source excerpt (first 23 physical lines; whitespace and line order retained):

```
 1 #!/usr/bin/env python3
 2 """Exploratory boundary/source-localization audit of frozen 3D delta-source.
 3 No preregistered B01-B24 result, no physical boundary validation.
 4 """
 5 import json, hashlib
 6 from pathlib import Path
 7 import numpy as np
 8 root=Path('/mnt/data')
 9 src=root/'UDCT_v7_13_Hydrus_3D_DeltaPotential_Exploratory.py'
10 s=src.read_text().split("out={'status'")[0]
11 s=s.replace(' pot=solve(rhs,h)', '''
12  # Volume-integrated source and outer-shell contribution, before Poisson solve.
13  axis=(np.arange(n)-(n-1)/2)*h
14  xx,yy,zz=np.meshgrid(axis,axis,axis,indexing='ij')
15  rad=np.sqrt(xx**2+yy**2+zz**2)
16  shell=rad>=0.8*(L/2)
17  total_abs=float(np.sum(np.abs(rhs))*h**3)
18  shell_abs=float(np.sum(np.abs(rhs[shell]))*h**3)
19  monopole=float(np.sum(rhs)*h**3)
20  dipole=np.array([np.sum(rhs*xx),np.sum(rhs*yy),np.sum(rhs*zz)])*h**3
21  shell_rms=float(np.sqrt(np.mean(rhs[shell]**2)))
22  inner_rms=float(np.sqrt(np.mean(rhs[~shell]**2)))
23  return {'n':n,'box_side_over_rh':float(L/rh),'spacing_over_rh':float(h/rh),'law':law.name,'ell_pc':
```

Audit interpretation: Identify the exact inputs, units, normalization and frozen constants in the full script. Determine whether this file computes a new physical profile, reads prior profiles, or only checks a manufactured numerical identity. Inspect the handling of exceptions, finite values, boundaries and convergence; a successful process exit alone is not a scientific validation.

Reproduction note: compare any outputs with the companion JSON and/or NPZ by exact archive name and digest. The filename timestamp is metadata, not proof of run chronology. No independent execution of this script is claimed in this companion.

# Source 55/77 — v713_b01_six_arm_profile_audit

Full archive filename: v713_b01_six_arm_profile_audit_20261021_012444_551.py

Source length: 43 lines; bytes: 3168; SHA-256: e17af1355b0f01bc4196cfe0748b6b3f59504ddcc2b4c5c43bdd897437462be9.

Functions: load(path,name):).

Classes: none. Imports: from pathlib import Path; import json,csv,hashlib,importlib.util,sys,time; import numpy as np.

Opening source documentation: Exploratory B01 six-arm common-grid audit; not a preregistered confirmatory outcome.

Static implementation signals (presence in source, not proof of execution): JSON output, internal assertion, hash verification, solver path.

Source excerpt (first 23 physical lines; whitespace and line order retained):

```
 1 #!/usr/bin/env python3
 2 """Exploratory B01 six-arm common-grid audit; not a preregistered confirmatory outcome."""
 3 from pathlib import Path
 4 import json,csv,hashlib,importlib.util,sys,time
 5 import numpy as np
 6 R=Path('/mnt/data/v713_post_work')
 7 plan=json.loads((R/'preregistration_plan.json').read_text())
 8 for n,d in plan['frozen_hashes'].items():
 9  assert hashlib.sha256((R/n).read_bytes()).hexdigest()==d,n
10 def load(path,name):
11  spec=importlib.util.spec_from_file_location(name,path);m=importlib.util.module_from_spec(spec);sys.
12 old=load(R/'UDCT_v7_12_Preregistration_Jeans_Transition.py','v712_six_audit')
13 proto=load(R/'UDCT_v7_13_Six_Arm_Decision_Prototype.py','v713_six_proto')
14 m=old.load_frozen();b=m.load_baseline();obj=b.DWARFS['Hydrus I']
15 with (R/'adaptive_brackets.csv').open() as f: t=next(csv.DictReader(f))
16 assert t['axis']=='beta' and t['law']=='BASE'
17 configs=[('default',b.make_grid('default')),('refined',b.make_grid('refined')),('validation',b.Grid(
18 all_arms={float(t['lo']):{},float(t['hi']):{}}
19 per_arm=[];t0=time.monotonic()
20 for name,grid in configs:
21  e,h=b.V706.galactic_field(obj,grid)[:2]
22  for inner in (.002,.0005):
23   st=b.model_state(obj,'Tapered cusp',b.mesh('Tapered cusp',grid,rmin=inner),e,h,grid,b.BASE_LAW,flo
```

Audit interpretation: Identify the exact inputs, units, normalization and frozen constants in the full script. Determine whether this file computes a new physical profile, reads prior profiles, or only checks a manufactured numerical identity. Inspect the handling of exceptions, finite values, boundaries and convergence; a successful process exit alone is not a scientific validation.

Reproduction note: compare any outputs with the companion JSON and/or NPZ by exact archive name and digest. The filename timestamp is metadata, not proof of run chronology. No independent execution of this script is claimed in this companion.

# Source 56/77 — v713_b02_six_arm_profile_audit

Full archive filename: v713_b02_six_arm_profile_audit_20261021_012441_190.py

Source length: 43 lines; bytes: 3169; SHA-256: 6acbcc845b912aff0c3ff124c04b40119cc6b844355096dccf472cb2673d3d41.

Functions: load(path,name):).

Classes: none. Imports: from pathlib import Path; import json,csv,hashlib,importlib.util,sys,time; import numpy as np.

Opening source documentation: Exploratory B01 six-arm common-grid audit; not a preregistered confirmatory outcome.

Static implementation signals (presence in source, not proof of execution): JSON output, internal assertion, hash verification, solver path.

Source excerpt (first 23 physical lines; whitespace and line order retained):

```
 1 #!/usr/bin/env python3
 2 """Exploratory B01 six-arm common-grid audit; not a preregistered confirmatory outcome."""
 3 from pathlib import Path
 4 import json,csv,hashlib,importlib.util,sys,time
 5 import numpy as np
 6 R=Path('/mnt/data/v713_b02_run')
 7 plan=json.loads((R/'preregistration_plan.json').read_text())
 8 for n,d in plan['frozen_hashes'].items():
 9  assert hashlib.sha256((R/n).read_bytes()).hexdigest()==d,n
10 def load(path,name):
11  spec=importlib.util.spec_from_file_location(name,path);m=importlib.util.module_from_spec(spec);sys.
12 old=load(R/'UDCT_v7_12_Preregistration_Jeans_Transition.py','v712_six_audit')
13 proto=load(R/'UDCT_v7_13_Six_Arm_Decision_Prototype.py','v713_six_proto')
14 m=old.load_frozen();b=m.load_baseline();obj=b.DWARFS['Hydrus I']
15 with (R/'adaptive_brackets.csv').open() as f: t=list(csv.DictReader(f))[1]
16 assert t['axis']=='beta' and t['law']=='BASE'
17 configs=[('default',b.make_grid('default')),('refined',b.make_grid('refined')),('validation',b.Grid(
18 all_arms={float(t['lo']):{},float(t['hi']):{}}
19 per_arm=[];t0=time.monotonic()
20 for name,grid in configs:
21  e,h=b.V706.galactic_field(obj,grid)[:2]
22  for inner in (.002,.0005):
23   st=b.model_state(obj,'Tapered cusp',b.mesh('Tapered cusp',grid,rmin=inner),e,h,grid,b.BASE_LAW,flo
```

Audit interpretation: Identify the exact inputs, units, normalization and frozen constants in the full script. Determine whether this file computes a new physical profile, reads prior profiles, or only checks a manufactured numerical identity. Inspect the handling of exceptions, finite values, boundaries and convergence; a successful process exit alone is not a scientific validation.

Reproduction note: compare any outputs with the companion JSON and/or NPZ by exact archive name and digest. The filename timestamp is metadata, not proof of run chronology. No independent execution of this script is claimed in this companion.

# Source 57/77 — v713_b03_six_arm_profile_audit

Full archive filename: v713_b03_six_arm_profile_audit_20261021_012437_272.py

Source length: 43 lines; bytes: 3169; SHA-256: 85b1ac132c0de0ab448a934321bb9e427d714c5ba71caf6188b24f03a63cb718.

Functions: load(path,name):).

Classes: none. Imports: from pathlib import Path; import json,csv,hashlib,importlib.util,sys,time; import numpy as np.

Opening source documentation: Exploratory B03 six-arm common-grid audit; not a preregistered confirmatory outcome.

Static implementation signals (presence in source, not proof of execution): JSON output, internal assertion, hash verification, solver path.

Source excerpt (first 23 physical lines; whitespace and line order retained):

```
 1 #!/usr/bin/env python3
 2 """Exploratory B03 six-arm common-grid audit; not a preregistered confirmatory outcome."""
 3 from pathlib import Path
 4 import json,csv,hashlib,importlib.util,sys,time
 5 import numpy as np
 6 R=Path('/mnt/data/v713_b02_run')
 7 plan=json.loads((R/'preregistration_plan.json').read_text())
 8 for n,d in plan['frozen_hashes'].items():
 9  assert hashlib.sha256((R/n).read_bytes()).hexdigest()==d,n
10 def load(path,name):
11  spec=importlib.util.spec_from_file_location(name,path);m=importlib.util.module_from_spec(spec);sys.
12 old=load(R/'UDCT_v7_12_Preregistration_Jeans_Transition.py','v712_six_audit')
13 proto=load(R/'UDCT_v7_13_Six_Arm_Decision_Prototype.py','v713_six_proto')
14 m=old.load_frozen();b=m.load_baseline();obj=b.DWARFS['Hydrus I']
15 with (R/'adaptive_brackets.csv').open() as f: t=list(csv.DictReader(f))[2]
16 assert t['axis']=='beta' and t['law']=='BASE'
17 configs=[('default',b.make_grid('default')),('refined',b.make_grid('refined')),('validation',b.Grid(
18 all_arms={float(t['lo']):{},float(t['hi']):{}}
19 per_arm=[];t0=time.monotonic()
20 for name,grid in configs:
21  e,h=b.V706.galactic_field(obj,grid)[:2]
22  for inner in (.002,.0005):
23   st=b.model_state(obj,'Tapered cusp',b.mesh('Tapered cusp',grid,rmin=inner),e,h,grid,b.BASE_LAW,flo
```

Audit interpretation: Identify the exact inputs, units, normalization and frozen constants in the full script. Determine whether this file computes a new physical profile, reads prior profiles, or only checks a manufactured numerical identity. Inspect the handling of exceptions, finite values, boundaries and convergence; a successful process exit alone is not a scientific validation.

Reproduction note: compare any outputs with the companion JSON and/or NPZ by exact archive name and digest. The filename timestamp is metadata, not proof of run chronology. No independent execution of this script is claimed in this companion.

# Source 58/77 — v713_b04_six_arm_profile_audit

Full archive filename: v713_b04_six_arm_profile_audit_20261021_012434_168.py

Source length: 44 lines; bytes: 3201; SHA-256: 5efbf820496e151281b4b7fe78ae5d0f41a0afc9ea0cbd78272ae44676edd535.

Functions: load(path,name):).

Classes: none. Imports: from pathlib import Path; import json,csv,hashlib,importlib.util,sys,time; import numpy as np.

Opening source documentation: Exploratory B04 six-arm common-grid audit; not a preregistered confirmatory outcome.

Static implementation signals (presence in source, not proof of execution): JSON output, internal assertion, hash verification, solver path.

Source excerpt (first 23 physical lines; whitespace and line order retained):

```
 1 #!/usr/bin/env python3
 2 """Exploratory B04 six-arm common-grid audit; not a preregistered confirmatory outcome."""
 3 from pathlib import Path
 4 import json,csv,hashlib,importlib.util,sys,time
 5 import numpy as np
 6 R=Path('/mnt/data/v713_b02_run')
 7 plan=json.loads((R/'preregistration_plan.json').read_text())
 8 for n,d in plan['frozen_hashes'].items():
 9  assert hashlib.sha256((R/n).read_bytes()).hexdigest()==d,n
10 def load(path,name):
11  spec=importlib.util.spec_from_file_location(name,path);m=importlib.util.module_from_spec(spec);sys.
12 old=load(R/'UDCT_v7_12_Preregistration_Jeans_Transition.py','v712_six_audit')
13 proto=load(R/'UDCT_v7_13_Six_Arm_Decision_Prototype.py','v713_six_proto')
14 m=old.load_frozen();b=m.load_baseline();obj=b.DWARFS['Hydrus I']
15 with (R/'adaptive_brackets.csv').open() as f: t=list(csv.DictReader(f))[3]
16 assert t['axis']=='ell_pc' and t['law']=='BASE'
17 configs=[('default',b.make_grid('default')),('refined',b.make_grid('refined')),('validation',b.Grid(
18 all_arms={float(t['lo']):{},float(t['hi']):{}}
19 per_arm=[];t0=time.monotonic()
20 for name,grid in configs:
21  e,h=b.V706.galactic_field(obj,grid)[:2]
22  for inner in (.002,.0005):
23   for ell in all_arms:
```

Audit interpretation: Identify the exact inputs, units, normalization and frozen constants in the full script. Determine whether this file computes a new physical profile, reads prior profiles, or only checks a manufactured numerical identity. Inspect the handling of exceptions, finite values, boundaries and convergence; a successful process exit alone is not a scientific validation.

Reproduction note: compare any outputs with the companion JSON and/or NPZ by exact archive name and digest. The filename timestamp is metadata, not proof of run chronology. No independent execution of this script is claimed in this companion.

# Source 59/77 — v713_b05_six_arm_profile_audit

Full archive filename: v713_b05_six_arm_profile_audit_20261021_012430_260.py

Source length: 44 lines; bytes: 3201; SHA-256: b6567789b3bb36f36a03bb0f03aa53f7e1c2a7d1b05ec425d181c1cdd01c2d76.

Functions: load(path,name):).

Classes: none. Imports: from pathlib import Path; import json,csv,hashlib,importlib.util,sys,time; import numpy as np.

Opening source documentation: Exploratory B05 six-arm common-grid audit; not a preregistered confirmatory outcome.

Static implementation signals (presence in source, not proof of execution): JSON output, internal assertion, hash verification, solver path.

Source excerpt (first 23 physical lines; whitespace and line order retained):

```
 1 #!/usr/bin/env python3
 2 """Exploratory B05 six-arm common-grid audit; not a preregistered confirmatory outcome."""
 3 from pathlib import Path
 4 import json,csv,hashlib,importlib.util,sys,time
 5 import numpy as np
 6 R=Path('/mnt/data/v713_b02_run')
 7 plan=json.loads((R/'preregistration_plan.json').read_text())
 8 for n,d in plan['frozen_hashes'].items():
 9  assert hashlib.sha256((R/n).read_bytes()).hexdigest()==d,n
10 def load(path,name):
11  spec=importlib.util.spec_from_file_location(name,path);m=importlib.util.module_from_spec(spec);sys.
12 old=load(R/'UDCT_v7_12_Preregistration_Jeans_Transition.py','v712_six_audit')
13 proto=load(R/'UDCT_v7_13_Six_Arm_Decision_Prototype.py','v713_six_proto')
14 m=old.load_frozen();b=m.load_baseline();obj=b.DWARFS['Hydrus I']
15 with (R/'adaptive_brackets.csv').open() as f: t=list(csv.DictReader(f))[4]
16 assert t['axis']=='ell_pc' and t['law']=='BASE'
17 configs=[('default',b.make_grid('default')),('refined',b.make_grid('refined')),('validation',b.Grid(
18 all_arms={float(t['lo']):{},float(t['hi']):{}}
19 per_arm=[];t0=time.monotonic()
20 for name,grid in configs:
21  e,h=b.V706.galactic_field(obj,grid)[:2]
22  for inner in (.002,.0005):
23   for ell in all_arms:
```

Audit interpretation: Identify the exact inputs, units, normalization and frozen constants in the full script. Determine whether this file computes a new physical profile, reads prior profiles, or only checks a manufactured numerical identity. Inspect the handling of exceptions, finite values, boundaries and convergence; a successful process exit alone is not a scientific validation.

Reproduction note: compare any outputs with the companion JSON and/or NPZ by exact archive name and digest. The filename timestamp is metadata, not proof of run chronology. No independent execution of this script is claimed in this companion.

# Source 60/77 — v713_b07_six_arm_profile_audit

Full archive filename: v713_b07_six_arm_profile_audit_20261021_012418_439.py

Source length: 43 lines; bytes: 3209; SHA-256: f81f02bc7a61c25fb56213f79fa435a9665fe534193dde07bd43aedf88985c24.

Functions: load(path,name):).

Classes: none. Imports: from pathlib import Path; import json,csv,hashlib,importlib.util,sys,time; import numpy as np.

Opening source documentation: Exploratory B07 six-arm common-grid audit; not a preregistered confirmatory outcome.

Static implementation signals (presence in source, not proof of execution): JSON output, internal assertion, hash verification, solver path.

Source excerpt (first 23 physical lines; whitespace and line order retained):

```
 1 #!/usr/bin/env python3
 2 """Exploratory B07 six-arm common-grid audit; not a preregistered confirmatory outcome."""
 3 from pathlib import Path
 4 import json,csv,hashlib,importlib.util,sys,time
 5 import numpy as np
 6 R=Path('/mnt/data/v713_b07_run')
 7 plan=json.loads((R/'preregistration_plan.json').read_text())
 8 for n,d in plan['frozen_hashes'].items():
 9  assert hashlib.sha256((R/n).read_bytes()).hexdigest()==d,n
10 def load(path,name):
11  spec=importlib.util.spec_from_file_location(name,path);m=importlib.util.module_from_spec(spec);sys.
12 old=load(R/'UDCT_v7_12_Preregistration_Jeans_Transition.py','v712_six_audit')
13 proto=load(R/'UDCT_v7_13_Six_Arm_Decision_Prototype.py','v713_six_proto')
14 m=old.load_frozen();b=m.load_baseline();obj=b.DWARFS['Hydrus I']
15 with (R/'adaptive_brackets.csv').open() as f: t=list(csv.DictReader(f))[6]
16 assert t['axis']=='beta' and t['law']=='A-f2'
17 configs=[('default',b.make_grid('default')),('refined',b.make_grid('refined')),('validation',b.Grid(
18 all_arms={float(t['lo']):{},float(t['hi']):{}}
19 per_arm=[];t0=time.monotonic()
20 for name,grid in configs:
21  e,h=b.V706.galactic_field(obj,grid)[:2]
22  for inner in (.002,.0005):
23   st=b.model_state(obj,'Tapered cusp',b.mesh('Tapered cusp',grid,rmin=inner),e,h,grid,next(l for l i
```

Audit interpretation: Identify the exact inputs, units, normalization and frozen constants in the full script. Determine whether this file computes a new physical profile, reads prior profiles, or only checks a manufactured numerical identity. Inspect the handling of exceptions, finite values, boundaries and convergence; a successful process exit alone is not a scientific validation.

Reproduction note: compare any outputs with the companion JSON and/or NPZ by exact archive name and digest. The filename timestamp is metadata, not proof of run chronology. No independent execution of this script is claimed in this companion.

# Source 61/77 — v713_b08_six_arm_profile_audit

Full archive filename: v713_b08_six_arm_profile_audit_20261021_012414_332.py

Source length: 43 lines; bytes: 3247; SHA-256: eba91e72b893a70da198c19dc809efd7dfc410008d343f98891719644e5e3a69.

Functions: load(path,name):).

Classes: none. Imports: from pathlib import Path; import json,csv,hashlib,importlib.util,sys,time; import numpy as np.

Opening source documentation: Exploratory B07 six-arm common-grid audit; not a preregistered confirmatory outcome.

Static implementation signals (presence in source, not proof of execution): JSON output, internal assertion, hash verification, solver path.

Source excerpt (first 23 physical lines; whitespace and line order retained):

```
 1 #!/usr/bin/env python3
 2 """Exploratory B07 six-arm common-grid audit; not a preregistered confirmatory outcome."""
 3 from pathlib import Path
 4 import json,csv,hashlib,importlib.util,sys,time
 5 import numpy as np
 6 R=Path('/mnt/data/v713_b07_run')
 7 plan=json.loads((R/'preregistration_plan.json').read_text())
 8 for n,d in plan['frozen_hashes'].items():
 9  assert hashlib.sha256((R/n).read_bytes()).hexdigest()==d,n
10 def load(path,name):
11  spec=importlib.util.spec_from_file_location(name,path);m=importlib.util.module_from_spec(spec);sys.
12 old=load(R/'UDCT_v7_12_Preregistration_Jeans_Transition.py','v712_six_audit')
13 proto=load(R/'UDCT_v7_13_Six_Arm_Decision_Prototype.py','v713_six_proto')
14 m=old.load_frozen();b=m.load_baseline();obj=b.DWARFS['Hydrus I']
15 with (R/'adaptive_brackets.csv').open() as f: t=list(csv.DictReader(f))[7]
16 assert t['axis']=='beta' and t['law']=='A-f2' and float(t['fixed_coordinate'])==1.2
17 configs=[('default',b.make_grid('default')),('refined',b.make_grid('refined')),('validation',b.Grid(
18 all_arms={float(t['lo']):{},float(t['hi']):{}}
19 per_arm=[];t0=time.monotonic()
20 for name,grid in configs:
21  e,h=b.V706.galactic_field(obj,grid)[:2]
22  for inner in (.002,.0005):
23   st=b.model_state(obj,'Tapered cusp',b.mesh('Tapered cusp',grid,rmin=inner),e,h,grid,next(l for l i
```

Audit interpretation: Identify the exact inputs, units, normalization and frozen constants in the full script. Determine whether this file computes a new physical profile, reads prior profiles, or only checks a manufactured numerical identity. Inspect the handling of exceptions, finite values, boundaries and convergence; a successful process exit alone is not a scientific validation.

Reproduction note: compare any outputs with the companion JSON and/or NPZ by exact archive name and digest. The filename timestamp is metadata, not proof of run chronology. No independent execution of this script is claimed in this companion.

# Source 62/77 — v713_b09_six_arm_profile_audit

Full archive filename: v713_b09_six_arm_profile_audit_20261021_012410_509.py

Source length: 43 lines; bytes: 3247; SHA-256: 28d9a5de9d2e6556435cd25ff84c44599f5f61f1b653834bd28c5de59001e60e.

Functions: load(path,name):).

Classes: none. Imports: from pathlib import Path; import json,csv,hashlib,importlib.util,sys,time; import numpy as np.

Opening source documentation: Exploratory B09 six-arm common-grid audit; not a preregistered confirmatory outcome.

Static implementation signals (presence in source, not proof of execution): JSON output, internal assertion, hash verification, solver path.

Source excerpt (first 23 physical lines; whitespace and line order retained):

```
 1 #!/usr/bin/env python3
 2 """Exploratory B09 six-arm common-grid audit; not a preregistered confirmatory outcome."""
 3 from pathlib import Path
 4 import json,csv,hashlib,importlib.util,sys,time
 5 import numpy as np
 6 R=Path('/mnt/data/v713_b07_run')
 7 plan=json.loads((R/'preregistration_plan.json').read_text())
 8 for n,d in plan['frozen_hashes'].items():
 9  assert hashlib.sha256((R/n).read_bytes()).hexdigest()==d,n
10 def load(path,name):
11  spec=importlib.util.spec_from_file_location(name,path);m=importlib.util.module_from_spec(spec);sys.
12 old=load(R/'UDCT_v7_12_Preregistration_Jeans_Transition.py','v712_six_audit')
13 proto=load(R/'UDCT_v7_13_Six_Arm_Decision_Prototype.py','v713_six_proto')
14 m=old.load_frozen();b=m.load_baseline();obj=b.DWARFS['Hydrus I']
15 with (R/'adaptive_brackets.csv').open() as f: t=list(csv.DictReader(f))[8]
16 assert t['axis']=='beta' and t['law']=='A-f2' and float(t['fixed_coordinate'])==1.5
17 configs=[('default',b.make_grid('default')),('refined',b.make_grid('refined')),('validation',b.Grid(
18 all_arms={float(t['lo']):{},float(t['hi']):{}}
19 per_arm=[];t0=time.monotonic()
20 for name,grid in configs:
21  e,h=b.V706.galactic_field(obj,grid)[:2]
22  for inner in (.002,.0005):
23   st=b.model_state(obj,'Tapered cusp',b.mesh('Tapered cusp',grid,rmin=inner),e,h,grid,next(l for l i
```

Audit interpretation: Identify the exact inputs, units, normalization and frozen constants in the full script. Determine whether this file computes a new physical profile, reads prior profiles, or only checks a manufactured numerical identity. Inspect the handling of exceptions, finite values, boundaries and convergence; a successful process exit alone is not a scientific validation.

Reproduction note: compare any outputs with the companion JSON and/or NPZ by exact archive name and digest. The filename timestamp is metadata, not proof of run chronology. No independent execution of this script is claimed in this companion.

# Source 63/77 — v713_b10_six_arm_profile_audit

Full archive filename: v713_b10_six_arm_profile_audit_20261021_012405_948.py

Source length: 43 lines; bytes: 3255; SHA-256: 7122406b4a868b67fb2f8450806891305c03eb057479094da5d3fc7bc24caba7.

Functions: load(path,name):).

Classes: none. Imports: from pathlib import Path; import json,csv,hashlib,importlib.util,sys,time; import numpy as np.

Opening source documentation: Exploratory B10 six-arm common-grid audit; not a preregistered confirmatory outcome.

Static implementation signals (presence in source, not proof of execution): JSON output, internal assertion, hash verification, solver path.

Source excerpt (first 23 physical lines; whitespace and line order retained):

```
 1 #!/usr/bin/env python3
 2 """Exploratory B10 six-arm common-grid audit; not a preregistered confirmatory outcome."""
 3 from pathlib import Path
 4 import json,csv,hashlib,importlib.util,sys,time
 5 import numpy as np
 6 R=Path('/mnt/data/v713_b07_run')
 7 plan=json.loads((R/'preregistration_plan.json').read_text())
 8 for n,d in plan['frozen_hashes'].items():
 9  assert hashlib.sha256((R/n).read_bytes()).hexdigest()==d,n
10 def load(path,name):
11  spec=importlib.util.spec_from_file_location(name,path);m=importlib.util.module_from_spec(spec);sys.
12 old=load(R/'UDCT_v7_12_Preregistration_Jeans_Transition.py','v712_six_audit')
13 proto=load(R/'UDCT_v7_13_Six_Arm_Decision_Prototype.py','v713_six_proto')
14 m=old.load_frozen();b=m.load_baseline();obj=b.DWARFS['Hydrus I']
15 with (R/'adaptive_brackets.csv').open() as f: t=list(csv.DictReader(f))[9]
16 assert t['axis']=='ell_pc' and t['law']=='A-f2' and float(t['fixed_coordinate'])==0.45
17 configs=[('default',b.make_grid('default')),('refined',b.make_grid('refined')),('validation',b.Grid(
18 all_arms={float(t['lo']):{},float(t['hi']):{}}
19 per_arm=[];t0=time.monotonic()
20 for name,grid in configs:
21  e,h=b.V706.galactic_field(obj,grid)[:2]
22  for inner in (.002,.0005):
23   for ell in all_arms:
```

Audit interpretation: Identify the exact inputs, units, normalization and frozen constants in the full script. Determine whether this file computes a new physical profile, reads prior profiles, or only checks a manufactured numerical identity. Inspect the handling of exceptions, finite values, boundaries and convergence; a successful process exit alone is not a scientific validation.

Reproduction note: compare any outputs with the companion JSON and/or NPZ by exact archive name and digest. The filename timestamp is metadata, not proof of run chronology. No independent execution of this script is claimed in this companion.

# Source 64/77 — v713_b11_six_arm_profile_audit

Full archive filename: v713_b11_six_arm_profile_audit_20261021_012359_216.py

Source length: 43 lines; bytes: 3255; SHA-256: c14ddf61a266c47a4c8ac72da7a149ee116c6508fd206382072eb70f8b4e3efd.

Functions: load(path,name):).

Classes: none. Imports: from pathlib import Path; import json,csv,hashlib,importlib.util,sys,time; import numpy as np.

Opening source documentation: Exploratory B11 six-arm common-grid audit; not a preregistered confirmatory outcome.

Static implementation signals (presence in source, not proof of execution): JSON output, internal assertion, hash verification, solver path.

Source excerpt (first 23 physical lines; whitespace and line order retained):

```
 1 #!/usr/bin/env python3
 2 """Exploratory B11 six-arm common-grid audit; not a preregistered confirmatory outcome."""
 3 from pathlib import Path
 4 import json,csv,hashlib,importlib.util,sys,time
 5 import numpy as np
 6 R=Path('/mnt/data/v713_b07_run')
 7 plan=json.loads((R/'preregistration_plan.json').read_text())
 8 for n,d in plan['frozen_hashes'].items():
 9  assert hashlib.sha256((R/n).read_bytes()).hexdigest()==d,n
10 def load(path,name):
11  spec=importlib.util.spec_from_file_location(name,path);m=importlib.util.module_from_spec(spec);sys.
12 old=load(R/'UDCT_v7_12_Preregistration_Jeans_Transition.py','v712_six_audit')
13 proto=load(R/'UDCT_v7_13_Six_Arm_Decision_Prototype.py','v713_six_proto')
14 m=old.load_frozen();b=m.load_baseline();obj=b.DWARFS['Hydrus I']
15 with (R/'adaptive_brackets.csv').open() as f: t=list(csv.DictReader(f))[10]
16 assert t['axis']=='ell_pc' and t['law']=='A-f2' and float(t['fixed_coordinate'])==0.5
17 configs=[('default',b.make_grid('default')),('refined',b.make_grid('refined')),('validation',b.Grid(
18 all_arms={float(t['lo']):{},float(t['hi']):{}}
19 per_arm=[];t0=time.monotonic()
20 for name,grid in configs:
21  e,h=b.V706.galactic_field(obj,grid)[:2]
22  for inner in (.002,.0005):
23   for ell in all_arms:
```

Audit interpretation: Identify the exact inputs, units, normalization and frozen constants in the full script. Determine whether this file computes a new physical profile, reads prior profiles, or only checks a manufactured numerical identity. Inspect the handling of exceptions, finite values, boundaries and convergence; a successful process exit alone is not a scientific validation.

Reproduction note: compare any outputs with the companion JSON and/or NPZ by exact archive name and digest. The filename timestamp is metadata, not proof of run chronology. No independent execution of this script is claimed in this companion.

# Source 65/77 — v713_b12_six_arm_profile_audit

Full archive filename: v713_b12_six_arm_profile_audit_20261021_011800_309.py

Source length: 43 lines; bytes: 3251; SHA-256: a9d3b181a2bdfe33f0ce70896bfbf7e3ac3ddda21d16c1bae01460237fcf9caa.

Functions: load(path,name):).

Classes: none. Imports: from pathlib import Path; import json,csv,hashlib,importlib.util,sys,time; import numpy as np.

Opening source documentation: Exploratory B12 six-arm common-grid audit; not a preregistered confirmatory outcome.

Static implementation signals (presence in source, not proof of execution): JSON output, internal assertion, hash verification, solver path.

Source excerpt (first 23 physical lines; whitespace and line order retained):

```
 1 #!/usr/bin/env python3
 2 """Exploratory B12 six-arm common-grid audit; not a preregistered confirmatory outcome."""
 3 from pathlib import Path
 4 import json,csv,hashlib,importlib.util,sys,time
 5 import numpy as np
 6 R=Path('/mnt/data/v713_b07_run')
 7 plan=json.loads((R/'preregistration_plan.json').read_text())
 8 for n,d in plan['frozen_hashes'].items():
 9  assert hashlib.sha256((R/n).read_bytes()).hexdigest()==d,n
10 def load(path,name):
11  spec=importlib.util.spec_from_file_location(name,path);m=importlib.util.module_from_spec(spec);sys.
12 old=load(R/'UDCT_v7_12_Preregistration_Jeans_Transition.py','v712_six_audit')
13 proto=load(R/'UDCT_v7_13_Six_Arm_Decision_Prototype.py','v713_six_proto')
14 m=old.load_frozen();b=m.load_baseline();obj=b.DWARFS['Hydrus I']
15 with (R/'adaptive_brackets.csv').open() as f: t=list(csv.DictReader(f))[11]
16 assert t['axis']=='beta' and t['law']=='A-Qs' and float(t['fixed_coordinate'])==1.0
17 configs=[('default',b.make_grid('default')),('refined',b.make_grid('refined')),('validation',b.Grid(
18 all_arms={float(t['lo']):{},float(t['hi']):{}}
19 per_arm=[];t0=time.monotonic()
20 for name,grid in configs:
21  e,h=b.V706.galactic_field(obj,grid)[:2]
22  for inner in (.002,.0005):
23   for ell in all_arms:
```

Audit interpretation: Identify the exact inputs, units, normalization and frozen constants in the full script. Determine whether this file computes a new physical profile, reads prior profiles, or only checks a manufactured numerical identity. Inspect the handling of exceptions, finite values, boundaries and convergence; a successful process exit alone is not a scientific validation.

Reproduction note: compare any outputs with the companion JSON and/or NPZ by exact archive name and digest. The filename timestamp is metadata, not proof of run chronology. No independent execution of this script is claimed in this companion.

# Source 66/77 — v713_b13_six_arm_profile_audit

Full archive filename: v713_b13_six_arm_profile_audit_20261021_012350_682.py

Source length: 43 lines; bytes: 3251; SHA-256: c5027120d2e01596e58a7323852ee2fcbf9ce10e3e5cb97879f7f6ec7cd51c8f.

Functions: load(path,name):).

Classes: none. Imports: from pathlib import Path; import json,csv,hashlib,importlib.util,sys,time; import numpy as np.

Opening source documentation: Exploratory B13 six-arm common-grid audit; not a preregistered confirmatory outcome.

Static implementation signals (presence in source, not proof of execution): JSON output, internal assertion, hash verification, solver path.

Source excerpt (first 23 physical lines; whitespace and line order retained):

```
 1 #!/usr/bin/env python3
 2 """Exploratory B13 six-arm common-grid audit; not a preregistered confirmatory outcome."""
 3 from pathlib import Path
 4 import json,csv,hashlib,importlib.util,sys,time
 5 import numpy as np
 6 R=Path('/mnt/data/v713_b07_run')
 7 plan=json.loads((R/'preregistration_plan.json').read_text())
 8 for n,d in plan['frozen_hashes'].items():
 9  assert hashlib.sha256((R/n).read_bytes()).hexdigest()==d,n
10 def load(path,name):
11  spec=importlib.util.spec_from_file_location(name,path);m=importlib.util.module_from_spec(spec);sys.
12 old=load(R/'UDCT_v7_12_Preregistration_Jeans_Transition.py','v712_six_audit')
13 proto=load(R/'UDCT_v7_13_Six_Arm_Decision_Prototype.py','v713_six_proto')
14 m=old.load_frozen();b=m.load_baseline();obj=b.DWARFS['Hydrus I']
15 with (R/'adaptive_brackets.csv').open() as f: t=list(csv.DictReader(f))[12]
16 assert t['axis']=='beta' and t['law']=='A-Qs' and float(t['fixed_coordinate'])==1.2
17 configs=[('default',b.make_grid('default')),('refined',b.make_grid('refined')),('validation',b.Grid(
18 all_arms={float(t['lo']):{},float(t['hi']):{}}
19 per_arm=[];t0=time.monotonic()
20 for name,grid in configs:
21  e,h=b.V706.galactic_field(obj,grid)[:2]
22  for inner in (.002,.0005):
23   for ell in all_arms:
```

Audit interpretation: Identify the exact inputs, units, normalization and frozen constants in the full script. Determine whether this file computes a new physical profile, reads prior profiles, or only checks a manufactured numerical identity. Inspect the handling of exceptions, finite values, boundaries and convergence; a successful process exit alone is not a scientific validation.

Reproduction note: compare any outputs with the companion JSON and/or NPZ by exact archive name and digest. The filename timestamp is metadata, not proof of run chronology. No independent execution of this script is claimed in this companion.

# Source 67/77 — v713_b14_six_arm_profile_audit

Full archive filename: v713_b14_six_arm_profile_audit_20261021_012340_232.py

Source length: 43 lines; bytes: 3251; SHA-256: 5750cf8c95bf0c9322d650f892da513a8870c64cfa85db544e5e2bdce151a802.

Functions: load(path,name):).

Classes: none. Imports: from pathlib import Path; import json,csv,hashlib,importlib.util,sys,time; import numpy as np.

Opening source documentation: Exploratory B14 six-arm common-grid audit; not a preregistered confirmatory outcome.

Static implementation signals (presence in source, not proof of execution): JSON output, internal assertion, hash verification, solver path.

Source excerpt (first 23 physical lines; whitespace and line order retained):

```
 1 #!/usr/bin/env python3
 2 """Exploratory B14 six-arm common-grid audit; not a preregistered confirmatory outcome."""
 3 from pathlib import Path
 4 import json,csv,hashlib,importlib.util,sys,time
 5 import numpy as np
 6 R=Path('/mnt/data/v713_b07_run')
 7 plan=json.loads((R/'preregistration_plan.json').read_text())
 8 for n,d in plan['frozen_hashes'].items():
 9  assert hashlib.sha256((R/n).read_bytes()).hexdigest()==d,n
10 def load(path,name):
11  spec=importlib.util.spec_from_file_location(name,path);m=importlib.util.module_from_spec(spec);sys.
12 old=load(R/'UDCT_v7_12_Preregistration_Jeans_Transition.py','v712_six_audit')
13 proto=load(R/'UDCT_v7_13_Six_Arm_Decision_Prototype.py','v713_six_proto')
14 m=old.load_frozen();b=m.load_baseline();obj=b.DWARFS['Hydrus I']
15 with (R/'adaptive_brackets.csv').open() as f: t=list(csv.DictReader(f))[13]
16 assert t['axis']=='beta' and t['law']=='A-Qs' and float(t['fixed_coordinate'])==1.5
17 configs=[('default',b.make_grid('default')),('refined',b.make_grid('refined')),('validation',b.Grid(
18 all_arms={float(t['lo']):{},float(t['hi']):{}}
19 per_arm=[];t0=time.monotonic()
20 for name,grid in configs:
21  e,h=b.V706.galactic_field(obj,grid)[:2]
22  for inner in (.002,.0005):
23   for ell in all_arms:
```

Audit interpretation: Identify the exact inputs, units, normalization and frozen constants in the full script. Determine whether this file computes a new physical profile, reads prior profiles, or only checks a manufactured numerical identity. Inspect the handling of exceptions, finite values, boundaries and convergence; a successful process exit alone is not a scientific validation.

Reproduction note: compare any outputs with the companion JSON and/or NPZ by exact archive name and digest. The filename timestamp is metadata, not proof of run chronology. No independent execution of this script is claimed in this companion.

# Source 68/77 — v713_b15_six_arm_profile_audit

Full archive filename: v713_b15_six_arm_profile_audit_20261021_012336_457.py

Source length: 43 lines; bytes: 3251; SHA-256: 055883ebd1d7c86303438c71310713bc28db9dcd415f72e4f3bfaed01c4e4ffb.

Functions: load(path,name):).

Classes: none. Imports: from pathlib import Path; import json,csv,hashlib,importlib.util,sys,time; import numpy as np.

Opening source documentation: Exploratory B15 six-arm common-grid audit; not a preregistered confirmatory outcome.

Static implementation signals (presence in source, not proof of execution): JSON output, internal assertion, hash verification, solver path.

Source excerpt (first 23 physical lines; whitespace and line order retained):

```
 1 #!/usr/bin/env python3
 2 """Exploratory B15 six-arm common-grid audit; not a preregistered confirmatory outcome."""
 3 from pathlib import Path
 4 import json,csv,hashlib,importlib.util,sys,time
 5 import numpy as np
 6 R=Path('/mnt/data/v713_b07_run')
 7 plan=json.loads((R/'preregistration_plan.json').read_text())
 8 for n,d in plan['frozen_hashes'].items():
 9  assert hashlib.sha256((R/n).read_bytes()).hexdigest()==d,n
10 def load(path,name):
11  spec=importlib.util.spec_from_file_location(name,path);m=importlib.util.module_from_spec(spec);sys.
12 old=load(R/'UDCT_v7_12_Preregistration_Jeans_Transition.py','v712_six_audit')
13 proto=load(R/'UDCT_v7_13_Six_Arm_Decision_Prototype.py','v713_six_proto')
14 m=old.load_frozen();b=m.load_baseline();obj=b.DWARFS['Hydrus I']
15 with (R/'adaptive_brackets.csv').open() as f: t=list(csv.DictReader(f))[14]
16 assert t['axis']=='beta' and t['law']=='A-Qs' and float(t['fixed_coordinate'])==2.0
17 configs=[('default',b.make_grid('default')),('refined',b.make_grid('refined')),('validation',b.Grid(
18 all_arms={float(t['lo']):{},float(t['hi']):{}}
19 per_arm=[];t0=time.monotonic()
20 for name,grid in configs:
21  e,h=b.V706.galactic_field(obj,grid)[:2]
22  for inner in (.002,.0005):
23   for ell in all_arms:
```

Audit interpretation: Identify the exact inputs, units, normalization and frozen constants in the full script. Determine whether this file computes a new physical profile, reads prior profiles, or only checks a manufactured numerical identity. Inspect the handling of exceptions, finite values, boundaries and convergence; a successful process exit alone is not a scientific validation.

Reproduction note: compare any outputs with the companion JSON and/or NPZ by exact archive name and digest. The filename timestamp is metadata, not proof of run chronology. No independent execution of this script is claimed in this companion.

# Source 69/77 — v713_b16_six_arm_profile_audit

Full archive filename: v713_b16_six_arm_profile_audit_20261021_012332_233.py

Source length: 43 lines; bytes: 3255; SHA-256: 1d6c4e8982b3347ec45618de0fa4652e1fbac7120ac0e2a61b921c34062d1eb4.

Functions: load(path,name):).

Classes: none. Imports: from pathlib import Path; import json,csv,hashlib,importlib.util,sys,time; import numpy as np.

Opening source documentation: Exploratory B16 six-arm common-grid audit; not a preregistered confirmatory outcome.

Static implementation signals (presence in source, not proof of execution): JSON output, internal assertion, hash verification, solver path.

Source excerpt (first 23 physical lines; whitespace and line order retained):

```
 1 #!/usr/bin/env python3
 2 """Exploratory B16 six-arm common-grid audit; not a preregistered confirmatory outcome."""
 3 from pathlib import Path
 4 import json,csv,hashlib,importlib.util,sys,time
 5 import numpy as np
 6 R=Path('/mnt/data/v713_b07_run')
 7 plan=json.loads((R/'preregistration_plan.json').read_text())
 8 for n,d in plan['frozen_hashes'].items():
 9  assert hashlib.sha256((R/n).read_bytes()).hexdigest()==d,n
10 def load(path,name):
11  spec=importlib.util.spec_from_file_location(name,path);m=importlib.util.module_from_spec(spec);sys.
12 old=load(R/'UDCT_v7_12_Preregistration_Jeans_Transition.py','v712_six_audit')
13 proto=load(R/'UDCT_v7_13_Six_Arm_Decision_Prototype.py','v713_six_proto')
14 m=old.load_frozen();b=m.load_baseline();obj=b.DWARFS['Hydrus I']
15 with (R/'adaptive_brackets.csv').open() as f: t=list(csv.DictReader(f))[15]
16 assert t['axis']=='ell_pc' and t['law']=='A-Qs' and float(t['fixed_coordinate'])==0.4
17 configs=[('default',b.make_grid('default')),('refined',b.make_grid('refined')),('validation',b.Grid(
18 all_arms={float(t['lo']):{},float(t['hi']):{}}
19 per_arm=[];t0=time.monotonic()
20 for name,grid in configs:
21  e,h=b.V706.galactic_field(obj,grid)[:2]
22  for inner in (.002,.0005):
23   for ell in all_arms:
```

Audit interpretation: Identify the exact inputs, units, normalization and frozen constants in the full script. Determine whether this file computes a new physical profile, reads prior profiles, or only checks a manufactured numerical identity. Inspect the handling of exceptions, finite values, boundaries and convergence; a successful process exit alone is not a scientific validation.

Reproduction note: compare any outputs with the companion JSON and/or NPZ by exact archive name and digest. The filename timestamp is metadata, not proof of run chronology. No independent execution of this script is claimed in this companion.

# Source 70/77 — v713_b17_six_arm_profile_audit

Full archive filename: v713_b17_six_arm_profile_audit_20261021_012328_469.py

Source length: 43 lines; bytes: 3256; SHA-256: a74f21a3b9df309491b59f4122b245a5a080dadd20a3d5484b39df497ecde608.

Functions: load(path,name):).

Classes: none. Imports: from pathlib import Path; import json,csv,hashlib,importlib.util,sys,time; import numpy as np.

Opening source documentation: Exploratory B17 six-arm common-grid audit; not a preregistered confirmatory outcome.

Static implementation signals (presence in source, not proof of execution): JSON output, internal assertion, hash verification, solver path.

Source excerpt (first 23 physical lines; whitespace and line order retained):

```
 1 #!/usr/bin/env python3
 2 """Exploratory B17 six-arm common-grid audit; not a preregistered confirmatory outcome."""
 3 from pathlib import Path
 4 import json,csv,hashlib,importlib.util,sys,time
 5 import numpy as np
 6 R=Path('/mnt/data/v713_b07_run')
 7 plan=json.loads((R/'preregistration_plan.json').read_text())
 8 for n,d in plan['frozen_hashes'].items():
 9  assert hashlib.sha256((R/n).read_bytes()).hexdigest()==d,n
10 def load(path,name):
11  spec=importlib.util.spec_from_file_location(name,path);m=importlib.util.module_from_spec(spec);sys.
12 old=load(R/'UDCT_v7_12_Preregistration_Jeans_Transition.py','v712_six_audit')
13 proto=load(R/'UDCT_v7_13_Six_Arm_Decision_Prototype.py','v713_six_proto')
14 m=old.load_frozen();b=m.load_baseline();obj=b.DWARFS['Hydrus I']
15 with (R/'adaptive_brackets.csv').open() as f: t=list(csv.DictReader(f))[16]
16 assert t['axis']=='ell_pc' and t['law']=='A-Qs' and float(t['fixed_coordinate'])==0.45
17 configs=[('default',b.make_grid('default')),('refined',b.make_grid('refined')),('validation',b.Grid(
18 all_arms={float(t['lo']):{},float(t['hi']):{}}
19 per_arm=[];t0=time.monotonic()
20 for name,grid in configs:
21  e,h=b.V706.galactic_field(obj,grid)[:2]
22  for inner in (.002,.0005):
23   for ell in all_arms:
```

Audit interpretation: Identify the exact inputs, units, normalization and frozen constants in the full script. Determine whether this file computes a new physical profile, reads prior profiles, or only checks a manufactured numerical identity. Inspect the handling of exceptions, finite values, boundaries and convergence; a successful process exit alone is not a scientific validation.

Reproduction note: compare any outputs with the companion JSON and/or NPZ by exact archive name and digest. The filename timestamp is metadata, not proof of run chronology. No independent execution of this script is claimed in this companion.

# Source 71/77 — v713_b19_six_arm_profile_audit

Full archive filename: v713_b19_six_arm_profile_audit_20261021_012316_290.py

Source length: 43 lines; bytes: 3256; SHA-256: 873edd23f52b735795074fc3897dcc6f9f1653568336f4e63e81cefaf97264a2.

Functions: load(path,name):).

Classes: none. Imports: from pathlib import Path; import json,csv,hashlib,importlib.util,sys,time; import numpy as np.

Opening source documentation: Exploratory B19 six-arm common-grid audit; not a preregistered confirmatory outcome.

Static implementation signals (presence in source, not proof of execution): JSON output, internal assertion, hash verification, solver path.

Source excerpt (first 23 physical lines; whitespace and line order retained):

```
 1 #!/usr/bin/env python3
 2 """Exploratory B19 six-arm common-grid audit; not a preregistered confirmatory outcome."""
 3 from pathlib import Path
 4 import json,csv,hashlib,importlib.util,sys,time
 5 import numpy as np
 6 R=Path('/mnt/data/v713_b07_run')
 7 plan=json.loads((R/'preregistration_plan.json').read_text())
 8 for n,d in plan['frozen_hashes'].items():
 9  assert hashlib.sha256((R/n).read_bytes()).hexdigest()==d,n
10 def load(path,name):
11  spec=importlib.util.spec_from_file_location(name,path);m=importlib.util.module_from_spec(spec);sys.
12 old=load(R/'UDCT_v7_12_Preregistration_Jeans_Transition.py','v712_six_audit')
13 proto=load(R/'UDCT_v7_13_Six_Arm_Decision_Prototype.py','v713_six_proto')
14 m=old.load_frozen();b=m.load_baseline();obj=b.DWARFS['Hydrus I']
15 with (R/'adaptive_brackets.csv').open() as f: t=list(csv.DictReader(f))[18]
16 assert t['axis']=='beta' and t['law']=='A-Qd' and float(t['fixed_coordinate'])==1.0
17 configs=[('default',b.make_grid('default')),('refined',b.make_grid('refined')),('validation',b.Grid(
18 all_arms={float(t['lo']):{},float(t['hi']):{}}
19 per_arm=[];t0=time.monotonic()
20 for name,grid in configs:
21  e,h=b.V706.galactic_field(obj,grid)[:2]
22  for inner in (.002,.0005):
23   for beta in all_arms:
```

Audit interpretation: Identify the exact inputs, units, normalization and frozen constants in the full script. Determine whether this file computes a new physical profile, reads prior profiles, or only checks a manufactured numerical identity. Inspect the handling of exceptions, finite values, boundaries and convergence; a successful process exit alone is not a scientific validation.

Reproduction note: compare any outputs with the companion JSON and/or NPZ by exact archive name and digest. The filename timestamp is metadata, not proof of run chronology. No independent execution of this script is claimed in this companion.

# Source 72/77 — v713_b20_six_arm_profile_audit

Full archive filename: v713_b20_six_arm_profile_audit_20261021_012309_515.py

Source length: 43 lines; bytes: 3256; SHA-256: 71c57c3aea0d208b266987ef032ece9ff239cb3bd15397d1be488603112a34bf.

Functions: load(path,name):).

Classes: none. Imports: from pathlib import Path; import json,csv,hashlib,importlib.util,sys,time; import numpy as np.

Opening source documentation: Exploratory B20 six-arm common-grid audit; not a preregistered confirmatory outcome.

Static implementation signals (presence in source, not proof of execution): JSON output, internal assertion, hash verification, solver path.

Source excerpt (first 23 physical lines; whitespace and line order retained):

```
 1 #!/usr/bin/env python3
 2 """Exploratory B20 six-arm common-grid audit; not a preregistered confirmatory outcome."""
 3 from pathlib import Path
 4 import json,csv,hashlib,importlib.util,sys,time
 5 import numpy as np
 6 R=Path('/mnt/data/v713_b07_run')
 7 plan=json.loads((R/'preregistration_plan.json').read_text())
 8 for n,d in plan['frozen_hashes'].items():
 9  assert hashlib.sha256((R/n).read_bytes()).hexdigest()==d,n
10 def load(path,name):
11  spec=importlib.util.spec_from_file_location(name,path);m=importlib.util.module_from_spec(spec);sys.
12 old=load(R/'UDCT_v7_12_Preregistration_Jeans_Transition.py','v712_six_audit')
13 proto=load(R/'UDCT_v7_13_Six_Arm_Decision_Prototype.py','v713_six_proto')
14 m=old.load_frozen();b=m.load_baseline();obj=b.DWARFS['Hydrus I']
15 with (R/'adaptive_brackets.csv').open() as f: t=list(csv.DictReader(f))[19]
16 assert t['axis']=='beta' and t['law']=='A-Qd' and float(t['fixed_coordinate'])==1.2
17 configs=[('default',b.make_grid('default')),('refined',b.make_grid('refined')),('validation',b.Grid(
18 all_arms={float(t['lo']):{},float(t['hi']):{}}
19 per_arm=[];t0=time.monotonic()
20 for name,grid in configs:
21  e,h=b.V706.galactic_field(obj,grid)[:2]
22  for inner in (.002,.0005):
23   for beta in all_arms:
```

Audit interpretation: Identify the exact inputs, units, normalization and frozen constants in the full script. Determine whether this file computes a new physical profile, reads prior profiles, or only checks a manufactured numerical identity. Inspect the handling of exceptions, finite values, boundaries and convergence; a successful process exit alone is not a scientific validation.

Reproduction note: compare any outputs with the companion JSON and/or NPZ by exact archive name and digest. The filename timestamp is metadata, not proof of run chronology. No independent execution of this script is claimed in this companion.

# Source 73/77 — v713_b21_six_arm_profile_audit

Full archive filename: v713_b21_six_arm_profile_audit_20261021_012303_573.py

Source length: 43 lines; bytes: 3256; SHA-256: b22c4baf0f9d4e8711dad6cf2bf91978770fe09a167796a85b206f3fd80f16e9.

Functions: load(path,name):).

Classes: none. Imports: from pathlib import Path; import json,csv,hashlib,importlib.util,sys,time; import numpy as np.

Opening source documentation: Exploratory B21 six-arm common-grid audit; not a preregistered confirmatory outcome.

Static implementation signals (presence in source, not proof of execution): JSON output, internal assertion, hash verification, solver path.

Source excerpt (first 23 physical lines; whitespace and line order retained):

```
 1 #!/usr/bin/env python3
 2 """Exploratory B21 six-arm common-grid audit; not a preregistered confirmatory outcome."""
 3 from pathlib import Path
 4 import json,csv,hashlib,importlib.util,sys,time
 5 import numpy as np
 6 R=Path('/mnt/data/v713_b07_run')
 7 plan=json.loads((R/'preregistration_plan.json').read_text())
 8 for n,d in plan['frozen_hashes'].items():
 9  assert hashlib.sha256((R/n).read_bytes()).hexdigest()==d,n
10 def load(path,name):
11  spec=importlib.util.spec_from_file_location(name,path);m=importlib.util.module_from_spec(spec);sys.
12 old=load(R/'UDCT_v7_12_Preregistration_Jeans_Transition.py','v712_six_audit')
13 proto=load(R/'UDCT_v7_13_Six_Arm_Decision_Prototype.py','v713_six_proto')
14 m=old.load_frozen();b=m.load_baseline();obj=b.DWARFS['Hydrus I']
15 with (R/'adaptive_brackets.csv').open() as f: t=list(csv.DictReader(f))[20]
16 assert t['axis']=='beta' and t['law']=='A-Qd' and float(t['fixed_coordinate'])==1.5
17 configs=[('default',b.make_grid('default')),('refined',b.make_grid('refined')),('validation',b.Grid(
18 all_arms={float(t['lo']):{},float(t['hi']):{}}
19 per_arm=[];t0=time.monotonic()
20 for name,grid in configs:
21  e,h=b.V706.galactic_field(obj,grid)[:2]
22  for inner in (.002,.0005):
23   for beta in all_arms:
```

Audit interpretation: Identify the exact inputs, units, normalization and frozen constants in the full script. Determine whether this file computes a new physical profile, reads prior profiles, or only checks a manufactured numerical identity. Inspect the handling of exceptions, finite values, boundaries and convergence; a successful process exit alone is not a scientific validation.

Reproduction note: compare any outputs with the companion JSON and/or NPZ by exact archive name and digest. The filename timestamp is metadata, not proof of run chronology. No independent execution of this script is claimed in this companion.

# Source 74/77 — v713_b22_six_arm_profile_audit

Full archive filename: v713_b22_six_arm_profile_audit_20261021_012257_380.py

Source length: 43 lines; bytes: 3253; SHA-256: ef2dc2de328d1e0a448f6734f29574ac373bebfb0b16483742c4fe0a343c1be4.

Functions: load(path,name):).

Classes: none. Imports: from pathlib import Path; import json,csv,hashlib,importlib.util,sys,time; import numpy as np.

Opening source documentation: Exploratory B22 six-arm common-grid audit; not a preregistered confirmatory outcome.

Static implementation signals (presence in source, not proof of execution): JSON output, internal assertion, hash verification, solver path.

Source excerpt (first 23 physical lines; whitespace and line order retained):

```
 1 #!/usr/bin/env python3
 2 """Exploratory B22 six-arm common-grid audit; not a preregistered confirmatory outcome."""
 3 from pathlib import Path
 4 import json,csv,hashlib,importlib.util,sys,time
 5 import numpy as np
 6 R=Path('/mnt/data/v713_b07_run')
 7 plan=json.loads((R/'preregistration_plan.json').read_text())
 8 for n,d in plan['frozen_hashes'].items():
 9  assert hashlib.sha256((R/n).read_bytes()).hexdigest()==d,n
10 def load(path,name):
11  spec=importlib.util.spec_from_file_location(name,path);m=importlib.util.module_from_spec(spec);sys.
12 old=load(R/'UDCT_v7_12_Preregistration_Jeans_Transition.py','v712_six_audit')
13 proto=load(R/'UDCT_v7_13_Six_Arm_Decision_Prototype.py','v713_six_proto')
14 m=old.load_frozen();b=m.load_baseline();obj=b.DWARFS['Hydrus I']
15 with (R/'adaptive_brackets.csv').open() as f: t=list(csv.DictReader(f))[21]
16 assert t['axis']=='ell_pc' and t['law']=='A-Qd' and float(t['fixed_coordinate'])==0.4
17 configs=[('default',b.make_grid('default')),('refined',b.make_grid('refined')),('validation',b.Grid(
18 all_arms={float(t['lo']):{},float(t['hi']):{}}
19 per_arm=[];t0=time.monotonic()
20 for name,grid in configs:
21  e,h=b.V706.galactic_field(obj,grid)[:2]
22  for inner in (.002,.0005):
23   for ell in all_arms:
```

Audit interpretation: Identify the exact inputs, units, normalization and frozen constants in the full script. Determine whether this file computes a new physical profile, reads prior profiles, or only checks a manufactured numerical identity. Inspect the handling of exceptions, finite values, boundaries and convergence; a successful process exit alone is not a scientific validation.

Reproduction note: compare any outputs with the companion JSON and/or NPZ by exact archive name and digest. The filename timestamp is metadata, not proof of run chronology. No independent execution of this script is claimed in this companion.

# Source 75/77 — v713_b23_six_arm_profile_audit

Full archive filename: v713_b23_six_arm_profile_audit_20261021_012252_402.py

Source length: 43 lines; bytes: 3254; SHA-256: 87ec05d7cb1501da14b8164d6902077a7092ba1d7d7d6edfdce2920d2bc1429b.

Functions: load(path,name):).

Classes: none. Imports: from pathlib import Path; import json,csv,hashlib,importlib.util,sys,time; import numpy as np.

Opening source documentation: Exploratory B23 six-arm common-grid audit; not a preregistered confirmatory outcome.

Static implementation signals (presence in source, not proof of execution): JSON output, internal assertion, hash verification, solver path.

Source excerpt (first 23 physical lines; whitespace and line order retained):

```
 1 #!/usr/bin/env python3
 2 """Exploratory B23 six-arm common-grid audit; not a preregistered confirmatory outcome."""
 3 from pathlib import Path
 4 import json,csv,hashlib,importlib.util,sys,time
 5 import numpy as np
 6 R=Path('/mnt/data/v713_b07_run')
 7 plan=json.loads((R/'preregistration_plan.json').read_text())
 8 for n,d in plan['frozen_hashes'].items():
 9  assert hashlib.sha256((R/n).read_bytes()).hexdigest()==d,n
10 def load(path,name):
11  spec=importlib.util.spec_from_file_location(name,path);m=importlib.util.module_from_spec(spec);sys.
12 old=load(R/'UDCT_v7_12_Preregistration_Jeans_Transition.py','v712_six_audit')
13 proto=load(R/'UDCT_v7_13_Six_Arm_Decision_Prototype.py','v713_six_proto')
14 m=old.load_frozen();b=m.load_baseline();obj=b.DWARFS['Hydrus I']
15 with (R/'adaptive_brackets.csv').open() as f: t=list(csv.DictReader(f))[22]
16 assert t['axis']=='ell_pc' and t['law']=='A-Qd' and float(t['fixed_coordinate'])==0.45
17 configs=[('default',b.make_grid('default')),('refined',b.make_grid('refined')),('validation',b.Grid(
18 all_arms={float(t['lo']):{},float(t['hi']):{}}
19 per_arm=[];t0=time.monotonic()
20 for name,grid in configs:
21  e,h=b.V706.galactic_field(obj,grid)[:2]
22  for inner in (.002,.0005):
23   for ell in all_arms:
```

Audit interpretation: Identify the exact inputs, units, normalization and frozen constants in the full script. Determine whether this file computes a new physical profile, reads prior profiles, or only checks a manufactured numerical identity. Inspect the handling of exceptions, finite values, boundaries and convergence; a successful process exit alone is not a scientific validation.

Reproduction note: compare any outputs with the companion JSON and/or NPZ by exact archive name and digest. The filename timestamp is metadata, not proof of run chronology. No independent execution of this script is claimed in this companion.

# Source 76/77 — v713_b24_six_arm_profile_audit

Full archive filename: v713_b24_six_arm_profile_audit_20261021_012248_516.py

Source length: 43 lines; bytes: 3253; SHA-256: 56f6a447378f7e8cd51b0debff15ee3e74d7c30bd7249b14d979782d415367d9.

Functions: load(path,name):).

Classes: none. Imports: from pathlib import Path; import json,csv,hashlib,importlib.util,sys,time; import numpy as np.

Opening source documentation: Exploratory B24 six-arm common-grid audit; not a preregistered confirmatory outcome.

Static implementation signals (presence in source, not proof of execution): JSON output, internal assertion, hash verification, solver path.

Source excerpt (first 23 physical lines; whitespace and line order retained):

```
 1 #!/usr/bin/env python3
 2 """Exploratory B24 six-arm common-grid audit; not a preregistered confirmatory outcome."""
 3 from pathlib import Path
 4 import json,csv,hashlib,importlib.util,sys,time
 5 import numpy as np
 6 R=Path('/mnt/data/v713_b07_run')
 7 plan=json.loads((R/'preregistration_plan.json').read_text())
 8 for n,d in plan['frozen_hashes'].items():
 9  assert hashlib.sha256((R/n).read_bytes()).hexdigest()==d,n
10 def load(path,name):
11  spec=importlib.util.spec_from_file_location(name,path);m=importlib.util.module_from_spec(spec);sys.
12 old=load(R/'UDCT_v7_12_Preregistration_Jeans_Transition.py','v712_six_audit')
13 proto=load(R/'UDCT_v7_13_Six_Arm_Decision_Prototype.py','v713_six_proto')
14 m=old.load_frozen();b=m.load_baseline();obj=b.DWARFS['Hydrus I']
15 with (R/'adaptive_brackets.csv').open() as f: t=list(csv.DictReader(f))[23]
16 assert t['axis']=='ell_pc' and t['law']=='A-Qd' and float(t['fixed_coordinate'])==0.5
17 configs=[('default',b.make_grid('default')),('refined',b.make_grid('refined')),('validation',b.Grid(
18 all_arms={float(t['lo']):{},float(t['hi']):{}}
19 per_arm=[];t0=time.monotonic()
20 for name,grid in configs:
21  e,h=b.V706.galactic_field(obj,grid)[:2]
22  for inner in (.002,.0005):
23   for ell in all_arms:
```

Audit interpretation: Identify the exact inputs, units, normalization and frozen constants in the full script. Determine whether this file computes a new physical profile, reads prior profiles, or only checks a manufactured numerical identity. Inspect the handling of exceptions, finite values, boundaries and convergence; a successful process exit alone is not a scientific validation.

Reproduction note: compare any outputs with the companion JSON and/or NPZ by exact archive name and digest. The filename timestamp is metadata, not proof of run chronology. No independent execution of this script is claimed in this companion.

# Source 77/77 — v713_validate_b01

Full archive filename: v713_validate_b01_20261021_012448_352.py

Source length: 24 lines; bytes: 2020; SHA-256: 0bb7e1d64c3a275d942b735b6b6cf446255b7825bc72c05b9eab7d648d827e95.

Functions: No named function definitions found.

Classes: none. Imports: from pathlib import Path; import sys,importlib.util,json,hashlib,time,numpy as np.

Static implementation signals (presence in source, not proof of execution): JSON output, internal assertion, hash verification, solver path.

Source excerpt (first 23 physical lines; whitespace and line order retained):

```
 1 from pathlib import Path
 2 import sys,importlib.util,json,hashlib,time,numpy as np
 3 R=Path('/mnt/data/v713_post_work');plan=json.loads((R/'preregistration_plan.json').read_text())
 4 for n,h in plan['frozen_hashes'].items():assert hashlib.sha256((R/n).read_bytes()).hexdigest()==h
 5 spec=importlib.util.spec_from_file_location('frozen_712_validation',R/'UDCT_v7_12_Preregistration_Je
 6 m=v.load_frozen();b=m.load_baseline();obj=b.DWARFS['Hydrus I'];t=time.monotonic()
 7 grid=b.Grid(radial=1440,polar=96,azimuth=144,disk_radial=480,disk_azimuth=256,disk_height=48)
 8 e,h,mass=b.V706.galactic_field(obj,grid);print('FIELD seconds',round(time.monotonic()-t,2),flush=Tru
 9 results=[]
10 for inner in (.002,.0005):
11  st=b.model_state(obj,'Tapered cusp',b.mesh('Tapered cusp',grid,rmin=inner),e,h,grid,b.BASE_LAW,1.,0
12  _,force=b.V706.work_from_state(st)
13  _,ref=b.V706.aperture_jeans(st,force,obj.re_pc)
14  pressure,_,_=m.solve(st['r'],st['rho'],force,0.)
15  sigma=np.divide(pressure,st['rho'],out=np.zeros_like(pressure),where=st['rho']>1e-280)
16  vref2=float(np.interp(1.,st['r']/st['rh'],st['gn']))*st['rh']
17  gate=float(np.max(abs(sigma-ref))/vref2)
18  assert np.isfinite(gate) and gate<1e-10,gate
19  for beta in (.35,.4):
20   rec=m.summary(st['r'],st['rho'],force,beta,st['rh'],st['gn'])
21   results.append({'inner_rh':inner,'beta':beta,'min_sigma2_kms2':rec['min_sigma2_kms2'],'min_normali
22  print('inner',inner,'done',flush=True)
23 out={'status':'EXPLORATORY validation-grid B01 two arms; NOT six-arm full profile classification','s
```

Audit interpretation: Identify the exact inputs, units, normalization and frozen constants in the full script. Determine whether this file computes a new physical profile, reads prior profiles, or only checks a manufactured numerical identity. Inspect the handling of exceptions, finite values, boundaries and convergence; a successful process exit alone is not a scientific validation.

Reproduction note: compare any outputs with the companion JSON and/or NPZ by exact archive name and digest. The filename timestamp is metadata, not proof of run chronology. No independent execution of this script is claimed in this companion.

# Part IV — Evidence ledger and outstanding tests

## Confirmed archival coverage

24 target IDs, six-arm common-domain profiles and a 162-member source archive are present. The target inventory reports 24 unresolved strict brackets and zero physical 3-D Hessian executions.

## Endpoint audit

The accompanying archived evidence audit reports 18 robust-positive, 30 unresolved and zero robust-negative endpoints. These are classifications of model computations, not empirical detection rates.

## Incomplete per-target JSON

B18 lacks an individual exploratory JSON, although its profiles occur in the common NPZ. B19 has duplicate individual JSON records. The missing native-F entries for seven endpoints cannot be independently recovered from common-domain NPZ alone.

## Implementation tests

Finite-difference Hessian stencils, Poisson manufactured solutions and corrected action variation are implementation checks. Their existence does not demonstrate a physically converged total-potential Hessian.

## Direct observations

The Hydrus I model uses observationally informed inputs. No fit to the complete individual-star line-of-sight velocity sample is established here.

## Future work

Independent solver implementation, physical exterior boundary specification, convergence at three Hessian resolutions, shared-radius negative witness checks, distribution-function admissibility and direct stellar-data likelihood fitting remain priorities.

# Appendix — Exact source file inventory

UDCT_v7_13_3D_Constitutive_Poisson_Coupling_Exploratory_20261021_012132_569.py | 3965 bytes | SHA-256
9117e6f13b30b97180e1d93797d797425ff1435ec9e310373c05ce45b1eb28bd

UDCT_v7_13_3D_Constitutive_Poisson_Coupling_Factor_Corrected_20261021_012116_770.json | 4188 bytes | SHA-256
ac6c9896a88ccbe707b3b541a186a35c98a09b4914a144d78c566cad7ab3f2e8

UDCT_v7_13_3D_Constitutive_Poisson_Coupling_Factor_Corrected_20261021_012118_509.py | 4091 bytes | SHA-256
732cf09e9eb5bc001451804d025dec25471e24af90df8abc06e602cb1d0fed87

UDCT_v7_13_3D_Hessian_Gate_Audit_20261020_20261021_011634_216.json | 1302 bytes | SHA-256
58e33822cab3b81548620bc96f880ccd97b89fa5c9572f9c2b1c9b310d97d52b

UDCT_v7_13_3D_Poisson_Manufactured_Control_20261021_012151_118.json | 1779 bytes | SHA-256
c8c70349d11345f358d57a1f7e7c5dadc66fc2ac38463438899fa22f151df877

UDCT_v7_13_3D_Poisson_Solver_Manufactured_Control_20261021_012151_799.py | 3212 bytes | SHA-256
b96bdabce1bbd8d51d5509391ba7b979fdaca16c53a05577f2dceb851b660d34

UDCT_v7_13_3D_Potential_Implementation_Gate_20261021_012207_049.json | 1991 bytes | SHA-256
3a2f42a372d8c9f20cf6124ffe36e562175338f870ba722c0a61b52371ab0214

UDCT_v7_13_3D_Potential_Implementation_Gate_20261021_012207_715.py | 2006 bytes | SHA-256
1d2b75de19a8dd99cf2ebfbeb12edb46afba4b5d25eee7162f4886233c435948

UDCT_v7_13_Acceleration_Richardson_Diagnostic_20261021_011856_979.json | 2545 bytes | SHA-256
05e5d672799274a77d8f679a015359d659d9e2c40ce27d812af561fd2a58e545

UDCT_v7_13_Acceleration_Richardson_Diagnostic_20261021_011857_779.py | 1977 bytes | SHA-256
41ddb033101b49f1fc869662b30eef789cf88f5fe21a07bd89fa4aa04994397b

UDCT_v7_13_Action_Variation_Factor_Audit_20261021_012119_839.json | 1330 bytes | SHA-256
b2c5440345862c153e11621b464b232fab7cf3840c8571bf3ded7f023b64318c

UDCT_v7_13_Action_Variation_Factor_Audit_20261021_012121_374.py | 3025 bytes | SHA-256
9ea62ea5348764013bbc86d7c00c5a48d9f93fa0f6ed27bf30dfb9bc50f56ab3

UDCT_v7_13_B01_B24_All_Common_Profiles_20261021_011738_243.npz | 2661383 bytes | SHA-256
a6a040bd7987f99b2f699b0cc776c088253f0d8777b4714b29386ab43f7e2bec

UDCT_v7_13_B01_B24_Six_Arm_Inventory_20261020_20261021_011759_252.json | 8745 bytes | SHA-256
2691edabb8b69a49f3dad41967f7fc597a7c56050dc5a97448001791a86f5739

UDCT_v7_13_B01_B24_Trace_Inventory_20261021_011739_280.json | 3994 bytes | SHA-256
100d324e9d8648a68b3fc942cd3b721b9eecd1d899d2feaea32af7680583a880

UDCT_v7_13_B01_B24_Trace_Inventory_20261021_011740_803.py | 2092 bytes | SHA-256
f5e1a15d44d7f0952728eff1daaaee8dd056865bb8de46b3738120123d7a7016

UDCT_v7_13_B01_Explicit_Hessian_Ablation_20261021_011652_218.json | 3213 bytes | SHA-256
f65d3f213a9b51e6ce918a12004921b075f9dca83193da60923bc8b93a71f75f

UDCT_v7_13_B01_Explicit_Hessian_Ablation_20261021_011656_464.py | 3003 bytes | SHA-256
b7779b7678a52369d3435a4566b54b81df24c98ea698c03bf573d606d67ed508

UDCT_v7_13_B01_Full_Common_Profiles_20261021_011746_763.npz | 110342 bytes | SHA-256
12cf1f5265f125a8dd075e8179718a417e5aeba2c13d57f91e77ee8e1e8c1461

UDCT_v7_13_B01_Full_Profile_Trace_20261021_011747_575.json | 4804 bytes | SHA-256
70a7e9cbd11a23d2d029437069fd4398b3a0d81d4c4d0347040c4102a9a1f010

UDCT_v7_13_B01_Hessian_Law_Control_Exploratory_20261021_011700_929.json | 5661 bytes | SHA-256
876f17e32e02fe32096fbc0db9d428194ba2b96bfb6e26f14fb065dda50cede0

UDCT_v7_13_B01_Hessian_Law_Control_Exploratory_20261021_011702_712.py | 2648 bytes | SHA-256
2b8083fd691f1e39a7dfce89d36ff2b5c983af7be59f1558a9d66262143f2cf2

UDCT_v7_13_B01_Independent_Jeans_Quadrature_Audit_20261021_012453_346.py | 2409 bytes | SHA-256
22ffabd0e7434d3b05e715214bfdbaa72356e942b10d3b6b24c1866032b81f06

UDCT_v7_13_B01_Inner_Cutoff_Stability_20261021_011709_985.json | 2654 bytes | SHA-256
44fbac8e50d4d3823dab2f139d5133b971fad01921a9e13e8229e48eba688d2e

UDCT_v7_13_B01_Inner_Cutoff_Stability_20261021_011711_520.py | 2112 bytes | SHA-256
96937328f38b24cfe7c922c3c33cc703868035ba0dfa5041263e53ebe46c478b

UDCT_v7_13_B01_Quadrature_Audit_20261021_012454_325.json | 2747 bytes | SHA-256
131723fee2d6bc914bf4a6058c1294a5f49edcbcfed7459c46093aa59bd7b1b9

UDCT_v7_13_B01_Radial_Domain_Audit_20261021_011729_130.json | 2866 bytes | SHA-256
411b227f891161576d9ea6b3d6362d393015500223a4212367818d52005d7e18

UDCT_v7_13_B01_Radial_Domain_Audit_20261021_011730_078.py | 2119 bytes | SHA-256
17f6a90bef9f02a945e19d3750ad75edcaab0ce34ec0bae8949b89314dff80bd

UDCT_v7_13_B01_Radial_Zero_Exploratory_20261021_011717_972.json | 3093 bytes | SHA-256
6ce205f6c41fe0931e8c8c83b1dd0613b9c6e64be95093fb7b65ef6dbf07f303

UDCT_v7_13_B01_Radial_Zero_Exploratory_20261021_011719_053.py | 2632 bytes | SHA-256
9cdc1cf246ad68a68b988c985bfa97fd9b8031c0ee92d56496616a6b7eaf5948

UDCT_v7_13_B01_Six_Arm_Profile_Exploratory_20261021_012443_978.json | 3256 bytes | SHA-256
a17c57eeca8bb1762b7b37b22683f371180059e070784213c3dd7e73a2ef34da

UDCT_v7_13_B01_Stress_Derivative_Decomposition_20261021_011643_661.json | 7355 bytes | SHA-256
dcc79cae4c8c19002e3df90a587ba7927c48696b799f95b529d73c0e047e6839

UDCT_v7_13_B01_Stress_Derivative_Decomposition_20261021_011646_125.py | 3727 bytes | SHA-256
5c0a5d8a00ae9374bf8830181f99c1a9377ff4aa5a6d53eddd05273536d67ed6

UDCT_v7_13_B01_validation_grid_exploratory_20261021_012447_920.json | 1246 bytes | SHA-256
6e4a2f01f9c881b89c6ba1f254e1680673922c4ed5f80aefdae921f96857202e

UDCT_v7_13_B02_B24_Full_Profile_Trace_20261021_011741_455.py | 5067 bytes | SHA-256
9a521ddee5a211759b5353628f6dfc2988ba38322655cefb3856271e7754acb5

UDCT_v7_13_B02_Six_Arm_Profile_Exploratory_20261021_012440_907.json | 3230 bytes | SHA-256
ca7122df533d58b9f612da76fd0a54e1dc3efe07f539def1192bd565205f01d2

UDCT_v7_13_B03_Six_Arm_Profile_Exploratory_20261021_012436_828.json | 3236 bytes | SHA-256
089d98253c58a70efe3fcc960cab3e2ccabd195e37bcfa544274c05b2dc8cd00

UDCT_v7_13_B04_Six_Arm_Profile_Exploratory_20261021_012433_635.json | 3492 bytes | SHA-256
f009dec73106ce24a31e5d30e46983f1bcb9d1d51a8592896a2ca3b82721955b

UDCT_v7_13_B05_Six_Arm_Profile_Exploratory_20261021_012429_424.json | 3501 bytes | SHA-256
15a658d28edf6e7a76e2aca6f6c6c723701b1b99cab5692e6ec8a99ef15d58a6

UDCT_v7_13_B06_Six_Arm_Profile_Exploratory_20261021_012425_495.json | 3480 bytes | SHA-256
407c381cbc2358872a8527b6d8950e4354c1e1ac01154262ed6b7eba5b7bd081

UDCT_v7_13_B07_Six_Arm_Profile_Exploratory_20261021_012418_036.json | 3211 bytes | SHA-256
9e376b4e7cb3d553012a0e9131d7319cb07a9f94e69d3d11bfff46130d36060c

UDCT_v7_13_B08_Six_Arm_Profile_Exploratory_20261021_012413_596.json | 3236 bytes | SHA-256
2f1ff5d693a93790f0e82776cf971dd152105a0001185905c346d481ceed52b3

UDCT_v7_13_B09_Six_Arm_Profile_Exploratory_20261021_012409_760.json | 3248 bytes | SHA-256
f04ef7a87a6ac72d00e935df585023a166423fa36513ea80a1c17e37156fc229

UDCT_v7_13_B10_Six_Arm_Profile_Exploratory_20261021_012405_115.json | 3246 bytes | SHA-256
326ce8d33916d37535a3d97b25877a7f2fc7031b22e20e60e5571ad07992c979

UDCT_v7_13_B11_Six_Arm_Profile_Exploratory_20261021_012358_483.json | 3247 bytes | SHA-256
5b3b247ad4d17c92687a217f320ddd532c603a7878e1fa27b6aecfeef163ca9e

UDCT_v7_13_B12_Six_Arm_Profile_Exploratory_20261021_011759_654.json | 3243 bytes | SHA-256
8da915f3044b2f142aaa8730438d5b84d912c34945528356c0efc8bc563fddcc

UDCT_v7_13_B13_Six_Arm_Profile_Exploratory_20261021_012349_829.json | 3276 bytes | SHA-256
1285098b8ab4b2804b6d6b3b01d2e283a76d8e626f4cf4f40c5b1021ec5685fc

UDCT_v7_13_B14_Six_Arm_Profile_Exploratory_20261021_012339_663.json | 3238 bytes | SHA-256
2de7a9844738bbf7c05b54d2562ce207303e4992d91599ee83252076a26e34f6

UDCT_v7_13_B15_Six_Arm_Profile_Exploratory_20261021_012335_481.json | 3242 bytes | SHA-256
9f53aba24413edca1a5f96442849406e3cdce5837afd6708ec8194541d2a3ad9

UDCT_v7_13_B16_Six_Arm_Profile_Exploratory_20261021_012331_628.json | 3289 bytes | SHA-256
0a0584538782ec683fd850b59a5f90e9608ead5d8a1ceaf45f3aab7951b61de3

UDCT_v7_13_B17_Six_Arm_Profile_Exploratory_20261021_012327_430.json | 3263 bytes | SHA-256
2fce5b4a0ec159f59af5fab8cd6d3fb6a0c63b1d6230b86e43a7c67e1a37c877

UDCT_v7_13_B19_Six_Arm_Profile_Exploratory_20261021_012314_716.json | 3259 bytes | SHA-256
732de77b0bfa2c4049497d01ce30665f9b7cebd89ff7a094bcf2e576b4cda342

UDCT_v7_13_B19_Six_Arm_Profile_Exploratory_20261021_012315_327.json | 3259 bytes | SHA-256
732de77b0bfa2c4049497d01ce30665f9b7cebd89ff7a094bcf2e576b4cda342

UDCT_v7_13_B20_Six_Arm_Profile_Exploratory_20261021_012308_699.json | 3239 bytes | SHA-256
2131b8633287e71dfb75fc37df127dc17990de17e9749bcb20d8794815af06eb

UDCT_v7_13_B21_Six_Arm_Profile_Exploratory_20261021_012302_765.json | 3250 bytes | SHA-256
9bd3837c3f50b26ef7488b285ca71f88c97e4d90e60152d39b5b35be8f82366a

UDCT_v7_13_B22_Six_Arm_Profile_Exploratory_20261021_012256_617.json | 3284 bytes | SHA-256
b4e851c61cf76b3a5d075995243f34ee47a43ceb8c8b20459d7137c731f7d2fe

UDCT_v7_13_B23_Six_Arm_Profile_Exploratory_20261021_012251_935.json | 3253 bytes | SHA-256
f7b4ff42c03d34d7c6aeb137fc90eeeedc94e908cc0c81fb1385b8bfeb3417d6

UDCT_v7_13_B24_Six_Arm_Profile_Exploratory_20261021_012247_408.json | 3261 bytes | SHA-256
732e8d0bb2e49c49ef9bc4739b0c5982ab7452d3b998c2852994f47352e7e63e

UDCT_v7_13_Boundary_Moment_Cancellation_Audit_20261021_011833_079.json | 2685 bytes | SHA-256
8965715d7e5d8efd85ceb7b6ec88be801897ba37511be4199bae8cd66ec84644

UDCT_v7_13_Boundary_Moment_Cancellation_Audit_20261021_011833_901.py | 2223 bytes | SHA-256
955e301b4c8c1b2a1b77c309bd5bc34227414b1306572a27f15584cf8515ee61

UDCT_v7_13_Common_Coordinate_Acceleration_Audit_20261021_011846_682.json | 8588 bytes | SHA-256
601014a62975d6bf1d5be403dcb3b2692ac28106dea91ba27a6c93f3f35be61f

UDCT_v7_13_Common_Coordinate_Acceleration_Audit_20261021_011847_335.py | 2582 bytes | SHA-256
8a86040ef901bc557465cfcec1d36b2e89683c41cfc3df64d2da4b4ad57ab0d4

UDCT_v7_13_Confirmatory_Readiness_Audit_20261021_011752_281.json | 11735 bytes | SHA-256
24dd0f4de408f944aa944212cac2e583572496974c0be2360e067fd3a880eb81

UDCT_v7_13_Confirmatory_Readiness_Audit_20261021_011753_298.py | 3060 bytes | SHA-256
6d68e443ce0e0e71ec1d190d50b76bedc1c394acf2edd0747f61373d90915b67

UDCT_v7_13_Constitutive_3D_Source_Prototype_20261021_012202_683.json | 3245 bytes | SHA-256
6d2cebbeac02d320ea886552f931323c0d69d998a70f8efa6a81cf8ddc399ba4

UDCT_v7_13_Constitutive_3D_Source_Prototype_20261021_012203_512.py | 3166 bytes | SHA-256
c8cc277b301d2acf3e0c7c3889bd59f24ff8957987e9bbb612a7bb17087c2d8c

UDCT_v7_13_Constitutive_3D_Source_Revalidated_20261021_012139_451.json | 3245 bytes | SHA-256
6d2cebbeac02d320ea886552f931323c0d69d998a70f8efa6a81cf8ddc399ba4

UDCT_v7_13_Constitutive_3D_Source_Revalidated_20261021_012140_566.py | 3178 bytes | SHA-256
dd2f014916449be1905e3c5152a3250257c35119a05f53e516e1f3ab6eb176e1

UDCT_v7_13_Direct_Boundary_59cube_Audit_20261021_011807_885.json | 3394 bytes | SHA-256
ab681dc93f5bc78e3bd4467eeacc5898dc220aa0a262a6940c24800b4ab275ad

UDCT_v7_13_Direct_Boundary_59cube_Audit_20261021_011808_455.py | 3073 bytes | SHA-256
6c7a6730538b61b2ebff91dc9cd2562c27208664e81c9df07acf763ae9267b2e

UDCT_v7_13_Direct_Boundary_Radial_Audit_20261021_011814_868.json | 6697 bytes | SHA-256
ab3a7b7540ee005d87a69b4c157647716aadd4cb15803738931f7d2310d8587e

UDCT_v7_13_Direct_Boundary_Radial_Audit_20261021_011815_545.py | 2529 bytes | SHA-256
1b7382db45a68bc844deac62546f231b44027a2e414064269a9b4469b31c11bf

UDCT_v7_13_Direct_Boundary_Resolution_Audit_20261021_011811_208.json | 6652 bytes | SHA-256
5532afc962e4ea471fe8deac2090c6f08932182f4e5764f16a131dd625b55938

UDCT_v7_13_Direct_Boundary_Resolution_Audit_20261021_011812_014.py | 2772 bytes | SHA-256
42a43e28ff2b965e22eb0817dba90f3429b014d02396d9970d2b2d1a4ea98e2c

UDCT_v7_13_Direct_Source_Boundary_Audit_20261021_011819_759.json | 5345 bytes | SHA-256
959fa9edb6fdb86f711273bb34cf915ed0a1370e9453c53155b90a897aa236d3

UDCT_v7_13_Direct_Source_Boundary_Audit_20261021_011820_499.py | 4232 bytes | SHA-256
7e37682cc0de3e6428f4bebae2178fd252d14d678fe7de645a4ef59ca3fcd6e1

UDCT_v7_13_Fixed_Box_Resolution_Audit_20261021_011900_790.json | 3702 bytes | SHA-256
a8ffdac950935416d2a81e20f5f3faa210e369a4d8f82ac9fcd497f85f67684c

UDCT_v7_13_Fixed_Box_Resolution_Audit_20261021_011901_260.py | 1521 bytes | SHA-256
1dd084d6a784cd74537f79ec374d2731d40c489046fb682b629176c10a02f5b7

UDCT_v7_13_Frozen_v710_Selftest_Log_20261021_012141_009.txt | 5006 bytes | SHA-256
81ad969b2baed784b0919e3e09fe8dbdb61b5945efbc2ee0ac3c0a3bd487ef7f

UDCT_v7_13_Green_Cell_Quadrature_Audit_20261021_011803_844.json | 2941 bytes | SHA-256
8f01142a521291d9fa610614bdb229fc1f8571d02ece8be64aaa3085a6e43a5e

UDCT_v7_13_Green_Cell_Quadrature_Audit_20261021_011804_539.py | 2278 bytes | SHA-256
2e4d2036a8453e8b0e803f20e04053f7fd1773ba909928b50878a60fc8e87c63

UDCT_v7_13_Hessian_Equivalent_Density_Audit_20261021_012045_768.json | 1023 bytes | SHA-256
28e12730d8053233162b7818c048036617dd22e0b870317e0f1cc9ffc1ae20db

UDCT_v7_13_Hessian_Equivalent_Density_Audit_20261021_012046_765.py | 1430 bytes | SHA-256
10072e73fbd6217c98ca935f3c9c00b08448860ce5f0310417a469b1bcc13e8e

UDCT_v7_13_Hessian_External_Field_Audit_20261021_012216_909.py | 2809 bytes | SHA-256
27f5c1e9cea2895a36e28d1fea721fdfca3013e86f20b2d6e38683ea43ed05d5

UDCT_v7_13_Hessian_Factor_Grid_Convergence_20261021_012058_970.json | 3769 bytes | SHA-256
627c6eb174a7fb74fdf9c38150b94b0f829887532dc3d845e046f87483199f79

UDCT_v7_13_Hessian_Factor_Grid_Convergence_20261021_012100_129.py | 3729 bytes | SHA-256
67be43fcea2bc12dffa2fc4754b9b040de2f5fac5fcaeb41880b8b0415188331

UDCT_v7_13_Hessian_Factor_Stress_Audit_20261021_012103_946.json | 7243 bytes | SHA-256
2685714a837c186035100b8cf792adc0405622cccaeb60fe4ffba086831cf535

UDCT_v7_13_Hessian_Factor_Stress_Audit_20261021_012105_059.py | 3177 bytes | SHA-256
620479a54966e61192f96f2323836d3483dd42a97f33905ab02a017429c0da5b

UDCT_v7_13_Hessian_Preflight_and_Estimator_Selftest_20261021_012224_191.py | 3508 bytes | SHA-256
7fe6f26020246587fe891b641df164a8bf775fb3fea463484f3e58f0245b6977

UDCT_v7_13_Hessian_Preflight_and_Estimator_Selftest_20261021_012458_489.py | 3508 bytes | SHA-256
7fe6f26020246587fe891b641df164a8bf775fb3fea463484f3e58f0245b6977

UDCT_v7_13_Hessian_Preflight_Results_20261021_012224_972.json | 2903 bytes | SHA-256
a51a12c506e0d28aed97a56860b9cd352bb6af27d8d5da729cae25c467b6cb20

UDCT_v7_13_Hessian_Preflight_Results_20261021_012459_097.json | 2903 bytes | SHA-256
a51a12c506e0d28aed97a56860b9cd352bb6af27d8d5da729cae25c467b6cb20

UDCT_v7_13_Hydrus_3D_Angular_Newtonian_Gate_20261021_011957_586.json | 4162 bytes | SHA-256
0d08c38fcb7d0ce76cd42ed15cdac8e9bd6fef85d75aaf2399460d839b54f824

UDCT_v7_13_Hydrus_3D_Angular_Newtonian_Gate_20261021_011958_087.py | 2852 bytes | SHA-256
9e7369841bb9d940ba3e835fd98e9c02e1eb265b3d773374db9c47cec2c9c359

UDCT_v7_13_Hydrus_3D_DeltaPotential_Exploratory_20261021_011907_010.json | 14237 bytes | SHA-256
8a20e3595f07751e845d810a4f13daba8cedad9f65bf7b8f28819b0a7f6eed87

UDCT_v7_13_Hydrus_3D_DeltaPotential_Exploratory_20261021_011907_412.py | 4784 bytes | SHA-256
531e28f3e5bae1643c19c4f8e8b43c0e0c933f873c101dcfbb3cf5921ce8dc80

UDCT_v7_13_Hydrus_Box_Boundary_Sensitivity_20261021_011903_745.json | 6971 bytes | SHA-256
e1f01205e55409c8be47ce679dc1aaad725feea486818deb634760565e3afc78

UDCT_v7_13_Hydrus_Box_Boundary_Sensitivity_20261021_011904_569.py | 1632 bytes | SHA-256
071209f8821298647e50e18c0f0cc5977376dd5a51f8f90df4c301bb5fb42c5d

UDCT_v7_13_Hydrus_Common_Frame_Newtonian_Audit_20261021_012019_022.json | 2709 bytes | SHA-256
6da6601d0eaa93658e117728ffd9887ac2331e793812c5c649e316fed2353728

UDCT_v7_13_Hydrus_Common_Frame_Newtonian_Audit_20261021_012020_218.py | 3842 bytes | SHA-256
d4fceefd7ae6cae91fcfdb7db7b3e06db445e891571e20aadb7a9fe6fa2de9ce

UDCT_v7_13_Hydrus_Ell_Sensitivity_Local_Source_20261021_011947_819.json | 53561 bytes | SHA-256
689ee7d3e66936e9affa324847277e0426b14524599fc1c9dc6c1875ff1e02bb

UDCT_v7_13_Hydrus_Ell_Sensitivity_Local_Source_20261021_011948_903.py | 4025 bytes | SHA-256
3e61d6103c36d65899958f9d5b4e28331a68b972e8fa28e16f19613f559890be

UDCT_v7_13_Hydrus_External_Field_3D_Input_Gate_20261021_012001_726.json | 7096 bytes | SHA-256
2ff93e4f6ef151207377e74406199016f340c7ea49158daee36eb757e2a26029

UDCT_v7_13_Hydrus_External_Field_3D_Input_Gate_20261021_012002_349.py | 3904 bytes | SHA-256
fedde7a28704bee3fa1d26c771c79926864e680427959bbdf06de697618be7aa

UDCT_v7_13_Hydrus_External_Field_vs_Tide_Audit_20261021_012006_151.json | 4782 bytes | SHA-256
55812e53be4426453e69a1dcfac528ccb73af23e5413eadfebec10658fa22ba4

UDCT_v7_13_Hydrus_External_Field_vs_Tide_Audit_20261021_012006_844.py | 3312 bytes | SHA-256
1ddc7e6528796aa9e150734ed16aeceae0579448a18d3909e6589259718f0885

UDCT_v7_13_Hydrus_External_Grid_Source_Audit_20261021_011910_184.json | 28294 bytes | SHA-256
a93107db65093a704732334c2011397b5dfcf8e6fcb9c3838b864f772c0d1bf2

UDCT_v7_13_Hydrus_External_Grid_Source_Audit_20261021_011910_302.py | 3795 bytes | SHA-256
7ae7ffa8b81b9af1f337b993a18aef040283909cc8fc4050f29c78f15d7cc376

UDCT_v7_13_Hydrus_External_Newtonian_Hessian_Exploratory_20261021_012217_257.json | 3088 bytes | SHA-256
90477b73dcf6314332ffcd21f27cd9245e7396fbec4081a0a914c7c1ce07205f

UDCT_v7_13_Hydrus_Hessian_Increment_Convergence_Audit_20261021_011930_841.json | 56075 bytes | SHA-256
f0eccb1ffe86451c7d6750eed6de1e91f0b7b87cfd55933ab8c2cda6353ab9f2

UDCT_v7_13_Hydrus_Hessian_Increment_Convergence_Audit_20261021_011931_333.py | 2464 bytes | SHA-256
63b8b67f3b157e491144d995514b449237df31af3acc72b2b83b6374ae34159a

UDCT_v7_13_Hydrus_Hessian_Increment_Convergence_Audit_summary_20261021_011929_452.txt | 9838 bytes | SHA-256
96cad32e79621f5ec4676bc2bec3479f642d426a824060264cd5b745b5ad19a3

UDCT_v7_13_Hydrus_Hessian_Orientation_Bounds_20261021_012028_843.json | 1393 bytes | SHA-256
3135bb219884652fa2a2edc0d29e1a0d97022fc8defa60656d13c7b6f54d67c2

UDCT_v7_13_Hydrus_Hessian_Orientation_Bounds_20261021_012030_205.py | 2908 bytes | SHA-256
4b73bad127b4dd5e88193de646e5a333df48487303335c93518d125deebb3164

UDCT_v7_13_Hydrus_Hessian_Scale_Comparison_20261021_012039_950.json | 1588 bytes | SHA-256
9978aca7790c0b36973efeaa810d72a3bbbcfcb123024f752cf7be87458694f1

UDCT_v7_13_Hydrus_Hessian_Scale_Comparison_20261021_012041_002.py | 2696 bytes | SHA-256
61ac3eef2ed4bc660e96188b26c5cee1fc8cd6375fc60454d0a8b255fc05d6ec

UDCT_v7_13_Hydrus_Local_3D_Source_Gate_20261021_011952_641.py | 4288 bytes | SHA-256
46a8009600d0f7549a61d93d3fb905d0966bf59252ced1e0cdff3f13e385f27a

UDCT_v7_13_Hydrus_Local_3D_Source_Gate_20261021_011953_357.json | 11339 bytes | SHA-256
a6787659fe17752a296177c971c87b382112a8b3cd85d8acb0d8691554a80745

UDCT_v7_13_Hydrus_Plummer_3D_Hessian_Control_20261021_012212_626.json | 2761 bytes | SHA-256
d9f1262bb2be1b3dcd35d27c786b1c385c3ed49f233c67572531cd33dcb3e160

UDCT_v7_13_Hydrus_Plummer_3D_Hessian_Control_20261021_012213_249.py | 4021 bytes | SHA-256
3fba5c142eb20d289fac71889344a5033d31fb9693f24ae60348fa2a57c2bec6

UDCT_v7_13_Hydrus_Radial_Tidal_Scan_20261021_012010_487.json | 6542 bytes | SHA-256
d82f67c0c4c139fceb8d776182bfcda415626df02731a0e8704a5fdf59c0e6dc

UDCT_v7_13_Hydrus_Radial_Tidal_Scan_20261021_012010_956.py | 2274 bytes | SHA-256
0a906597563a4a12fed95c16f13df2071080fc74c87e4dc4d9484536d7433cef

UDCT_v7_13_Independent_Endpoint_Decision_Audit_20261021_011733_110.json | 23588 bytes | SHA-256
ae99e13b92e51d8986b8566418414f2c2dfb9dadc5f9550b40b5451b6254630b

UDCT_v7_13_Independent_Endpoint_Decision_Audit_20261021_011734_252.py | 3716 bytes | SHA-256
ec902e4fd983c973352a92cc0d09d857e2a4aee5ddbc8043b8078fac667a50fd

UDCT_v7_13_Joint_Grid_Box_StopGate_Audit_20261021_011843_265.json | 4029 bytes | SHA-256
cbcae90fb8ab6e4f635a4d53beecdbaee173bbea42b4c439030ff8c964d02236

UDCT_v7_13_Joint_Grid_Box_StopGate_Audit_20261021_011843_977.py | 2929 bytes | SHA-256
6a563d967c40878d19dd73c0f278826d117ca6dd506b0eeaf9af918cbf386a75

UDCT_v7_13_Multipole_Boundary_Exploratory_20261021_011826_685.json | 4127 bytes | SHA-256
ad6685e9c0338866bdf9cb51af8d700d8334f527733711727805ae69012c5a1f

UDCT_v7_13_Multipole_Boundary_Exploratory_20261021_011827_040.py | 3576 bytes | SHA-256
25e4ed1340291a0335e1816797986940cfb5e1634d3019f374e34bccbe7c25ed

UDCT_v7_13_Poisson_Sign_Units_Boundary_Gate_20261021_012145_753.json | 3342 bytes | SHA-256
68d990d05e5b2917728d9e2c08fa13d2c68f3c3f4ac4a72d6d842520f9255298

UDCT_v7_13_Poisson_Sign_Units_Boundary_Gate_20261021_012146_283.py | 3454 bytes | SHA-256
7b310af88ad1323c1277f7457c8894a2c4f43507b75553b07160a4e100f6e3e7

UDCT_v7_13_Quadrupole_Boundary_Diagnostic_20261021_011829_919.json | 3693 bytes | SHA-256
f17772db2bac1568279b96348d7c57b8657016c3e2a385886aa18fdfcbc5104a

UDCT_v7_13_Quadrupole_Boundary_Diagnostic_20261021_011830_763.py | 2644 bytes | SHA-256
7904cffc875d250c06dde5124f538a7925d69f00a8b33128ae0d344d1f9b4909

UDCT_v7_13_Radial_Acceleration_Convergence_20261021_011853_198.json | 11662 bytes | SHA-256
bc0bd2cdae2d4153dcf651f3b110caae54249d792cb7de3531ab23b4db9d2624

UDCT_v7_13_Radial_Acceleration_Convergence_20261021_011853_533.py | 2819 bytes | SHA-256
bb8281c2c7dea363b30401d1a92ea6d795bba27f92b62b4e20e87e7f98cb34ac

UDCT_v7_13_Radial_Box_Size_Audit_20261021_011848_100.json | 6037 bytes | SHA-256
6ae3e56ae6bd4eb6985180679dc38017cf196102dacabb40d6e192b7577114ca

UDCT_v7_13_Radial_Box_Size_Audit_20261021_011848_697.py | 2587 bytes | SHA-256
bc23e7865073da8001e02aa969e4c404a66255d4a31b982d6116fde3a2fabea2

UDCT_v7_13_Source_Localization_Boundary_Audit_20261021_011839_531.json | 2784 bytes | SHA-256
c6558c5c2d5b8b9c317781fe69fd1b419463354f37322be5abd2508fcfa42518

UDCT_v7_13_Source_Localization_Boundary_Audit_20261021_011840_423.py | 2635 bytes | SHA-256
c2292a794d7ed00bd214dffad7206311ffb87c268d15166b9f39dc7b11e6765d

v713_b01_six_arm_profile_audit_20261021_012444_551.py | 3168 bytes | SHA-256
e17af1355b0f01bc4196cfe0748b6b3f59504ddcc2b4c5c43bdd897437462be9

v713_b02_six_arm_profile_audit_20261021_012441_190.py | 3169 bytes | SHA-256
6acbcc845b912aff0c3ff124c04b40119cc6b844355096dccf472cb2673d3d41

v713_b03_six_arm_profile_audit_20261021_012437_272.py | 3169 bytes | SHA-256
85b1ac132c0de0ab448a934321bb9e427d714c5ba71caf6188b24f03a63cb718

v713_b04_six_arm_profile_audit_20261021_012434_168.py | 3201 bytes | SHA-256
5efbf820496e151281b4b7fe78ae5d0f41a0afc9ea0cbd78272ae44676edd535

v713_b05_six_arm_profile_audit_20261021_012430_260.py | 3201 bytes | SHA-256
b6567789b3bb36f36a03bb0f03aa53f7e1c2a7d1b05ec425d181c1cdd01c2d76

v713_b07_six_arm_profile_audit_20261021_012418_439.py | 3209 bytes | SHA-256
f81f02bc7a61c25fb56213f79fa435a9665fe534193dde07bd43aedf88985c24

v713_b08_six_arm_profile_audit_20261021_012414_332.py | 3247 bytes | SHA-256
eba91e72b893a70da198c19dc809efd7dfc410008d343f98891719644e5e3a69

v713_b09_six_arm_profile_audit_20261021_012410_509.py | 3247 bytes | SHA-256
28d9a5de9d2e6556435cd25ff84c44599f5f61f1b653834bd28c5de59001e60e

v713_b10_six_arm_profile_audit_20261021_012405_948.py | 3255 bytes | SHA-256
7122406b4a868b67fb2f8450806891305c03eb057479094da5d3fc7bc24caba7

v713_b11_six_arm_profile_audit_20261021_012359_216.py | 3255 bytes | SHA-256
c14ddf61a266c47a4c8ac72da7a149ee116c6508fd206382072eb70f8b4e3efd

v713_b12_six_arm_profile_audit_20261021_011800_309.py | 3251 bytes | SHA-256
a9d3b181a2bdfe33f0ce70896bfbf7e3ac3ddda21d16c1bae01460237fcf9caa

v713_b13_six_arm_profile_audit_20261021_012350_682.py | 3251 bytes | SHA-256
c5027120d2e01596e58a7323852ee2fcbf9ce10e3e5cb97879f7f6ec7cd51c8f

v713_b14_six_arm_profile_audit_20261021_012340_232.py | 3251 bytes | SHA-256
5750cf8c95bf0c9322d650f892da513a8870c64cfa85db544e5e2bdce151a802

v713_b15_six_arm_profile_audit_20261021_012336_457.py | 3251 bytes | SHA-256
055883ebd1d7c86303438c71310713bc28db9dcd415f72e4f3bfaed01c4e4ffb

v713_b16_six_arm_profile_audit_20261021_012332_233.py | 3255 bytes | SHA-256
1d6c4e8982b3347ec45618de0fa4652e1fbac7120ac0e2a61b921c34062d1eb4

v713_b17_six_arm_profile_audit_20261021_012328_469.py | 3256 bytes | SHA-256
a74f21a3b9df309491b59f4122b245a5a080dadd20a3d5484b39df497ecde608

v713_b19_six_arm_profile_audit_20261021_012316_290.py | 3256 bytes | SHA-256
873edd23f52b735795074fc3897dcc6f9f1653568336f4e63e81cefaf97264a2

v713_b20_six_arm_profile_audit_20261021_012309_515.py | 3256 bytes | SHA-256
71c57c3aea0d208b266987ef032ece9ff239cb3bd15397d1be488603112a34bf

v713_b21_six_arm_profile_audit_20261021_012303_573.py | 3256 bytes | SHA-256
b22c4baf0f9d4e8711dad6cf2bf91978770fe09a167796a85b206f3fd80f16e9

v713_b22_six_arm_profile_audit_20261021_012257_380.py | 3253 bytes | SHA-256
ef2dc2de328d1e0a448f6734f29574ac373bebfb0b16483742c4fe0a343c1be4

v713_b23_six_arm_profile_audit_20261021_012252_402.py | 3254 bytes | SHA-256
87ec05d7cb1501da14b8164d6902077a7092ba1d7d7d6edfdce2920d2bc1429b

v713_b24_six_arm_profile_audit_20261021_012248_516.py | 3253 bytes | SHA-256
56f6a447378f7e8cd51b0debff15ee3e74d7c30bd7249b14d979782d415367d9

v713_validate_b01_20261021_012448_352.py | 2020 bytes | SHA-256 0bb7e1d64c3a275d942b735b6b6cf446255b7825bc72c05b9eab7d648d827e95

UDCT_v7_13_3D_Constitutive_Poisson_Coupling_Exploratory_20261021_012131_309.json | 4088 bytes | SHA-256
8badf231e2df0258e90229475bc84d2cbbeedb76c708d60dfb7bc8af66119b8b

# References


v7.13 preregistration DOI https://doi.org/10.5281/zenodo.23342971

v7.12 post-registration DOI https://doi.org/10.5281/zenodo.23296444

v7.11 post-registration DOI https://doi.org/10.5281/zenodo.23263099

v7.10 post-registration DOI https://doi.org/10.5281/zenodo.23233837

v7.09 post-registration DOI https://doi.org/10.5281/zenodo.23130843

Primary evidence: unchanged user-provided 7.13 Hessian.zip